AN UNOFFICIAL GUIDE

Deep Agents

The Complete Guide to LangChain's Deep Agents

Architecture, execution, context, delegation and steering, deployment, protocols and OpenWiki, with build-alongs and a full interview preparation section.

WRITTEN BY

Sadiq G Pasha

FIRST EDITION · OCTOBER 2026 · NOT AFFILIATED WITH LANGCHAIN, INC.

Deep Agents

The Complete Guide to LangChain's Deep Agents

AN UNOFFICIAL GUIDE

A complete study guide to architecture, execution environment, context management, delegation, steering, middleware, deployment and protocols, with build-alongs and interview preparation.

Sadiq G Pasha

First edition · October 2026

Deep Agents: The Complete Guide to LangChain's Deep Agents (Unofficial)

by Sadiq G Pasha

First edition, version 1.0, October 2026.

© 2026 Sadiq G Pasha. All rights reserved. This copy is licensed to the purchaser for personal use. Please do not share it, resell it or post it online. Short quotations for review are welcome. Material adapted from MIT-licensed sources remains available under that licence, as set out in Appendix F.

Unofficial guide. This book is independent. It is not published, sponsored or endorsed by LangChain, Inc., and it is not an official part of the LangChain documentation. LangChain, LangGraph, LangSmith and Deep Agents are product names of LangChain, Inc.; all other names belong to their owners and are used only to identify the software described.

Made with AI assistance. The text, diagrams and code were drafted with the assistance of an AI model (Claude, from Anthropic), working from public documentation and under the author's direction. See Appendix F.

No warranty. The book and its code samples are provided for learning, as is, without warranty. This framework changes quickly, so check names, signatures and versions against the release you install.

Companion code and corrections: github.com/sadiqgpasha89/deep-agents-guide-code

Preface

A deep agent is an ordinary tool-calling agent that has been given the things a long-running agent actually needs: somewhere to work, a way to manage what it remembers, a way to hand work to specialists, and a way for a person to stay in control. None of those ideas is exotic on its own. The craft is in how they fit together, and that is what the Deep Agents documentation sets out, page by page, as reference material. Reference material is excellent for checking a detail. It is a poor way to learn a subject for the first time.

This book is the guide I would want on the desk while learning it. It starts from first principles, covering what an agent is and what LangChain and LangGraph each contribute, and then works outward through the four families of capability the harness provides: the execution environment, context management, delegation and steering. It goes on to the material that surrounds those ideas in practice: middleware, frontends, protocols, deployment (including Managed Deep Agents), fault tolerance and OpenWiki. Every chapter explains what a feature is, why it exists, when to use it, how it looks in code, and where it breaks.

I had two readers in mind while writing. The first is the engineer who has to build something real this quarter and needs a map of the terrain, with its defaults and its sharp edges marked. The second is the candidate preparing for an interview, who must explain not only how a feature works but why it was designed that way, and what happens in the unusual cases. For that reader, Part XII is a full interview section: one hundred and twenty-four direct, concept and scenario questions, each answered with an elaborated explanation, step-by-step reasoning, advantages, limitations, trade-offs, edge cases and likely follow-ups, followed by system-design walkthroughs, bug-spotting exercises, rapid-fire numbers and mock interview rounds.

A few notes on how to read it. The figures are drawn for this book from the architectures they describe; where the official documentation has a figure of its own, I have redrawn the idea rather than copying the image. Code is condensed to show its essential shape, so check names and signatures against the version you install, because this framework moves quickly. I have marked the version gates I know of, and where a feature is in beta the text says so.

Finally, my thanks to the LangChain team for publishing documentation that is open, detailed and candid about its own rough edges, and to everyone who answers questions in the community. Any errors in this book are mine, and I would be glad to hear about them.

Sadiq G Pasha
October 2026

About This Book

How the book is organised

The twelve parts follow the way a deep agent is built. Parts I and II get you from nothing to a running, deployable agent. Parts III to VI cover the four families of capability: execution environment, context management, delegation and steering. Parts VII to IX cover the mechanism and the connections: middleware, frontends and protocols. Part X covers OpenWiki. Part XI turns the ideas into complete projects, and Part XII prepares you to explain them under questioning. The appendices collect the reference material.

Three ways to read it

- **To build.** Read Parts I to VI in order, working through the build-alongs in Part XI as you go. Dip into Parts VII to IX when you need hooks, a UI or a protocol.
- **To prepare for an interview.** Read the first chapter of Part XII for the answer method, then work through the direct, concept and scenario questions by topic. When an answer feels thin, follow its pointer back to the chapter that teaches it. Finish with the spot-the-bug exercises and the mock rounds.
- **To look something up.** Use the contents page, the cheat sheet, the troubleshooting catalogue and the glossary in the appendices.

Reading the page

Most sections follow the same order: what a feature is, why it exists, when to use it, how it works, and what its limits and gotchas are. Coloured boxes mark the things worth noticing.

At a glance. A short summary that opens a chapter or section.

Edge case. A situation where the obvious behaviour is not what happens.

In an interview. How a point is likely to be asked, and what a strong answer includes.

Short answer. In Part XII, the thirty-second version of an answer, before the walkthrough.

Scenario. In Part XII, the situation a scenario question puts you in.

Companion code

Every code sample in the book is also available as files, organised by part and chapter, together with the complete build-along projects. Find it at github.com/sadiqgpasha89/deep-agents-guide-code. Open an issue there to report a mistake or ask for an update.

Figures, code and versions

Figures are numbered through the whole book. A few figures are labelled *schematic illustration*: they show the shape of a user interface or console rather than a literal screenshot. Code samples are Python unless a label says otherwise, and some are condensed for length. Version numbers, beta labels and example model names reflect the documentation as it stood in October 2026 and will drift; confirm them against the release you use.

Lessons List

Each part of the book is a lesson. After each one you should be able to do what its row says.

LESSON 1	**Getting Started**	Explain what an agent harness is and how Deep Agents builds on LangChain and LangGraph. Build, customise and configure a first deep agent, and choose a model for it. (8 chapters)	1
LESSON 2	**Deployment**	Take an agent from a laptop to production, managed or self-hosted, with authentication, durable execution, sandboxes, memory, channels, schedules and fault tolerance. (16 chapters)	23
LESSON 3	**Execution Environment**	Control where and how the agent acts: its tools, virtual filesystem and backends, permission rules, sandboxes, interpreters and streaming output. (8 chapters)	72
LESSON 4	**Context Management**	Decide what the agent knows and remembers: skills, memory, retrieval, offloading, summarization and model-specific profiles. (5 chapters)	93
LESSON 5	**Delegation**	Split large problems across specialists with synchronous, dynamic and asynchronous subagents, and know when to choose each. (3 chapters)	110
LESSON 6	**Steering**	Keep humans and graders in control with approvals, edits, rejections and rubric-driven quality loops. (2 chapters)	126
LESSON 7	**Middleware**	Use the hook system that powers the harness: prebuilt middleware, ordering rules, and how to write your own. (3 chapters)	133
LESSON 8	**Frontend**	Stream agent state to user interfaces: subagent cards, todo lists, sandbox views and approval flows. (2 chapters)	147
LESSON 9	**Protocols**	Connect the agent to the world with ACP, MCP, A2A and AG-UI, and know which protocol fits which job. (4 chapters)	154
LESSON 10	**OpenWiki**	Generate and maintain Markdown wikis that give coding agents durable knowledge of a repository. (6 chapters)	166
LESSON 11	**Build-Alongs**	Build four complete agents from scratch, then combine every layer in a capstone project. (5 chapters)	180
LESSON 12	**Interview Preparation**	Answer direct, concept and scenario questions with depth: advantages, limitations, trade-offs and edge cases, then practise with system-design walkthroughs, bug-spotting and mock rounds. (16 chapters)	195
APPENDICES	**Appendices**	Keep the cheat sheet, trap list, version notes, troubleshooting catalogue and glossary within reach; the notices record the licences that apply. (6 entries)	344

Contents

PART I **Getting Started** 1

1 The Big Picture 2
2 Background: Agents, LangChain and LangGraph 3
3 Overview: What Deep Agents Is 7
4 Quickstart: Your First Deep Agent 9
5 Customization: Configuring a Deep Agent 11
6 Models and Providers 16
7 Deep Agents and the Claude Agent SDK 18
8 Release History and What Changed 20

PART II **Deployment** 23

9 Managed Deep Agents: Overview 24
10 Managed Deep Agents: Quickstart and Project Structure 28
11 Managed Deep Agents: Defining the Agent 31
12 Managed Deep Agents: Instructions, Skills and Context Hub 33
13 Managed Deep Agents: Tools, Middleware and MCP Connectors 35
14 Managed Deep Agents: Durable Memory 38
15 Managed Deep Agents: Sandboxes 41
16 Managed Deep Agents: Connections, Identity and the Runtime 43
17 Managed Deep Agents: Channels and Agent-Owned Interrupts 47
18 Managed Deep Agents: Schedules 52
19 Managed Deep Agents: Deploying, the CLI and Local Development 54
20 Managed Deep Agents: Exposing the Agent over MCP, and Built-in Search 57
21 Managed Deep Agents: Evals and Migration 59
22 Tutorial: Extending the Research Assistant with Search, Memory and a Schedule 61
23 Going to Production 63
24 Fault Tolerance 69

PART III **Execution Environment** 72

25 Tools: The Built-in Toolbox 73
26 Backends: Where Files Live 75
27 Permissions: Controlling File Access 79
28 Multimodality 81
29 Sandboxes: Isolated Execution 83
30 Interpreters (Beta): Code Inside the Agent 87
31 Event Streaming (Beta) 89
32 Streaming: Watching the Agent Work 91

PART IV **Context Management** 93

33 Skills: Procedures on Demand 94
34 Memory: What the Agent Always Knows 97
35 Retrieval: Bringing in Outside Knowledge 100
36 Context Engineering: Offloading, Summarization and Isolation 104
37 Profiles (Beta): Defaults per Model and Provider 108

PART V **Delegation** 110

38 Subagents: Delegating to Specialists 111
39 Dynamic Subagents (Beta): Delegation from Code 116
40 Async Subagents: Background Work 122

PART VI **Steering** 126

41 Human-in-the-Loop: Approvals and Interrupts 127
42 Grading Rubrics (Beta) 130

PART VII **Middleware** 133
43 Middleware: Overview 134
44 Prebuilt Middleware 137
45 Custom Middleware 143

PART VIII **Frontend** 147
46 Frontend: Overview 148
47 Frontend Patterns 150

PART IX **Protocols** 154
48 ACP: Agent Client Protocol 155
49 MCP with LangChain 157
50 A2A with LangSmith 161
51 AG-UI: Agent User Interaction 164

PART X **OpenWiki** 166
52 OpenWiki: Overview and Quickstart 167
53 OpenWiki: Code Mode in Depth 169
54 OpenWiki: Personal Mode 172
55 OpenWiki: Coding-Agent Integrations, Search and Workspaces 174
56 OpenWiki: Providers, Configuration and Telemetry 176
57 OpenWiki: Visualizing and Automating Updates 178

PART XI **Build-Alongs** 180
58 Build-Along 1: A Deep Research Agent 181
59 Build-Along 2: A Data Analysis Agent 184
60 Build-Along 3: A Content Builder Agent 187
61 Build-Along 4: Retrieval-Augmented Generation with Deep Agents 189
62 Capstone: A Release-Notes Assistant that Uses Every Layer 192

PART XII **Interview Preparation** 195
63 How to Answer Deep Agents Interview Questions 196
64 Interview Questions: Foundations and Architecture 199
65 Interview Questions: Tools, Backends and Permissions 212
66 Interview Questions: Sandboxes, Interpreters and Streaming 224
67 Interview Questions: Context Management 236
68 Interview Questions: Delegation and Subagents 248
69 Interview Questions: Steering with Human-in-the-Loop and Rubrics 260
70 Interview Questions: Middleware and Reliability 270
71 Interview Questions: Frontend and Protocols 281
72 Interview Questions: Deployment and Production 291
73 Interview Questions: Managed Deep Agents 301
74 Interview Questions: OpenWiki, Build-Alongs and the Capstone 315
75 System-Design Walkthroughs 329
76 Spot the Bug 335
77 Rapid-Fire Numbers and Defaults 339
78 Mock Interview Rounds 342

Appendices 344
A Cheat Sheet: Code, Defaults and Decisions 345
B The Trap List: Thirty Mistakes to Avoid 347
C Version Notes and Documentation Differences 349
D Troubleshooting and Edge-Case Catalogue 350
E Glossary 356
F Notices, Licences and Acknowledgements 360

PART I

01

Getting Started

What an agent harness is, how Deep Agents sits on LangChain and LangGraph, and how to build, configure and choose a model for your first deep agent.

IN THIS PART

1 The Big Picture

2 Background: Agents, LangChain and LangGraph

3 Overview: What Deep Agents Is

4 Quickstart: Your First Deep Agent

5 Customization: Configuring a Deep Agent

6 Models and Providers

7 Deep Agents and the Claude Agent SDK

8 Release History and What Changed

CHAPTER 1

The Big Picture

At a glance: Deep Agents is an *agent harness*: the familiar tool-calling loop, pre-wired with everything a long-running agent needs. It is built on **LangChain** (the framework) and **LangGraph** (the runtime), and `create_deep_agent(...)` returns a compiled LangGraph graph.

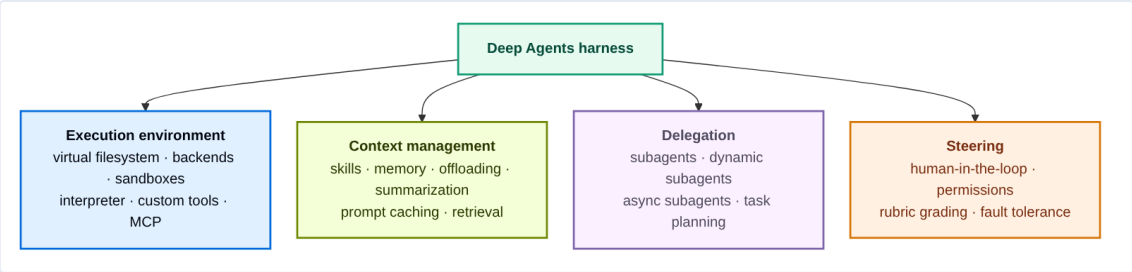


Figure 1. Deep Agents harness capabilities, grouped into four categories.

A deep agent is an ordinary agent that has been given four things ordinary agents lack: somewhere to work and a bounded way to act (the **execution environment**), the right knowledge without drowning in tokens (**context management**), the ability to hand heavy subtasks to specialists (**delegation**), and a person who stays in control (**steering**).

Category	What lives here	Part
Execution environment	Tools, virtual filesystem and backends, permissions, sandboxes, interpreters, streaming	Part III
Context management	Skills, memory, retrieval, offloading and summarization, profiles	Part IV
Delegation	Synchronous, dynamic and asynchronous subagents	Part V
Steering	Human-in-the-loop approvals, rubric grading	Part VI

Three cross-cutting topics tie these together: **middleware** (Part VII) implements most of the capabilities above, **frontends and protocols** (Parts VIII and IX) expose the agent, and **deployment** (Part II) runs it in production.

Mental model: skills tell the agent *how* to do something, memory tells it *what to remember*, summarization is *how it forgets gracefully*, subagents are *how it splits work*, and interrupts are *how a person says no*.

CHAPTER 2

Background: Agents, LangChain and LangGraph

> **At a glance:** An agent is a language model running in a loop with tools. LangChain gives you the building blocks (models, messages, tools, middleware), LangGraph gives you the runtime (state, persistence, interrupts, streaming), and Deep Agents is the harness assembled on top. Knowing the five LangGraph ideas in this chapter (state, reducers, threads, checkpoints and interrupts) makes every later chapter easier.

The agent loop

A *tool-calling agent* repeats one cycle until the model stops asking for tools:

1. Send the conversation (system prompt + messages) to the model.
2. If the reply contains **tool calls**, run each tool and append the results as `ToolMessage`s.
3. Go back to step 1. If the reply has no tool calls, that reply is the answer.

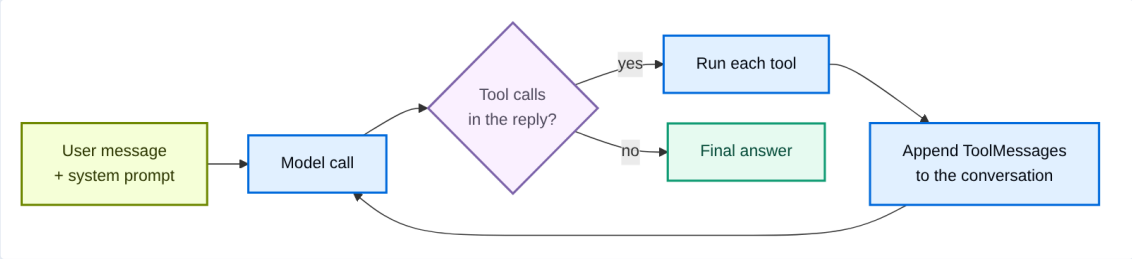


Figure 2. The tool-calling agent loop.

This loop is the *core agent loop* that middleware hooks into later. It is simple and general, and it breaks down on long tasks for four predictable reasons:

Problem	What goes wrong	Deep Agents' answer
Context fills up	Tool outputs and history exceed the model's window; quality and cost degrade	Offload large results to files; summarize old history
No place to put work	Intermediate artifacts live only in the chat	A virtual filesystem behind pluggable backends
One thread of thought	A single context handles every subtask	Subagents with isolated context windows
No control or durability	A crash loses progress; risky actions run unchecked	Checkpointing, human-in-the-loop, permissions

The three layers

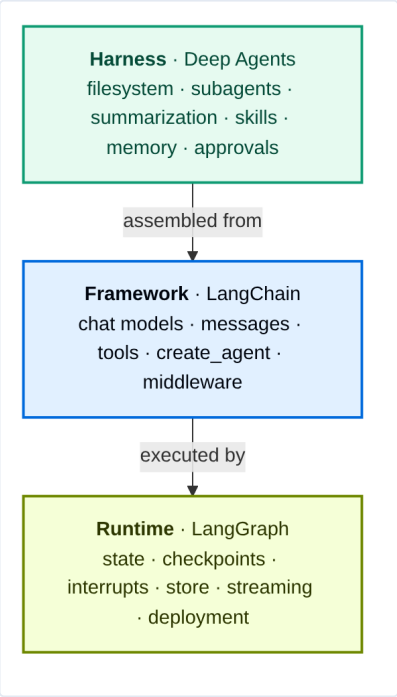


Figure 3. Framework, runtime and harness.

Layer	Product	Role
Runtime	LangGraph	Executes graphs: state, persistence, streaming, interrupts, deployment
Framework	LangChain	Abstractions: chat models, messages, tools, agents, middleware
Harness	Deep Agents	An opinionated, pre-assembled agent: filesystem, subagents, summarization, skills, memory, approvals

Rule of thumb: pick the **harness** when you want a capable agent now, the **framework** when you want to assemble your own, and the **runtime** when you need to control every node and edge.

LangChain in five minutes

- **Chat models** are created with `init_chat_model("provider:model")`, for example `"anthropic:claude-sonnet-4-6"`. The prefix selects the integration; everything after the first colon is the provider's model ID.
- **Messages** are typed: `HumanMessage`, `AIMessage` (which may carry `tool_calls`), `ToolMessage`, `SystemMessage`. Newer content is a list of **content blocks** (text, image, audio, file), which is how multimodality works.
- **Tools** are functions the model may call. The `@tool` decorator turns a typed function with a docstring into a tool; the schema the model sees is inferred from the **type hints and docstring**, so a vague docstring means a confused model.
- **Agents.** `create_agent(model, tools, ...)` builds the tool-calling loop above as a LangGraph graph.
- **Middleware** hooks into the loop (before/after/around the model and tools). Deep Agents is, in effect, a curated middleware stack.

PYTHON

```python
from langchain.agents import create_agent
from langchain.tools import tool

@tool
def get_weather(city: str) -> str:
    """Return the current weather for a city."""
    return f"It is sunny in {city}."
```

```
agent = create_agent("anthropic:claude-sonnet-4-6", tools=[get_weather])
result = agent.invoke({"messages": [{"role": "user", "content": "Weather in Pune?"}]})
print(result["messages"][-1].content)
```

LangGraph in five ideas

A LangGraph program is a graph of **nodes** (functions) that read and update a shared, typed **state**. `create_agent` and `create_deep_agent` both return a *compiled* graph, so everything below applies to them.

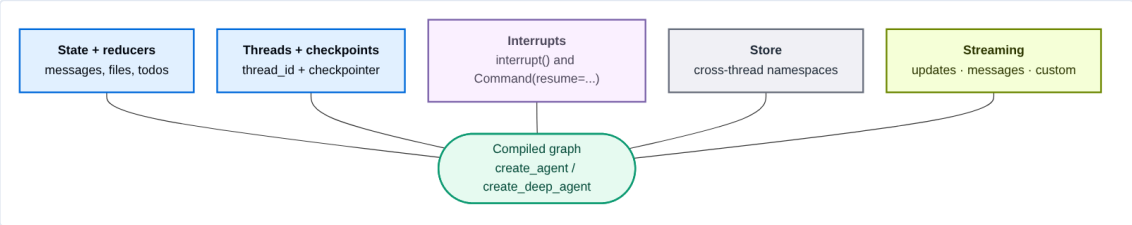


Figure 4. The LangGraph primitives that Deep Agents relies on.

1. **State and reducers.** State is a typed dictionary (`messages`, `files`, `todos`, …). When a node returns an update, a **reducer** decides how it merges. The message list uses an *append* reducer; most other keys simply overwrite. This is why middleware can add state keys and why parallel branches can safely write messages.
2. **Threads and checkpoints.** Give a run a `thread_id` and a **checkpointer** (for example `InMemorySaver` in tests, a database-backed saver in production) and LangGraph saves a checkpoint after every step. The same `thread_id` later resumes the conversation. Without a checkpointer there is no memory between calls, no resume and no interrupts.
3. **Interrupts.** A node or tool can call `interrupt(value)` to pause the graph. The run stops, the value is returned to the caller under `__interrupt__`, and `Command(resume=...)` continues from the checkpoint, even days later. Human-in-the-loop is built on this.
4. **Store (long-term memory).** A checkpoint belongs to one thread. A **store** is a key-value space shared across threads, organized by *namespaces* such as `(user_id, "memories")`. Deep Agents' `StoreBackend` writes files into it.
5. **Streaming.** `stream` and `stream_events` emit updates, messages (token by token) and custom events while the graph runs; sub-graphs (subagents) appear under *namespaces*.

Also worth knowing: **runtime context** is an immutable, per-run object (user ID, role, API keys) separate from state; **durable execution** means a crashed worker can resume from the last checkpoint; and **Agent Server** (LangSmith Deployment) hosts graphs behind an API with threads, runs, a store, cron jobs and authentication. **LangSmith Studio** is its debugging UI and **LangSmith tracing** records every model and tool call.

PYTHON

```python
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command, interrupt

# 1. A checkpointer makes the thread resumable.
agent = create_agent("anthropic:claude-sonnet-4-6", tools=[get_weather], checkpointer=InMemorySaver())
cfg = {"configurable": {"thread_id": "demo-1"}}
agent.invoke({"messages": [{"role": "user", "content": "Hi, I'm Sadiq"}]}, cfg)
agent.invoke({"messages": [{"role": "user", "content": "What is my name?"}]}, cfg)   # remembers

# 2. A tool can pause for a human, then resume with a value.
@tool
def transfer(amount: float) -> str:
    """Transfer money."""
    ok = interrupt({"question": f"Approve transferring {amount}?"})
    return "sent" if ok else "cancelled"
# agent.invoke(Command(resume=True), cfg)   # continues from the checkpoint
```

> **Edge case:** `interrupt()` re-executes the interrupted node *from the top* when resumed. Anything before the `interrupt` call in that tool runs twice, so keep that part idempotent (or guard side effects), a rule that reappears in the human-in-the-loop and Managed Deep Agents chapters.

How the rest of the book maps onto these ideas

LangGraph idea	Where it surfaces in Deep Agents
State + reducers	The `files` and `todos` state keys, `state_schema`
Thread + checkpoint	`StateBackend` (thread-scoped files), short-term memory, `thread_id` on every resume
Interrupt	`interrupt_on`, `FilesystemPermission(mode="interrupt")`, in-tool `interrupt()`, MCP elicitation
Store	`StoreBackend`, `/memories/`, `MemoryMiddleware`
Runtime context	`context_schema`, namespace factories (`rt.server_info.user.identity`)
Streaming + namespaces	`subgraphs=True`, `stream.subagents`, `tools:<id>` namespaces
Agent Server	LangSmith Deployment, Managed Deep Agents, async subagents, A2A and MCP endpoints

> **In an interview:** be able to say in one breath: "Deep Agents is an agent harness on LangChain's agent abstractions and LangGraph's durable runtime; `create_deep_agent` returns a compiled LangGraph graph, so checkpointers, stores, interrupts and streaming all behave as in LangGraph."

CHAPTER 3

Overview: What Deep Agents Is

> **At a glance:** Deep Agents is an *agent harness*: the same tool-calling loop as any agent, but pre-wired with a virtual filesystem, subagents, summarization, skills, memory, human-in-the-loop and sandboxes. `create_deep_agent(...)` returns a compiled **LangGraph** graph.

What is Deep Agents?

What. An open-source (MIT) **agent harness** built on LangChain (the agent framework) and run on LangGraph (the runtime). You call one function, `create_deep_agent`, and get an agent that is already equipped for long, multi-step, context-heavy work.

Why. A bare tool-calling loop falls apart on long tasks: the context window fills with tool output, there's nowhere to keep scratch work, and one agent has to do everything. The harness adds the missing parts:

Built-in capability	Problem it solves
Virtual filesystem (`ls`, `read_file`, `write_file`, `edit_file`, `glob`, `grep`, `delete`)	Scratch space, durable memory, offloading big results
Subagents (`task` tool)	Context isolation, parallelism, specialisation
Summarization + offloading	Keeps the window from overflowing
Skills + memory	On-demand procedures; always-on facts and preferences
Human-in-the-loop, permissions, rubric grading	Steering and safety
Sandboxes / interpreters	Running code safely
Task planning (`write_todos`, **opt-in since v0.7**)	Visible progress tracking

When. Long-running, complex, multi-step tasks: research, coding, data analysis, content pipelines. For a simple tool-calling agent, `langchain.agents.create_agent` is lighter. For fine-grained control over orchestration, use LangGraph directly.

HOW

PYTHON

```python
from deepagents import create_deep_agent

def get_weather(city: str) -> str:
    """Get weather for a given city."""
    return f"It's always sunny in {city}!"

agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",   # "provider:model" string, or a chat model object
    tools=[get_weather],
    system_prompt="You are a helpful assistant",
)
agent.invoke({"messages": [{"role": "user", "content": "what is the weather in sf"}]})
```

Advantages. Batteries included; model-agnostic (any LangChain chat model with tool calling); still "just a LangGraph graph", so you inherit streaming, checkpointing, interrupts and deployment.

LIMITATIONS / GOTCHAS

- The model **must support tool calling**.
- Opinionated defaults: for example, planning (`write_todos`) is off unless you add `TodoListMiddleware`.
- More moving parts (middleware stack, backends, subagents) than a bare agent; you need to understand the defaults to debug.

LangChain vs LangGraph vs Deep Agents

Layer	Role	Examples of what it gives you
LangChain	*Framework*: abstractions	models, tools, messages, `create_agent`, middleware
LangGraph	*Runtime*: execution engine	durable execution, streaming, checkpointers, interrupts, stores, state graphs
Deep Agents	*Harness*: opinionated, preconfigured	filesystem, subagents, planning, skills, memory, summarization, HITL

Rule of thumb. Harness for "give me a capable agent now", framework for "I'll assemble my own", runtime for "I need to control every node and edge".

The four capability categories

(See the diagram in the README.) **Execution environment** (where the agent acts) · **Context management** (what it knows and how it fits in the window) · **Delegation** (how work is split) · **Steering** (how humans stay in control).

The six headline capabilities

1. Take actions in an environment (tools, files, code execution).
2. Connect to your data (memories, skills, domain knowledge, loaded at the right moment).
3. Manage growing context (summarize history, offload big results).
4. Parallelize tasks (delegate to isolated subagents).
5. Stay in the loop (pause for human approval).
6. Improve over time (update memory, skills, prompts from real usage).

CHAPTER 4

Quickstart: Your First Deep Agent

> **At a glance:** Install `deepagents`, give the agent a search tool, call `create_deep_agent`, `invoke` it. The docs build a *research agent* that searches, offloads big results to files, can spawn subagents, and writes a report.

Prerequisites & install

What / Why. You need a model provider key and a model that supports tool calling.

HOW

```bash
pip install deepagents          # or: uv add deepagents
# provider key, e.g.
export ANTHROPIC_API_KEY=...    # or GOOGLE_API_KEY / OPENAI_API_KEY / OPENROUTER_API_KEY ...
# optional, for non-built-in search
pip install tavily-python
```

The docs show tabs for Google, OpenAI, Anthropic, OpenRouter, Fireworks, Baseten, Ollama, and "Other" (any LangChain chat model).

Gotcha. The LangSmith Gateway can route most providers through LangSmith (bring your own keys or use gateway credits).

Creating a search tool

What. The agent's only "domain" tool in the quickstart.

TWO OPTIONS

Option	What it is	Extra package/key?
Provider built-in search (recommended)	A tool *dict* handled server-side by the provider	No
Tavily (any provider)	A plain Python function wrapping `TavilyClient.search`	`tavily-python` + `TAVILY_API_KEY`

HOW

```python
# Provider built-in search: pass the dict straight into tools=[...]
google_search    = {"google_search": {}}
openai_search    = {"type": "web_search"}
anthropic_search = {"type": "web_search_20260209", "name": "web_search"}
```

```python
# Tavily wrapper (works with any model)
import os
from typing import Literal
from tavily import TavilyClient

tavily_client = TavilyClient(api_key=os.environ["TAVILY_API_KEY"])
```

```python
def internet_search(
    query: str,
    max_results: int = 5,
    topic: Literal["general", "news", "finance"] = "general",
    include_raw_content: bool = False,
):
    """Run a web search."""
    return tavily_client.search(
        query, max_results=max_results,
        include_raw_content=include_raw_content, topic=topic,
    )
```

Gotcha. The tool schema is inferred from the **type hints + docstring**, so write a clear docstring.

Creating and running the agent

HOW

PYTHON

```python
from deepagents import create_deep_agent

research_instructions = """You are an expert researcher. Conduct thorough research,
then write a polished report."""

agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
    tools=[anthropic_search],                # or [internet_search]
    system_prompt=research_instructions,
)

result = agent.invoke({"messages": [{"role": "user", "content": "What is langgraph?"}]})
print(result["messages"][-1].content)
```

OPTIONAL: LANGSMITH TRACING

BASH

```bash
export LANGSMITH_TRACING=true
export LANGSMITH_API_KEY=...
```

Traces show tool calls, subagent delegation and LLM responses.

How it works (what the agent does by itself)

1. **Conducts research** by calling the search tool.
2. **Manages context** by using `write_file` / `read_file` to offload large search results.
3. **Spawns subagents** for complex subtasks.
4. **Synthesizes** a report.

> **In an interview:** The quickstart shows the "big three" built-ins in action: filesystem offloading, subagents, synthesis. Task planning is **not** on by default.

Limitations. Quality depends on the model and prompt; Tavily-style search returns snippets unless you fetch full pages (see Build-Along 1: A Deep Research Agent).

When to use it. First run, demos, and as the skeleton for any research-style agent: swap the search tool for your own domain tools and keep the rest.

Advantages. Minimal setup (one tool, one prompt); file offloading, subagents and summarization come preconfigured, so you don't write the agent loop.

CHAPTER 5

Customization: Configuring a Deep Agent

> **At a glance:** Everything is a parameter of `create_deep_agent`, plus middleware. This is the page to know cold for "how would you configure X?" questions.

The `create_deep_agent` signature

PYTHON

```python
create_deep_agent(
    model: str | BaseChatModel | None = None,
    tools: Sequence[BaseTool | Callable | dict] | None = None,
    *,
    system_prompt: str | SystemMessage | None = None,
    middleware: Sequence[AgentMiddleware] = (),
    subagents: Sequence[SubAgent | CompiledSubAgent | AsyncSubAgent] | None = None,
    skills: list[str] | None = None,
    memory: list[str] | None = None,
    permissions: list[FilesystemPermission] | None = None,
    backend: BackendProtocol | None = None,
    interrupt_on: dict[str, bool | InterruptOnConfig] | None = None,
    response_format: ResponseFormat | type | dict | None = None,
    state_schema: type[DeepAgentState] | None = None,
    context_schema: type[ContextT] | None = None,
    checkpointer: Checkpointer | None = None,
    store: BaseStore | None = None,
    debug: bool = False,
    name: str | None = None,
    cache: BaseCache | None = None,
) -> CompiledStateGraph
```

Parameter	Does	Notes
`model`	The LLM	`provider:model` string or chat model object
`tools`	Your domain tools	Built-ins are added automatically
`system_prompt`	Custom instructions	`str` or `SystemMessage` (structured blocks preserved)
`middleware`	Extra middleware	Same `.name` as a built-in **replaces** it
`subagents`	Custom subagent specs	General-purpose subagent auto-added
`skills`	Directories containing skill folders	Progressive disclosure
`memory`	`AGENTS.md` paths loaded at startup	Always in context
`permissions`	Path-level read/write rules	First match wins, default allow
`backend`	Where files live	Default `StateBackend()`
`interrupt_on`	Pause before tool calls	**Needs a checkpointer**
`response_format`	Structured final output	Pydantic / `ToolStrategy` / `ProviderStrategy`
`state_schema`	Custom graph state	Must subclass `DeepAgentState`
`context_schema`	Per-run immutable context	Passed as `context=` on invoke
`checkpointer` / `store`	LangGraph persistence	Needed for HITL, resume, cross-thread memory

Model

What. Which LLM drives the agent.

How. A `provider:model` string (resolved via `init_chat_model`), or a configured instance for full control.

```python
from langchain.chat_models import init_chat_model

agent = create_deep_agent(model="openai:gpt-5.5")                # string form

model = init_chat_model(model="google_genai:gemini-3.6-flash", thinking_level="medium")
agent = create_deep_agent(model=model)                             # configured instance
```

Advantages. Switching providers is a one-string change. **Limitations.** Parameters differ per provider (check the integration page). Chat models auto-retry transient failures with exponential backoff (`max_retries`, `timeout` on the LangChain Models page).

Tools

What. Custom functions, `@tool`s, or tool dicts in `tools=[...]`; plus MCP tools.

Why. Give the agent domain actions; built-ins handle files, delegation, etc.

HOW

```python
from langchain.mcp import MCPAdapter          # pip install "langchain[mcp]"

config = {"mcpServers": {"my_server": {"url": "http://localhost:8000/mcp"}}}
async with MCPAdapter(config) as adapter:
    tools = await adapter.list_tools()
    agent = create_deep_agent(model="openai:gpt-5.5", tools=tools)
```

Gotchas. Docstrings drive tool use. MCP setup is async. Full detail in Part III (Tools) and Part IX (MCP).

System prompt

What. `system_prompt=` as a `str` or `SystemMessage`.

How (assembly order). custom `system_prompt` → base agent prompt → memory → skills list → virtual-filesystem prompt (+ `execute` docs when sandboxed) → subagent (`task`) prompt → custom middleware prompts → HITL prompt (if `interrupt_on`).

Notes. A harness profile can add `base_system_prompt` and `system_prompt_suffix` (always last). For per-run variation use `@dynamic_prompt` middleware, because `system_prompt=` is static.

Middleware

What. Middleware wraps the agent loop. Deep Agents ships its own stack and accepts any LangChain middleware (prebuilt, provider-specific, custom).

Why. It is how filesystem tools, subagents, summarization, skills, memory, HITL and caching are implemented, and how you extend them.

THE BUILT-IN STACK, IN ORDER

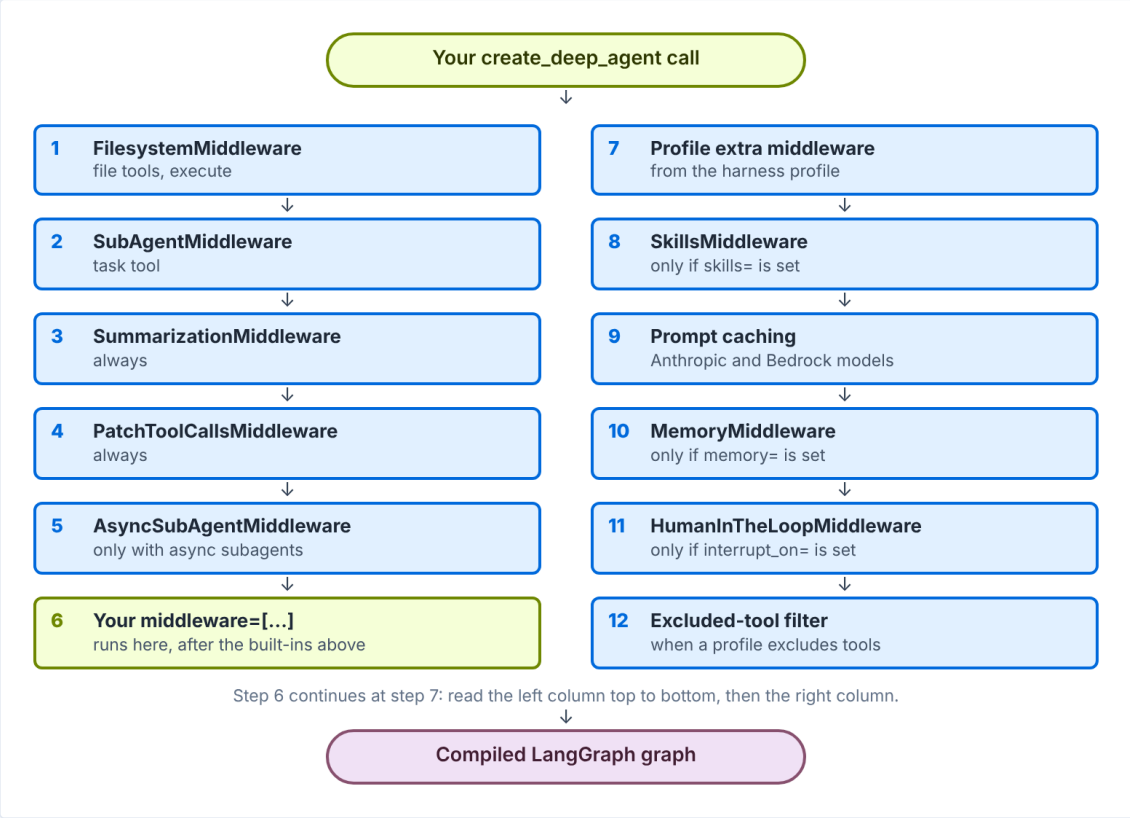


Figure 5. Deep Agents middleware stack order.

#	Middleware	Present when
1	`FilesystemMiddleware`	Always (enforces `permissions=`)
2	`SubAgentMiddleware`	≥1 sync subagent (general-purpose is auto-added, so usually)
3	`SummarizationMiddleware`	Always
4	`PatchToolCallsMiddleware`	Always (repairs dangling tool calls after interrupts/malformed args)
5	`AsyncSubAgentMiddleware`	Async subagents configured
6	**Your `middleware=[...]`**	
7	Harness-profile extras	Profile defines them
8	`SkillsMiddleware`	`skills=` set
9	Prompt caching	Anthropic / Bedrock models (no-op elsewhere)
10	`MemoryMiddleware`	`memory=` set (after caching so memory edits don't bust the cache prefix)
11	`HumanInTheLoopMiddleware`	`interrupt_on=` set
12	Excluded-tool filtering	A profile excludes tools

HOW: ADD OR OVERRIDE

PYTHON

```python
from langchain.agents.middleware import wrap_tool_call, SummarizationMiddleware
from deepagents.middleware.filesystem import FilesystemMiddleware

@wrap_tool_call
```

```python
def log_tool_calls(request, handler):
    result = handler(request)
    return result

agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        log_tool_calls,                                  # inserted after PatchToolCallsMiddleware
        SummarizationMiddleware(                         # same .name -> REPLACES the built-in
            model="anthropic:claude-haiku-4-5",
            backend=my_backend,
            trigger=("tokens", 100_000),
            keep=("messages", 20),
        ),
        # read-only filesystem (read_file is mandatory in the list; deepagents>=0.7)
        FilesystemMiddleware(backend=my_backend, tools=["read_file", "ls", "glob", "grep"]),
    ],
)
```

Other documented overrides: `AnthropicPromptCachingMiddleware(ttl="1h")` (default 5 min); `SkillsMiddleware(..., system_prompt=template)` where the template needs `{skills_locations}`, `{skills_list}`, `{skills_load_warnings}` (`None` suppresses the catalog).

Advantages. Full control over behaviour without forking the library. **Limitations / gotchas.**

- **Override ≠ merge.** A replacement must be fully configured (e.g. a custom `FilesystemMiddleware` needs its own `backend=` and `permissions=`).
- **Do not mutate `self`** inside hooks (race conditions across subagents, parallel tools, threads). Put values in graph state.
- `FilesystemMiddleware`, `SubAgentMiddleware` and permission middleware can't be excluded (raises `ValueError`).
- `trigger` accepts `("fraction", x)`, `("tokens", n)`, `("messages", n)`; a list combines with OR.

More in Part VII (Middleware).

Subagents

What. Specialist child agents the main agent can call through `task`.

HOW

PYTHON

```python
research_subagent = {
    "name": "research-agent",
    "description": "Used to research more in depth questions",
    "system_prompt": "You are a great researcher",
    "tools": [internet_search],
    "model": "openai:gpt-5.5",        # optional; default = main agent's model
}
agent = create_deep_agent(model="google_genai:gemini-3.6-flash", subagents=[research_subagent])
```

Details in Part V (Delegation).

Backends

What. Where the virtual filesystem's bytes live. Default `StateBackend` (thread-scoped LangGraph state). Others: `FilesystemBackend`, `LocalShellBackend`, `StoreBackend`, `ContextHubBackend`, `CompositeBackend`, sandboxes. Details in Part III.

Sandboxes and interpreters

What. Sandbox backends add an `execute` shell tool; `CodeInterpreterMiddleware` (from `langchain_quickjs`) adds an in-process JavaScript `eval` tool. Details in Part III.

Human-in-the-loop

```python
from langgraph.checkpoint.memory import MemorySaver

agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
    tools=[remove_file, fetch_file, notify_email],
    interrupt_on={
        "remove_file": True,                                    # approve / edit / reject / respond
        "fetch_file": False,                                    # never interrupt
        "notify_email": {"allowed_decisions": ["approve", "reject"]},  # no editing
    },
    checkpointer=MemorySaver(),                                 # REQUIRED
)
```

Details in Part VI (Steering).

Skills and memory

```python
agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
    skills=["/skills/"],                # directory that CONTAINS skill folders
    memory=["/AGENTS.md"],              # always loaded
    # seed the files into the backend (StateBackend example):
    # agent.invoke({...}, files={"/AGENTS.md": create_file_data(text)})
)
```

Skills = procedures loaded on demand; memory = facts loaded always. Details in Part IV.

Structured output

What. `response_format=` (a Pydantic model, `ToolStrategy`, `ProviderStrategy`, or raw type) makes the final answer a validated object. **Gotcha.** Subagents can use their own `response_format` and return JSON to the parent (`deepagents>=0.5.3`).

State schema, runtime context

```python
from dataclasses import dataclass
from deepagents import DeepAgentState        # import path: confirm against your installed version

@dataclass
class Context:
    user_id: str

class MyState(DeepAgentState):          # must subclass DeepAgentState to keep the messages reducer
    extra_field: str

agent = create_deep_agent(model=..., context_schema=Context, state_schema=MyState)
agent.invoke({"messages": [...], "extra_field": "x"}, context=Context(user_id="u1"))
```

State = mutable and checkpointed. Runtime context = immutable per-run input that is **not** added to the prompt by itself. (`state_schema` needs `deepagents>=0.6.6`.)

CHAPTER 6

Models and Providers

> **At a glance:** Any LangChain chat model with tool calling works. Use `provider:model` strings; use provider profiles for shared init kwargs; `openai:` strings use the Responses API by default.

Supported models

What. Any LangChain chat model that supports tool calling. Format `provider:model`: the prefix picks the integration, everything after the first colon is the provider's model ID (e.g. `baseten:zai-org/GLM-5.2`). Resolution goes through `init_chat_model`.

Providers in the docs: OpenAI, Anthropic, Azure, Google Gemini, AWS Bedrock, HuggingFace, plus "Other" (DeepSeek, Ollama, OpenRouter, Fireworks, Baseten, ...).

Suggested models (Deep Agents eval suite)

Provider	Models the docs list
Google	`gemini-3.1-pro-preview`, `gemini-3.6-flash`
OpenAI	`gpt-5.5`, `gpt-5.4`
Anthropic	`claude-opus-4-8`, `claude-opus-4-7`, `claude-opus-4-6`
Open-weight	`GLM-5.2`, `Kimi-K2.7 Code`, `MiniMax-M3` (via Baseten, Fireworks, OpenRouter, Ollama)

Eval categories: File Ops, Retrieval, Tool Use, Memory, Conversation, Summarization. **Caveat from the docs:** passing evals is *necessary but not sufficient* for strong performance on long, complex tasks; scores change each run, so use them as relative guidance.

Configuring model parameters

```python
from langchain.chat_models import init_chat_model
model = init_chat_model(model="google_genai:gemini-3.6-flash", thinking_level="medium")
agent = create_deep_agent(model=model)
```

Gotcha. Available parameters vary by provider.

Provider profiles (construction defaults)

What. A `ProviderProfile` packages init kwargs applied when you pass a `provider:model` **string**. **Why.** Set temperature / reasoning effort once for all agents.

```python
from deepagents import ProviderProfile, register_provider_profile

register_provider_profile("openai", ProviderProfile(init_kwargs={"temperature": 0}))             # every
```

```
openai model
register_provider_profile("openai:gpt-5.5", ProviderProfile(init_kwargs={"reasoning_effort": "medium"}))  #
one model; inherits temperature=0
```

Gotcha. Not applied to pre-built model objects. Register before creating agents. Full merge semantics in Part IV (Profiles).

OpenAI and the Responses API

- `openai:...` strings use the built-in OpenAI provider profile, which turns on the **Responses API** by default (supports OpenAI conversation state).
- **Chat Completions instead:** pass a pre-built model with `use_responses_api=False`.
- **Keep Responses but disable data retention:** `use_responses_api=True, store=False, include=["reasoning.encrypted_content"]`.

PYTHON

```python
from langchain.chat_models import init_chat_model
model = init_chat_model("openai:gpt-5.5", use_responses_api=True, store=False,
                        include=["reasoning.encrypted_content"])
```

Selecting a model at runtime

What / Why. Let users choose a model from a dropdown without rebuilding the agent.

PYTHON

```python
from dataclasses import dataclass
from langchain.agents.middleware import wrap_model_call
from langchain.chat_models import init_chat_model

@dataclass
class Context:
    model: str

@wrap_model_call
def configurable_model(request, handler):
    model = init_chat_model(request.runtime.context.model)
    return handler(request.override(model=model))

agent = create_deep_agent(model="anthropic:claude-sonnet-4-6",
                          middleware=[configurable_model], context_schema=Context)
agent.invoke({"messages": [...]}, context=Context(model="openai:gpt-5.5"))
```

Why it matters, when to choose, and trade-offs

Why. The harness is model-agnostic, so you can match the model to the job (for example a stronger model for the coordinator and a cheaper one for a subagent via the subagent spec's `model` field) and avoid vendor lock-in.

When. Pick by tool-calling reliability and context size first; use the docs' eval categories (File Ops, Retrieval, Tool Use, Memory, Conversation, Summarization) as relative guidance.

Advantages. One `provider:model` syntax; init kwargs set once through provider profiles; runtime switching with `wrap_model_call` + `context_schema`.

Limitations / gotchas. Parameters differ per provider; provider profiles ignore pre-built model objects; `openai:` strings default to the Responses API (use a pre-built model for Chat Completions); eval scores are relative, not guarantees.

CHAPTER 7

Deep Agents and the Claude Agent SDK

> **At a glance:** Deep Agents is model- and infrastructure-agnostic with a built-in multi-tenant deployment path. Claude Agent SDK is Claude-only and runs the agent inside a sandbox.

What / why / when

What. The docs' side-by-side of two agent harnesses: Deep Agents (LangChain; any model, pluggable backends) and the Claude Agent SDK (Anthropic; Claude models, built around the agent running inside a sandbox).

Why it matters. It is the standard "why not just use X?" interview question. The answer is about *model flexibility, where the agent runs, and who builds the server layer*.

When to pick which. Deep Agents for multi-provider, multi-tenant or hosted deployments and mixed backends. Claude Agent SDK for an Anthropic-only stack where you are happy to own the server layer.

Head-to-head

Dimension	Deep Agents	Claude Agent SDK
Where the agent runs	In a sandbox, *or* outside calling a remote sandbox as a tool	Inside a sandbox only
Execution backend	Pluggable: local, virtual FS, remote sandbox, custom	The sandbox's local filesystem
Model provider	Any (Anthropic, OpenAI, Google, 100+)	Claude (Anthropic, Bedrock, Vertex, Azure)
Per-model tuning	Harness profiles (beta)	Configure in code at each call site
Deployment	LangSmith Managed Deep Agents, or `langgraph build` to a Docker image	Self-host; you build server, auth, streaming
Multi-tenancy	Built in: scoped threads, per-user sandboxes, RBAC	Build it yourself
Agent server	Included (streaming, threads, run history, webhooks, auth)	You write the HTTP/WebSocket/SSE layer
License	MIT	MIT (Claude Code itself is proprietary)

Two patterns for connecting agents to sandboxes

1. **Agent runs *inside* the sandbox**: mirrors local dev; keys live in the sandbox; you build the comms layer. (The Claude Agent SDK's only model.)
2. **Agent runs *outside*, sandbox as a tool**: keys stay on your server, sandbox failures don't lose agent state, at the cost of per-call latency. (Deep Agents supports both; its examples use this one.)

> **In an interview:** Choose Deep Agents for model/infra flexibility and built-in multi-tenant deployment; choose Claude Agent SDK if you're all-in on Anthropic and happy to build the server layer.

ADVANTAGES / LIMITATIONS (FROM THE TABLE ABOVE)

- *Deep Agents:* flexible models and backends, a bundled agent server, built-in multi-tenancy. Cost: more pieces to configure (backends, profiles, deployment).
- *Claude Agent SDK:* mirrors local development, one provider. Cost: you write the HTTP/WebSocket/SSE layer, auth and multi-tenancy yourself, and the agent must live inside the sandbox.

Deep Agents vs LangChain vs LangGraph (concepts page)

Feature	LangGraph	LangChain	Deep Agents
Short-term memory	Short-term memory	Short-term memory	`StateBackend`
Long-term memory	Long-term memory	Long-term memory	Long-term memory (`StoreBackend`)
Skills	n/a	Multi-agent skills	Skills
Subagents	Subgraphs	Multi-agent subagents	Subagents
Human-in-the-loop	Interrupts	HITL middleware	`interrupt_on`
Streaming	Streaming	Agent streaming	Streaming

Deep Agents exposes these at the highest level; LangGraph at the lowest.

CHAPTER 8

Release History and What Changed

> **At a glance:** Four releases matter for interview "what changed" questions: v0.4 (sandboxes, summarization via `wrap_model_call`), v0.5 (async subagents, multimodal `read_file`), v0.6 (interpreter, stream v3, harness profiles, `ContextHubBackend`), v0.7 (leaner prompts, todos opt-in, middleware override, `delete` tool).

v0.7.0 · 24 Jul 2026

Optimizations

- **Leaner default prompts:** the base prompt starts empty and tool-usage prose duplicating schemas was trimmed. Tool-description tokens fell 43%; input tokens for a default-agent turn fell 65% (5,395 to 1,895) per the changelog's benchmark, with no reported quality regression.

Features

- **Middleware override:** an instance whose `.name` matches a built-in now **replaces** it instead of raising a duplicate error.
- **Filesystem tools:** new `delete` tool (file or directory, recursive); `write_file` overwrites instead of failing; `FilesystemMiddleware` accepts an allowlist of built-in tools; `read_file` pagination reports total/remaining lines and next `offset`; `grep` / `glob` return partial results with a `truncated` flag; `grep` has a 1,000-match cap, streamed output and optional context lines.
- **Prompt caching:** Bedrock caching via `deepagents[aws]`, automatic Fireworks session affinity.
- **NVIDIA:** built-in Nemotron 3 Ultra harness profile.

Breaking changes

- **Todos are opt-in.** `create_deep_agent` no longer adds `TodoListMiddleware`. Restore with `from langchain.agents.middleware import TodoListMiddleware`.
- **Backend shims removed:** `BackendFactory`, `BACKEND_TYPES`, `FileFormat`, `Unset` are gone. Pass concrete backend instances; give `StoreBackend` an explicit `namespace`. New files store string content; older `list[str]` content stays readable.
- **Output format changes:** empty `ls` / `glob` return "No files found" (not `[]`); `read_file` no longer adds a line-number gutter (update raw-output parsers).

v0.6.0 · 12 May 2026

- **`CodeInterpreterMiddleware` (experimental):** code execution + programmatic tool calling via a scoped QuickJS runtime.
- **Streaming:** `stream_events` / `astream_events` accept `version="v3"`.
- **`DeltaChannel` (beta):** message history and agent files store per-step *deltas*, keeping checkpoints small for long threads. **Warning:** rolling back to an earlier version is unsupported once threads persisted, because older versions can't read delta checkpoints.
- **Harness profiles:** `HarnessProfile` bundles per-provider/per-model prompt tweaks, tool overrides, middleware and subagent defaults.

- **`ContextHubBackend`:** filesystem backend on LangSmith Hub; every write is a Hub commit.

v0.5.0 · 7 Apr 2026

- **Async subagents:** non-blocking background tasks (require LangSmith Deployment).
- **Multimodal `read_file`:** PDFs, audio, video in addition to images.
- **Backend changes (compatible):** State/Store backends support binary files; better error propagation; `StateBackend()` / `StoreBackend()` can be instantiated directly (the factory form `backend=(lambda rt: StateBackend(rt))` is **deprecated**).
- **Anthropic prompt caching** performance improvements.

v0.4.0 · 10 Feb 2026

- **Sandbox integrations:** `langchain-modal`, `langchain-daytona`, `langchain-runloop`.
- **Summarization** now runs in the model node via `wrap_model_call`; full history stays in state; more accurate token counting; auto-trigger on `ContextOverflowError` (Anthropic and OpenAI integrations).
- **Responses API default** for `openai:`-prefixed model strings (`store=False` to disable retention).

LangChain / LangGraph changes that matter to Deep Agents

Date	Package	Change
14 Oct 2026	`langchain` 1.4.6	`HumanInTheLoopMiddleware` **per-call interrupt mode** (batched stays default; per-call planned as next major's default)
1 Sep 2026	`langchain` 1.4.0	MCP moves to `langchain.mcp` (beta, FastMCP-based), replacing `langchain-mcp-adapters`
12 May 2026	`langchain` 1.3.0	Event streaming `version="v3"` for agents
12 May 2026	`langgraph` 1.2.0	`DeltaChannel` (beta), per-node timeouts, node-level error handlers, graceful shutdown, v3 streaming API
10 Mar 2026	`langgraph` 1.1.0	`version="v2"` for streaming/invoke with typed output
15 Dec 2025	`langchain` 1.2.0	Tool `extras` attribute, strict schema for `response_format`
25 Nov 2025	`langchain` 1.1.0	Model profiles (drive context-aware summarization), model retry middleware

How to use the changelog: upgrade checklist (0.6 to 0.7)

1. If you relied on planning, add `TodoListMiddleware()` explicitly.
2. Replace backend factories / `BackendFactory` with concrete backend instances; give `StoreBackend` an explicit `namespace`.
3. Update parsers for `read_file` (no line-number gutter) and for empty `ls` / `glob` results ("No files found").
4. Remove any workaround for duplicate-middleware errors: a same-`.name` middleware now replaces the built-in.
5. Don't plan to roll back across 0.6 delta checkpoints.

Advantages of upgrading. Leaner prompts (input tokens down 65% in the changelog's benchmark), a `delete` tool, richer `read_file` / `grep` / `glob` behaviour. **Risk.** The breaking changes above, plus beta APIs (interpreter, profiles, event streaming) that may still move.

Why the changelog matters in an interview

Be ready to say: "Todos became opt-in in 0.7 because default prompts were slimmed (65% fewer input tokens)"; "Middleware override by name landed in 0.7"; "Async subagents arrived in 0.5 and need LangSmith Deployment"; "Interpreters/profiles/ `ContextHubBackend` arrived in 0.6"; "Delta checkpoints in 0.6 make rollback unsafe". **Caveat:** dates and numbers are from the changelog on 22 Oct 2026.

PART II

02

Deployment

From prototype to production service: Managed Deep Agents, LangSmith Deployment, authentication, durable execution, sandboxes and fault tolerance.

IN THIS PART

9 Managed Deep Agents: Overview

10 Managed Deep Agents: Quickstart and Project Structure

11 Managed Deep Agents: Defining the Agent

12 Managed Deep Agents: Instructions, Skills and Context Hub

13 Managed Deep Agents: Tools, Middleware and MCP Connectors

14 Managed Deep Agents: Durable Memory

15 Managed Deep Agents: Sandboxes

16 Managed Deep Agents: Connections, Identity and the Runtime

17 Managed Deep Agents: Channels and Agent-Owned Interrupts

18 Managed Deep Agents: Schedules

19 Managed Deep Agents: Deploying, the CLI and Local Development

20 Managed Deep Agents: Exposing the Agent over MCP, and Built-in Search

21 Managed Deep Agents: Evals and Migration

22 Tutorial: Extending the Research Assistant with Search, Memory and a Schedule

23 Going to Production

24 Fault Tolerance

CHAPTER 9

Managed Deep Agents: Overview

At a glance: A LangSmith service. You write the agent as a **project of files** (`agent.py` , `instructions.md` , `skills/` , `tools/` ...); the `mda` CLI uploads it and LangSmith runs it on managed infrastructure (the open-source Deep Agents harness + the LangSmith Agent Server runtime). *"You focus on what your agent does. MDA runs it."*

Status: Managed Deep Agents is in public beta on LangSmith Cloud. Treat names and behaviour as subject to change and check the current documentation before you depend on a detail.

What it is

What. A hosted way to deploy Deep Agents without operating servers. You provide *business logic*; MDA provides the *harness* and the *runtime*.

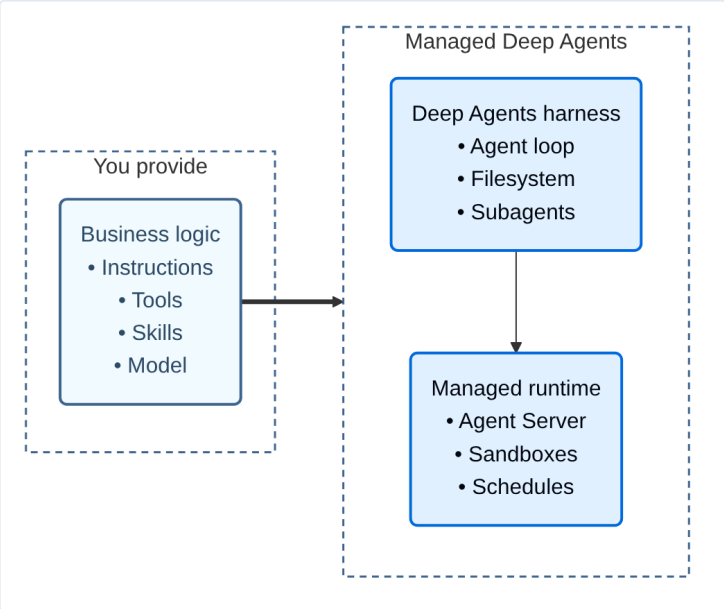


Figure 6. Managed Deep Agents: what you provide vs what MDA provides.

You provide	MDA provides
Instructions, tools, skills, middleware, model	**Deep Agents harness:** agent loop, filesystem, subagents
	Managed runtime: Agent Server (API, threads, runs, streaming, MCP endpoint), sandboxes, schedules

Why. Run a production agent without wiring up a server, store, checkpointer, auth and streaming yourself; keep agent logic as a readable file layout; expose the deployed agent to other tools (e.g. as an MCP tool).

WHEN

- You want a production agent with no infrastructure to run.

- You already use Deep Agents and want the runtime handled for you.
- You want to trigger the agent from **channels** (e.g. Slack) or **schedules** (cron), or call it from an MCP client.

Stay with open-source `create_deep_agent` if you run the agent yourself or need your own backend, store or checkpointer.

Project structure

How. Each capability maps to a file or directory. Only `agent.py` is required; add the rest as needed.

Capability	Path	Purpose
Model + core config	`agent.py`	**Required**
Instructions	`instructions.md`	System prompt
Skills	`skills/`	Task playbooks loaded when relevant
Tools	`tools/`	Functions for app logic or external services
MCP connectors	`tools/mcp.py`	Remote MCP servers that supply tools
Middleware	`middleware/`	Logic around model and tool calls
Sandbox	`sandbox/`	Isolated filesystem + shell for agent-written code
Memory	`memory.py`	Preferences/knowledge persisting across threads
Identity	`identity.py`	Per-caller private threads, memory, credentials
Channels	`channels/`	Messaging connections (e.g. Slack) that start runs and receive replies
Schedules	`schedules/`	Managed cron schedules
Evals	`evals/`	Harbor tasks that test the agent

Instructions, skills and optional durable memory are stored in **Context Hub**. **Connections** handle OAuth so end users authorize their own accounts.

Example project

PYTHON

```python
# agent.py
from managed_deepagents import define_deep_agent

from middleware.audit import log_tool_calls
from tools.search import internet_search

agent = define_deep_agent(
    name="research-assistant",          # static name is REQUIRED
    model="openai:gpt-5.5",
    tools=[internet_search],
    middleware=[log_tool_calls],
)
```

MARKDOWN

```markdown
<!-- instructions.md -->
# Assistant

You are a helpful assistant.
```

MARKDOWN

```markdown
<!-- skills/research/SKILL.md -->
---
name: research
```

```
description: Gather and synthesize context before answering complex questions.
---

# Research

Use this skill when a task needs more than a direct answer.

1. Identify what information is missing.
2. Search LangChain docs when the question is about LangChain, LangGraph, or LangSmith.
3. Summarize findings before responding to the user.
```

PYTHON

```python
# tools/search.py
from langchain.tools import tool

@tool(parse_docstring=True)
def internet_search(query: str) -> str:
    """Search the internet for relevant sources.

    Args:
        query: The search query.
    """
    return f"Results for: {query}"
```

PYTHON

```python
# middleware/audit.py
from collections.abc import Awaitable, Callable
from langchain.agents.middleware import wrap_tool_call
from langchain.messages import ToolMessage
from langchain.tools.tool_node import ToolCallRequest
from langgraph.types import Command

@wrap_tool_call
async def log_tool_calls(
    request: ToolCallRequest,
    handler: Callable[[ToolCallRequest], Awaitable[ToolMessage | Command]],
) -> ToolMessage | Command:
    print(f"Calling tool: {request.tool_call['name']}")
    result = await handler(request)
    print(f"Finished tool: {request.tool_call['name']}")
    return result
```

PYTHON

```python
# tools/mcp.py : remote MCP servers as tools
from managed_deepagents import define_mcp

mcp = define_mcp(
    servers={
        "langchainDocs": {
            "transport": "http",
            "url": "https://docs.langchain.com/mcp",
            "include_tools": ["search_docs_by_lang_chain"],
        },
    },
)
```

`define_deep_agent` VS `create_deep_agent`

What. The same harness, a different owner of the compile step.

	`create_deep_agent`	`define_deep_agent`
Returns	A compiled agent	A **definition**
Who runs it	You	The managed runtime
Backend, store, checkpointer	You own them	Supplied by the runtime

	`create_deep_agent`	`define_deep_agent`
Extras required	None	A static `name`

How it works. The `mda` CLI passes the definition to the runtime; the runtime supplies memory, skills and the system prompt, then compiles the agent **with** `create_deep_agent`. Every `define_*` function follows this pattern: *declare what the agent needs; the runtime manages the lifecycle.* The `mda` CLI and its commands are covered in the chapters on the quickstart and on deploying.

Advantages, limitations, gotchas

Advantages. No servers to run · agent logic is a readable file layout · same harness, so no new API · sandboxes, memory, channels and schedules declared in files · OAuth connections let end users authorize their own accounts · deployed agents are callable through an **MCP endpoint**.

Limitations. You give up control of the backend, store and checkpointer, which the runtime supplies. The service is in public beta, so names and behaviour can change between releases; check the current documentation for quotas, regions and pricing before you commit.

> **In an interview:** *Managed Deep Agents = Deep Agents + LangSmith Agent Server, declared as files.* `define_deep_agent` *returns a definition; the platform compiles it with* `create_deep_agent`.

Migration. The chapter on evals and migration explains how to move an existing `create_deep_agent` project onto the managed runtime.

CHAPTER 10

Managed Deep Agents: Quickstart and Project Structure

> **At a glance:** Managed Deep Agents (MDA) is a code-first way to build a Deep Agent and let LangSmith host it. You scaffold a project with the `mda` CLI, edit a few well-known files, test in LangSmith Studio with `mda dev`, and ship with `mda deploy`. MDA is in **public beta on LangSmith Cloud**.

What the quickstart builds

The quickstart creates a *research assistant*: an agent with a model, a set of instructions and a web-search tool. The workflow has five stages: scaffold the project, configure the model and instructions, add search, test locally in Studio, and deploy to a hosted runtime. A follow-up tutorial (later in this part) adds durable memory and a daily schedule to the same project.

Prerequisites. Python and `uv`; a model-provider API key (the quickstart defaults to OpenAI); and a LangSmith account and API key, which `mda dev`, `mda deploy` and Studio all need.

Step by step

1. **Scaffold.** Run `uvx --from managed-deepagents mda init research-assistant`, then `cd research-assistant`. The command creates the project directory and refuses to overwrite one that already exists.
2. **Add provider keys.** Put the key in `.env` (the template has `OPENAI_API_KEY` active and commented entries for Anthropic, Google and Azure variables). `mda deploy` later forwards non-reserved `.env` values to the deployment as secrets. Never commit `.env`.
3. **Add a LangSmith key.** Create a key under *Settings → API Keys* and set `LANGSMITH_API_KEY` in `.env`. It authenticates `mda dev` and `mda deploy` and opens the agent in Studio.
4. **Edit `instructions.md`.** This Markdown file is the agent's always-on behaviour. On deploy it syncs to LangSmith Context Hub, so it can later be edited without redeploying.
5. **Configure `agent.py`.** The agent is defined with `define_deep_agent` from `managed_deepagents`. The agent's `name` is also its default deployment name.
6. **Run locally.** `uv sync`, then `uv run mda dev`. This loads `.env`, starts a local Agent Server and opens Studio. Try: "What were the main announcements from the latest LangChain release?" You should see a search tool call and a cited answer.
7. **Deploy.** `uv run mda deploy` packages the project, runs it as a hosted deployment on LangSmith Agent Server and prints the dashboard URL.

Model and search tool per provider

The provider's built-in search tool is a plain dictionary passed in `tools=[...]`. It **must match the model's provider**, otherwise the search never appears in the trace.

Provider	Model string	Built-in search tool
OpenAI	`openai:gpt-5.5`	`{"type": "web_search"}`
Google	`google_genai:gemini-3.6-flash`	`{"google_search": {}}`

Provider	Model string	Built-in search tool
Anthropic	`anthropic:claude-sonnet-4-6`	`{"type": "web_search_20260209", "name": "web_search"}`
Microsoft Foundry	a model object (`AzureAIOpenAIApiChatModel` , from `langchain-azure-ai`) built from endpoint, key and deployment-name variables	`{"type": "web_search"}`

PYTHON

```python
# agent.py
from managed_deepagents import define_deep_agent

agent = define_deep_agent(
    name="research-assistant",          # graph ID and default deployment name
    model="openai:gpt-5.5",
    tools=[{"type": "web_search"}],     # provider built-in search; must match the model
)
```

If you prefer a provider-independent search, add `TAVILY_API_KEY` to `.env` , `uv add tavily-python` , and write a LangChain `@tool` named `internet_search` under `tools/search.py` (the tutorial chapter shows the complete code).

The project layout

An MDA project is an ordinary Python package with **one required file**, `agent.py` , which must export a variable named `agent` created with `define_deep_agent` . Everything else is either ordinary code you import, or a file whose *presence* switches on a managed capability.

Path	Role
`agent.py`	The agent definition (required; exactly one agent entry per project)
`instructions.md`	Managed context: the system prompt
`skills/<name>/SKILL.md`	One directory per skill, with optional supporting files
`tools/`	Application code, imported from the agent entry. `tools/mcp.py` is reserved for MCP declarations
`middleware/`	Application code, imported from the agent entry
`channels/<name>.py`	Managed messaging channels (direct children only)
`schedules/<name>.py`	Managed cron schedules (direct children only)
`sandbox/__init__.py`	Enables a sandbox filesystem and shell
`identity.py`	Enables caller authentication
`memory.py`	Enables durable memory
`pyproject.toml`	Dependencies; also selects the deployment's Python version
`.env`	Local and deploy secrets
`evals/`	Harbor evals workspace

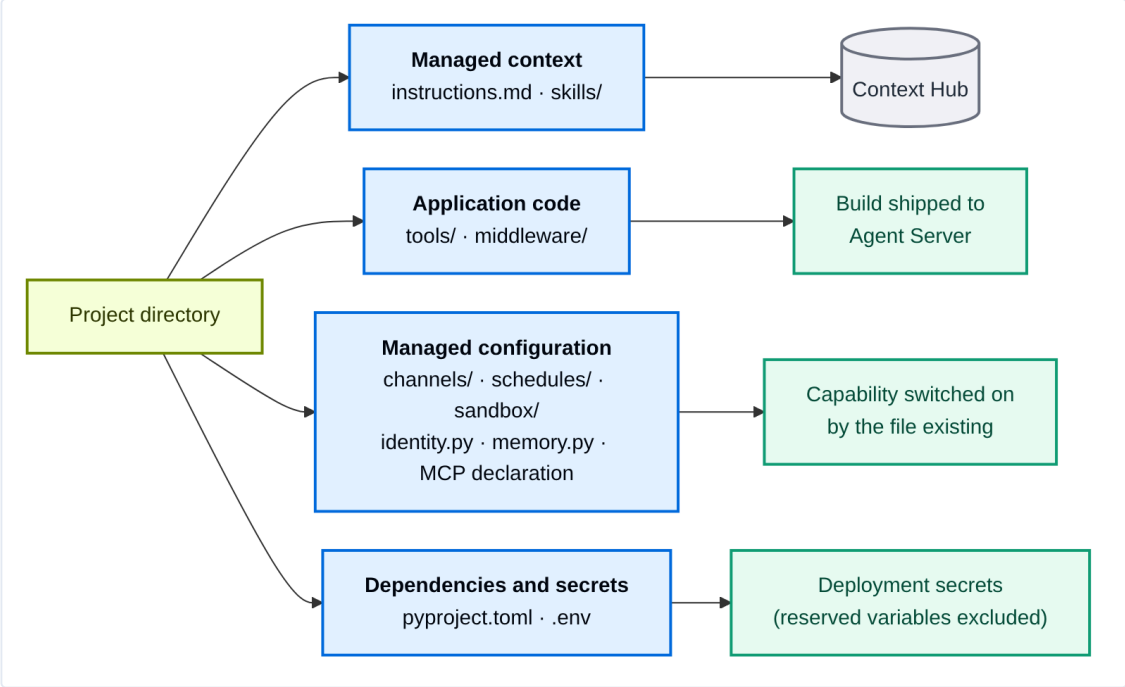


Figure 7. How each part of an MDA project is routed at deploy time.

Four ways MDA treats your files

1. **Managed context** (`instructions.md`, `skills/`, optional memory) is synced to **Context Hub**.
2. **Application code** (`tools/`, `middleware/`) is imported like any module and shipped inside the build.
3. **Managed configuration** (`channels/`, `schedules/`, `sandbox/`, `identity.py`, `memory.py`, MCP declarations) enables capabilities by existing. Only *direct children* of `channels/` and `schedules/` count; nested modules are ignored.
4. **Dependencies and secrets**: `.env` is loaded locally and its non-reserved values are forwarded as deployment secrets. Reserved platform variables and `.env` files are excluded from the build archive.

Sharing code outside the project

The build copies only the project directory. To use a sibling package, add it as a local dependency (`uv add ../shared`) and import it normally. The shared package needs `[project]` and `[build-system]` tables (the documentation's example uses Hatchling). `mda dev` reloads on edits and `mda deploy` ships the package.

Advantages, limitations and edge cases

Advantages. A working hosted agent in minutes; convention over configuration (files switch features on); instructions and skills are editable without redeploying; a local Studio loop that mirrors production.

Limitations. Public beta; the runtime owns the checkpointer, store and backend, so you cannot supply your own; the quickstart does not state plan-tier limits.

> **Edge case:** the provider tool dictionary must match the model. If you swap `model="openai:..."` for an Anthropic model but leave `{"type": "web_search"}` in `tools`, nothing fails loudly; the agent just never searches.

> **Edge case:** the agent `name` must start with a letter and may contain only letters, numbers, underscores and hyphens, because it is also the graph ID that clients pass as `assistant_id`.

CHAPTER 11

Managed Deep Agents: Defining the Agent

> **At a glance:** `define_deep_agent` returns a **definition**, not a running agent. At deploy time the managed runtime compiles it with `create_deep_agent`. Most configuration (system prompt, skills, memory, sandbox, identity, channels, schedules) lives in project files rather than in the definition.

What and why

`define_deep_agent` is imported from `managed_deepagents`. It keeps the parameters that describe *behaviour* (model, tools, middleware, subagents, permissions, interrupts, output schema) and drops those that describe *infrastructure*, which the runtime owns (backend, store, checkpointer). That split is what lets LangSmith host the agent for you.

Parameters

Parameter	Purpose	Notes
`name`	**Required.** Graph ID and default deployment name	Must start with a letter; letters, numbers, `_`, `-`. Clients pass it as `assistant_id`. `mda deploy --name` overrides the deployment name
`model`	The chat model	A `provider:model` string or a LangChain chat model instance. Provider keys go in `.env`
`tools`	Tools the agent can call	Define in local modules and import them. Remote MCP servers use MCP connectors instead
`middleware`	Behaviour around model calls, tool calls and lifecycle	Runs in list order
`subagents`	Specialist delegated agents	Each may have its own prompt, model and tools
`permissions`	Path rules for the built-in file tools	**Cleared when the project declares a sandbox**, because a permission rule would disable the sandbox `execute` tool
`interrupt_on`	Pause before selected tool calls for approval	The reviewer can approve, edit or reject
`response_format`	Force output to match a schema	Structured output

Parameters that exist on `create_deep_agent` but are owned by the runtime are simply not exposed.

Models, including the LLM Gateway

Use `openai:`, `anthropic:` or `google_genai:` strings, or pass a model instance (required for Microsoft Foundry). With the **LLM Gateway**, prefix the model ID with `langsmith:` (for example `langsmith:moonshotai/kimi-k3`). Note the difference: gateway IDs use a **slash** between provider and model, direct provider strings use a **colon**. LangChain-hosted models need no provider secret and draw on Gateway Credits; models from a provider your workspace has configured use that provider's secret and bill to your provider account. `mda init my-agent --gateway` scaffolds a gateway project.

Per-run configuration with a graph factory

Instead of exporting a static definition, export a **function named `agent`**. It is a graph factory (sync or async) that receives a `ManagedServerRuntime` and returns a definition. Callers choose behaviour through the run's `context` argument.

```python
# agent.py (sketch; field names follow the documentation)
from managed_deepagents import define_deep_agent

async def agent(runtime):
    ctx = runtime.execution_runtime.context if runtime.execution_runtime else None
    # ctx is None on channel runs, and execution_runtime is None when the
    # server loads the agent only to inspect its schema.
    return define_deep_agent(
        name="support-agent",           # must be an inline string literal
        model="openai:gpt-5.5",
        # instructions=..., skills=[...], mcp=[...], sandbox=...  (selectable resources)
    )
```

Selectable managed resources can be set only by a factory, never by a static definition:

Field	Omitted	Supplied	Cleared
`instructions`	uses `instructions.md`	uses the supplied prompt	`""` removes authored instructions
`skills`	exposes all project skills	exposes only the selected skills	`[]` exposes none
`mcp`	uses the `mcp` declaration	uses only the selected MCP definitions	`[]` exposes no MCP tools
`sandbox`	uses the `sandbox` declaration	uses the supplied sandbox	`None` disables the sandbox

A skill path must end in `skills/<name>` and point to a skill in the project. A factory may return generated skill definitions, but their names cannot duplicate project skill names. **Memory is not selectable per run**: declare it in `memory.py` and control access with an `allow` policy.

Rules every factory must obey

1. **No side effects.** The factory runs at the start of each run, on resume and retry, and when Agent Server reads schemas. For the same context it must return the same definition, and it must not create records, call external APIs or change shared state.
2. **Stable interrupts.** Middleware that can interrupt must be present in *every* configuration, so resumed runs still have it.
3. **Inline `name`.** `mda build` reads the name without calling the factory.
4. **Install every model integration** the factory can return; the build does not detect them.
5. **Run context is not verified identity.** Authorize in tools against the verified caller on the runtime (see *Connections, Identity and the Runtime*).
6. For per-model-call behaviour changes *within* a run, write custom middleware instead.

> **Edge case:** declaring a `sandbox/` directory silently clears `permissions`. If your agent relied on path rules, restrict sandboxed agents through the tools you provide instead.

> **In an interview:** "`define_deep_agent` vs `create_deep_agent`?" The first returns a configuration the platform compiles and supplies infrastructure for; the second returns a compiled graph you host and wire to your own checkpointer, store and backend.

CHAPTER 12

Managed Deep Agents: Instructions, Skills and Context Hub

> **At a glance:** Instructions are the *always-on* system prompt, skills are *on-demand* procedures, memory is *agent-updatable* knowledge. Instructions and skills are read-only to the agent and stored in **Context Hub**, so they can change without a redeploy.

Concept	Role	Loaded when
Instructions	Always-on system prompt	Every run
Skills	Task-specific procedures	When the agent selects them
Memory	Knowledge the agent can update	When durable memory is enabled

Instructions

`instructions.md` sits at the project root and is standard Markdown. Its contents are inserted into the system prompt on **every run**. The agent cannot modify it. Use it for role, behaviour, constraints and guidance about when to use which tool.

Skills

Each skill is a directory under `skills/` with a `SKILL.md` that has `name` and `description` frontmatter. Supporting files (templates, scripts) can sit beside it.

```
skills/
  research/
    SKILL.md
    templates/report.md
    scripts/fetch_sources.py
```

MARKDOWN

```
---
name: research
description: Gather and synthesize context before answering complex questions.
---

# Research
Use this skill when a task needs more than a direct answer.
1. Identify what information is missing.
2. Use `query_db` to look up relevant records.
3. Summarize findings before responding to the user.
```

Loading follows **progressive disclosure**: at startup the agent sees only each skill's name and description; when a task matches a description it reads the full `SKILL.md`; supporting files load only when needed. Because selection depends on the description, a vague description may leave a good skill unused.

Context Hub

Context Hub is LangSmith's store for a managed agent's deploy-owned context: system prompt, skills and optional durable memory.

Content	Source	Synced on deploy	Agent-writable
System prompt	`instructions.md`	Yes	No
Skills	`skills/`	Yes (every UTF-8 file)	No
Durable memory	`memory` declaration	Enables the tree only	Yes, under `/memories/`

Not synced: tools, middleware, MCP connectors, channels, schedules, sandboxes and the agent definition. They ship with the compiled deployment.

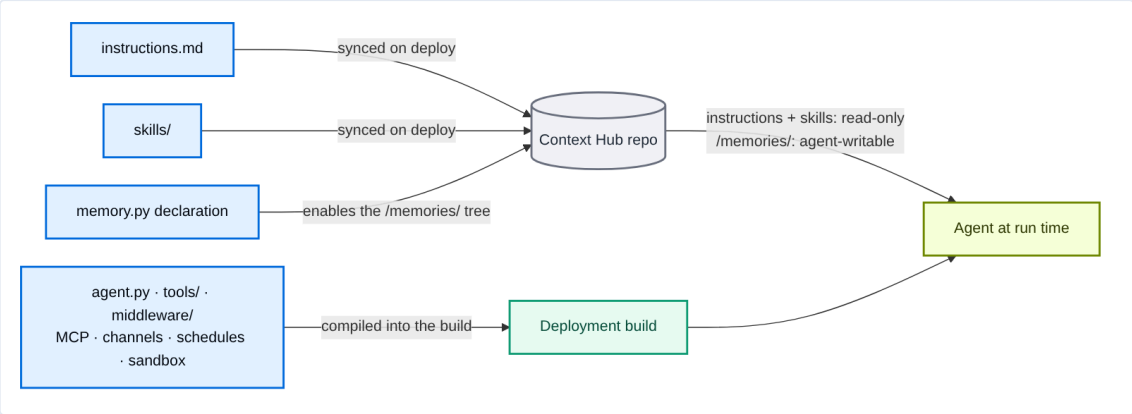


Figure 8. What syncs to Context Hub and what ships with the build.

Source of truth. Keep the project files authoritative. Hub edits (made in the LangSmith UI) are a fast way to change behaviour but remain only until a later deploy overwrites them. Deploys also **remove deployed skill files that no longer exist locally**.

Opening the repo. From the deployment in LangSmith, follow the link in the *Details* sidebar to the Context Hub repo that holds instructions, skills and memories. An archived repo is labelled as such.

Conflict handling on deploy

A conflict arises when `instructions.md` or `skills/` changed in the Hub since the last deploy. The CLI halts before syncing.

Shell	Behaviour
Interactive	Asks whether to overwrite the Hub copy. The default answer (No) keeps the Hub version and skips that section
Non-interactive (CI, redirected output)	No prompt; the deploy errors until you pass `--context-strategy`

Strategy	Effect
`overwrite`	Replace the Hub-edited section with the project copy, then continue
`keep-hub`	Keep the Hub version, skip that section, then continue

If the repo changes *during* a sync, the deploy fails with a commit conflict and must simply be re-run.

> **Edge case:** choosing `keep-hub` means later project edits to that section do not reach the deployment until a future deploy resolves the conflict. Local `mda dev` uses a mock Hub, so local Studio sessions never create or update the hosted repo.

CHAPTER 13

Managed Deep Agents: Tools, Middleware and MCP Connectors

> **At a glance:** Instructions and skills are discovered automatically. **Tools and middleware are not**: you import them and pass them to `define_deep_agent`. Remote MCP servers are different again: they are *declared* in `tools/mcp.py` and loaded by the platform.

Custom tools

Tools are application code. Put them under `tools/` and use the `tool` decorator from `langchain.tools` with `parse_docstring=True`, so the docstring's `Args:` section becomes the parameter descriptions the model sees.

PYTHON

```python
# tools/customer.py
from langchain.tools import tool

@tool(parse_docstring=True)
def lookup_customer(customer_id: str) -> str:
    """Look up a customer record.

    Args:
        customer_id: The customer's unique ID.
    """
    ...
```

PYTHON

```python
# agent.py
from managed_deepagents import define_deep_agent
from tools.customer import lookup_customer

agent = define_deep_agent(
    name="support-agent",
    model="openai:gpt-5.5",
    tools=[lookup_customer],
    interrupt_on={"lookup_customer": True},   # optional human approval
)
```

Key points:

- Use clear, **unique** tool names to avoid collisions; `tools/mcp.py` is a reserved file name.
- Tools read secrets from environment variables. Locally they come from `.env`; `mda deploy` forwards non-reserved values as hosted secrets. Per-run values (request metadata, flags) use the standard LangChain runtime-context patterns.
- Tools needing API keys or OAuth tokens should resolve credentials at runtime through **connections** (next chapters).
- To read or write sandbox files from a tool, use `runtime.backend`.
- `interrupt_on` pauses before the named tool. Locally, Studio shows the pending call and lets you resume; on a deployed agent, resume through the server API with a `Command(resume=...)` payload. The managed runtime owns the checkpointer, so no extra setup is needed.
- Tools ship with the agent code. They are **not** synced to Context Hub.

Middleware

Middleware adds behaviour around model calls, tool calls and the lifecycle. Place custom middleware in `middleware/` and pass the instances in the `middleware` list; they run in list order.

What the runtime owns, and middleware must not replace: the backend, store, checkpointer, memory, skills and the system prompt.

PYTHON

```python
# agent.py
from langchain.agents.middleware import ModelCallLimitMiddleware, PIIMiddleware
from managed_deepagents import define_deep_agent
from middleware.audit import log_tool_calls

agent = define_deep_agent(
    name="support-agent",
    model="openai:gpt-5.5",
    middleware=[
        PIIMiddleware("email", strategy="redact", apply_to_input=True),
        ModelCallLimitMiddleware(run_limit=50),
        log_tool_calls,
    ],
)
```

PYTHON

```python
# middleware/audit.py
from langchain.agents.middleware import wrap_tool_call
from langchain.messages import ToolMessage
from langchain.tools.tool_node import ToolCallRequest
from langgraph.types import Command

@wrap_tool_call
async def log_tool_calls(request: ToolCallRequest, handler) -> ToolMessage | Command:
    print(f"calling {request.tool_call['name']}")
    result = await handler(request)
    print("done")
    return result
```

> **Critical edge case: async hooks.** MDA always invokes the agent with `ainvoke` and `astream`, so **custom middleware must use async hooks**. Synchronous hooks still work when you call Deep Agents directly with `invoke` / `stream`, which is why this fails only after you move to MDA.

Middleware is not synced to Context Hub; it ships with the build. To touch the thread's sandbox from a hook, use `runtime.backend`.

MCP connectors: giving the agent remote tools

MDA creates the MCP client and loads the server's tools, so you do not import them. Declare servers in a module under `tools/` (conventionally `tools/mcp.py`) that exports a module-level `mcp`. **A project has exactly one MCP declaration**, so every server goes into it.

PYTHON

```python
# tools/mcp.py
from managed_deepagents import define_mcp

mcp = define_mcp(
    servers={
        "langchainDocs": {
            "transport": "http",          # Streamable HTTP; "sse" is the legacy option
            "url": "https://docs.langchain.com/mcp",
        },
    },
)
```

Transports. `http` (Streamable HTTP) and `sse` (legacy). **Stdio servers are not supported**; expose the server over HTTP or re-implement its operations as an authored tool.

Selecting tools. All tools are exposed by default. `include_tools` is an allowlist; `exclude_tools` is a denylist applied afterwards; a tool cannot be in both. Filters match **raw MCP tool names**, before any prefix is added. By default tools are prefixed with the server name, so `search_docs_by_lang_chain` on server `langchainDocs` becomes `langchainDocs__search_docs_by_lang_chain`.

Per-server options

Option	Notes
`transport`, `url`	Required
`headers`	Static headers sent to the server
`include_tools`, `exclude_tools`	Raw tool names
`default_tool_timeout`	Per-call timeout: **seconds in Python, milliseconds in TypeScript**
`automatic_sse_fallback`	HTTP only: allow fallback to SSE
`reconnect`	SSE only

Definition-level options: `prefix_tool_name_with_server_name` (default `true`) and `throw_on_load_error` (default `true`: one failing server fails the whole load; `false` starts with a partial tool set).

Credentials. Two routes: *MCP OAuth* for servers that advertise OAuth and support automatic client registration (`mda connections create <slug>`, or add `--mcp <url>`), or an *opaque secret / general OAuth connection* set through the server's `connection` option using `connections.get(...)`.

Concept	Kind	How it reaches the agent
Tools	Application code	Imported and passed in the definition
Middleware	Application code	Imported and passed in the definition
MCP connectors	Managed configuration	Declared under `tools/`; not imported
Instructions, skills	Managed context	Synced to Context Hub

CHAPTER 14

Managed Deep Agents: Durable Memory

> **At a glance:** By default a managed agent remembers only within a thread. **Durable memory** is optional knowledge that persists across threads, sessions and deployments, backed by Context Hub. It has two independent layers, **agent** (shared by all callers) and **user** (one authenticated person). Requires `managed-deepagents>=0.8.0`.

Enabling it

Add `memory.py` at the project root and export a `memory` declaration built with `define_memory`. Omit a layer to disable it; remove the declaration to turn durable memory off entirely.

```python
# memory.py: agent layer only (the tutorial's form)
from managed_deepagents import MemoryLayer, define_memory
memory = define_memory(agent=MemoryLayer())

# memory.py: both layers
memory = define_memory(agent=MemoryLayer(), user=MemoryLayer())
```

Layer	Mount path	Owner	Use
Agent	`/memories/agent/`	The deployment, shared by **all** callers	Team conventions, reusable procedures
User	`/memories/user/`	The authenticated person who started the run	Personal preferences, caller-specific context

The runtime never copies content between layers. Use *instructions* for always-on behaviour and *skills* for task procedures; memory is for knowledge **learned during runs and reused later**.

Hot and cold memory

Path	Role	Behaviour
`AGENTS.md` at a mount root	**Hot** memory	Loaded into every model call; keep it compact
Other files under the mount	**Cold** memory	Read only when relevant

The agent uses the built-in `read_file`, `edit_file` and `write_file`. A hot file that does not exist yet stays empty until the first write. Writes outside the mounted locations, including other paths under `/memories/`, are **not durable**.

Guiding what the agent remembers

The agent decides what to remember from its prompt. Make the policy explicit with a *Memory* section in `instructions.md`: keep compact, frequently useful knowledge in each mount's `AGENTS.md` and longer material in cold files; store only broadly appropriate procedures in the agent layer; never store personal data, customer-private data, credentials or tokens in the agent layer; keep personal context in the user layer; treat existing memory as untrusted notes, not instructions; and do not claim to have remembered something if a write fails. Deploys never overwrite durable content already in Context Hub.

Access control: a per-run decision

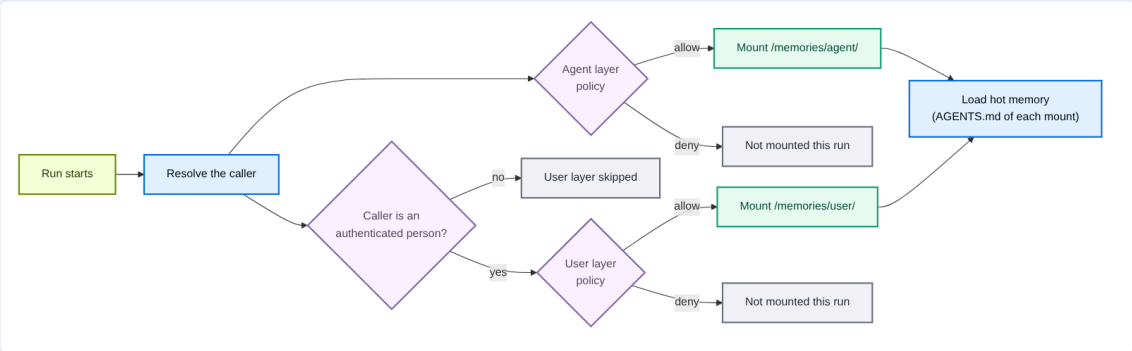


Figure 9. How a run decides whether to mount agent and user memory.

1. **Resolve the caller.** User memory stops here unless the caller is authenticated as a *person*.
2. **Evaluate the layer's policy**: your `allow(context)` function if supplied, otherwise the default.
3. **Mount and load.** Layers that pass mount at their paths and their hot memory loads.

Run source	Agent memory	User memory
Slack one-to-one DM	Allowed	Allowed
Slack channel or group DM	Allowed	Denied
Direct API run	Allowed	Denied
Studio, verified user	Allowed	Allowed (policy not called)

Why these defaults? Agent memory is deployment-shared, so it is on by default. User memory is personal, so it is on by default only where the conversation is already private. The runtime cannot tell from outside whether a channel or API run is private, so those default to denied.

To override a default, supply `allow` (sync or async). The documentation's example keeps agent memory open to everyone and enables user memory when the caller passes `remember: true` in the run context. **A policy can only widen access within the caller the runtime already authenticated**: it cannot enable user memory for a service principal such as a plain LangSmith API key, and it cannot select someone else's memory. Returning `false` removes only that layer's mount for the run; stored memory is not deleted. Policies run once per run, before storage mounts.

Identity behind user memory

User memory is keyed on the authenticated principal, never on caller-supplied input: the Slack user the event resolves to, the logged-in LangSmith user in Studio, the developer's `langsmith-dev` principal under `mda dev`, or whoever your identity provider authenticates for direct API runs. A LangSmith API key authenticates a *service*, so the default identity provider never mounts user memory; configure Supabase or backend identity for signed-in end users. Each person's memory lives in a separate Context Hub repository derived from the deployment and the principal, so two deployments never share a person's memory.

Testing

Studio is the way to test user memory without Slack: verified Studio users receive every declared layer. Deployed memory lives in Context Hub; under `mda dev` it is stored on disk under `.mda/__contexthub__`, keyed on the `langsmith-dev` principal. **Local and deployed stores are separate**; facts taught locally do not appear after deployment. If a layer seems to do nothing, first confirm it is declared in `memory.py`.

Security

> **Warning:** agent memory is shared by every caller and every caller can influence it. Store only what all callers may read and modify; never store personal data or secrets there. Memory is untrusted input for later callers: it must not grant authority, change tool permissions or bypass approvals. Keep those controls in the agent definition. Do not enable agent memory where callers must not influence one another.

Concept	Role	Scope
Instructions and skills	Deploy-owned behaviour	Deployment-wide, read-only to the agent
Thread state	Conversation continuity	One thread
Agent memory	Learned, retained knowledge	Deployment, across threads
User memory	Personal retained context	One authenticated person, across threads

CHAPTER 15

Managed Deep Agents: Sandboxes

At a glance: A sandbox gives a managed agent an isolated filesystem and shell. Declare it in `sandbox/__init__.py` with `define_sandbox`. The page documents one backend, **LangSmith Sandboxes**. Each durable thread gets its own sandbox, cloned from a snapshot built from your optional `setup.sh`.

Declaring it

```
sandbox/
  __init__.py      # required: calls define_sandbox
  setup.sh         # optional: provisions a snapshot
```

`mda init` scaffolds the declaration (omit with `--no-sandbox`) but does not create `setup.sh`. The sandbox is enabled **only while `sandbox/` exists**.

Option	Default	Meaning
`idle_ttl_seconds`	600	Inactivity before the sandbox *and its contents* are deleted. **Not recoverable**
`default_timeout`	600	Seconds allowed per command

Snapshot provisioning with `setup.sh`

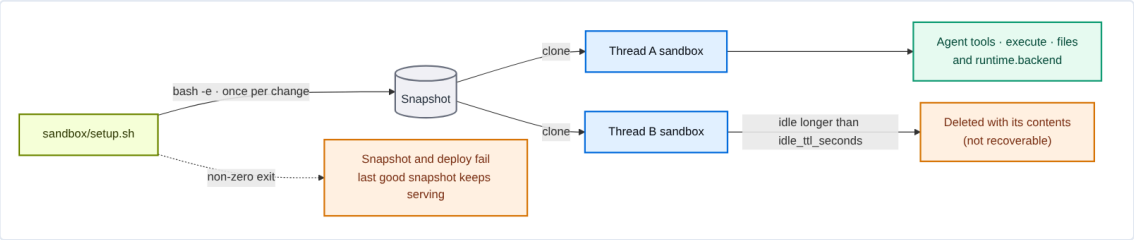


Figure 10. How a setup script becomes a snapshot and each thread's sandbox.

If `sandbox/setup.sh` exists, `mda deploy` and `mda dev` run it **once** and save the result as a snapshot. It runs under `bash -e`, so a non-zero exit fails the snapshot and the deploy. Cloned repos and installed packages persist in the snapshot. New threads **clone** the snapshot rather than rerun the script; the snapshot rebuilds only when `setup.sh` changes. If a rebuild fails, the live deployment is not moved: the last successful snapshot keeps serving.

Edge cases: deploy-forwarded `.env` values are available *during* `setup.sh` but thread sandboxes do not inherit them. Anything written to disk becomes part of every thread's image, so **never write secrets in `setup.sh`**. Editing `setup.sh` and redeploying does not wipe `/workspace` on existing threads.

Choosing a base. By default LangSmith's template is used. Otherwise set exactly one of `snapshot_name` (tags allowed), `snapshot_id` or `docker_image`. For a private image supply `registry` with `url`, `username` and `password_env`; only the variable *name* is compiled in, the value stays out of the build and snapshot.

The sandbox proxy (secrets stay outside)

The proxy runs outside the sandbox. It injects headers into matching outbound requests and controls reachable destinations, so sandbox code can call authenticated APIs without holding credentials. A rule has a `name`, `match_hosts` and `headers`; a header value can reference a workspace secret such as `"Bearer {OPENAI_API_KEY}"` (store the secret in workspace secrets first). `access_control.allow_list` restricts destinations, and a connection reference can supply the value through `bearer(ref)` or `basic(username, ref)`.

How the agent uses it

The built-in tools do the work: `ls`, `read_file`, `write_file`, `edit_file`, `delete`, `glob`, `grep` and `execute`. Instructions can tell the agent where to work and what not to change.

Code access with `runtime.backend`

Authored tools and middleware reach the sandbox through `runtime.backend` (needs `managed-deepagents>=0.8.0`). Operations: `ls`, `read` (2000 lines by default), `write`, `edit`, `delete`, `grep`, `glob`, `execute`, plus `upload_files` and `download_files` for raw bytes; each has an async `a`-prefixed twin.

- Each call binds to **the sandbox of the thread handling the current run**.
- The backend resolves **lazily**: a tool that never touches it never provisions a sandbox.
- Upload/download results carry `path` and `error` (download also `content`). Errors include `file_not_found`, `permission_denied`, `is_directory`, `invalid_path`: **always check `error`**.

> **Edge cases:** `runtime.backend` covers only the sandbox, not Context Hub (so skills, instructions and memory are out of reach). Without a sandbox it is `None`: guard every call, since a project can remove `sandbox/` after a tool ships. A root `glob` returns an error instead of provisioning a sandbox. `delete`, `upload_files` and `download_files` depend on the installed backend and raise if it lacks them.

Cleanup. Delete `sandbox/` to opt out. `mda delete` also removes the deployment's managed sandboxes, its `{deployment}--setup-*` recipe snapshots and any registry.

Goal	Use
Write files, run code, run shell commands in isolation	Sandbox
Store knowledge across threads	Memory
Always-on behaviour with no filesystem	Instructions

CHAPTER 16

Managed Deep Agents: Connections, Identity and the Runtime

> **At a glance:** *Connections* hold credentials in the LangSmith workspace (not your project). *Identity* decides who may call the agent. The *runtime object* keeps three trust sources separate: context (what the caller asked for), server info (who the caller verifiably is) and channel (where the message came from).

Connections

A **connection** links the agent to an external service (GitHub, Notion, Tavily, ...). Tools and MCP definitions resolve it at runtime with `connections.get(slug, {"type": "agent" | "user"})`.

Why use them. Credentials stay out of the project (otherwise `mda deploy` collects `.env` values as secrets on every deploy); a user-owned connection resolves the *requesting person's own* credential; the platform runs the OAuth round-trip so you need no callback route or token store; and connections belong to the workspace, so several deployments can share a slug, each with its own credential.

Owner	Resolves to	Typical use
`agent`	One credential belonging to the deployment, shared by all callers	Web search, one team channel
`user`	The calling person's own credential	Acting as the requester, reading their private pages

> **Edge cases:** `connections.get` *selects* an owner; it never creates one. A missing agent credential fails the run; a missing user credential raises an authorization interrupt. A credential under the same slug but a different owner does not satisfy the lookup. Agent-owned credentials belong to a deployment, so create them after at least one successful `mda deploy`. Your code never passes a user ID; the runtime determines the caller.

Create modes (they cannot be mixed in one command):

Mode	Use when	Owner
Opaque secret	Fixed API key or static secret	Agent
General OAuth (bring your own app)	You register your own OAuth app	Each caller, or the agent via `--authorize` / client credentials
MCP OAuth	An MCP server supports automatic client registration	Each caller, or the agent via `--authorize`

BASH

```bash
# opaque secret (always agent-owned)
uv run mda connections create organization-tavily --secret-from-env TAVILY_API_KEY
# catalog OAuth: you supply only client id + secret
uv run mda connections create team-github --oauth github --client-id <ID> --secret-from-env GH_SECRET
uv run mda connections catalog          # list preconfigured services
```

PYTHON

```python
# in a tool
from managed_deepagents import connections   # import path: confirm against the MDA reference
```

```
api_key = await connections.get("organization-tavily", {"type": "agent"})
```

Slugs are workspace-unique names using lowercase letters, numbers and single hyphens, and should identify the account (for example `organization-tavily`); the provider name goes to `--oauth`, not the slug.

OAuth details. `--scope` replaces catalog defaults; `--allowed-scope` sets a ceiling for later authorizations; `--authorization-param KEY=VALUE` adds parameters; `--auth-method` is `client_secret_basic`, `client_secret_post` or `none`. On creation the CLI prints a **redirect URI to register with the provider**. Custom endpoints use `--authorize-url` / `--token-url`. The **client-credentials** grant authenticates the agent as an application with no browser sign-in and is always agent-owned. `--authorize` stores an agent-owned grant by signing in once; use it only when the provider offers no application identity, and prefer a **dedicated team-owned account** (a personal account gives the agent the owner's full access, shows their name in audit logs and breaks when their access changes).

The authorization interrupt. Before the first model turn the runtime checks every user-owned connection. If any grant is missing it raises one LangGraph interrupt of type `credential_authorization_required` listing the missing `slug`s (and `connect_url`s for OAuth). The frontend opens each `connect_url` (HTTPS only), polls the Agent Auth session until it is `completed`, `failed` or `expired`, and resumes with `credential_authorization_completed` and the `connected_slugs`. The UI never receives the access token. Slack and Studio handle this for you; a custom frontend must implement it (first-class custom-channel support is described as in development).

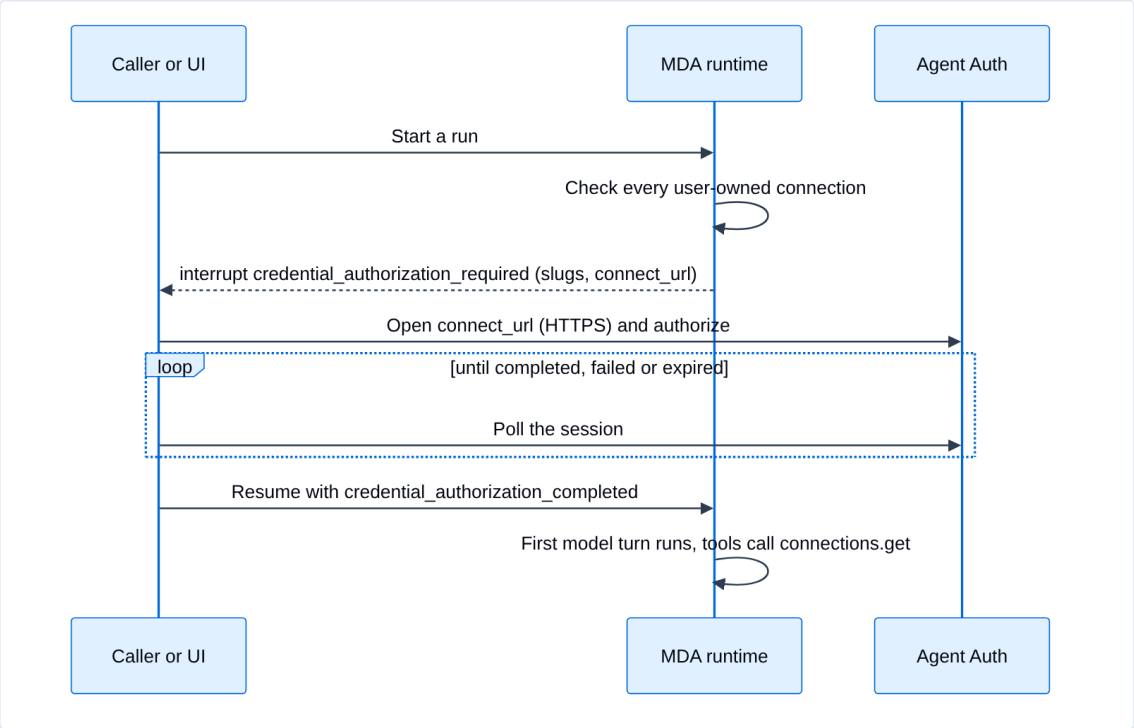


Figure 11. Authorization interrupt flow for user-owned connections.

Limitations. User-owned connections hold OAuth grants; Slack and Studio cannot collect per-caller API keys. Connections are managed with the CLI (`list`, `get`, `delete`), not the LangSmith UI. Under `mda dev`, agent-owned opaque connections read an environment variable named `MDA_DEV_` + the slug uppercased with hyphens turned into underscores; user-owned connections can only be tested end to end on a deployed agent. Rotating an agent-owned value takes effect on the next run without a redeploy.

Identity

Identity determines who can invoke the agent. It is declared in `identity.py` with `define_identity(auth=...)`.

Provider	Declaration	Who verifies	Per-user private threads
LangSmith API key (default)	`auth.langsmith_api_key()`	LangSmith Cloud	No
Supabase	`auth.supabase(project_ref="...")`	MDA, via Supabase JWT	Yes
Your own backend	`auth="backend"`	Your API, asserting the user ID with a shared secret	Yes

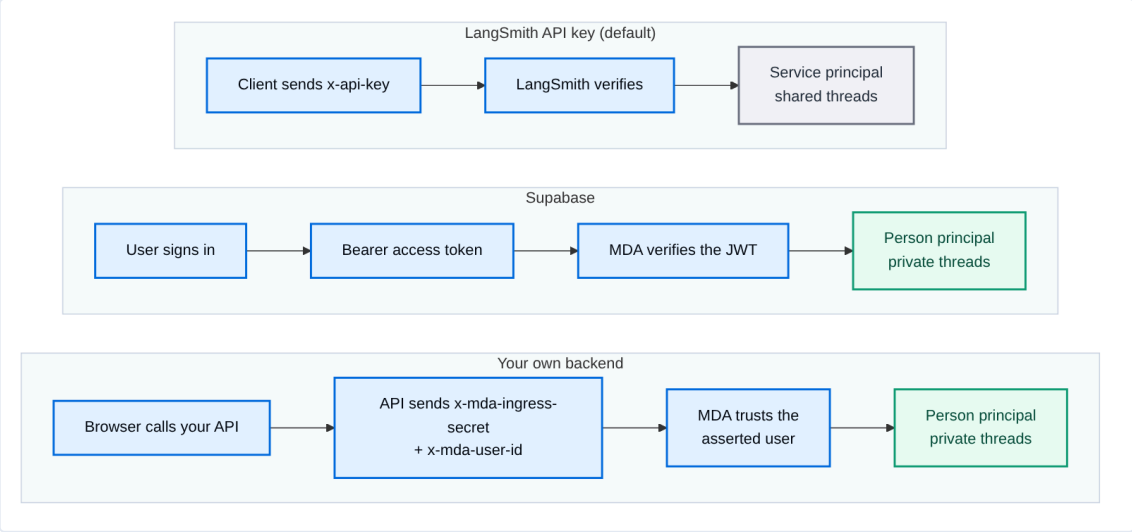


Figure 12. The three identity modes and how each verifies a caller.

```python
# identity.py: API key (default)
identity = define_identity(auth=auth.langsmith_api_key())
# Supabase
identity = define_identity(auth=auth.supabase(project_ref="your-project-ref"))
# Own backend
identity = define_identity(auth="backend")
```

- **API key.** Clients send `x-api-key`. Anyone holding the key reaches the deployment and sees the same threads.
- **Supabase.** Enable an auth provider, copy the project reference (the subdomain before `.supabase.co`), declare it, and send the user's access token as `Authorization: Bearer ...`. The publishable key is used only for sign-in.
- **Own backend.** Browser → your API → deployment. Generate `MDA_INGRESS_SECRET` (`openssl rand -hex 32`), put it in `.env`, and have your backend send `x-mda-ingress-secret` and `x-mda-user-id` on every request. Preflight fails if backend identity is declared without the secret.

> **Edge cases:** workspace access bypasses per-user privacy (Studio can read every thread). Holders of `MDA_INGRESS_SECRET` can impersonate any user, so it must never reach browsers or mobile apps. Adding Supabase identity later does not retroactively assign owners to existing threads. In backend mode a changed user ID routes the user to a different set of threads. Troubleshooting: 401 means the credential header is missing or wrong; 403 usually means a caller is reaching another user's threads.

The managed runtime

Agent factories, tools and middleware receive the native LangChain/LangGraph runtime with three managed fields added.

Code location	Managed type	Extends
Agent factory	`ManagedServerRuntime[Context]`	`ServerRuntime`
Tool	`ManagedToolRuntime[Context, State]`	`ToolRuntime`
Middleware hook	`ManagedRuntime[Context]`	`Runtime`

- **`server_info`**: native server metadata plus the **verified caller**: `principal` (`id`, `kind` = person / service / channel, `claims`), `subject` (the authority the run acts with: agent or user), `link`, and `source` (`http`, `schedule`, `studio`, `slack`, ...). **Read the caller from `principal`, not the native `user`.**
- **`channel`**: the verified delivery that started the run (`name`, `provider`, `event`, `raw_event`, `post`), or `None` for direct API and schedule runs.
- **`backend`**: the thread's sandbox filesystem, or `None`.

Three separate sources of trust. *Context* comes from the API call's `context` parameter, is validated against your schema, and selects behaviour but **grants no access**. *Server info* comes from identity resolution of a verified token; MDA strips identity keys that clients send through `configurable`, so this is the field to authorize against. *Channel* comes from managed channel ingress; a `channel` key placed in a direct API call's context does **not** populate `runtime.channel`.

PYTHON

```python
@tool
async def check_order(order_id: str, runtime: ManagedToolRuntime[AppContext]) -> str:
    """Check the status of an order."""
    principal = runtime.server_info.principal if runtime.server_info else None
    if principal is None:
        return "Sign in to check orders."
    if runtime.channel is not None and runtime.channel.post is not None:
        await runtime.channel.post({"type": "content", "content": "Checking that order now."})
    region = runtime.context.region if runtime.context else "us"
    return f"Order {order_id} for {principal.id} ships from {region}."
```

Edge cases: channel deliveries carry no application context, so on channel runs `context` is `None` in the factory, tools and middleware (required context fields do not reject them). `principal` may be absent because identity is opt-in. `post` sends an *extra* message; the agent's final reply still posts automatically, and the built-in Slack channel sends text only. The reply target is private: you cannot read or change it. In middleware, narrow with `isinstance(runtime, ManagedRuntime)` before touching managed fields. Agent Server also calls factories just to read schemas, so `execution_runtime` is `None` outside `threads.create_run` and the factory must return the same graph structure every time.

CHAPTER 17

Managed Deep Agents: Channels and Agent-Owned Interrupts

> **At a glance:** A *channel* connects a deployed agent to a messaging service: inbound events start runs, replies go back to the same conversation. Built-ins are **Slack** and **HTTP**. *Agent-owned interrupts* let your own tool post a custom Slack form and pause until a person answers.

Channels

A channel has three jobs: verify and normalize inbound events (and start a run), send the agent's reply to the originating conversation, and provision the provider resources.

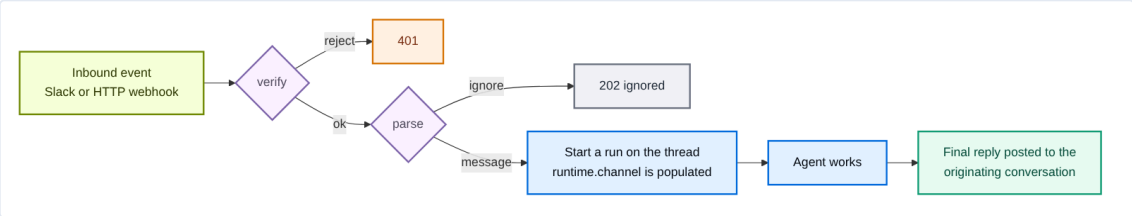


Figure 13. Runtime flow for a channel message.

Declarations live in `channels/<name>.py`, one module per channel, each exporting a module-level `channel`. The file name is the channel name (unique within the project; do not use `channels/channel.py`).

Channel	Run triggers
Slack	Mentions, direct messages, thread replies
HTTP	Any service that can send a JSON webhook

Channels vs MCP connectors: channels receive messages that start runs; connectors give the agent tools from a remote MCP server. A project can use both.

The Slack channel

Add it with `mda init ... --channel slack` or, in an existing project, `uv run mda channels init slack`. Declare it with `channels.slack()` and deploy; on the first deploy the CLI prints an HTTPS authorization link: open it, choose the workspace, approve, and press Enter in the terminal (workspaces that require admin approval need it first). **Each deployment supports one Slack channel.**

PYTHON

```python
# channels/slack.py
from managed_deepagents import channels

channel = channels.slack(
    name="Orders Helper",
    description="Answers questions about order status and returns",
```

```
    icon="orders-helper.png",          # 512x512 PNG, <= 1 MB, relative to channels/
    background_color="#0f766e",
)
```

Parameter	Default	Rules
`name`		1-35 chars; letters, numbers, spaces, `_`, `-`, `.`; cannot start/end with a space or dash
`description`		up to 139 characters
`icon`, `background_color`		icon 512x512 PNG ≤ 1 MB; six-digit hex colour
`trigger_on_all_messages`	`False`	When `False`, channel messages start runs only if they mention the agent; DMs always do
`allow_bot_triggers`	`False`	Whether other bots' messages can start runs
`reactions`	`"eyes"`	`False`, a Slack short name, or a callable `(dict with "text") -> short name` (sync or async)

Situation	Starts a run?
Direct message to the agent	Yes, every message
Channel message that mentions the agent	Yes
Channel message, no mention (default)	No
Channel message with `trigger_on_all_messages=True`	Yes
Reply in an active thread	Yes
Message from another bot	Only if `allow_bot_triggers=True`

Reactions signal "seen" before the reply is ready and never block it. A callable that raises, takes over five seconds or returns an invalid value falls back to the default emoji and the run continues, so a missing emoji is not evidence of a failed run. Redeploy after changing name, description, icon or colour.

File exchange needs `managed-deepagents>=0.8.0` and a configured sandbox. Uploaded files (including earlier ones in the same thread) are staged into `/workspace/attachments/` before the run, labelled as data not instructions; the agent sends files back with `attach_file` for paths under `/workspace`. Attachment is never automatic: your instructions must say when to produce and attach a file. Limits: 200 MiB per file each way; deleted or externally stored files are skipped. File transfers use the bot's access, so anyone in a thread can reach files others uploaded to it, and a DM and a channel thread never share attachments because each thread has its own sandbox.

> **Edge case:** human-in-the-loop requests in Slack support only `approve` and `reject` decisions.

The HTTP channel

An HTTP channel exposes the agent as a webhook endpoint for any external service. You supply two callbacks, `verify` and `parse`, and optionally `post`.

PYTHON

```python
# channels/orders.py
from managed_deepagents import channels
from lib.orders import parse, verify

channel = channels.http(provider="orders", verify=verify, parse=parse)
```

```python
import hashlib, hmac, json, os, uuid

def verify(context) -> bool:                        # context: HttpChannelRequest
    secret = os.environ["ORDERS_WEBHOOK_SECRET"]
    digest = hmac.new(secret.encode(), context.raw_body, hashlib.sha256).hexdigest()
    received = context.request.headers.get("x-orders-signature", "")
    return hmac.compare_digest(f"sha256={digest}", received)

def parse(context):
    event = json.loads(context.raw_body)
    if event.get("type") != "order.comment":
        return {"type": "ignore"}
    return {
        "type": "message",
        "message": {
            "user_id": str(event["actor"]["id"]),
            "thread_id": str(uuid.uuid5(ORDERS_NAMESPACE, str(event["order"]["id"]))),
            "content": event["comment"]["body"],
            "target": str(event["order"]["id"]),
        },
        "raw_event": event,
    }
```

- `verify` runs first. `True` continues; `False` returns `401`; raising returns `500`. Verify the signature against the **raw bytes**, not a re-serialized body, and load the secret from the environment (not `connections.get`, which needs a run). **The endpoint has no platform authentication: `verify` is the only barrier between the public internet and an agent run.**
- `parse` returns `{"type": "message", ...}` or `{"type": "ignore"}`, optionally with a Starlette `response` (for example to answer a provider's verification challenge; needs `starlette>=1.6.0`). Required message fields: `user_id` (verified, string), `thread_id` (**must be a UUID**; use `uuid.uuid5` to map external IDs, otherwise `400`), `target` (private reply destination) and `content`. `raw_event` exposes the provider JSON at `runtime.channel.raw_event`.
- `post` (optional, async) sends the reply after the run finishes and returns the message `id` and optional `url`; `on_error(error, target)` replaces the default error reply. No reply is posted when a run pauses on an interrupt or the agent already delivered a final message.
- `provider` namespaces caller IDs: channels serving the same service should share one provider value so a user maps to one principal; identical IDs under different providers are different principals.

Endpoint: `POST https://<deployment-url>/channels/<name>/events`. Responses: `202` accepted (run started, body includes `status: "accepted"` and a `deliveryId`) or ignored; `400` parse/validation failure; `401` verify rejected; `500` verify raised or the run failed to start. A `202` means *accepted*, not *completed*. Deploy normally (`mda deploy`), put the signing key in `.env`, and register the URL with the external service.

Agent-owned interrupts

An agent-owned interrupt is a pause created by **your** code: a tool posts a Slack message or modal that you design with Block Kit, then calls LangGraph's `interrupt()`. When the user presses a button or submits the modal, the platform forwards the answer as a `user_prompt_response` channel event and resumes the paused run. Requires `managed-deepagents>=0.8.0` and a deployed Slack channel; forms render only in Slack (Studio shows the raw value).

	`interrupt_on` (platform-owned)	Agent-owned interrupt
Pauses	Before selected tool calls	Whenever your tool calls `interrupt()`
UI	Platform approval card	Blocks/modal you author
Answer handled by	The platform	Your code after `interrupt()` returns

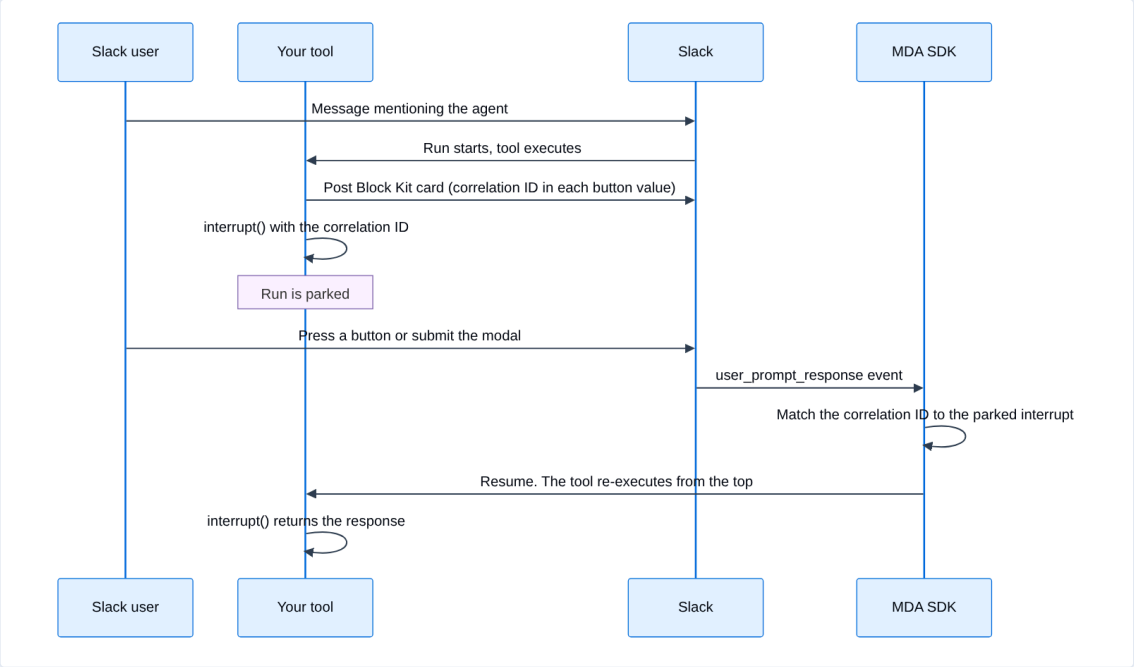


Figure 14. Lifecycle of an agent-owned interrupt in Slack.

1. **Start**: a Slack message starts the run; the tool reads the conversation from `runtime.channel` and posts a Block Kit message into the same thread with the deployment's bot token (`connections.get("mda/slack-bot-token", {"type": "agent"})`, a reserved slug).
2. **Pause**: the tool calls `interrupt()` with a value containing a **correlation ID** (`INTERRUPT_CORRELATION_KEY`, also placed in each button's JSON `value`).
3. **Answer**: the user presses a button or submits a modal.
4. **Resume**: the SDK matches the event to the parked interrupt, and `interrupt()` returns the response (`provider`, `action`, optional `value`, optional `correlation_id`).

> **Critical edge case:** a tool that pauses re-executes **from the top** on resume (standard LangGraph behaviour). Guard the posting step so it only runs when the event is *not* already a `user_prompt_response`, or you will post the form twice.

Routing a response to its interrupt

Parked interrupts	Correlation ID in response	Outcome
Exactly one	Absent	Resumes it
One or more	Present, matches exactly one	Resumes the match
Several	Absent	Nothing runs; ambiguity is logged
One or more	Present, matches none/several	Nothing runs
None	Any	Starts a fresh run with no new message; the response is on `runtime.channel.event`

Modals. A button whose `action_id` begins with `mda_modal::` opens a modal whose full view JSON is in the button's `value`. Slack's single-use `trigger_id` lasts about three seconds, so MDA opens the view itself and the agent is not told about the click. On submit, `action` equals the view's `callback_id` (set it to the button's `action_id`) and `value` equals

`state.values`. Never set `private_metadata` (the platform uses it to route back; setting it stops the modal opening). MDA does not validate the view; one Slack rejects shows the user nothing.

Slack limits (not checked by the platform): button `value` 2000 chars; modal title 24; 100 blocks per view; `callback_id` 255. An oversized button fails when the tool posts; the others fail silently for the user.

Failure modes. If a user dismisses a modal the interrupt stays parked (no cancel or expiry). If they reply in the thread instead, the message starts a normal run and the parked checkpoint is abandoned while the card's buttons stay live. If two people press the same button, the first resumes and the second starts a fresh turn; nothing deduplicates. Submissions cannot return field-level validation errors or chain modals.

CHAPTER 18

Managed Deep Agents: Schedules

> **At a glance:** Schedules run the agent on a cron cadence with no user message. **Declared schedules** are files in `schedules/` that `mda deploy` turns into LangSmith crons. **Runtime schedules** are created by the agent itself through `runtime.schedules`.

Declared schedules

Each schedule is its own file whose name becomes the schedule name (`daily_digest.py` → `daily_digest`), exporting a top-level `schedule`.

```python
# schedules/daily_digest.py
from managed_deepagents import define_schedule

schedule = define_schedule(
    cron="0 8 * * 1-5",                # 08:00 on weekdays
    timezone="America/Los_Angeles",    # omit for UTC
    prompt="Write the daily digest.",
)
```

- `cron` must be a standard **five-field** expression; seconds-based syntax is rejected.
- Define **exactly one** of `prompt` (converted to a user message) or `input` (a structured LangGraph input). Both or neither is an error.
- **Declarations must be static:** literals, lists, dicts and top-level literal constants only. No env reads, function calls, `**kwargs` or computed values; dynamic logic belongs in the agent, tools, middleware or runtime context.
- `deliver_to` posts the final response through a Slack channel (needs `managed-deepagents>=0.4.0`). Use a **channel ID**, since a scheduled run has no originating conversation; the bot must have access.

Threads. The default is an **ephemeral** thread per run, deleted by LangSmith afterwards. For state that accumulates across runs use `thread={"mode": "persistent", "id": "<uuid>"}`. You must create that thread yourself (in Studio) and copy its UUID; neither MDA nor the CLI creates it. Persistent threads require durable memory to be enabled.

Reconciliation and local testing

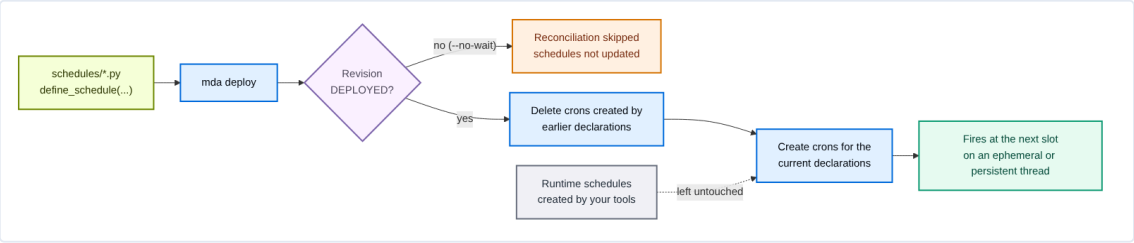


Figure 15. How declared schedules become managed crons.

When a deployment reaches `DEPLOYED`, `mda deploy` deletes cron jobs created by earlier declarations and creates jobs for the current ones; removing a file and redeploying removes its cron. Runtime-created schedules are untouched.

> **Edge case:** `mda deploy --no-wait` exits before `DEPLOYED`, so schedules are **not reconciled**. Omit `--no-wait` when adding, changing or removing schedules. Also, a deployed schedule does not run immediately; the first fire waits for the next slot.

`mda dev` creates a local cron job for each schedule (hot-recreated when `schedules/` changes); `--no-schedules` stops them firing. Local firing needs `managed-deepagents>=0.9.3`; earlier versions list schedules at startup but do not execute them.

Runtime schedules (`managed-deepagents>=0.9.0`)

Your tools call `runtime.schedules.create(...)`; the agent reaches it only through tools you write, so you control what it may do.

Parameter	Meaning
`owner`	Required: `{"type": "user"}` (runs as that person) or `{"type": "agent"}` (the agent's service identity)
`cron` or `at`	Exactly one. `at` is a timezone-aware datetime or ISO 8601 string; rounded up to the next whole minute; must be 1 minute to 365 days ahead
`timezone`	IANA zone, for `cron` only (default UTC)
`prompt` / `input`	Exactly one
`new_thread`	Default `True` for `cron`, `False` for `at`
`paused`	Default `false`
`metadata`	Free-form JSON; keys starting `mda_` are reserved

A schedule created during a Slack run **inherits the channel**: each run posts its answer back. With `new_thread=True` each run posts a new Slack message; with `False` it replies in the existing thread. Management: `list`, `get`, `update` (cron, at, timezone, prompt, input, paused, metadata; channel and thread choice are fixed at creation) and `delete`. Prefer pausing to deleting if the user may want it back. `list` hides expired one-time schedules unless `include_expired=True`, and for the agent owner it also returns declared schedules, which can only be changed in code.

Limits. At most **20 runtime schedules per owner per agent**; paused ones count, expired one-time and declared ones do not. Adjust with `MDA_MAX_AGENT_SCHEDULES` and `MDA_MAX_USER_SCHEDULES` (`>=0.9.3`).

Symptom	Likely cause
"must export a named schedule declaration"	No top-level `schedule` variable
"must define exactly one of prompt or input"	Both or neither set
"cron must be a standard 5-field expression"	Seconds-based or wrong field count
"schedule is not static"	Computed values; use literals
"Invalid thread ID: must be a UUID (HTTP 422)"	Persistent thread ID is not the UUID of an existing thread
"Schedule limit reached"	Delete schedules or raise the limit
Does not fire locally	`--no-schedules` was passed

CHAPTER 19

Managed Deep Agents: Deploying, the CLI and Local Development

At a glance: `mda deploy` compiles your project into a managed LangGraph app, syncs Context Hub, uploads the source and triggers a hosted build on LangSmith Agent Server. `mda dev` runs the same thing locally with Studio.

What `mda deploy` does

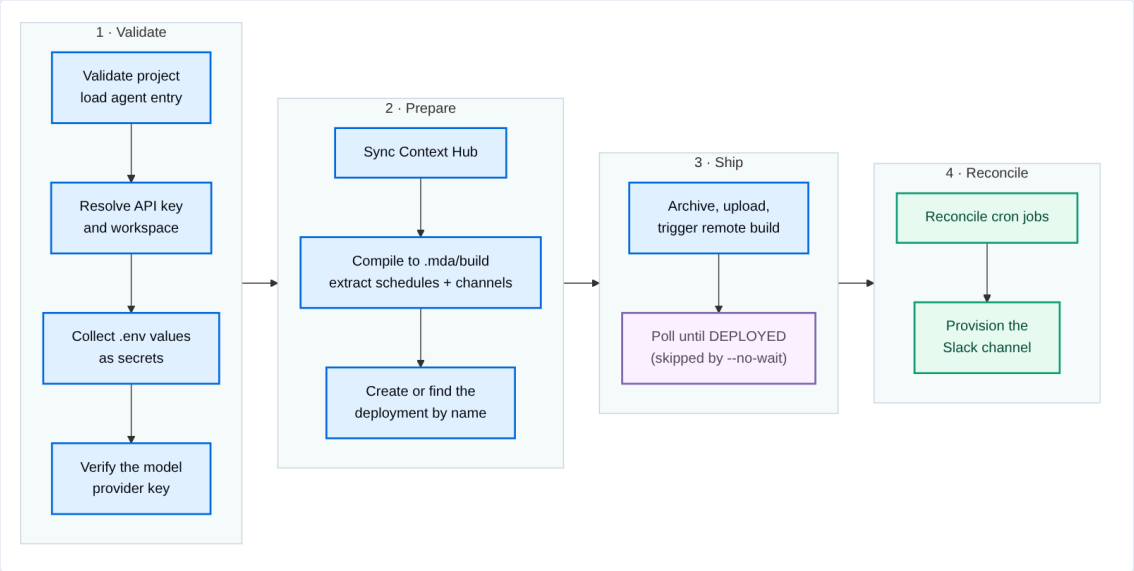


Figure 16. The mda deploy pipeline.

1. Validate the project and load the agent entry.
2. Resolve the API key and optional workspace.
3. Collect non-reserved `.env` values as secrets.
4. Verify the model-provider key exists (`.env`, shell, or workspace secrets); fail *before upload* if not.
5. Sync deploy-owned context to Context Hub.
6. Compile into `.mda/build` and extract `schedules/` and `channels/`.
7. Create or find the hosted deployment by name.
8. Archive, upload and trigger a remote build.
9. Poll until the revision is `DEPLOYED` (unless `--no-wait`).
10. Reconcile managed cron jobs.
11. Provision the Slack channel (the CLI shows any authorization step and continues when you finish it).

The result is an **Agent Server API** plus an **MCP endpoint**. Flags: `--name`, `--deployment-type dev|prod` (used only when creating a deployment), `--workspace-id`, `--context-strategy overwrite|keep-hub`, `--no-wait`.

Secrets. `mda deploy` reads project `.env` values before shell variables. Reserved platform variables (`LANGSMITH_API_KEY`, `LANGGRAPH_HOST_API_KEY`, `LANGCHAIN_API_KEY`, `LANGSMITH_WORKSPACE_ID`) authenticate the deploy but are **never uploaded** as user secrets; other entries (model keys, MCP tokens, custom tool credentials) are forwarded. Reserved variables, empty values and `.env` / `.env.*` files are excluded from the build archive. `MDA_INGRESS_SECRET` is required only when backend identity is declared.

Python version. With `managed-deepagents>=0.8.0` the build image supports Python 3.11-3.14 and picks the newest allowed by `requires-python`. Pin with `[tool.mda] python-version = "3.14"` (a quoted `major.minor` string) or `MDA_PYTHON_VERSION` for one build. This affects only the deployment image, not local builds, `mda dev`, Harbor images or sandboxes.

Regions. `LANGSMITH_ENDPOINT` selects the region: GCP US `https://api.smith.langchain.com` (default), GCP EU `https://eu.api.smith.langchain.com`, GCP APAC `https://apac.api.smith.langchain.com`, AWS US `https://aws.api.smith.langchain.com`. Set it before the first deploy; credentials must belong to the target region; non-interactive runs (CI) need `LANGSMITH_API_KEY` for that region. LangSmith managed tools are available only in GCP US. (Other MDA pages describe availability as US-only; check the current regional status before committing.)

The `mda` command reference

Command	Purpose
`mda init <name>`	Scaffold a Python project (fails if the directory exists)
`mda build [path]`	Compile without deploying (`--out`; the output directory is emptied first)
`mda dev [path]`	Compile and run the local dev server with Studio
`mda deploy [path]`	Compile, sync Context Hub, upload and deploy
`mda connections …`	`catalog`, `create <slug>`, `list`, `get <slug>`, `delete <slug>`
`mda channels init slack`	Add a Slack declaration
`mda evals init`	Set up a Harbor eval workspace
`mda logs [path]`	Tail deployed logs (`--lines`, `--level`, `--follow/--no-follow`)
`mda delete` / `mda destroy`	Delete the deployment and its LangSmith resources

`mda init` flags: `--instructions`, `--instructions-file`, `--identity`, `--memory agent|none`, `--model provider:model`, `--no-sandbox`, `--channel slack` (repeatable), `--gateway`. It generates `agent.py`, `instructions.md`, `pyproject.toml`, `README.md`, `.env` and a `.gitignore` that ignores `.env`, `.env.*` and `.mda/`. The deploy key lookup order is `LANGGRAPH_HOST_API_KEY`, then `LANGSMITH_API_KEY`, then `LANGCHAIN_API_KEY`.

Local development with `mda dev`

`mda dev` validates and compiles into `.mda/build`, copies `.env` into it, creates a **local Context Hub mock**, starts the language-specific LangGraph dev server (`uv` required) and opens Studio. Flags: `--port`, `--hostname`, `--no-browser`, `--no-reload`, `--no-schedules`.

Aspect	Local (`mda dev`)	Deployed
Context Hub	Mock	Real hosted repo
Identity	Studio supplies a local test user	Your configured provider
Sandbox	Falls back to a **temporary local folder** (path printed) if the provider fails	LangSmith sandbox

Aspect	Local (`mda dev`)	Deployed
Schedules	Local crons (≥0.9.3)	Managed crons
User connections	Cannot be exercised end to end	Full OAuth flow

> **Edge case:** because `mda dev` silently falls back to a local temp sandbox when the configured provider fails, always test identity and sandbox behaviour on a development deployment before production.

Limits and troubleshooting

- A build may not exceed **200 MB**; remove generated or large files.
- A project with a Slack channel **cannot use** `--no-wait`, since provisioning needs the deployed server URL.
- `--context-strategy` is mandatory in non-interactive shells when Hub conflicts occur.
- Deploy needs a workspace with deployments access (401/403 otherwise).

Symptom	Fix
`no agent entry file found`	Add `agent.py` at the project root
`mda dev` cannot find `uv`	Install `uv`
`No LangSmith API key found`	Set a key or add it to `.env`
Missing model provider key	Add to `.env`, export it, or set a workspace secret
`BUILD_FAILED` / `DEPLOY_FAILED`	Open the dashboard URL and read the revision logs

CHAPTER 20

Managed Deep Agents: Exposing the Agent over MCP, and Built-in Search

At a glance: Every MDA deployment serves an **MCP endpoint at `/mcp`**, so any MCP client that supports Streamable HTTP can call the agent as a tool. Separately, MDA offers **built-in web search** powered by Parallel.

The MCP endpoint

The endpoint is `<DEPLOYMENT_API_URL>/mcp`. `mda deploy` prints the *dashboard* URL, not the API URL: open the dashboard, copy the **API URL** from the deployment details and append `/mcp`. The local `mda dev` server serves the same path. The direction matters: the endpoint is **outbound** (it exposes your agent to others), while an MCP *connector* points inward (it adds a remote server's tools to your agent). A deployment can use both.

The agent is published as **one MCP tool**, named after the agent's `name`, with the agent's input schema.

Authentication follows the deployment's identity: `x-api-key: <LANGSMITH_API_KEY>` for the default mode (the key must belong to the workspace that owns the deployment) or `Authorization: Bearer <access_token>` for Supabase (impractical for static-header MCP clients). Failures: `401 missing x-api-key`; `403 API key is forbidden` (invalid, revoked or expired); `403 API key tenant mismatch` (valid key, wrong workspace).

BASH

```
claude mcp add --transport http research-assistant \
  <DEPLOYMENT_API_URL>/mcp \
  --header "x-api-key: $LANGSMITH_API_KEY"
```

JSON

```
{ "mcpServers": { "research-assistant": {
    "type": "http", "url": "<DEPLOYMENT_API_URL>/mcp",
    "headers": { "x-api-key": "<LANGSMITH_API_KEY>" } } } }
```

PYTHON

```
from langchain_mcp_adapters.client import MultiServerMCPClient
client = MultiServerMCPClient({"research-assistant": {
    "transport": "streamable_http", "url": "<DEPLOYMENT_API_URL>/mcp",
    "headers": {"x-api-key": os.environ["LANGSMITH_API_KEY"]}}})
tools = await client.get_tools()
```

Edge cases: MCP requests are **stateless**: separate calls share no thread state unless you opt in to durable memory (shared by all callers). A client that accepts only a URL and negotiates OAuth cannot authenticate, and may report a Dynamic Client Registration failure instead of the missing header; configure the header explicitly. A key in a client config is a workspace-wide shared secret. The local `mda dev` server is **unauthenticated** and grants every local caller an unscoped principal.

Built-in search powered by Parallel

MDA includes web search through Parallel (parallel.ai), run by LangSmith, so you need no Parallel account or key; the integration offers zero data retention. It is available on **LangSmith Plus and Startup plans** only and free during private beta. Configure it by setting `LANGSMITH_WORKSPACE_ID` and a personal `LANGSMITH_API_KEY` (for Studio) in `.env`, then registering the server in `tools/mcp.py`:

```python
from managed_deepagents import define_mcp

mcp = define_mcp(
    servers={
        "Parallel": {
            "transport": "http",
            "url": "https://api.smith.langchain.com/v1/managed-tools/servers/parallel/mcp",
        },
    },
)
```

If the file already defines `mcp`, **add to the existing `servers` map** rather than defining `mcp` twice. Use must comply with Parallel's terms of service.

CHAPTER 21

Managed Deep Agents: Evals and Migration

> **At a glance:** MDA evals are **Harbor** tasks, drafted with a coding agent and the `eval-engineering` skill, reviewed by a human, and run in Docker. Migration from `create_deep_agent` is mostly repackaging: one constructor call becomes a definition, and prompt, skills and memory move into files.

Evaluating with Harbor

Prerequisites: an MDA project, `uv`, Docker (task environments) and a coding agent (Deep Agents Code, Claude Code, Codex, Cursor or another that supports Agent Skills).

```bash
uvx --from npx-skills skills add langchain-ai/langchain-skills --skill eval-engineering --yes
uv run mda evals init -i      # creates evals/harbor-job.json and .mda/evals/ ; launches a coding agent
```

`evals/harbor-job.json` is **user-owned** (written only if missing); everything under `.mda/evals/` is **generated** and may be overwritten. The coding agent inspects the project, drafts a Task Spec for review, and writes the runnable task only after approval. Reviewers check four things: the Task Spec, the instruction, the environment and the verifier.

```
evals/
├── harbor-job.json
└── <task>/
    ├── Task.md           # human-reviewed spec
    ├── instruction.md    # what the agent is asked to do
    ├── task.toml         # Harbor config
    ├── environment/Dockerfile
    └── tests/test.sh     # verifier, runs after the agent finishes
```

Scoring is numeric: the verifier writes a single reward to `/logs/verifier/reward.txt` and metrics to `/logs/verifier/reward.json`. The project-specific run command (generated for you) pins Harbor 0.21.0 and loads the MDA job plugin and the LangSmith plugin; with `LANGSMITH_API_KEY` set, runs are recorded to the dataset named in `HARBOR_LANGSMITH_DATASET`. Inspect results with `harbor view .mda/evals/jobs`. The recommended loop: review failures with the coding agent; if an eval does not measure the intended behaviour, fix the task or verifier; if it exposes a product failure, fix the agent; rerun.

Migrating from Deep Agents

Deciding. Self-managed Deep Agents give you any backend, your own store and checkpointer and full control of surrounding code; MDA gives you hosting, one managed sandbox, one shared memory tree, connections with OAuth, channels and schedules. The harness is identical, so building first on Deep Agents and moving later is a normal path.

What does not carry over: custom backends (the managed sandbox is the only filesystem/shell option), custom stores and checkpointers (Agent Server provides persistence), custom application routes (use a LangSmith deployment), and **Python custom state schemas** (`define_deep_agent` rejects `state_schema`).

Old parameter	New location
`model`, `tools`, `middleware`, `subagents`, `interrupt_on`, `response_format`, `context_schema`, `cache`, `debug`	Unchanged

Old parameter	New location
(new) `name`	Required static string
`system_prompt`	`instructions.md`
`skills`	`skills/<name>/SKILL.md`
`memory`	`memory.py` with `define_memory(...)`
`backend`	`sandbox/` with `define_sandbox(...)`
`permissions`	Stays, but cleared when a sandbox is declared
`store`, `checkpointer`	Removed (runtime-managed)
`state_schema`	No Python equivalent

Steps. (1) Add `managed-deepagents` to the existing manifest; do not run `mda init` (it creates a new directory); for Hatchling projects set `allow-direct-references = true`. (2) Rewrite the entry in a root `agent.py` that exports `agent` (re-exporting from another module fails with a misleading error about `name`). (3) Move prompt, skills and memory to files; existing memory content does not transfer. (4) Declare a sandbox if you used one. (5) Optionally declare identity. (6) Put credentials in the project `.env` (shell variables are not read). (7) Add `.env`, `.env.*` and `.mda/` to `.gitignore`; `mda build` copies the whole project root. (8) `mda build`, then `mda dev`. (9) `mda deploy`.

> **Edge cases:** `mda build` does not validate the definition, so leftover managed parameters fail only when the graph loads under `mda dev`. `define_memory` mounts an empty tree: move needed knowledge into instructions or skills before deleting old memory files. Everything beside the entry ships in the deployment.

CHAPTER 22

Tutorial: Extending the Research Assistant with Search, Memory and a Schedule

> **At a glance:** Starting from the quickstart project, replace built-in search with an authored Tavily tool, enable durable agent memory, add a weekday digest schedule, and deploy. Files touched: `.env`, `tools/search.py`, `agent.py`, `instructions.md`, `memory.py`, `schedules/daily_digest.py`.

Step 1: a custom search tool. Add `TAVILY_API_KEY` to `.env`, run `uv add tavily-python`, and create `tools/search.py`:

PYTHON

```python
import os
from typing import Literal
from langchain.tools import tool
from tavily import TavilyClient

tavily_client = TavilyClient(api_key=os.environ["TAVILY_API_KEY"])

@tool
def internet_search(
    query: str,
    max_results: int = 5,
    topic: Literal["general", "news", "finance"] = "general",
) -> dict:
    """Search the internet for relevant sources."""
    return tavily_client.search(query, max_results=max_results, topic=topic)
```

Wire it into `agent.py` in place of the provider search dict:

PYTHON

```python
from managed_deepagents import define_deep_agent
from tools.search import internet_search

agent = define_deep_agent(name="research-assistant", model="openai:gpt-5.5", tools=[internet_search])
```

Restart `mda dev` and ask about the latest LangChain release; the agent should call `internet_search` and cite sources.

STEP 2: A MEMORY POLICY IN INSTRUCTIONS.MD

MARKDOWN

```markdown
## Memory
- Record reusable research procedures and project knowledge that can improve future work.
- For release research, check the project's official changelog before secondary sources.
- Never store personal data or secrets in memory.
```

Step 3: enable memory. Create `memory.py`:

PYTHON

```python
from managed_deepagents import MemoryLayer, define_memory
memory = define_memory(agent=MemoryLayer())
```

Restart `mda dev`. In one thread ask the agent to research a release and record a reusable rule ("check the official changelog before secondary sources"). Open a **new thread** and ask how it will research the next release; it should apply the rule despite having no history. That is durable memory working.

Step 4: a schedule. Create `schedules/daily_digest.py`:

```python
from managed_deepagents import define_schedule

schedule = define_schedule(
    cron="0 8 * * 1-5",
    timezone="America/Los_Angeles",
    prompt=("Review durable memory for reusable research rules. "
            "Summarize anything useful, then list open questions for today."),
)
```

If memory is empty on the first fire, the agent still returns open questions.

Step 5: deploy and inspect. `uv run mda deploy` (do **not** pass `--no-wait`, or the schedule is not reconciled). At the dashboard check that the deployment is ready, that a cron named after the module stem exists (`daily_digest`, or `daily-digest` in TypeScript), and that a test chat shows model calls, `internet_search` calls and memory reads/writes in the trace. No run happens immediately; the first fire waits for the next weekday 8 a.m. Pacific.

Caveats. Memory is shared across the deployment and visible to all callers. Leave scaffolded `identity` and `sandbox/` alone for this exercise. **Next steps:** add custom middleware (logging, retries, guardrails), identity, and Harbor evals.

CHAPTER 23

Going to Production

> **At a glance:** Production = **thread** + **user** + **assistant** scoping, **durable execution**, **multi-tenant auth**, **sandboxes with an auth proxy**, **guardrails** (permissions, fault tolerance, PII), and a **streaming frontend**.

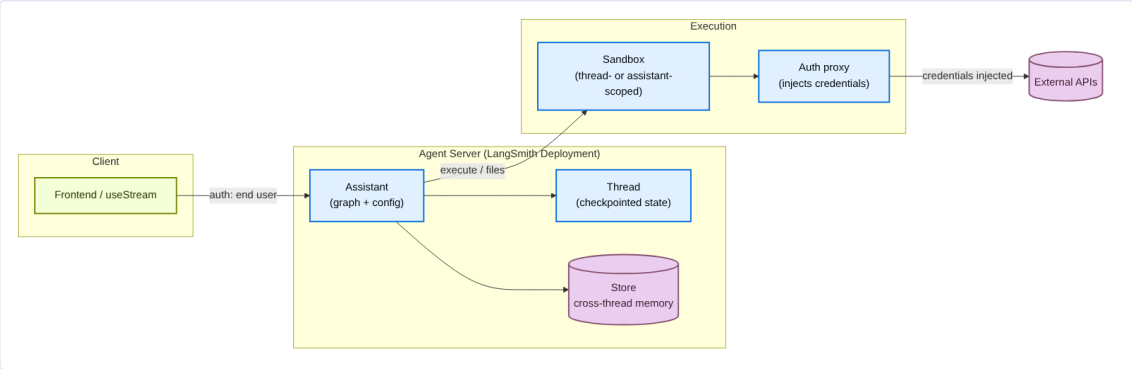


Figure 17. Production architecture for a deployed Deep Agent.

The three primitives: threads, users, assistants

WHAT

Primitive	Meaning	Scope rule
Thread	One conversation	Messages and scratch files are thread-scoped by default
User	Someone using the agent	Memory/files can be private (per user) or shared; identity comes from your auth layer
Assistant	A configured agent instance	Memory/files can be tied to one assistant or shared across all

Why. Almost every production bug is a scoping bug ("whose memory is this?"). Deciding scope up front drives backend namespaces, sandbox lifecycle and auth.

Deployment options

WHAT / WHEN

Option	When
Managed Deep Agents (recommended path; CLI-first)	Hosted harness + runtime; agent as a project of files
LangSmith Deployment (custom code)	Custom app code, custom routes, advanced auth. Provisions threads, runs, a **store** and a **checkpointer**; gives auth, webhooks, cron, observability; can expose the agent via **MCP** or **A2A**
Self-host	`langgraph build` produces a standalone Docker image

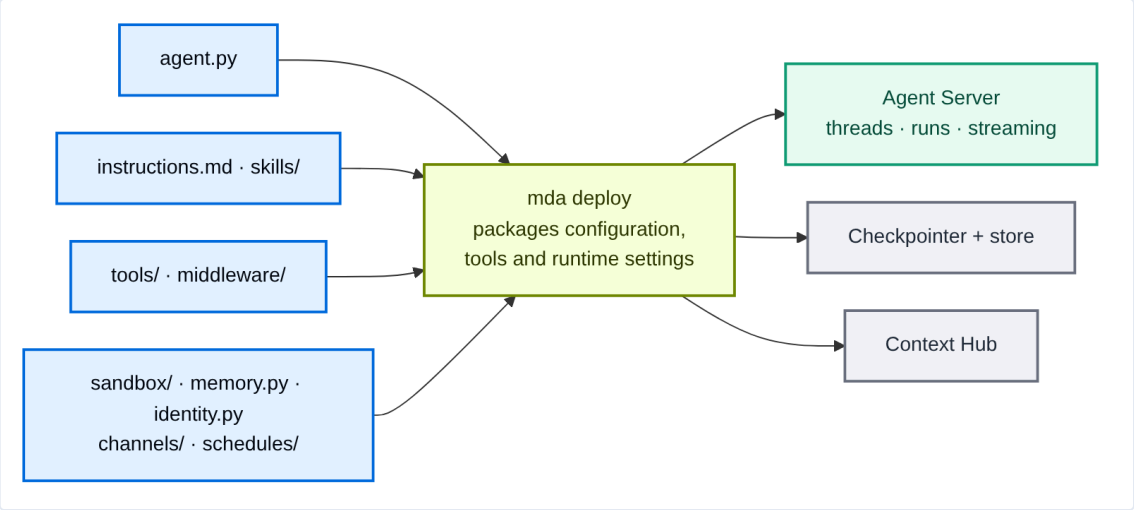


Figure 18. Managed Deep Agents packages configuration, tools and runtime settings for LangSmith.

HOW: LANGGRAPH.JSON

```json
{
  "dependencies": ["."],
  "graphs": { "agent": "./agent.py:agent" },
  "env": ".env"
}
```

`dependencies` = packages (`["."]` = current dir); `graphs` = `"<id>": "./<file>:<variable>"`; `env` = path to a `.env` file.

Invoking the agent: `thread_id` and `context`

What. Every production invocation carries two **independent** things:

- `thread_id` (in `config={"configurable": {...}}`): the conversation; the checkpointer uses it to persist/resume.
- `context`: per-run data (`user_id`, API keys, flags) read by tools/middleware via `runtime.context`.

HOW

```python
from langgraph_sdk import get_client

client = get_client(url="https://my-deployment", api_key="...")
thread = await client.threads.create()                    # new conversation; reuse thread_id for follow-ups

async for chunk in client.runs.stream(
    thread["thread_id"], "agent",
    input={"messages": [{"role": "user", "content": "hi"}]},
    context={"user_id": "u1"},
    stream_mode="updates",
):
    print(chunk)
```

Tip. Generate a fresh `thread_id` (e.g. `uuid7()`) for a new conversation.

Multi-tenancy and the three auth layers

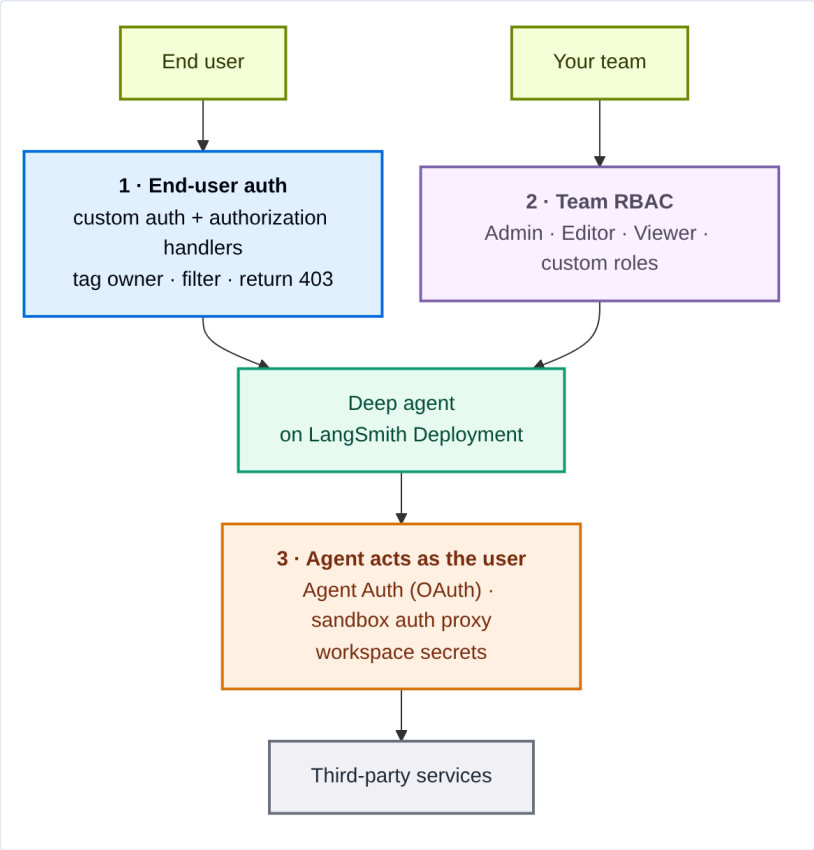


Figure 19. Three authentication layers compose: end-user auth, the agent acting as the user, and team RBAC.

#	Concern	Mechanism
1	**User identity & access control**	LangSmith **custom auth** verifies users; **authorization handlers** run after auth to tag ownership (`owner: user_id`), filter resources so users see only their own, or return HTTP 403
2	**Team RBAC** (separate from end-user auth)	Workspace **Admin** (full) · **Editor** (create/modify; cannot delete runs or manage members) · **Viewer** (read-only) · custom roles on Enterprise
3	**End-user credentials** (agent acts on the user's behalf)	**Agent Auth** (managed OAuth 2.0: first use interrupts with a consent URL, then resumes with a token that is stored and refreshed) · **Sandbox auth proxy** (below) · **Workspace secrets** for org-wide keys

Gotcha. End-user auth and RBAC are different things: RBAC is who on *your team* can touch the deployment; custom auth is who among *your users* can touch which thread.

Async everywhere

What / Why. LLM apps are I/O-bound. Use async tools (native async avoids thread overhead), async middleware hooks (`abefore_agent`), and `async` **graph factories** for resource lifecycle (sandboxes, MCP connections). Convention: `a`-prefixed methods (`ainvoke`, `astream`, `abefore_agent`).

Durability

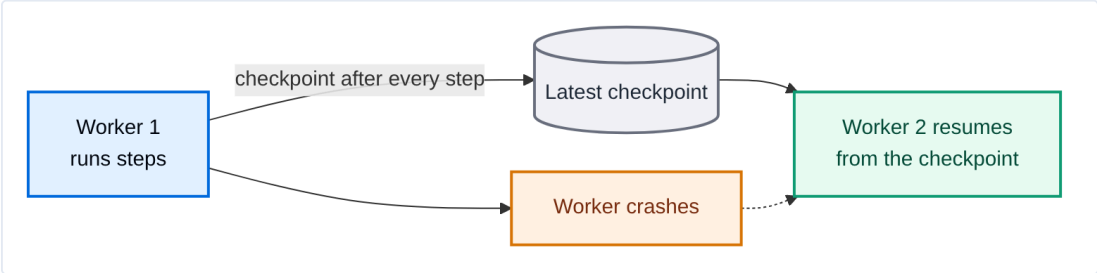


Figure 20. Durable execution: when a worker crashes mid-run, another worker resumes from the latest checkpoint.

What. Deep Agents run on LangGraph, which checkpoints at every step. A run interrupted by failure, timeout or HITL resumes from the last state without reprocessing.

Why. Crash recovery; **indefinite interrupts** (pause for days awaiting approval); **time travel** (rewind/replay); an audit/recovery point before irreversible actions (payments).

How. LangSmith Deployments configure a persistent checkpointer automatically. Self-hosting: see LangGraph persistence.

Memory scoping in production

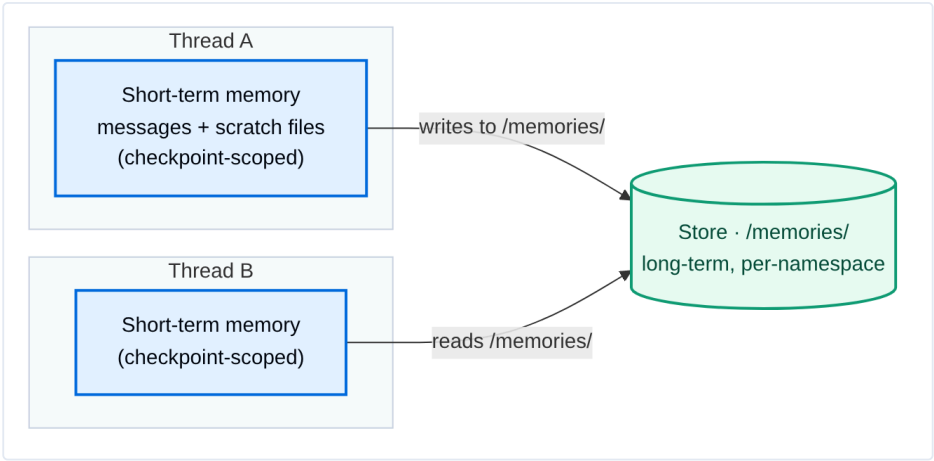


Figure 21. Short-term memory is thread-scoped through checkpoints; long-term memory persists across threads through the store.

What. Memory is always persistent across conversations; the question is **scope**. Recommended default: **user-scoped**.

HOW

PYTHON

```python
from deepagents.backends import CompositeBackend, StateBackend, StoreBackend

backend = CompositeBackend(
    default=StateBackend(),                                    # thread scratch
    routes={"/memories/": StoreBackend(
        namespace=lambda rt: (rt.server_info.assistant_id, rt.server_info.user.identity)
    )},
)
```

Gotcha. Writes to **shared** scopes are a prompt-injection vector: enforce read-only (permissions / policy hooks) and gate sensitive writes with HITL. Full treatment in Part IV (Memory).

Execution environment in production

What / When. Filesystem backends are enough for pure file I/O. Use **sandboxes** when the agent must run code or install packages; isolation also contains crashes/OOMs. ⚠️ `FilesystemBackend` and `LocalShellBackend` touch the host directly: **don't use them in deployed agents**.

SANDBOX LIFECYCLE DECISION

Scope	Sandbox ID stored on	Lifecycle	Example
Thread-scoped (common)	Thread metadata	Fresh per conversation; cleaned up on TTL	Data-analysis bot
Assistant-scoped	Assistant config	Shared across conversations	Coding assistant with a cloned repo

How. An **async graph factory** reads `thread_id` or `assistant_id` from `config["configurable"]`, looks up the sandbox by name (`thread-{id}` / `assistant-{id}`), creates it if missing (with `idle_ttl_seconds`), and returns the agent. *(Illustrative sketch; SDK method names vary by version.)*

PYTHON

```python
from langchain_core.runnables import RunnableConfig
# SandboxClient / LangSmithSandbox come from the `langsmith[sandbox]` package (check import paths for your version)

async def make_graph(config: RunnableConfig):
    thread_id = config["configurable"]["thread_id"]
    name = f"thread-{thread_id}"                                  # assistant-scoped: f"assistant-{assistant_id}"
    client = SandboxClient()
    sandbox = find_sandbox_by_name(client, name)                  # pseudocode: look up an existing sandbox
    if sandbox is None:
        sandbox = client.create_sandbox(name=name, idle_ttl_seconds=3600)
    return create_deep_agent(model="anthropic:claude-sonnet-4-6",
                             backend=LangSmithSandbox(sandbox=sandbox))
```

Gotchas. Graph factories receive a `RunnableConfig`, **not** a full `Runtime`. Assistant-scoped sandboxes accumulate state → set a TTL, use snapshots, or clean up.

File transfer. App code uses `upload_files` (seed user files, skill scripts, configs, memories) before the run and `download_files` (artifacts, updated memories) after. Middleware can automate it: `abefore_agent` uploads from the store; `aafter_agent` downloads changes.

Managing secrets: the sandbox auth proxy

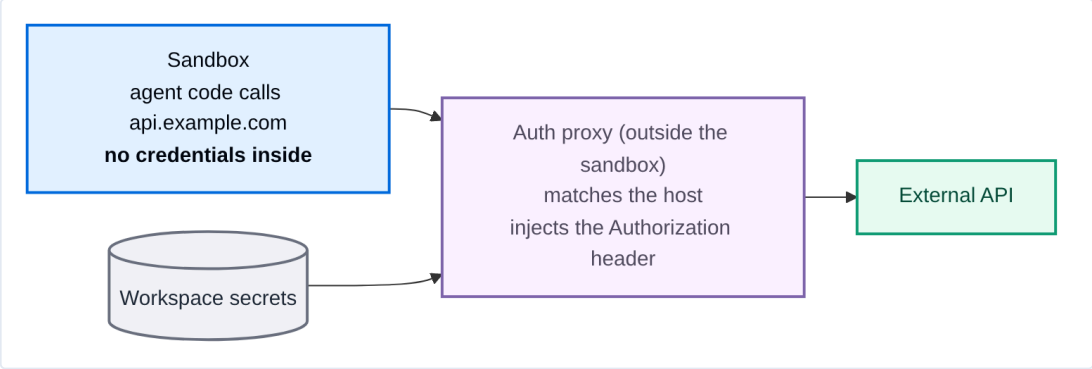


Figure 22. Sandbox auth proxy: credentials are injected into outbound requests so secrets never enter the sandbox.

What. The LangSmith sandbox auth proxy intercepts outbound requests and **injects headers per destination host** (e.g. `api.openai.com` → `Authorization: Bearer ${OPENAI_API_KEY}`). Secrets resolve from workspace settings; keys never appear in sandbox code, env vars or logs.

Why. A sandbox protects the host but not against prompt injection or exfiltration; anything inside it can be read by the agent.

Rules. ✅ Auth proxy or host-side tools. ✅ Workspace secrets for keys the *server* uses. ❌ Never env vars or uploaded files with secrets.

Guardrails

Guardrail	Tool	Where
Path-level read/write rules	`permissions=[FilesystemPermission(...)]`	Part III
Rate limits, retries, fallbacks, error handling	Prebuilt middleware	Part VII
Runaway loops	`ModelCallLimitMiddleware`, `ToolCallLimitMiddleware`	Part VII
PII	`PIIMiddleware`	below

PYTHON

```python
from langchain.agents.middleware import PIIMiddleware

middleware = [
    PIIMiddleware("email", strategy="redact", apply_to_input=True),        # -> [REDACTED_EMAIL]
    PIIMiddleware("credit_card", strategy="mask", apply_to_input=True),    # -> ****-****-****-1234
]
```

Strategies: `redact` (default), `mask`, `hash` (deterministic), `block` (raise). Custom detectors allowed (regex or function).

Connecting a frontend

`useStream` (React, Vue, Svelte, Angular) streams messages, subagent progress and custom state. Point `apiUrl` at the deployment (locally `http://localhost:2024`), set `assistantId`, configure reconnection. For deep-agent workflows raise `recursionLimit` (Deep Agents defaults to **10,000**) and use `streamSubgraphs: true`. See Part VIII.

Advantages, limitations, gotchas (production)

Advantages. Durability, scoping and auth are largely provided by the platform; same code locally and deployed.

Limitations. Platform features (Agent Auth, auth proxy, RBAC custom roles) are LangSmith-specific; self-hosting means you own checkpointer/store/auth.

Gotchas. `thread_id` ≠ `context`; omitted `namespace` on `StoreBackend` shares data across users; don't ship `FilesystemBackend` / `LocalShellBackend`; assistant-scoped sandboxes leak state across users unless carefully scoped.

CHAPTER 24

Fault Tolerance

At a glance: Route each error to whoever can fix it: the **system** (retries, fallbacks, limits), the **model** (error messages it can read), a **human** (interrupts), or the **developer** (uncaught exceptions). Deep Agents expresses all of this as middleware, so it composes with everything else.

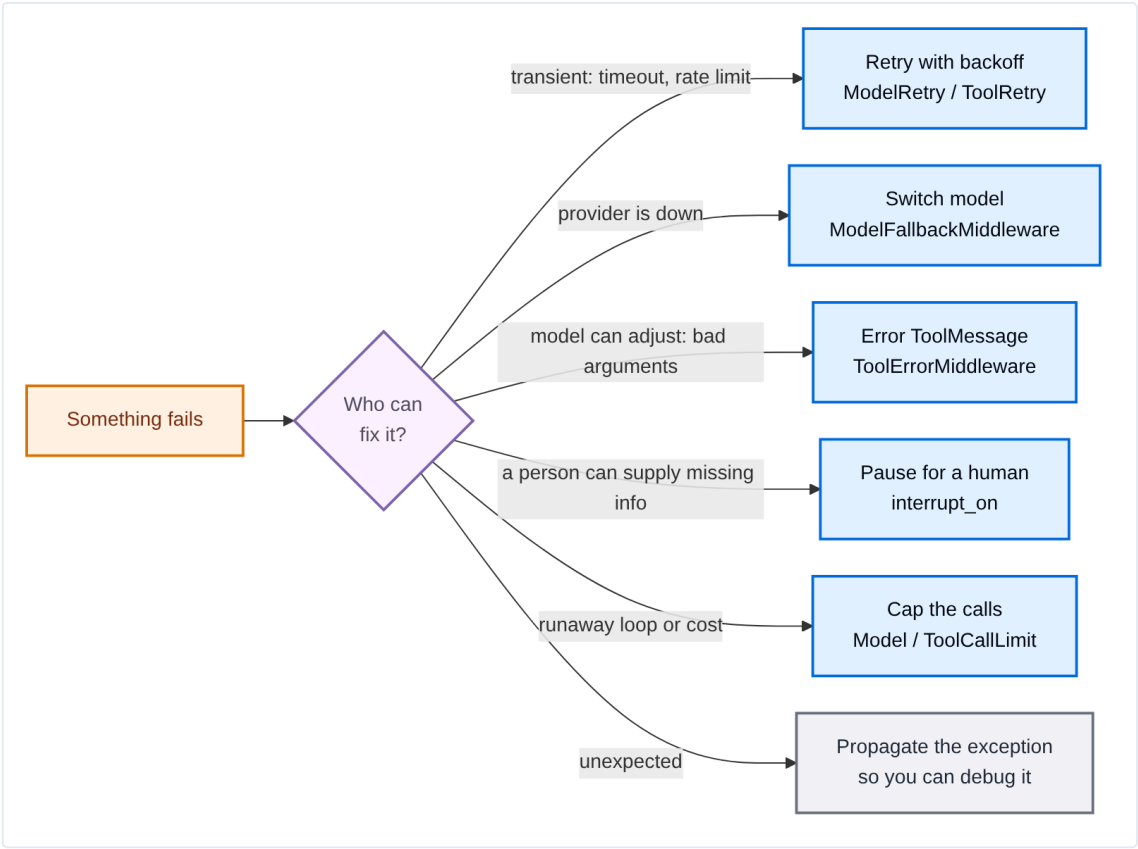


Figure 23. Which handler fixes which kind of error.

Rate limiting

Provider rate limiting controls how fast you call the model provider. Pass an `InMemoryRateLimiter` to `init_chat_model` through `rate_limiter`:

PYTHON

```python
from langchain_core.rate_limiters import InMemoryRateLimiter
from langchain.chat_models import init_chat_model

limiter = InMemoryRateLimiter(
    requests_per_second=0.5,        # target request rate
    check_every_n_seconds=0.1,      # how often to check whether a request is allowed
    max_bucket_size=5,              # maximum burst
)
model = init_chat_model("anthropic:claude-sonnet-4-6", rate_limiter=limiter)
```

Call limits cap total calls and stop runaway loops:

```python
from langchain.agents.middleware import ModelCallLimitMiddleware, ToolCallLimitMiddleware

middleware = [
    ModelCallLimitMiddleware(run_limit=50),     # resets every turn
    ToolCallLimitMiddleware(run_limit=200),
    # thread_limit=... caps calls across a whole conversation (needs a checkpointer)
]
```

`run_limit` resets each turn; `thread_limit` accumulates across a conversation and therefore requires a checkpointer.

Retries (transient failures)

Retries handle timeouts and rate limits with exponential backoff.

```python
from langchain.agents.middleware import ModelRetryMiddleware, ToolRetryMiddleware

middleware = [
    ModelRetryMiddleware(max_retries=3, backoff_factor=2.0, initial_delay=1.0),
    ToolRetryMiddleware(
        max_retries=2,
        tools=["search", "fetch_url"],                    # scope to tools that call external APIs
        retry_on=(TimeoutError, ConnectionError),
    ),
]
```

- Scope `ToolRetryMiddleware` to tools that call **external services**; retrying a failed local file read rarely helps.
- Integration exceptions such as `ModelRateLimitError` and `ModelTimeoutError` carry an `is_retryable` flag that the retry middleware respects by default.

Fallbacks (provider outages)

`ModelFallbackMiddleware("gpt-5.5")` switches to another model when the primary provider is fully down. Retries come first (same model, backoff); the fallback only matters when retries are exhausted. A common stack is retry → fallback → call limits.

Error handling the model can act on

By default a tool exception **halts the run**. `ToolErrorMiddleware(on_error)` (needs `langchain>=1.3.14`) converts selected exceptions into error `ToolMessage`s so the model sees the failure and adjusts.

```python
from langchain.agents.middleware import ToolErrorMiddleware

def on_error(exc: Exception, request) -> str | None:
    if isinstance(exc, ValueError):
        return f"Bad arguments: {exc}. Fix them and call the tool again."
    return None                       # None = let it propagate

middleware = [ToolErrorMiddleware(on_error)]
```

Return a **string** to surface the error to the model, or `None` to propagate the exception. The guidance in the documentation is blunt: *do not catch what you cannot handle.* Unhandled exceptions should propagate unchanged for debugging.

Humans

`create_deep_agent(..., interrupt_on={"send_email": True})` pauses before the named tool so a user can supply missing information or approve the action. See the Human-in-the-loop chapter.

Strategy summary

Error type	Handler
Transient (network, rate limit)	`ModelRetryMiddleware` / `ToolRetryMiddleware`
LLM-recoverable (tool failures, parsing)	`ToolErrorMiddleware`
User-fixable (missing information)	`interrupt_on`
Provider outage	`ModelFallbackMiddleware`
Runaway loops	`ModelCallLimitMiddleware` / `ToolCallLimitMiddleware`
Unexpected	Let the exception propagate

Ordering and edge cases

Edge case: placement. Put `ToolRetryMiddleware` *inside* `ToolErrorMiddleware` with `on_failure="error"` so that, once retries are exhausted, the exception reaches the error handler and becomes a message the model can read. If the order is reversed, an exception can be turned into a message before the retry layer ever sees it, so the retry never fires.

Edge case: retries multiply cost. `max_retries=3` on a model call that fails for a *non-transient* reason (a malformed request) just burns four calls. Retry only retryable errors, and pair retries with a call limit.

Edge case: non-idempotent tools. Retrying a tool that sends an email or charges a card can duplicate the effect. Give such tools idempotency keys, exclude them from `ToolRetryMiddleware(tools=[...])`, or gate them with `interrupt_on`.

Edge case: durable recovery. Retries and fallbacks handle a *call* failing. A *process* crash is handled by checkpointing: with a checkpointer on durable infrastructure, another worker resumes from the latest checkpoint (see Going to Production).

PART III

03

Execution Environment

The execution environment is *where and how the agent acts*: tools, a virtual filesystem on pluggable backends, permission rules, code execution (sandbox shell and in-process interpreter) and streaming.

IN THIS PART

25 Tools: The Built-in Toolbox

26 Backends: Where Files Live

27 Permissions: Controlling File Access

28 Multimodality

29 Sandboxes: Isolated Execution

30 Interpreters (Beta): Code Inside the Agent

31 Event Streaming (Beta)

32 Streaming: Watching the Agent Work

CHAPTER 25

Tools: The Built-in Toolbox

> **At a glance:** Built-in harness tools (files, `task`, optional `execute` / `eval` / `write_todos`) + your own tools + MCP tools. The schema comes from **type hints + docstring**.

Built-in harness tools

What. Tools `create_deep_agent` adds automatically alongside your `tools=`.

Tool	Purpose	Availability
`ls`	List files with metadata (size, modified time)	Always
`read_file`	Read with line numbers, `offset` / `limit` pagination; **multimodal blocks** for image/video/audio/PDF	Always
`write_file`	Create a file or overwrite	Always
`edit_file`	Exact string replacement (`replace_all` for global)	Always
`delete`	Delete a file or a directory recursively	`deepagents>=0.7`; hidden if the backend can't delete
`glob`	Find files by pattern (`**/*.py`)	Always
`grep`	Search contents (files-only, content+context, or counts)	Always
`execute`	Run shell commands	**Sandbox backends only** (and `LocalShellBackend`)
`task`	Spawn a subagent	≥1 sync subagent (general-purpose by default)
`write_todos`	Structured task list	**Opt-in** via `TodoListMiddleware`
`eval`	Run JavaScript in QuickJS	**Opt-in** via `CodeInterpreterMiddleware` (beta)

Gotcha. `write_todos` does not appear by default (opt-in since v0.7).

Custom tools

What. A plain function, a LangChain `@tool`, or a tool dict, passed as `tools=[...]`.

Why. Domain actions (search, DB queries, APIs).

HOW

```python
from typing import Literal

def internet_search(
    query: str,
    max_results: int = 5,
    topic: Literal["general", "news", "finance"] = "general",
) -> str:
```

```
    """Run a web search and return results as text."""    # docstring + hints => schema
    ...

agent = create_deep_agent(model="anthropic:claude-sonnet-4-6", tools=[internet_search])
```

Advantages. No separate schema; works with any provider. **Limitations.** Vague docstrings degrade tool selection; details of `StructuredTool`, return types and error handling live in the LangChain Tools docs.

MCP tools

What. Deep Agents fully supports the Model Context Protocol: databases, APIs, filesystems, browsers through a standard interface.

How. `pip install "langchain[mcp]"`, then:

PYTHON

```python
import asyncio
from langchain.mcp import MCPAdapter

config = {"mcpServers": {"my_server": {"url": "http://localhost:8000/mcp"}}}

async def main():
    async with MCPAdapter(config) as adapter:
        tools = await adapter.list_tools()
        agent = create_deep_agent(model="openai:gpt-5.5", tools=tools)
        return await agent.ainvoke({"messages": [{"role": "user", "content": "..."}]})

asyncio.run(main())            # the docs' example defines main() but doesn't call it
```

Limitations. Async only. Stdio servers, OAuth, filtering and stateful sessions are in Part IX (Protocols), in the chapter on MCP.

Multimodal tool results

What. A custom tool may return a **list of content blocks** (text, image, audio, video, file) instead of a string. **Requires** a model that supports multimodal tool results. See *Multimodality* below.

When to use which kind of tool

- **Built-in file tools** for anything the agent reads or writes (scratch files, memory, skills, offloaded results).
- `execute` only on a sandbox (or `LocalShellBackend`): installs, tests, CLIs.
- **Custom tools** for domain actions (search, DB queries, APIs).
- **MCP tools** for third-party tool servers.
- **Provider tool dicts** (e.g. built-in web search) when you don't want to host the tool yourself.

CHAPTER 26

Backends: Where Files Live

At a glance: A backend implements the file operations. The agent sees one filesystem; *you* choose where bytes live. Default = `StateBackend` (thread-scoped LangGraph state). Use `CompositeBackend` to mix durable and scratch storage.

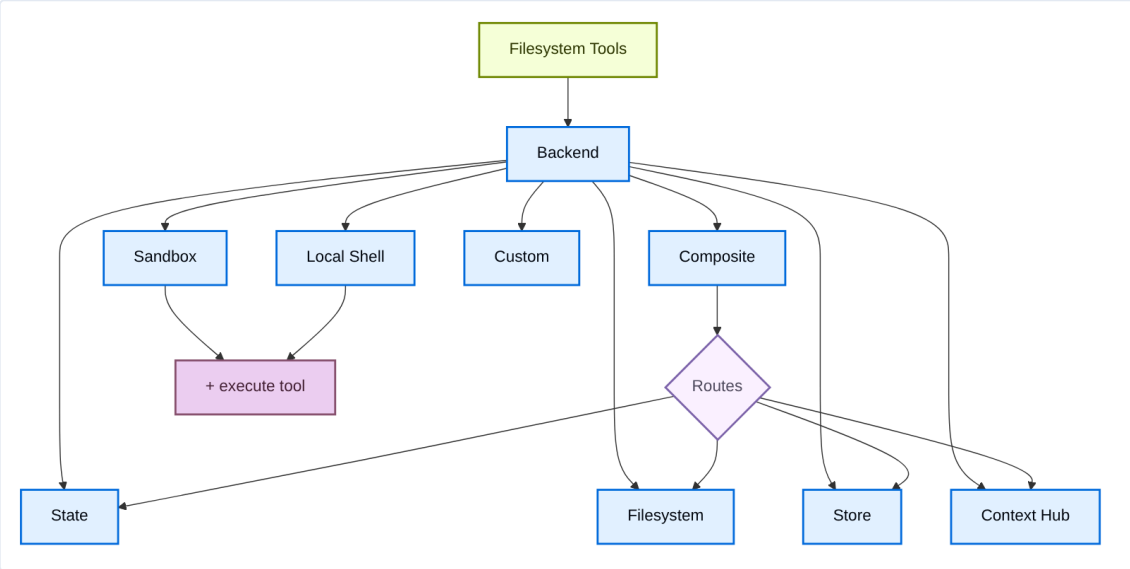


Figure 24. Backends architecture: filesystem tools talk to a backend; Composite routes by path prefix.

What a backend is, and why it exists

What. The pluggable layer behind `ls/read_file/write_file/edit_file/glob/grep/delete`. It also stores internal artifacts: skills, memory, offloaded tool results, summarized history.

Why. One agent, many storage realities: ephemeral scratch, local disk, durable cross-thread store, isolated sandbox, your own S3/DB. **Paths** are virtual and conventionally start with `/` (`/skills/...`, `/memories/...`).

Backend overview

Backend	Storage / scope	`execute`?	Typical use
`StateBackend` (default)	LangGraph **state**, per thread (persisted via checkpointer)	No	Scratch space, offloaded results
`FilesystemBackend`	Real **local disk** under `root_dir`	No	Local dev, CI, mounted volumes
`LocalShellBackend`	Local disk **+ host shell**	Yes	Trusted dev only (**no sandboxing**)
`StoreBackend`	LangGraph **Store**, cross-thread	No	Durable memory, multi-thread data
`ContextHubBackend`	**LangSmith Hub** repo (commit history)	No	Hub-native persistence; no separate store
`CompositeBackend`	**Path-prefix router** over other backends	Depends on routes	Mix scratch + durable

Backend	Storage / scope	`execute`?	Typical use
Sandbox backends	Isolated remote environment	Yes	Production code execution
Custom	Anything (DB, S3, ...)	Optional	Integrations

```python
StateBackend()
FilesystemBackend(root_dir: str, virtual_mode: bool = False)
LocalShellBackend(root_dir, virtual_mode, env, inherit_env)   # timeout=120s, max_output_bytes=100_000
StoreBackend(namespace: Callable[[Runtime], tuple[str, ...]], store=...)
ContextHubBackend("owner/name" | "name")
CompositeBackend(default=<backend>, routes={"/prefix/": <backend>})
```

StateBackend

What. Files live in LangGraph state; per-thread; persisted via the checkpointer. **Shared** by supervisor and subagents: a subagent's files remain in state after it finishes.

When. Default. Scratch work, offloaded results.

Limitations. Not shared across threads; checkpointed every step, so **avoid large files**; calls outside a graph run don't take effect until the graph runs.

FilesystemBackend

What. Reads/writes real disk under `root_dir`. Secure path resolution; tries to block unsafe symlinks; uses ripgrep for `grep` when available.

```python
from deepagents.backends import FilesystemBackend
backend = FilesystemBackend(root_dir="/work/project", virtual_mode=True)
```

Gotchas. `virtual_mode=False` (default) → **no path restriction**, even with `root_dir`. `virtual_mode=True` confines paths under `root_dir` but gives no protection once shell access exists. It also writes internal artifacts (offloaded results, history) to **real disk**, so wrap it in a `CompositeBackend`. **Never in deployed agents.**

LocalShellBackend

What. Local disk **+ host shell** via `subprocess.run(shell=True)`; **no sandboxing**; cwd = `root_dir` but commands can reach any path. Default timeout 120 s, `max_output_bytes=100_000`.

When. Trusted development only.

StoreBackend

What. Cross-thread durable storage on the LangGraph Store. Needs a **namespace factory** (below). Omit `store=` when deploying to LangSmith (the platform provisions one).

Gotcha. Missing `namespace` = all users share data (legacy default keyed by assistant). Always set one in multi-user apps. As of v0.7 you must give `StoreBackend` an explicit namespace.

ContextHubBackend

What. Files live in a **LangSmith Hub** repo. Lazily pulls the tree and serves reads from cache; **writes become Hub commits** with optimistic `parent_commit` concurrency control.

Why. Version history for free; no separate store.

Gotchas. Stale writes fail: re-pull and retry. Non-UTF-8 files rejected. Missing repo treated as empty; first write may create it. Needs `LANGSMITH_API_KEY`. Agent repo mounts at root; linked skill repos appear under `/skills/`.

CompositeBackend (routing)

What. Routes by **path prefix**; the **longest matching prefix wins** (`/memories/projects/` beats `/memories/`). `ls` , `glob` , `grep` aggregate across backends and keep original prefixes. Unmatched paths go to `default` .

HOW: THE CANONICAL "MEMORIES" PATTERN

```python
from deepagents.backends import CompositeBackend, StateBackend, StoreBackend

backend = CompositeBackend(
    default=StateBackend(),                                    # ephemeral thread scratch
    routes={"/memories/": StoreBackend(
        namespace=lambda rt: (rt.server_info.user.identity,)   # per-user isolation
    )},
)
agent = create_deep_agent(model=..., backend=backend, store=store)  # pass store= HERE, not to the composite
```

Tips. Use `StateBackend` as default to keep internal data ephemeral.

Namespace factories (multi-tenant isolation)

What. `NamespaceFactory = Callable[[Runtime], tuple[str, ...]]` . `Runtime` exposes `rt.context` (your context), `rt.server_info` (`assistant_id` , `graph` , `user.identity`), `rt.execution_info` (`thread_id` , `run_id` , `checkpoint_id`).

Scope	Factory
Per user	`lambda rt: (rt.server_info.user.identity,)`
Per assistant	`lambda rt: (rt.server_info.assistant_id,)`
Per thread	`lambda rt: (rt.execution_info.thread_id,)`
User + assistant	`lambda rt: (rt.server_info.assistant_id, rt.server_info.user.identity)`
Org	`lambda rt: (rt.context.org_id,)`

Gotchas. Allowed characters: alphanumerics and `- _ . @ + : ~` ; wildcards `*` / `?` rejected. `rt.server_info` needs `deepagents>=0.5.0` .

The backend protocol (custom backends)

What. Implement `BackendProtocol` . Results carry an `error` field: **return errors, don't raise**.

Method	Signature (abridged)	Rules
`ls`	`ls(path) -> LsResult`	Include `is_dir` , `size` , `modified_at` when available; sort by path
`read`	`read(file_path, offset=0, limit=2000) -> ReadResult`	Missing file → error result
`write`	`write(file_path, content) -> WriteResult`	Create-only; conflict → error
`edit`	`edit(file_path, old_string, new_string, replace_all=False) -> EditResult`	`old_string` must be unique unless `replace_all` ; report `occurrences`
`glob`	`glob(pattern, path=None) -> GlobResult`	Empty list if none
`grep`	`grep(pattern, path=None, glob=None) -> GrepResult`	**Literal-string** search
`delete`	`delete(file_path) -> DeleteResult`	**Optional**; hidden from the model if unsupported

`SandboxBackendProtocol` adds `execute`. Result types: `FileInfo(path, is_dir?, size?, modified_at?)`, `GrepMatch(path, line, text)`, `FileData(content, encoding ∈ {utf-8, base64}, created_at, modified_at)`.

```python
# Sketch: map virtual paths to object-store keys (S3-style). Import paths/types: check your version.
class S3Backend(BackendProtocol):
    def __init__(self, bucket, prefix): ...
    def ls(self, path):        ...   # list keys under prefix + path -> LsResult(entries=[FileInfo(...)])
    def read(self, file_path, offset=0, limit=2000): ...   # get_object -> ReadResult
    def write(self, file_path, content): ...               # put_object if absent else error result
    def edit(self, file_path, old_string, new_string, replace_all=False): ...
    def glob(self, pattern, path=None): ...
    def grep(self, pattern, path=None, glob=None): ...     # literal search
```

Policy hooks, and binary offloading

- **Policy hooks.** Subclass `FilesystemBackend` and override `write` / `edit` to reject protected prefixes, **or** wrap any backend in a delegating class that checks deny-prefixes before forwarding. Declarative `FilesystemPermission` is evaluated **before** the backend is called, so prefer it for simple path rules; use hooks for rate limits, audit logs, custom logic.
- **Binary content offloading** (`deepagents>=0.7.21`, opt-in): `offload_binary_content=True` on `FilesystemMiddleware`. Blobs go to `/blobs/<sha256>` (or `artifacts_root`); message history keeps references; media is restored before each model call. No effect if the blob path routes to `StateBackend`; earlier checkpoints still hold the original payloads.

Advantages and limitations of the backend model

Advantages. One tool surface (`ls`, `read_file`, `write_file`, `edit_file`, `glob`, `grep`, `delete`) regardless of where the bytes live; swap storage without touching the prompt; route by path with `CompositeBackend`; write a custom backend for your own storage.

Limitations. `StateBackend` is thread-scoped and checkpointed (large files add up); `FilesystemBackend` / `LocalShellBackend` aren't safe in deployed agents; durable storage needs an explicit store and namespace; every path you permission-gate must fall under a route when the default backend is a sandbox.

Backend gotchas (interview favourites)

- `FilesystemBackend` with `virtual_mode=False` → no path restriction.
- `virtual_mode=True` offers no protection once shell access is enabled.
- A bare `FilesystemBackend` writes internal artifacts to real disk.
- Missing `StoreBackend` namespace → shared data across users.
- `files_update` and callable `backend=` are deprecated; `BackendContext` → `Runtime` (`deepagents>=0.5.2`); `.runtime` / `.state` accessors removed in `>=0.7`. The docs are inconsistent about `files_update`; check your version.

CHAPTER 27

Permissions: Controlling File Access

> **At a glance:** Declarative **allow / deny / interrupt** rules on the *built-in filesystem tools*. First match wins; **no match = allow**. They do **not** cover `execute`, MCP tools or custom tools.

What / why / when

What. `FilesystemPermission` rules evaluated *before* the backend runs. Needs `deepagents>=0.5.2`.

Why. Isolate the agent to a working directory, protect `.env`/credentials, make memory or skills read-only, require approval for sensitive paths.

When. Any agent with a real filesystem; always with shared memory.

Rule structure

PYTHON

```python
from deepagents import FilesystemPermission

FilesystemPermission(
    operations=["read", "write"],        # "read" = ls, read_file, glob, grep; "write" = write_file, edit_file, delete
    paths=["/workspace/**"],             # glob patterns; ** recursive, {a,b} alternatives
    mode="deny",                         # "allow" (default) | "deny" | "interrupt" (needs checkpointer + deepagents>=0.6.8)
)
agent = create_deep_agent(model=..., backend=..., permissions=[...], checkpointer=...)
```

Evaluation order

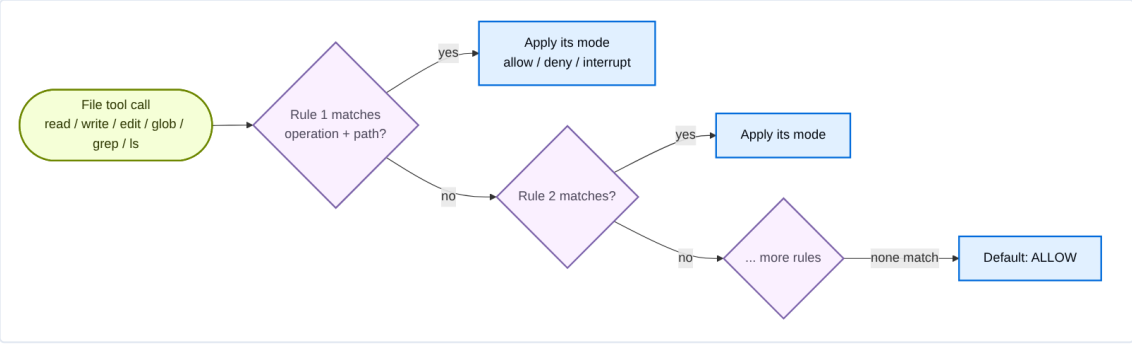


Figure 25. How permission rules are evaluated: first match wins, no match allows.

- Rules are checked **in declaration order**; the **first rule whose operations *and* paths match decides**.
- **No match → allowed.** The default is permissive: add a catch-all deny to be safe.

Recipes

```python
# Read-only agent
[FilesystemPermission(operations=["write"], paths=["/**"], mode="deny")]

# Workspace isolation: allow /workspace, deny everything else
[FilesystemPermission(operations=["read", "write"], paths=["/workspace/**"], mode="allow"),
 FilesystemPermission(operations=["read", "write"], paths=["/**"], mode="deny")]

# Protect secrets: specific denies BEFORE the broad allow
[FilesystemPermission(operations=["read", "write"], paths=["/workspace/.env", "/workspace/examples/**"],
mode="deny"),
 FilesystemPermission(operations=["read", "write"], paths=["/workspace/**"], mode="allow"),
 FilesystemPermission(operations=["read", "write"], paths=["/**"], mode="deny")]

# Read-only memory/policies (with a CompositeBackend)
[FilesystemPermission(operations=["write"], paths=["/memories/**", "/policies/**"], mode="deny")]

# Approval required for writes to /secrets
[FilesystemPermission(operations=["write"], paths=["/secrets/**"], mode="interrupt")]
```

Subagents, interrupts and delete semantics

- **Subagents** inherit the parent's rules. Setting `permissions` in a subagent spec **replaces** (does not merge with) the parent's.
- **Interrupt mode:** a matching `write_file` / `edit_file` / `delete` raises a HITL interrupt instead of running; needs a checkpointer; merges with `interrupt_on`.
- **Delete semantics.** Directories are **all-or-nothing**: `delete` checks `write` on the target and every descendant and denies the whole operation if any is denied (even empty dirs). Plain files use exact first-match-wins (an earlier narrow allow beats a later catch-all deny; `deepagents>=0.7.3`).
- **Composite caveat:** if the default backend is a sandbox, every permission path must fall under a route prefix. Paths outside any route (including `/**`) raise `NotImplementedError`.

Advantages

Declarative and auditable (a list of rules, no code); evaluated **before** the backend is called; three outcomes (allow / deny / interrupt for human approval); inherited by subagents; works with any backend.

Limitations and gotchas

- ⚠️ **Only built-in filesystem tools.** Not custom tools, MCP tools, or sandbox `execute` (which can run arbitrary commands and bypass path rules).
- Order matters: a broad allow above a specific deny makes the deny unreachable.
- Omitting the catch-all deny leaves paths open.
- Unanchored interrupt patterns (e.g. `/**/secrets`) over-fire on bulk tools (`ls`, `glob`, `grep`, directory `delete`); anchor with a literal prefix (`/secrets/**`).

CHAPTER 28

Multimodality

> **At a glance:** Support depends on the **model**. `read_file` returns media as content blocks; tools can return blocks; **summarization drops media** (text-only summary); **offloading keeps it**.

What and how

What. Inputs and outputs beyond text: images, audio, video, files.

WHERE IT APPEARS

1. **User input:** put blocks in the message `content` list, using standard LangChain content blocks.
2. `read_file`**:** returns blocks for supported media. The agent sees them only if the model supports that modality.
3. **Custom tool output:** return a list mixing text and media; it becomes a `ToolMessage` the model reads next turn (read normalized blocks via `content_blocks`). MCP tools also support multimodal content.

PYTHON

```python
# 1. user input with an image
agent.invoke({"messages": [{"role": "user", "content": [
    {"type": "text", "text": "What does this screenshot show?"},
    {"type": "image", "url": "https://example.com/screenshot.png"},
]}]})

# 3. tool returning text + image
@tool
def capture_screenshot() -> list[dict]:
    """Capture a screenshot of the page."""
    return [{"type": "text", "text": "Screenshot attached."},
            {"type": "image", "url": "https://example.com/page.png"}]
```

Supported file types for `read_file`

Kind	Extensions
Image	`.png .jpg .jpeg .gif .webp .heic .heif`
Video	`.mp4 .mpeg .mov .avi .flv .mpg .webm .wmv .3gpp`
Audio	`.wav .mp3 .aiff .aac .ogg .flac`
File	`.pdf .ppt .pptx`

Context-compression gotchas (very testable)

- **Offloading** counts only text tokens; non-text blocks (images) are **kept**, not compressed.
- **Summarization** replaces older turns with a **text-only** summary; images/audio/video/files in that range **no longer reach the model**. The full conversation is still saved as text on the filesystem.

Why / when / advantages / limitations

Why. Agents often need to look at screenshots, PDFs, charts or audio, not only text.

When. Visual QA, document or slide review, transcription, reading data from images, and any tool that returns rendered output.

Advantages. Standard LangChain content blocks for input and tool output; `read_file` handles many media types directly; offloading preserves media.

Limitations. Depends on the model's modalities and the provider's MIME support; summarization drops media; base64 inflates token use quickly.

Best practices

- Save large media to a backend/object store; return a short text description + path/URL.
- Prefer references over base64 in long conversations.
- Delegate image-heavy inspection to subagents (they return compact text).
- Tune summarization thresholds or supply a custom token counter if image tokens are expensive.

Checklist. Model accepts the input modality ✔ · accepts it in tool results ✔ · provider supports your MIME types ✔.

CHAPTER 29

Sandboxes: Isolated Execution

At a glance: A sandbox is an isolated environment (own filesystem + shell). In Deep Agents it's a **backend**; sandbox backends add the `execute` tool on top of the file tools. Use when the agent must write files, install deps, run tests, call CLIs.

What / why / when

What. Isolation from your host, credentials and network. **Why.** Run untrusted or agent-written code safely; contain crashes. **When.** Coding, data analysis, anything needing an OS. For pure file I/O a plain filesystem backend is enough.

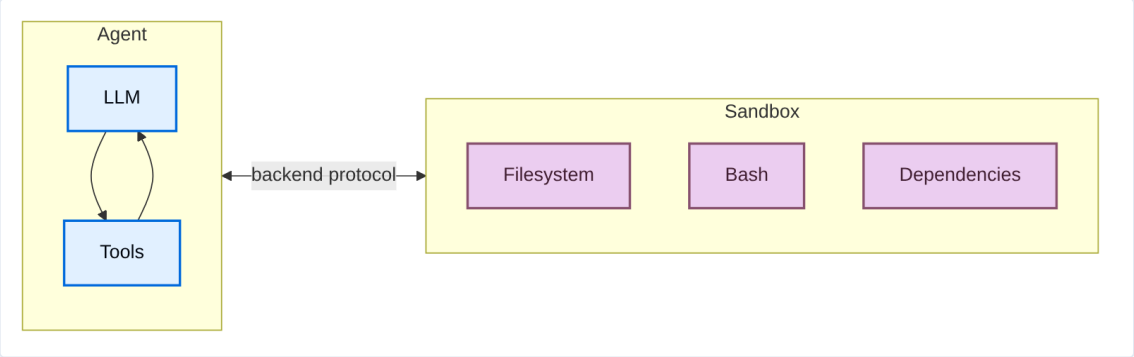


Figure 26. Agent talks to a sandbox through the backend protocol.

Providers

Provider	Package	Class
LangSmith	`langsmith`	`LangSmithSandbox`
Daytona	`langchain-daytona`	`DaytonaSandbox`
E2B	`langchain-e2b`	`E2BSandbox`
Modal	`langchain-modal`	`ModalSandbox`
Runloop	`langchain-runloop`	`RunloopSandbox`
Vercel	`langchain-vercel-sandbox`	`VercelSandbox`
AgentCore	`langchain-agentcore-codeinterpreter`	`AgentCoreSandbox`
NVIDIA OpenShell	`langchain-nvidia-openshell`	`OpenShellSandbox`

How to use one

```python
from langchain_anthropic import ChatAnthropic

client = SandboxClient()                                    # LangSmith example
ls_sandbox = client.create_sandbox()
backend = LangSmithSandbox(sandbox=ls_sandbox)              # DaytonaSandbox(sandbox=...),
```

```
E2BSandbox(sandbox=...), ...

agent = create_deep_agent(
    model=ChatAnthropic(model="claude-sonnet-4-6"),
    backend=backend,
    system_prompt="You are a coding assistant with sandbox access.",
)
try:
    agent.invoke({"messages": [{"role": "user", "content": "Create a hello-world Python file and run
it"}]})
finally:
    client.delete_sandbox(ls_sandbox.name)                # ALWAYS tear down (it costs money until shut
down)
```

Teardown differs by provider: LangSmith `delete_sandbox`, Daytona/Vercel `stop()`, E2B `kill()`, Modal `terminate()`, Runloop `shutdown()`, AgentCore `interpreter.stop()`.

The `execute` tool

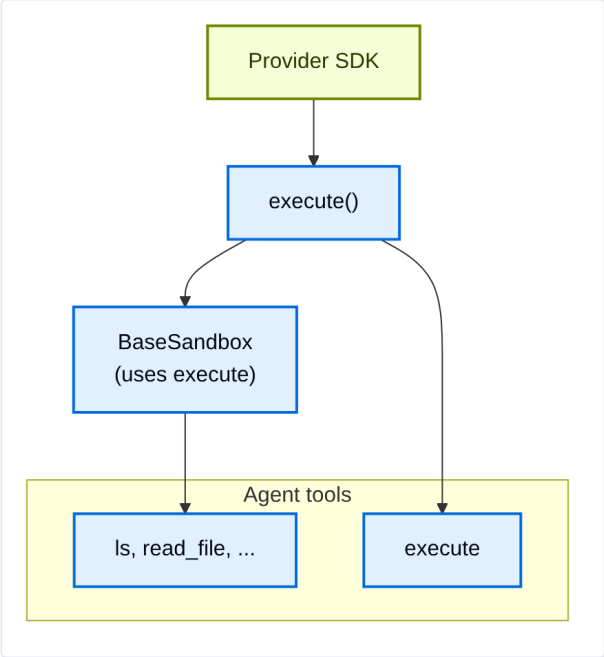


Figure 27. How execute() underpins the agent's tools via BaseSandbox.

- Input: `command` (string). Output: combined stdout/stderr, exit code, truncation notice.
- Provider integrations implement `execute()`; `BaseSandbox` builds the file tools on top of it.
- If the backend doesn't implement the sandbox protocol, `execute` is **filtered out**; the agent never sees it.
- Very large output is written to a file the agent reads incrementally via `read_file` (protects the context).
- App code may call `backend.execute(cmd)` directly (`result.output`).

Two planes of file access

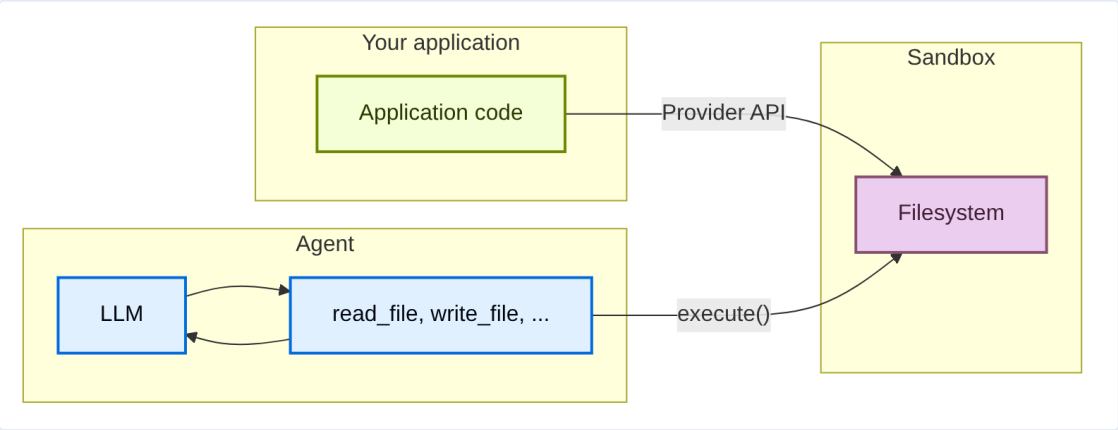


Figure 28. Agent tools use execute(); app code uses the provider's file-transfer API.

```python
backend.upload_files([("/data/sales.csv", csv_bytes)])        # seed before the run (native provider API, not shell)
# ... agent runs ...
results = backend.download_files(["/out/report.png"])          # per-path .path / .content (bytes|None) / .error
```

Why two planes? Agent tools run *through* `execute()` inside the sandbox; your app bypasses the agent via the provider's transfer API, which is safer and works for binary files.

Lifecycle

Scope	Behaviour
Thread-scoped (default)	One sandbox per conversation (`thread-{thread_id}`)
Assistant-scoped	One shared sandbox per assistant; files/packages persist → needs TTL / snapshots / cleanup

Always set an **idle TTL**.

Integration patterns

Pattern	Pros	Cons
Agent runs *in* the sandbox	Mirrors local dev	Keys live in the sandbox; you build the comms layer
Sandbox as a tool (docs' examples)	Keys stay outside; failures don't lose agent state	Per-call latency

Security: the part interviewers ask about

- A sandbox protects the **host**. It does **not** stop **context injection** (someone steering the agent into running commands) or **network exfiltration** (HTTP/DNS).
- **"Never put secrets inside a sandbox"**: no env vars, mounted files, or `secrets` options, even short-lived.
- Prefer **host-side tools** (agent calls by name, never sees the secret) or a provider **auth proxy** (see Part II, sandbox auth proxy).

- If you must inject secrets anyway: HITL on all tool calls, blocked/restricted network, narrow short-lived creds, traffic monitoring.
- Treat all sandbox output as **untrusted input**; redact sensitive patterns with middleware; block network when unneeded.

Advantages

Keeps agent-run code away from the host; gives a real shell (`execute`) for installs, tests and CLIs; `upload_files` / `download_files` move data between your app and the sandbox; two integration patterns (agent inside vs sandbox as a tool); thread- or assistant-scoped lifecycles; several provider integrations (e.g. Modal, Daytona, Runloop, LangSmith sandboxes).

Limitations

Cost until shut down; provider-specific teardown; permissions don't cover `execute`; the docs mention both `SandboxBackendProtocol` and `SandboxBackendProtocolV2`, so check your version.

CHAPTER 30

Interpreters (Beta): Code Inside the Agent

> **At a glance:** `CodeInterpreterMiddleware` (package `langchain-quickjs`) adds an **`eval`** tool that runs JavaScript in an in-memory **QuickJS (WebAssembly)** workspace inside the agent loop. Intermediate values stay out of context. It enables **programmatic tool calling (PTC)** and dynamic subagents.

What / why / when

What. A JavaScript workspace the agent writes to; only relevant results return to the model.

Why. Compose tools, hold state, loop/batch/transform data, without paying tokens for every intermediate value.

When. Orchestration and data shaping. **Not** for shell, package installs, tests or filesystem edits (use a sandbox).

Requires. `langchain-quickjs>=0.3.0`, Python 3.11+. Beta; APIs may change.

	Interpreter	Sandbox
Good for	Composing tools, state, shaping model-visible output	Shell, installs, tests, filesystem edits
Isolation	Wasm engine per thread	OS-level, remote
Network/FS	None by default	Yes

Runtime

QuickJS compiled to WebAssembly (`quickjs-rs`) on **wasmtime**; **each LangGraph thread gets its own Wasm instance** (no cross-thread leakage). Default: no host filesystem, network, shell, package manager or clock.

How to set it up (parameters)

PYTHON

```python
from langchain_quickjs import CodeInterpreterMiddleware
from langgraph.checkpoint.memory import MemorySaver

agent = create_deep_agent(
    model="openai:gpt-5.5",
    middleware=[CodeInterpreterMiddleware(ptc=["web_search"])],
    checkpointer=MemorySaver(),          # optional: durable threads, time travel
)
```

Param	Default	Meaning
`memory_limit`	64 MB	Heap cap per thread
`timeout`	5.0 s	Per `eval` call
`tool_name`	`"eval"`	Tool name
`capture_console`	`True`	Capture `console.log/warn/error`
`max_result_chars`	4000	Truncate returned text
`ptc`	`None`	Allowlist of tools exposed as `tools.*`
`max_ptc_calls`	256	`tools.*` calls per eval

Param	Default	Meaning
`subagents`	`True`	Expose the `task()` global
`mode`	`"thread"`	Persistence: `"thread"` / `"turn"` / `"call"`
`max_snapshot_bytes`	= `memory_limit`	Snapshot cap

Programmatic tool calling (PTC)

What. Allowlisted tools appear as async functions under the `tools` global, in **camelCase** (`web_search` → `tools.webSearch({ query })`). Code can loop, branch, retry and use `Promise.all`; only the final output returns to the model.

```ts
const results = await Promise.all(
  topics.map((t) => tools.webSearch({ query: `${t} best practices` })),
);
```

Gotcha. PTC calls **skip the normal tool-calling path**, so `interrupt_on` approvals aren't enforced per call. Treat the allowlist as a **permission boundary**.

Subagents from code

With subagents configured, `task()` dispatches them from code (default `subagents=True`). See Part V (Dynamic Subagents).

```ts
const reviews = await Promise.all(
  paths.map((path) => task({ description: `Review ${path}`, subagentType: "reviewer" })),
);
```

Persistence modes

`mode`	State lives
`"thread"` (default)	Across turns (snapshot after each turn, restored before the next)
`"turn"`	Across `eval` calls in one turn, then resets
`"call"`	Fresh REPL on every `eval`

Snapshots keep only **serializable** data (functions become inaccessible after restore). Snapshots restore **memory, not external effects**. Cross-turn persistence doesn't need a checkpointer; adding one also stores snapshots in checkpoint history.

Advantages

Intermediate values never enter the context (cheaper, less noise); loops, batching and retries live in code; parallel tool and subagent calls with `Promise.all`; per-thread persistent state; no host access by default.

Limitations, limits and security

Limits: 64 MB heap · 5 s per eval · 4,000 returned chars · 256 PTC calls · snapshot ≤ memory limit.

Does not provide by default: agent tools, filesystem, network, datetime, shell, package installs.

Security. The Wasm sandbox isolates the *engine*, not the *capabilities you grant*. Keep the PTC allowlist narrow, keep wasmtime updated, and for hostile multi-tenant workloads also isolate in separate processes/containers. It's a scoped interpreter, not a full production sandbox.

CHAPTER 31

Event Streaming (Beta)

> **At a glance:** Typed **projections** (`messages` , `tool_calls` , `values` , `output`) and a first-class **`stream.subagents`** : one handle per delegated `task` call. Recommended for new apps (`deepagents` v0.6+, `version="v3"`).

What / why

What. Instead of one mixed chunk stream you branch on, you get separate iterators per projection, on the parent stream *and* on each subagent handle. Python uses snake_case (`tool_calls`).

Why. UIs need "what is each specialist doing right now?" without parsing `stream_mode` chunks or internal graph nodes.

`stream.subagents`

- One handle per `task` call. Fields: `name` (= `subagent_type`), `path` , `status` (`started | completed | failed | interrupted`), `messages` , `tool_calls` , `subagents` (nested), `output` .
- Tool-call fields: `tool_name` , `input` , `output_deltas` , `completed` , `error` , `output` .
- **Discovery is lazy**: message/tool streams open only when accessed.
- Reading `subagent.output` on a failed subagent can raise; wrap it.

PYTHON

```
stream = agent.stream_events(
    {"messages": [{"role": "user", "content": "Research LangGraph and write a summary"}]},
    version="v3",
)
for subagent in stream.subagents:
    print(f"[{subagent.name}] status={subagent.status}")
    for call in subagent.tool_calls:
        print("  tool:", call.tool_name, call.input)
    for nested in subagent.subagents:           # nested subagents: recurse
        print("  nested:", nested.name)
print(stream.output)
```

Concurrency and ordering

- **Async:** `stream = await agent.astream_events(input, version="v3")` + `asyncio.gather` over coordinator and subagent consumers.
- **Sync:** `stream.interleave("messages", "subagents")` yields `(name, item)` pairs.
- **Exact arrival order:** iterate raw events; filter `method == "messages"` , then the payload's `content-block-delta` / `text-delta` . An empty `namespace` = the coordinator.

`stream.subagents` vs `stream.subgraphs`

`subgraphs` reflects **graph structure** (internal nodes). `subagents` reflects **product-level delegation** and hides internals → **prefer `subagents` for user-facing UI**.

How to use it, advantages, limitations

How. Call `agent.stream_events(input, version="v3")` (or `astream_events`), then iterate `stream.subagents`, `stream.messages`, `stream.tool_calls` or `stream.values` as needed (examples above).

Advantages. Typed per-projection iterators; first-class subagent handles (status, tool calls, nested subagents); lazy discovery; no namespace parsing.

Limitations. Beta; needs `deepagents` v0.6+ and `version="v3"`; reading `output` on a failed subagent can raise; a different mental model from classic `stream_mode` code.

CHAPTER 32

Streaming: Watching the Agent Work

> **At a glance:** Classic LangGraph streaming extended with first-class subagent streams. Pass `subgraphs=True` and `version="v2"`, then route events by **namespace** (`ns`). Prefer event streaming for new apps.

What / why / advantages / limitations

What. LangGraph's `stream` / `astream` with subagent events surfaced through namespaces.

Why. Show token-level output, tool progress and per-subagent activity as it happens instead of waiting for the final answer.

Advantages. Works with existing LangGraph code; modes can be combined; namespaces make subagent routing deterministic.

Limitations. You parse namespaces and chunk shapes yourself, and shapes differ by `version` and mode (see Gotchas). Event streaming (above) is the recommended path for new apps.

Enabling subagent events

PYTHON

```python
for chunk in agent.stream(
    {"messages": [{"role": "user", "content": "Research quantum computing"}]},
    stream_mode="updates",
    subgraphs=True,           # without this, NO subagent events
    version="v2",
):
    ...
```

Namespaces

`ns`	Meaning
`()`	Main agent
`("tools:<id>",)`	Subagent spawned by `task` call `<id>`
`("tools:<id>", "model_request:<id>")`	A model call inside that subagent

Route UI by checking for a segment starting with `tools:`.

Stream modes

Mode	Emits	Use for
`updates`	Per-node state updates after each step	Subagent progress, lifecycle, interrupts
`messages`	`(token, metadata)` as LLM output arrives	Token streaming, tool-call chunks, tool results
`custom`	Payloads from `get_stream_writer()`	Custom progress signals
a list	Combined	Full picture

Patterns

- **Subagent lifecycle.** *Pending* = the `task` call appears in the main `model` node; *running* = first event from a `tools:` namespace; *complete* = matching `ToolMessage` (`msg.name == "task"`) in the main `tools` node.
- **Tool-call chunks** arrive as `AIMessageChunk.tool_call_chunks` (name once, args streamed); skip plain content when chunks are present.
- **Custom updates:**

```python
from langgraph.config import get_stream_writer
from langchain.tools import tool

@tool
def analyze(data: str) -> str:
    """Analyze data."""
    writer = get_stream_writer()
    writer({"status": "starting", "progress": 0})
    ...
    writer({"status": "done", "progress": 100})
    return "ok"
# pass `analyze` to a subagent via its "tools" key; events surface under that subagent's namespace
```

- **HITL.** Interrupts appear as `__interrupt__` in `updates` ; resume with `{"decisions": [...]}` as with `.invoke()` , using a stable `thread_id` .
- **Summarization tokens** show up in streams; filter with `metadata.get("lc_source") == "summarization"` .

Gotchas

The docs mix dict-style chunks (`chunk["type"]`) with tuple unpacking. With `subgraphs=True` : single mode → `(namespace, data)` , multiple modes → `(namespace, mode, data)` . Without it: `data` or `(mode, data)` . LangGraph 1.1+ also offers a unified `version="v2"` `StreamPart` dict. **Confirm the shape your installed version returns.**

PART IV

04

Context Management

Context management controls **what the agent knows**, **how it fits in the window**, and **what persists**.

Layer	Mechanism	Loaded when
Skills	Folders with `SKILL.md` (procedures)	Progressive disclosure: metadata at startup, body on demand
Memory	`AGENTS.md`-style files	Always, at startup
Retrieval	Search tools / files / subagents	When the agent needs external knowledge
Offloading + summarization	Compresses big tool I/O and history	Automatically as the window fills
Prompt caching	Caches the static prompt prefix	Each turn (Anthropic / Bedrock)

> **Mental model:** skills = *how to do X (on demand)* · memory = *what to always remember* · summarization = *how to forget gracefully* · caching = *how to pay less for the static prefix*.

IN THIS PART

33 Skills: Procedures on Demand

34 Memory: What the Agent Always Knows

35 Retrieval: Bringing in Outside Knowledge

36 Context Engineering: Offloading, Summarization and Isolation

37 Profiles (Beta): Defaults per Model and Provider

CHAPTER 33

Skills: Procedures on Demand

> **At a glance:** A skill is a folder with a `SKILL.md` (YAML frontmatter + instructions) plus optional scripts/references/assets, following the open **Agent Skills** standard (agentskills.io). Only the name + description are loaded at startup; the body loads when the agent decides it's relevant (**progressive disclosure**).

What / why / when

What. Packaged specialised workflows, domain knowledge and instructions.

Why. Keep startup context small while still giving the agent deep playbooks; share and version procedures as files.

When. Procedures and playbooks (how to write a blog post, how to run a code review). *Facts and preferences belong in memory.*

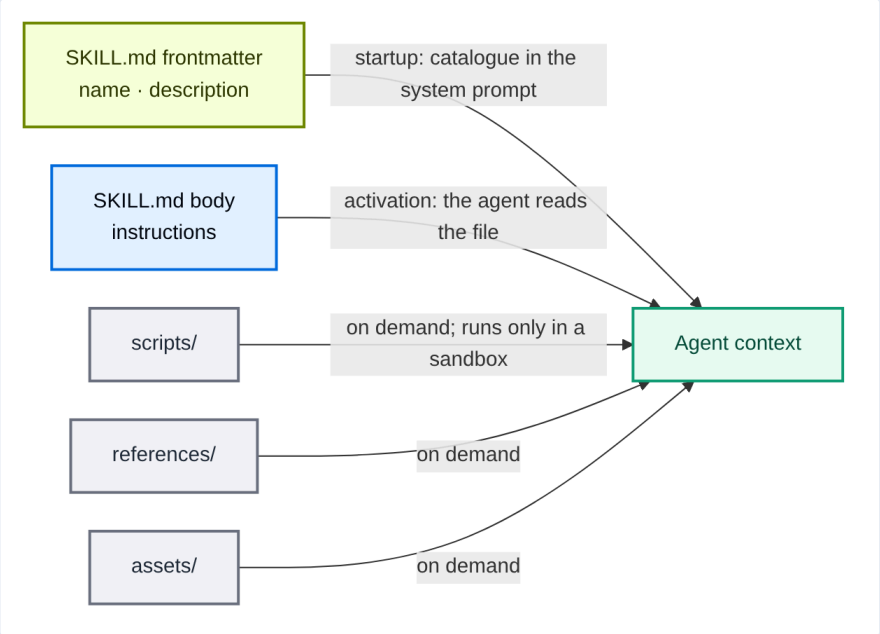


Figure 29. Skills composition: how skill components map into the agent's context at startup and on activation.

Directory layout and SKILL.md

```
skills/
  <skill-name>/           # directory name MUST match frontmatter `name`
    SKILL.md              # REQUIRED: frontmatter + instructions
    scripts/              # optional: executable code
    references/           # optional: on-demand docs
    assets/               # optional: templates, schemas, images
```

Reference supporting files from `SKILL.md` with **relative paths, one level deep**.

MARKDOWN

```
---
name: blog-post
description: Write a long-form blog post with research, a hero image and a quality checklist. Use when the
user asks for a blog article.
```

```
metadata:
  include_tools: "web_search generate_cover"   # space-separated string, NOT a YAML list
---

# Blog post workflow
1. Delegate research to the `researcher` subagent.
2. Read the findings, then write `blogs/<slug>/post.md`.
3. Generate a cover image into the same folder.
4. Run the quality checklist in `references/checklist.md`.
```

Frontmatter	Required	Rules
`name`	✅	lowercase alphanumeric + hyphens, **1–64 chars**, must match the directory
`description`	✅	**≤ 1,024 chars**; what it does *and when to use it*
`license`	❌	name or bundled file ref
`compatibility`	❌	environment needs, ≤ 500 chars
`metadata`	❌	key-values; Deep Agents reads `include_tools` here
`allowed-tools`	❌	experimental; unrelated to skill tools

Size guidance: body **< ~5,000 tokens and < 500 lines**; files **> 10 MB are skipped**.

Progressive disclosure (the core idea)

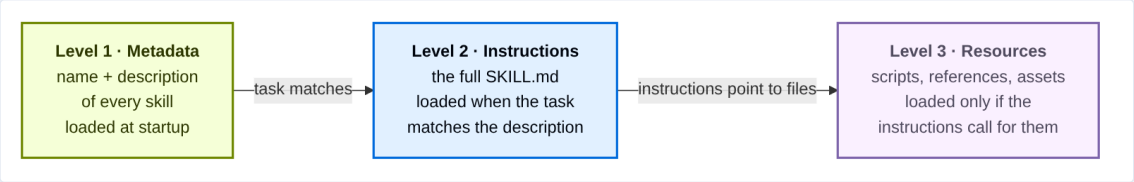


Figure 30. Progressive disclosure: metadata first, then instructions, then resources.

1. **Metadata (startup):** `name` + `description` of every skill goes into the system prompt.
2. **Instructions (on invocation):** the agent reads the full `SKILL.md` via `read_file`.
3. **Resources (as needed):** scripts/references/assets load only when the instructions call for them.

`SkillsMiddleware` handles levels 1–2; the LLM handles level 3.

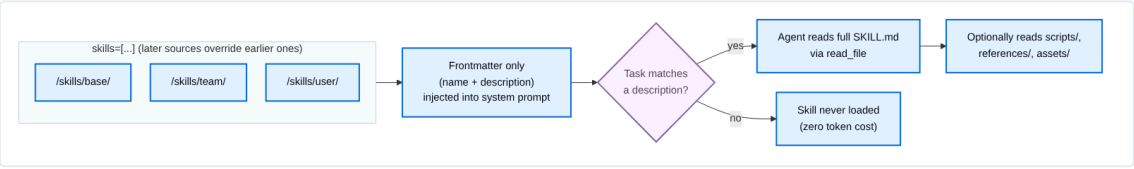


Figure 31. Skill sources layered, then metadata, match, load.

Sources and backends

```python
agent = create_deep_agent(model=..., backend=..., skills=["/skills/"])
```

- `skills` is a `list[str]`, forward slashes, **relative to the backend root**. Omitted = no skills.
- Each entry must point to a directory that **contains** skill directories, **not** a skill directory itself (passing the skill folder directly silently loads nothing).

Backend	How skills get in
`StateBackend` (default)	`invoke(..., files={"/skills/x/SKILL.md": create_file_data(text)})` (raw strings rejected)
`StoreBackend`	`backend.upload_files([(path, bytes)])`, then pass the backend
`FilesystemBackend`	Same upload; `virtual_mode=True` confines to `root_dir`; existing on-disk skills load without upload

Precedence. Multiple sources are applied in order; for duplicate names the **later source wins**. Use it to layer project overrides on base skills (and beware silent shadowing).

```python
result = agent.invoke(
    {"messages": [{"role": "user", "content": "Write a post about LangGraph"}]},
    files={"/skills/blog-post/SKILL.md": create_file_data(skill_md_text)},   # StateBackend seeding
)
```

Skill tools (`deepagents>=0.7.22`)

What. Tools bundled with a skill, declared in frontmatter via `include_tools` and supplied through middleware.

```python
from deepagents.middleware.skills import SkillsMiddleware   # import path: confirm for your version

SkillsMiddleware(backend=backend, sources=["/skills/"], tools=[list_issues, create_issue])
# or tools=resolver, where resolver: (name: str, runtime: Runtime) -> list[BaseTool]
```

Gotchas. Skill tools are **gated**: calls fail until the agent has read `SKILL.md`; summarization that drops that read re-gates them. If the agent already has a same-named tool, the agent's version wins. Deferred alternative: `@tool(extras={"defer_loading": True})` + `ProviderToolSearchMiddleware()` (not gated). Unmatched `include_tools` names only log at DEBUG.

Reloading and pinning

- Skills load **once per thread**, cached in state under `skills_metadata`. Reset with `{"skills_metadata": None}` in run input or `agent.update_state(...)` (`deepagents>=0.7.16`). Use `None`, **not** `[]`: `[]` means "no skills" for the rest of the thread. Reset from `after_agent`, not `after_model`. Without a checkpointer, skills reload every run.
- **Pinning** (`deepagents>=0.7.23`): pass `"pinned_skills": ["name"]` in the run input. Each skill's `SKILL.md` (without frontmatter) is injected as a `HumanMessage` inside `<skill name=... path=...>`; order follows your list; `additional_kwargs["lc_source"] == "pinned_skill"`. Pinned text is **frozen** (re-pin to pick up edits). Your app must parse names from user input; Deep Agents doesn't.

Subagents, permissions, and executing scripts

- The **general-purpose** subagent inherits `skills=`. **Custom subagents need their own** `"skills": [...]`; skill state is isolated between agent and subagents.
- Make a skill tree read-only: `FilesystemPermission(operations=["write"], paths=["/skills/**"], mode="deny")` (more specific rules first). `mode="interrupt"` needs a checkpointer (`>=0.6.8`).
- Scripts can be read from any backend but **run only in sandbox backends**; sandboxes don't see skill files unless you sync them (middleware: `abefore_agent` upload, `after_agent` download).

Advantages, limitations, gotchas

Advantages. Cheap at startup; modular and shareable; works with any backend.

Limitations / gotchas. Wrong path level → silent no-load · `name` must match directory · overlapping descriptions → wrong skill chosen · too many similar skills degrades selection · with a checkpointer, edits don't appear in an existing thread until reset · a later same-name source shadows an earlier one.

CHAPTER 34

Memory: What the Agent Always Knows

> **At a glance:** Persistent context via `AGENTS.md` files passed with `memory=[...]`. Unlike skills, memory files are **always loaded** and can be **updated by the agent**, so preferences persist across threads when routed to a store.

What / why / when

What. `AGENTS.md`-convention files stored in the configured backend (`StateBackend`, `StoreBackend`, `FilesystemBackend`).

Why. Carry preferences, facts and learned instructions across conversations.

When. Small, always-relevant context. Detailed workflows belong in skills.

PYTHON

```python
from deepagents.backends import CompositeBackend, StateBackend, StoreBackend

agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
    memory=["/memories/AGENTS.md"],
    skills=["/skills/"],
    backend=CompositeBackend(
        default=StateBackend(),
        routes={"/memories/": StoreBackend(namespace=lambda rt: (rt.server_info.user.identity,))},
    ),
    store=store,
)
```

Memory vs skills

	Memory	Skills
Purpose	Facts, preferences, context	Procedures: *how* to do a task
Loading	**Always** (startup)	**On demand**
Writability	Often writable by the agent	Usually read-only (developer-defined)
Size	Keep minimal (it's in every run)	Detailed workflows live here

Short-term vs long-term memory

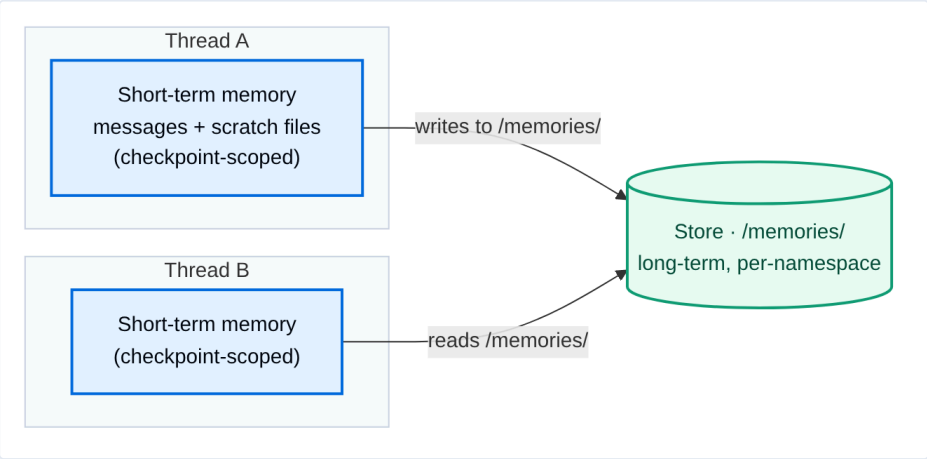


Figure 32. Short-term memory is thread-scoped through checkpoints; long-term memory persists across threads through the store.

Scoping: who sees what

Scope	Namespace	Use case	Example
Thread (short-term)	checkpoint-scoped	One conversation	In-progress context
User (recommended default)	`(user_id,)`	Per-user preferences	"I prefer concise responses"
Assistant	`(assistant_id,)`	Shared instructions for one agent	"Cap posts at 280 characters"
Global / Org	`(org_id,)`	Read-only policy for all	"Never disclose internal pricing"

⚠️ **Prompt-injection risk.** Shared writable memory (assistant/user/org) lets one user inject instructions into another's conversation. Make shared scopes read-only where possible (write org policy via app code, not the agent), deny writes with `permissions`, add backend policy hooks.

Usage patterns

- **Self-improving instructions.** Assistant-scoped memory lets the agent learn preferences across users, updating files with `edit_file`. Writes may be *hot-path* (during the conversation, adds latency) or *background* (consolidation between conversations).
- **Knowledge bases.** Shared read-only memory seeded from app code:

```python
from deepagents.backends.utils import create_file_data      # import path: confirm for your version
store.put(("org-123",), "/AGENTS.md", create_file_data("# Policies\n- Never disclose internal pricing."))
```

- **Episodic memory.** Past conversations are checkpointed threads; expose search over them as a tool so the agent recalls how it solved things before:

```python
threads = await client.threads.search(metadata={"user_id": "u1"}, limit=5)
history = await client.threads.get_history(thread_id=threads[0]["thread_id"])
```

- **Background consolidation ("sleep-time compute").** A scheduled consolidation agent reviews recent threads, extracts facts and merges them into memory.

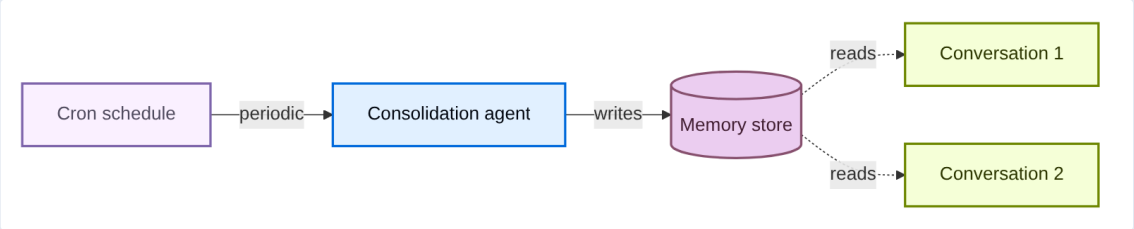


Figure 33. Consolidation: a cron-triggered agent reads conversations and writes the memory store.

```python
await client.crons.create(
    assistant_id="consolidator",
    schedule="0 */6 * * *",                       # UTC! every 6 hours
    input={"messages": [{"role": "user", "content": "Consolidate the last 6 hours of conversations"}]},
)
```

Advantages, limitations, gotchas

Advantages. Simple file model; user-scoped by default is safe; agent can self-improve.

LIMITATIONS / GOTCHAS

- **Cron interval must equal the lookback window** (6-hour job → 6-hour search) or you reprocess/drop memories. Cron is **UTC**. Don't consolidate far more often than users chat.
- Concurrent writes to one file → last-write-wins; serialize consolidation or use per-topic files.
- `InMemoryStore` is dev-only. Trace memory writes in LangSmith (they appear as tool calls).
- Memory is in **every** run: bloated memory hurts everything.

CHAPTER 35

Retrieval: Bringing in Outside Knowledge

At a glance: RAG with Deep Agents = retrieval as **tools**, results **offloaded to files**, analysed by **subagents**, guided by **skills**, optionally graded by a **rubric**. Pick a RAG shape: 2-step, agentic, or hybrid.

What / why / when

What. Giving the agent knowledge it wasn't trained on (your docs, tickets, wiki). **Why.** Without retrieval, answers come from training data (generic or outdated). **When.** Any domain-specific Q&A or research.

The indexing → retrieval pipeline

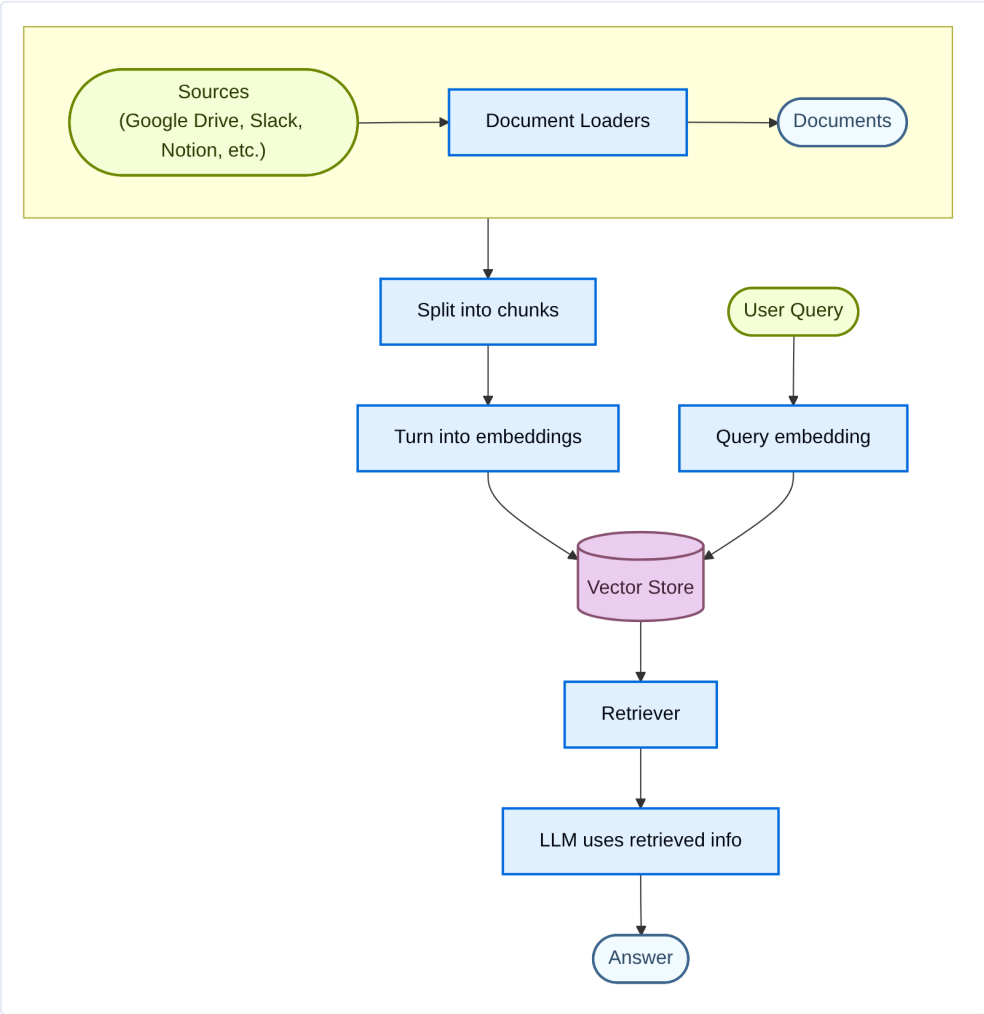


Figure 34. Indexing and retrieval pipeline: sources → loaders → chunks → embeddings → vector store → retriever → LLM.

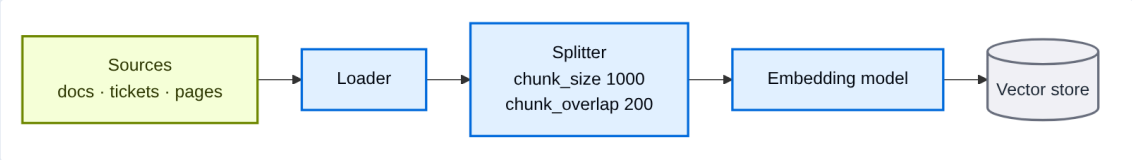


Figure 35. RAG indexing: sources are loaded, split into chunks, embedded and stored in a vector store.

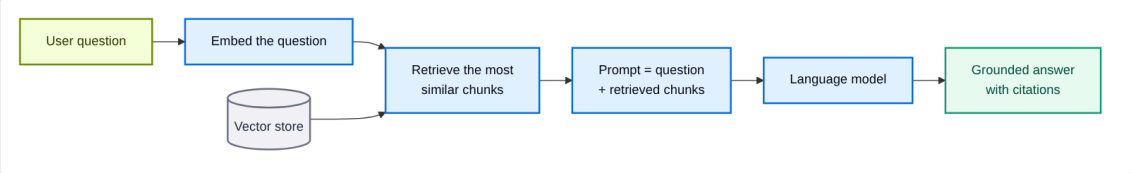


Figure 36. RAG retrieval and generation: embed the question, retrieve similar chunks, prompt the model.

Three RAG architectures

Pattern	Core idea	Trade-off	Good for
2-step RAG	Retrieval *always* runs before generation	Predictable, fast; less flexible	FAQs
Agentic RAG	The LLM decides *when and how* to retrieve	Flexible; variable latency	Multi-tool research
Hybrid	Query enhancement + validation loops	Balanced control; more iteration	Quality-sensitive Q&A

2-step RAG

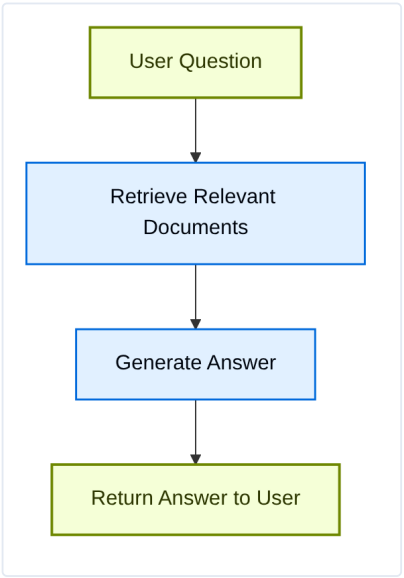


Figure 37. 2-step RAG: retrieve then generate.

Agentic RAG

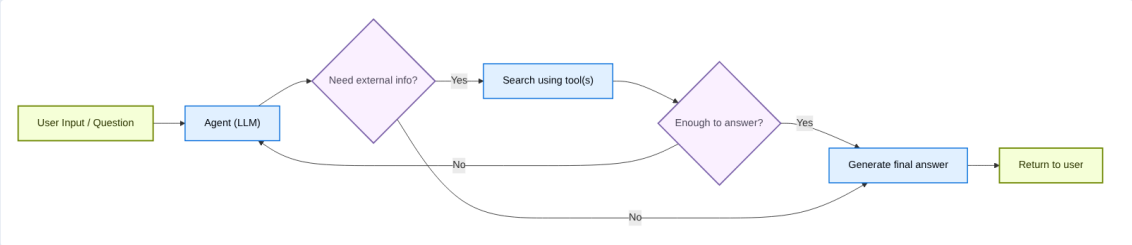


Figure 38. Agentic RAG: agent decides whether to search, loops until enough.

Hybrid RAG

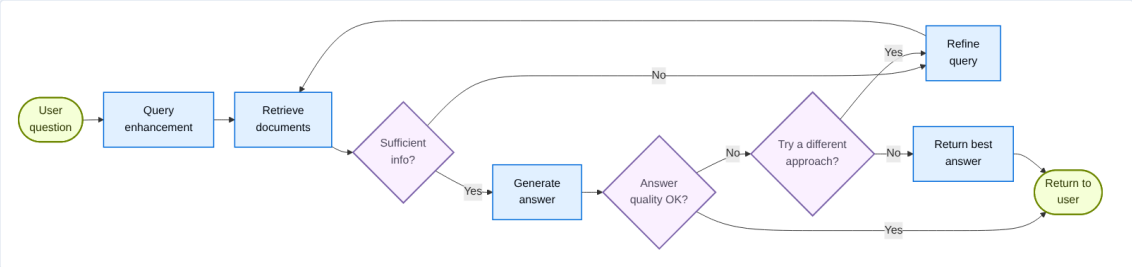


Figure 39. Hybrid RAG: query enhancement, retrieval, sufficiency check, answer-quality check, refine loop.

How Deep Agents integrates retrieval

- **Tools:** expose retrieval as a tool (`fetch_url` , `fetch_documentation` , a vector search).
- **Filesystem:** write retrieved chunks to files and pass **paths** around instead of text.
- **Subagents:** one subagent per chunk-file returns a short summary (parallel).
- **Skills:** a skill states which index, query phrasing and citation format to use.
- **Rubric grading:** a grader checks groundedness; revise until it passes.

```python
from langchain.tools import tool

@tool
def fetch_documentation(url: str) -> str:
    """Fetch a docs page and convert it to markdown."""
    ...

agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
    tools=[fetch_documentation],
    system_prompt="Answer using the documentation. Cite URLs. If a fetch fails, say so.",
    subagents=[{"name": "chunk-analyst", "description": "Reads ONE retrieved file and summarizes it",
                "system_prompt": "Read the file at the path you're given and return a 3-bullet summary."}],
)
```

Recommendations

Reuse existing data stores (SQL, CRM, docs) by wrapping them as tools · keep components modular (loaders, splitters, embeddings, vector stores swappable) · **restrict sources** (allowlist domains for URL fetching) · handle failures gracefully (report failed fetches, fall back to model knowledge) · persist and refresh the vector store in production.

Advantages

Grounded, citable answers from your own data; context stays small because chunks live in files and subagents summarise them; works with any retriever or vector store you wrap as a tool; skills can encode the retrieval strategy; a rubric can enforce groundedness.

Limitations / gotchas

Indirect prompt injection via retrieved documents (prompts reduce but don't eliminate it; validate answers, check citations). The *same backend instance* must be passed to the search tool and to `create_deep_agent`, so built-in tools like `read_file` can see saved chunks (see Build-Along 4: Retrieval-Augmented Generation).

CHAPTER 36

Context Engineering: Offloading, Summarization and Isolation

> **At a glance:** Four moving parts: **input context** (what you start with), **compression** (offloading + summarization), **isolation** (subagents), **long-term memory**. Offload at ~20k tokens; summarize at 85% of the window.

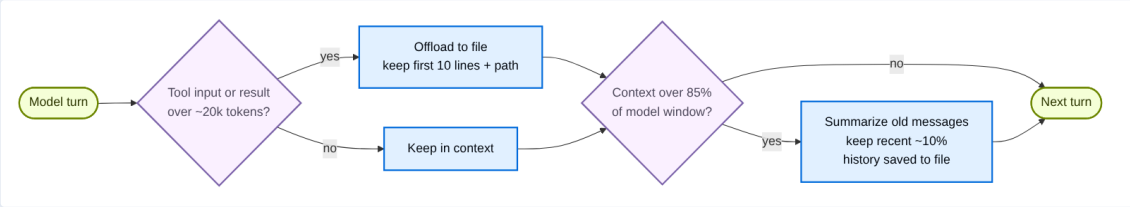


Figure 40. Context lifecycle: offload large I/O, summarize old history, carry on.

What / why / when / how

What. Everything that decides what the model sees on each call, and how the harness keeps that window small over a long run.

Why. Long runs fill the window with stale tool output; quality and cost both suffer.

When. Any agent that makes many tool calls, reads large files or web pages, or lives across many turns.

How. Four levers, each a section below: (1) shape the input context (prompt, memory, skills); (2) compress (offload above ~20k tokens, summarize at 85% of the window); (3) isolate (subagents return one report); (4) persist (long-term memory under `/memories/`).

Input context (static, at startup)

- **System prompt order:** custom prompt → base prompt → memory → skills list → filesystem prompt (+ `execute` docs) → `task` prompt → middleware prompts → HITL prompt.
- **Memory** is always injected (no progressive disclosure) → keep it small.
- **Skills:** frontmatter only at startup.
- **Tool prompts:** built-ins add their own instructions; custom tools rely on name, description and arg docs.
- **Dynamic prompts:** `@dynamic_prompt` middleware for per-run variation (`system_prompt=` is static).

Runtime context (per run)

What. Immutable per-run inputs (IDs, roles, keys, flags). **Not** added to the prompt automatically; a tool or middleware must surface it. **Propagates to all subagents.**

```python
from dataclasses import dataclass
from langchain.tools import tool, ToolRuntime

@dataclass
class Context:
    user_id: str
```

```python
@tool
def whoami(runtime: ToolRuntime[Context]) -> str:
    """Return the current user id."""
    return runtime.context.user_id

agent = create_deep_agent(model=..., tools=[whoami], context_schema=Context)
agent.invoke({"messages": [...]}, context=Context(user_id="u1"))
```

Custom state schema

PYTHON

```python
class MyState(DeepAgentState):          # must subclass DeepAgentState (keeps the messages reducer)
    extra_field: str

agent = create_deep_agent(model=..., state_schema=MyState)        # needs deepagents>=0.6.6
agent.invoke({"messages": [...], "extra_field": "hello"})
# inside a tool: runtime.state["extra_field"]
```

State = mutable, checkpointed. **Runtime context** = immutable per run. Compiled and remote subagents don't inherit the parent `state_schema`.

Compression 1: offloading (built in)

What. Big tool inputs/results are moved to the filesystem.

- **Tool inputs > 20,000 tokens** (file write/edit content): replaced by a file pointer once the context exceeds **85%** of the window.
- **Tool results > 20,000 tokens:** full result saved to the backend; the agent sees a **path + first 10 lines** and can re-read or `grep`.

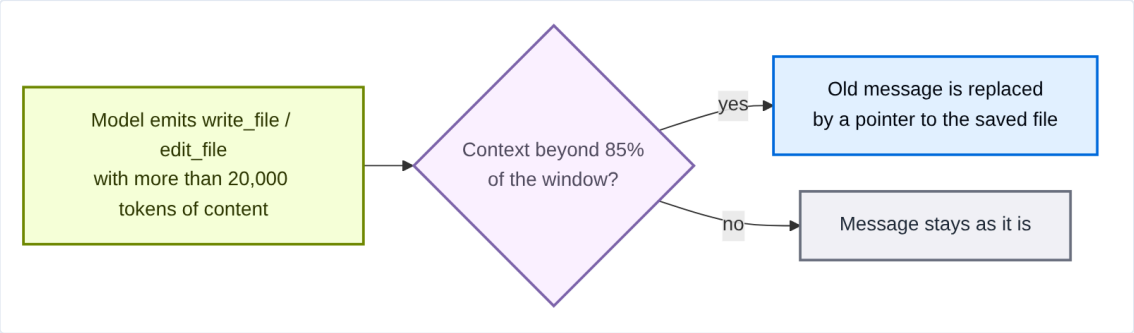


Figure 41. Offloading tool inputs.

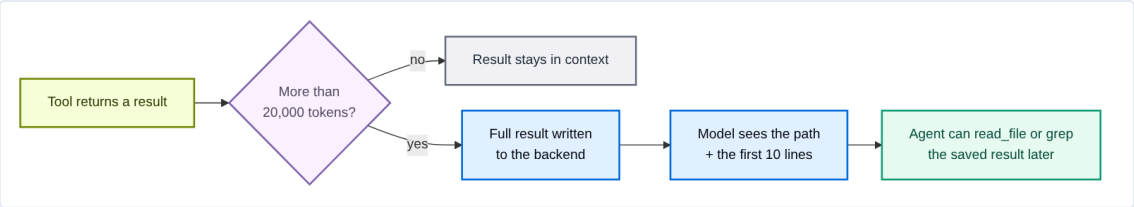


Figure 42. Offloading tool results.

Compression 2: summarization

What. `SummarizationMiddleware` compacts history when the window fills.

Setting	Behaviour
Trigger	**85% of** `max_input_tokens` (from the model profile), after offloading has nothing left to remove
Keep	most recent **10%** of tokens verbatim
Fallback (no model profile)	**170,000-token** trigger, keep **6 messages**
Overflow	`ContextOverflowError` → forced summarization + retry
Output	Structured summary (intent, artifacts, next steps) replaces history in context; **full text written to the filesystem**

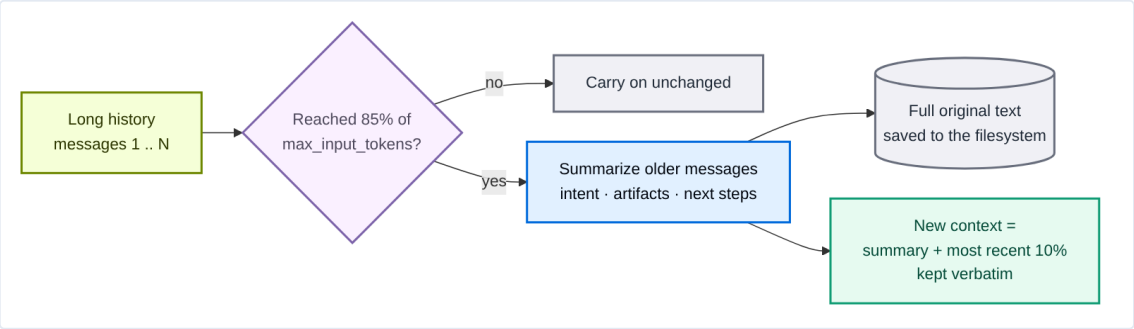


Figure 43. Summarization: older conversation steps compacted into a summary.

```python
from langchain.agents.middleware import SummarizationMiddleware

agent = create_deep_agent(
    model=...,
    middleware=[SummarizationMiddleware(
        model="anthropic:claude-haiku-4-5", backend=backend,
        trigger=("tokens", 100_000), keep=("messages", 20),
    )],
)
```

On-demand compaction. `create_summarization_tool_middleware(model, backend)` adds a `compact_conversation` tool so the agent can compact *between tasks*; it doesn't disable automatic summarization.

Gotchas. Summarization tokens appear in streams (filter `lc_source == "summarization"`); summaries are text-only (media is lost, see Multimodality).

Isolation: subagents

`task` runs a subagent in a **fresh context** and returns **one final report**, so intermediate tool calls never enter the parent window. Ask for concise summaries; keep raw data in files and pass **paths**. See Part V.

Long-term memory

```python
agent = create_deep_agent(
    store=store,
    backend=CompositeBackend(default=StateBackend(),
                             routes={"/memories/": StoreBackend(namespace=lambda _rt: ("memories",))}),
)
```

Paths under `/memories/` persist across threads; everything else is per-thread. The directory starts empty (pre-seed via the Store API).

Prompt caching

For **Anthropic and Amazon Bedrock** (Claude or Nova), `create_deep_agent` automatically caches the static prompt prefix (base instructions, memory, skills) → lower latency/cost. Override TTL: `AnthropicPromptCachingMiddleware(ttl="1h")` (default 5 min). Caching runs *after* Patch and your middleware; `MemoryMiddleware` sits *after* caching so memory edits don't invalidate the prefix.

Advantages

Offloading and summarization run automatically; the full originals are kept on the filesystem, so detail isn't lost; subagents give each subtask a clean window; prompt caching cuts latency and cost for the static prefix.

Gotchas (context)

Memory is always loaded (bloat hurts every run) · `system_prompt` is static · runtime context never reaches the model unless exposed · custom state must subclass `DeepAgentState` · without a Composite route to a store, `/memories/` isn't durable · built-in compression doesn't resize images · `excluded_tools` shrinks tool schemas each turn, but that's configuration, **not** compression.

CHAPTER 37

Profiles (Beta): Defaults per Model and Provider

> **At a glance:** Two kinds of per-model defaults. **Provider profile** = how the *model is constructed* (init kwargs). **Harness profile** = how the *agent is shaped* (prompts, tools, middleware, general-purpose subagent). Register before creating agents.

What / why / when

What. Registered, reusable defaults keyed by provider or model: a provider profile for building the model, a harness profile for shaping the agent.

Why. Different models need different init kwargs, prompt wording, tool sets or middleware; profiles keep that out of every call site.

When. You support several models, ship a model-specific prompt tweak, hide tools from a model, or distribute defaults as a plugin.

Provider profile vs harness profile

	Provider profile	Harness profile
Affects	Model construction (temperature, timeout, …)	Agent harness (prompts, tools, middleware, subagent)
Applied	Only for `provider:model` strings	At agent construction, after the model is built
Register	`register_provider_profile(key, ProviderProfile(...))`	`register_harness_profile(key, HarnessProfile(...))`

Keys. Provider-level (`"openai"`) or model-level (`"openai:gpt-6-astra"`). Only the first colon separates provider from model ID. Model-level **inherits** unset fields from provider-level. **Lookup order** (for model objects): exact `provider:identifier` → identifier-only (if it contains `:`) → provider defaults.

Fields

PYTHON

```
HarnessProfile(
    base_system_prompt: str,          # main agent: added after caller prompt; subagents: replaces authored
prompt
    system_prompt_suffix: str,        # appended at the very end
    tool_description_overrides: Mapping[str, str],
    excluded_tools: frozenset[str],   # by tool name, post-injection filter
    excluded_middleware: frozenset[type[AgentMiddleware] | str],
    extra_middleware: Sequence[AgentMiddleware] | Callable[[], Sequence[AgentMiddleware]],
    general_purpose_subagent: GeneralPurposeSubagentProfile,   # disable, rename, or re-prompt
)
ProviderProfile(
    init_kwargs: Mapping[str, Any],                  # forwarded to init_chat_model
    pre_init: Callable[[str], None],                 # e.g. credential checks
    init_kwargs_factory: Callable[[], dict[str, Any]], # runtime-derived kwargs
)
```

Merge rules

Field	Merge behaviour
prompts	new wins
`excluded_tools` , `excluded_middleware`	**set union**
`init_kwargs`	key-wise, new wins
`pre_init`	chained (existing first)
`extra_middleware`	same type replaces at the same position; new types append

Re-registering under an existing key merges onto the prior profile, so you can customise a built-in.

How to use

```python
from deepagents import HarnessProfile, register_harness_profile, GeneralPurposeSubagentProfile  # confirm import paths for your version

register_harness_profile(
    "anthropic:claude-sonnet-4-6",
    HarnessProfile(
        system_prompt_suffix="Prefer concise answers.",
        excluded_tools=frozenset({"glob"}),
        general_purpose_subagent=GeneralPurposeSubagentProfile(enabled=True),
    ),
)
agent = create_deep_agent(model="anthropic:claude-sonnet-4-6")   # profile applied automatically
```

Declarative + plugins. `HarnessProfileConfig` is a serializable subset (`from_dict` / `to_dict`). Package profiles via entry points:

```toml
[project.entry-points."deepagents.harness_profiles"]
my_provider = "my_pkg.profiles:register_harness"
[project.entry-points."deepagents.provider_profiles"]
my_provider = "my_pkg.profiles:register_provider"
```

Load order: built-ins → entry-point plugins → direct user calls (additive).

Advantages, limitations, gotchas

Advantages. Tune per-model quirks once, without touching call sites; ship as plugins.

GOTCHAS

- Register **before** creating agents. **No wildcard key**: register per provider (or override at the call site).
- `FilesystemMiddleware` , `SubAgentMiddleware` and permission middleware **can't be excluded** (`ValueError`).
- Remove `task` with `GeneralPurposeSubagentProfile(enabled=False)` + no sync subagents.
- Provider profiles don't touch existing model objects. Caller's `system_prompt` is always first; `system_prompt_suffix` always last. Subagents re-resolve profiles against *their own* model.

Rule of thumb. Global changes in code; profiles for model-dependent adjustments.

PART V

05

Delegation

Delegation answers *"how do big problems get split up?"* Three flavours, from simplest to most flexible:

Flavour	Who decides	Blocking?	Best for
Subagents (`task` tool)	The model, one call at a time	Yes (sync)	Isolating a heavy subtask
Dynamic subagents	The model writes *code* that dispatches many	Within one `eval`	Fan-out, verification, tournaments
Async subagents	The model, via task IDs	No	Long-running, steerable, parallel work

IN THIS PART

38 Subagents: Delegating to Specialists

39 Dynamic Subagents (Beta): Delegation from Code

40 Async Subagents: Background Work

CHAPTER 38

Subagents: Delegating to Specialists

> **At a glance:** The main agent calls the built-in `task` tool to spawn an **ephemeral child agent** in a fresh context. The subagent runs to completion and returns **one final report**, so the parent never sees the intermediate tool calls (**context quarantine**).

What / why / when

What. A subagent is a child agent defined by a spec (name, description, prompt, tools, ...). Sync subagents **block** the supervisor until they finish; multiple `task` calls in one turn run **in parallel**.

Why. Keeps the parent's window clean, enables specialisation (own prompt, tools, model, skills) and parallelism.

When. Isolated, long-running, multi-step or parallel subtasks, especially tool-heavy research or review.

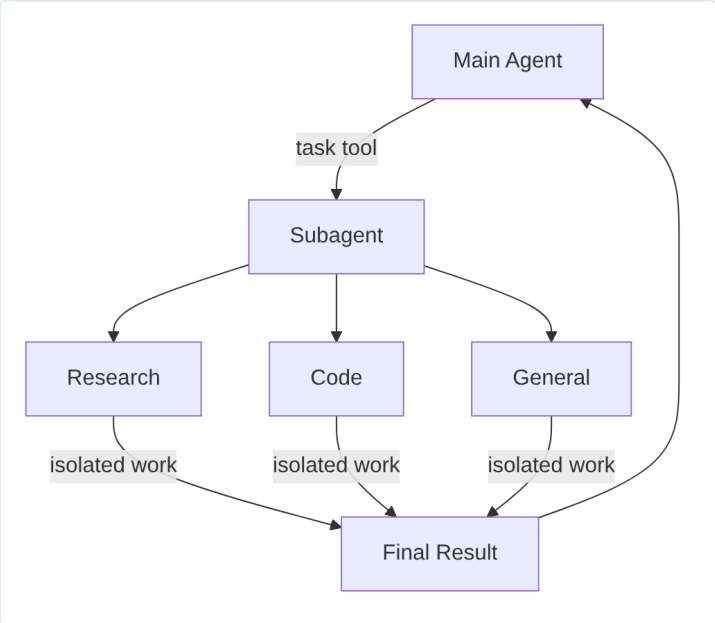


Figure 44. Subagent flow: main agent calls task; subagent does isolated work; one final result returns.

EXECUTION PROPERTIES

- **Isolated or inherited context:** default `mode: "isolated"` (new instance, own context); `mode: "fork"` inherits the full parent history.
- **Autonomous:** runs until completion. **Single handoff:** one final report. **Stateless messaging:** can't send multiple messages back. **Token-efficient:** heavy work compressed into a compact result.

The default `general-purpose` subagent

- Added automatically unless you pass a sync subagent named `general-purpose`. Has filesystem tools; inherits the main agent's skills.

- Customise via the harness profile (`general_purpose_subagent=GeneralPurposeSubagentProfile(...)` : rename, re-prompt, or `enabled=False`), or **replace it entirely** by passing your own `general-purpose` spec (this *replaces* the default).

Defining a subagent (the `SubAgent` spec)

```python
research_subagent = {
    "name": "research-agent",            # REQUIRED; used in task() calls and metadata
    "description": "Used to research more in depth questions",   # REQUIRED; drives delegation decisions
    "system_prompt": "You are a great researcher",   # REQUIRED for isolated; optional addendum for fork
    "mode": "isolated",                  # "isolated" (default) | "fork"
    "tools": [internet_search],          # INHERITS parent tools; if set, REPLACES them
    "model": "openai:gpt-5.5",           # INHERITS; "provider:model" or model object
    "middleware": [],                    # does NOT inherit; same .name replaces a default
    "interrupt_on": {"send_email": True},# INHERITS; needs a checkpointer
    "skills": ["/skills/research/"],     # does NOT inherit; gets its own SkillsMiddleware
    "response_format": ResearchResult,   # JSON result to parent (deepagents >= 0.5.3)
    "permissions": [FilesystemPermission(operations=["write"], paths=["/**"], mode="deny")],  # INHERITS;
REPLACES
}
agent = create_deep_agent(model="google_genai:gemini-3.6-flash", subagents=[research_subagent])
```

Field	Inherits?	Override semantics
`tools`	✅	**Replaces** parent tools (doesn't add)
`model`	✅	Defaults to the main agent's model
`middleware`	❌	A matching `.name` replaces a default instance
`interrupt_on`	✅	Subagent value used; needs a checkpointer
`skills`	❌	Must be set explicitly
`permissions`	✅	**Replaces** parent rules (no merge)

CompiledSubAgent (bring your own graph)

What. Wrap any compiled LangGraph graph as a subagent.

```python
from langchain.agents import create_agent
from deepagents import CompiledSubAgent

custom = CompiledSubAgent(
    name="data-analyzer",
    description="Specialized agent for data analysis",
    runnable=create_agent(model="anthropic:claude-sonnet-4-6", tools=[run_sql], system_prompt="..."),
    mode="isolated",                    # or "fork"
)
agent = create_deep_agent(model=..., subagents=[custom])
```

Gotchas. The graph must have a state key named `"messages"`. A compiled subagent keeps its **own** system prompt even in fork mode. Compiled and remote subagents don't inherit the parent `state_schema`.

Isolated vs fork

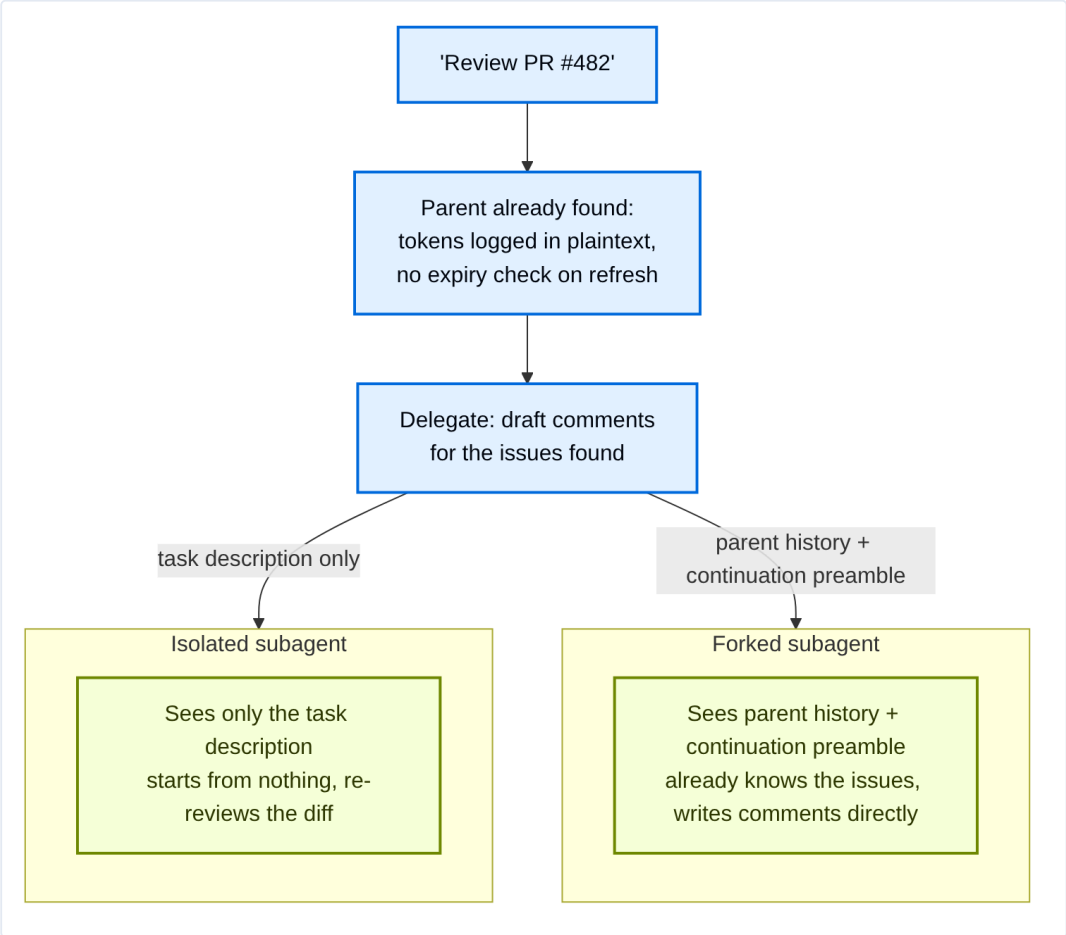


Figure 45. Isolated vs forked subagent for "Review PR #482": an isolated one starts from nothing; a fork already knows what the parent found.

Dimension	Isolated (default)	Fork (beta, `deepagents>=0.7.13`)
Context	Only the task description	Parent's full conversation and state
System prompt	You define it (required)	Parent's prompt; your addendum is appended
Skills	Via `skills`	Not supported; inherits parent's
Can call `task`	Yes	**No**
Best for	Focused, low-context work	Continuing work the parent already started

- **Fork mechanics:** the trailing `task` call is replaced by a short continuation preamble; the fork's final answer returns as a normal tool result.
- Leave `system_prompt` unset in fork mode to keep the **prompt-cache prefix shared** with the parent (a fork `system_prompt` *appends* and reduces cache reuse).
- `tools` replaces inherited tools, so pass the full set you want.

```python
reviewer_fork = {"name": "pr-commenter", "description": "Drafts review comments for issues already found",
                 "mode": "fork"}      # no system_prompt: shares the parent's cache prefix
```

Structured output from subagents

Set `response_format` (Pydantic model, `ToolStrategy`, `ProviderStrategy`, or raw type). The parent receives **JSON** as the `ToolMessage` content.

Running without subagents

1. `general_purpose_subagent=GeneralPurposeSubagentProfile(enabled=False)` on the harness profile, **and** 2. pass no sync subagents. `SubAgentMiddleware` and `task` are then not attached. **Async subagents are unaffected.** (Don't use `excluded_middleware`: it raises `ValueError`.)

Context propagation and observability

- Runtime context from the parent `invoke` flows to all subagents and their tools. For per-subagent settings, use namespaced keys or separate context fields.
- LangSmith traces carry `lc_agent_name` metadata so you can filter runs by agent (e.g. `coordinator`, or a subagent name).

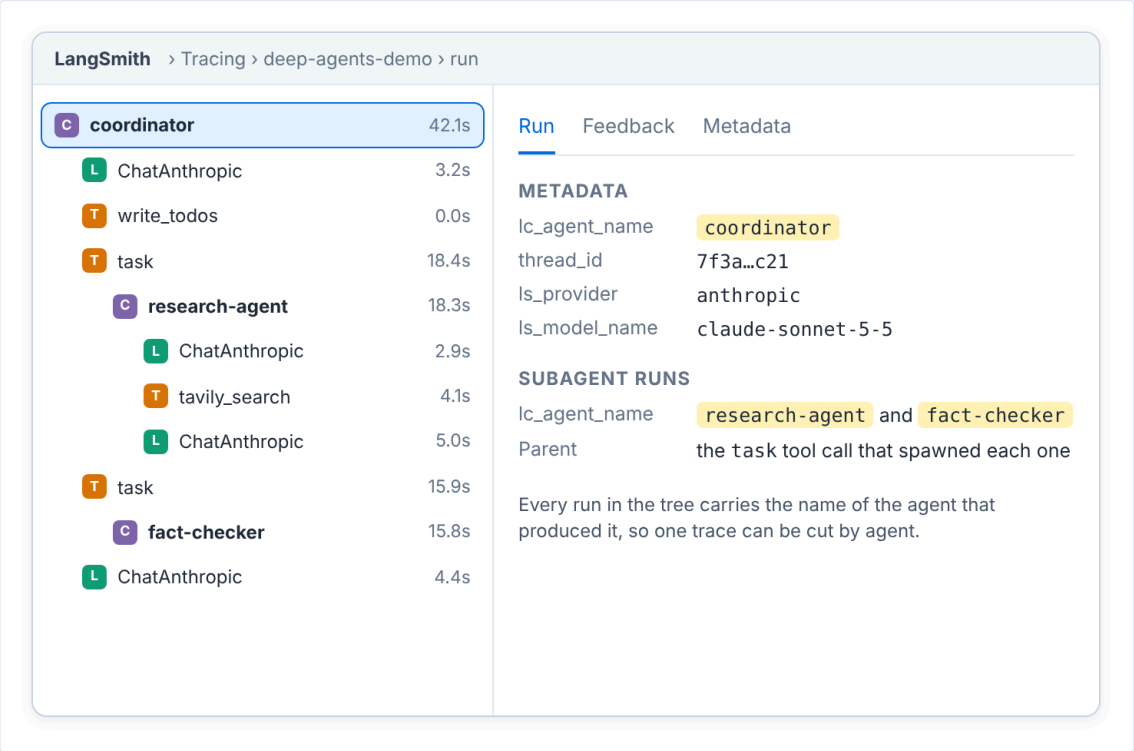


Figure 46. A LangSmith run tree in which every run carries the lc_agent_name of the agent that produced it (schematic illustration).

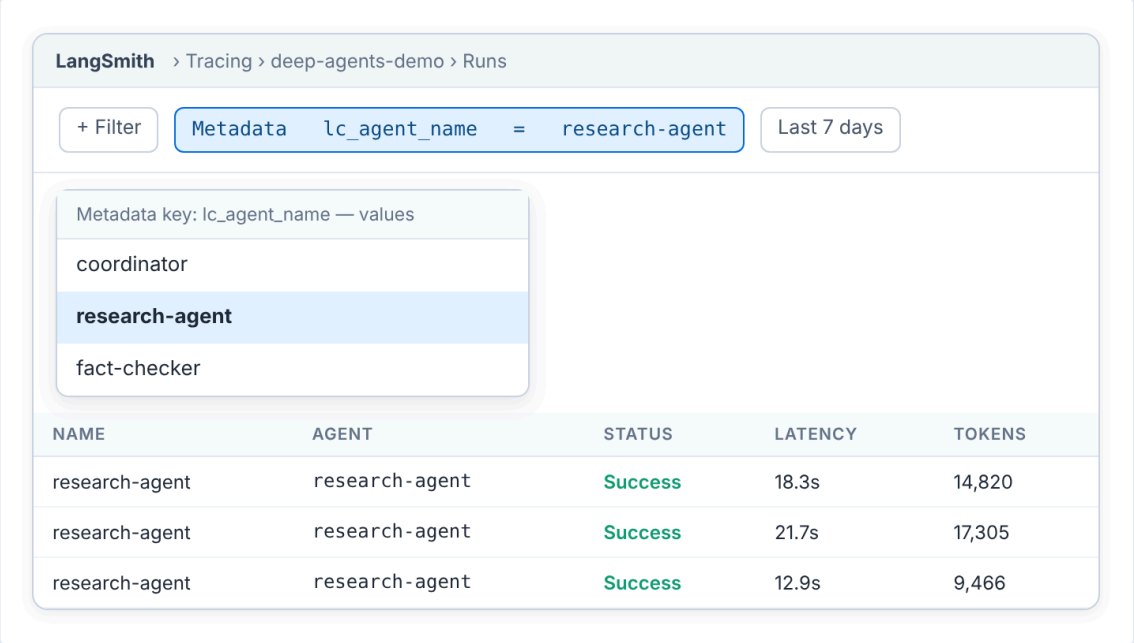


Figure 47. Filtering LangSmith runs by the lc_agent_name metadata key (schematic illustration).

Advantages, limitations, gotchas

Advantages. Context isolation, parallelism, per-subagent model/tools/skills, structured results.

Limitations. One report back, no mid-task messages (use async subagents to steer). Sync = blocks the supervisor.

GOTCHAS (INTERVIEW FAVOURITES)

- `tools` and `permissions` **replace**; they don't extend.
- Isolated subagents **don't inherit** the parent's system prompt, skills or middleware.
- `interrupt_on` requires a checkpointer.
- Re-adding a `general-purpose` spec replaces the default.
- Forks can't call `task`.
- Docs inconsistency: one example calls `task(name=..., task=...)`; the fork trace uses `subagent_type` and `description`. Check the real tool schema in your version.

Task planning (`write_todos`): the companion tool

What. An **opt-in** capability (since v0.7) giving the agent a structured task list in state with statuses `pending | in_progress | completed`.

HOW

```python
from langchain.agents.middleware import TodoListMiddleware

agent = create_deep_agent(model=..., middleware=[TodoListMiddleware()])
```

When. Long tasks, weaker models that benefit from an accountability tool, UIs that stream progress from state (`stream.values.todos`, see Part VIII).

CHAPTER 39

Dynamic Subagents (Beta): Delegation from Code

> **At a glance:** Instead of one model-chosen `task` call at a time, the agent writes **JavaScript** that runs in the QuickJS **interpreter** and calls `task()` many times, with loops, `Promise.all`, and code-level logic. Requires `CodeInterpreterMiddleware` (`langchain-quickjs>=0.2.0`, Python 3.11+).

What / why / when

What. A `task()` global inside `eval` that dispatches configured subagents and resolves to their result.

Why. Orchestration logic (fan-out, voting, tournaments, loops) is *deterministic code* instead of N model turns; intermediate results stay out of the model's context.

When. Many items to process in parallel; need verification or ranking; exhaustive sweeps.

Setup and the `task()` global

PYTHON

```python
from deepagents import create_deep_agent
from langchain_quickjs import CodeInterpreterMiddleware

agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
    subagents=[
        {"name": "reviewer", "description": "Reviews one file", "system_prompt": "You review code."},
        {"name": "verifier", "description": "Skeptically verifies a finding", "system_prompt": "Be
skeptical."},
    ],
    middleware=[CodeInterpreterMiddleware(ptc=["glob"], subagents=True)],   # subagents=False disables
dispatch
)
```

TS

```ts
// inside the eval tool
const result = await task({
  description: string,        // the subagent's prompt
  subagentType: string,       // name of a configured subagent
  responseSchema?: object,    // optional JSON schema → typed result (don't JSON.parse it)
});
```

- Each `task()` runs a **full agentic loop** and resolves to the subagent's result.
- **Trigger word:** say "**workflow**" to encourage code-based orchestration; use plain phrasing for a single delegation.
- Code that lists files needs the relevant tool on the PTC allowlist (`ptc=["glob"]`).
- A built-in general-purpose subagent handles basic fan-out with no extra setup.

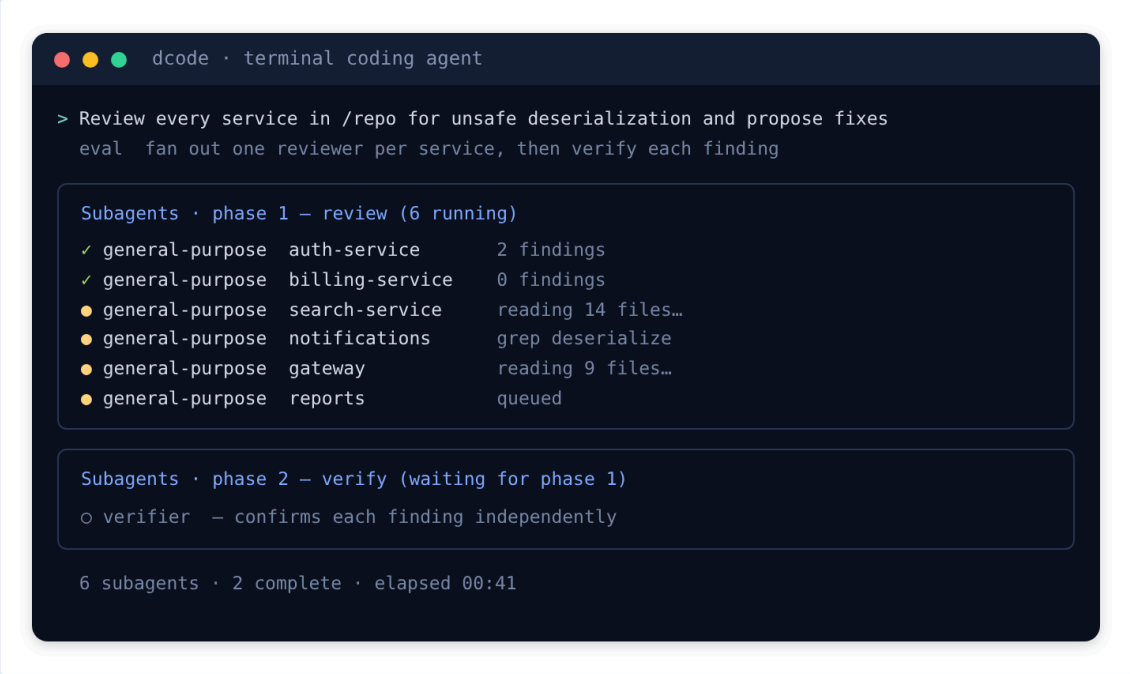


Figure 48. A terminal panel showing dynamically spawned subagents grouped by dispatch phase (schematic illustration).

The six orchestration patterns

> The six diagrams below show the shape of each pattern. The JavaScript is illustrative, assembled from the patterns the documentation describes.

Pattern	Shape	Typical use
Classify and act	Route each item to a specialist, then `Promise.all`	Ticket triage
Fan-out and synthesize	Parallel `task()` per item, then merge	Per-file code review
Adversarial verification	Find issues → verifiers confirm → keep confirmed	Security audits, low false positives
Generate and filter	Parallel proposals → score → pick best	Design alternatives
Tournament	Pairwise judge rounds until one winner	Subjective optimisation
Loop until done	`while` rounds, dedupe with a `Set`, stop when nothing new	Exhaustive sweeps (dead code)

1. Classify and act

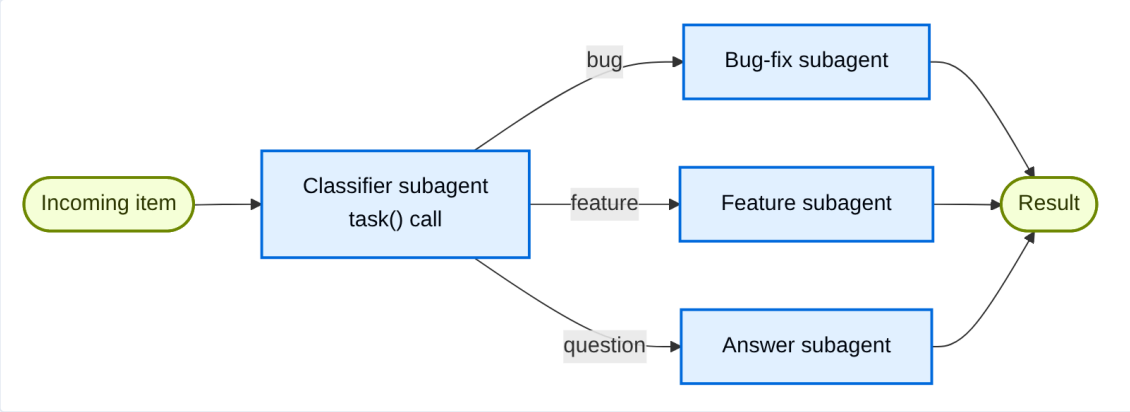


Figure 49. Classify and act.

```ts
const labels = await Promise.all(items.map((it) =>
  task({ description: `Classify: ${it}`, subagentType: "classifier", responseSchema: labelSchema })));
const results = await Promise.all(items.map((it, i) =>
  task({ description: `Handle: ${it}`, subagentType: labels[i].category })));   // route to the right specialist
```

2. Fan-out and synthesize

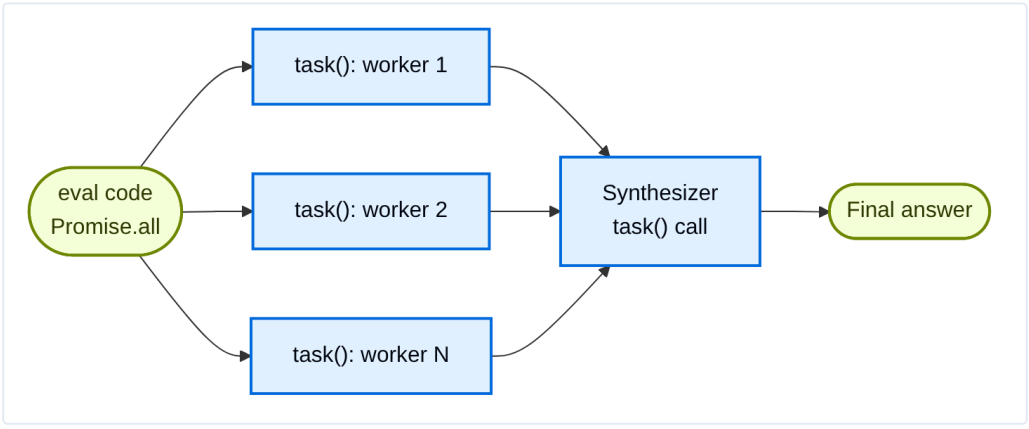


Figure 50. Fan-out and synthesize.

```ts
const reviews = await Promise.all(files.map((file) =>
  task({ description: `Review ${file}`, subagentType: "reviewer", responseSchema })));
const summary = await task({ description: `Synthesize: ${JSON.stringify(reviews.flatMap(r => r.issues))}`,
                             subagentType: "synthesizer" });
```

3. Adversarial verification (lowest false positives)

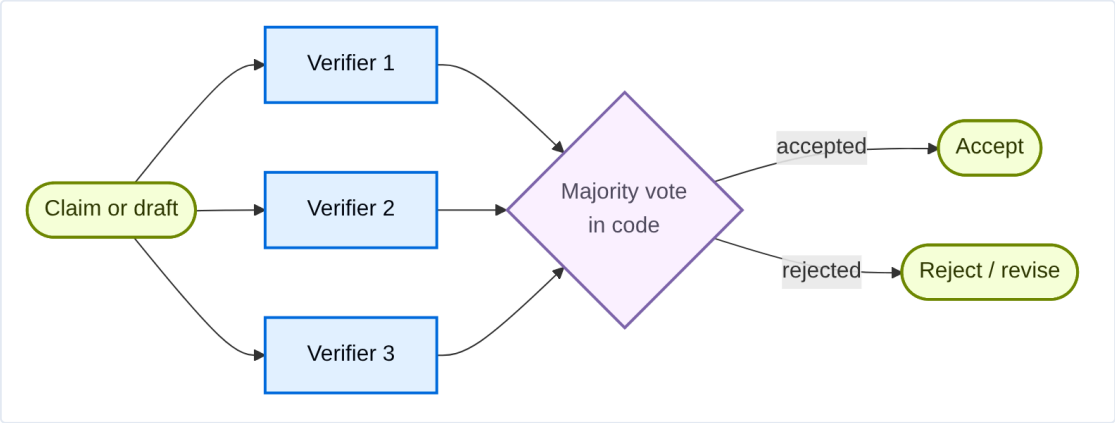


Figure 51. Adversarial verification with a majority vote.

```ts
const verdicts = await Promise.all(findings.map((f) =>
  task({ description: `Verify ${f.file}:${f.line}`, subagentType: "verifier", responseSchema: verdictSchema
})));
const confirmed = findings.filter((_, i) => verdicts[i]?.confirmed);
```

Use a **separate verifier with a skeptical system prompt** and a structured verdict. (The docs' diagram shows three verifiers voting by majority.)

4. Generate and filter

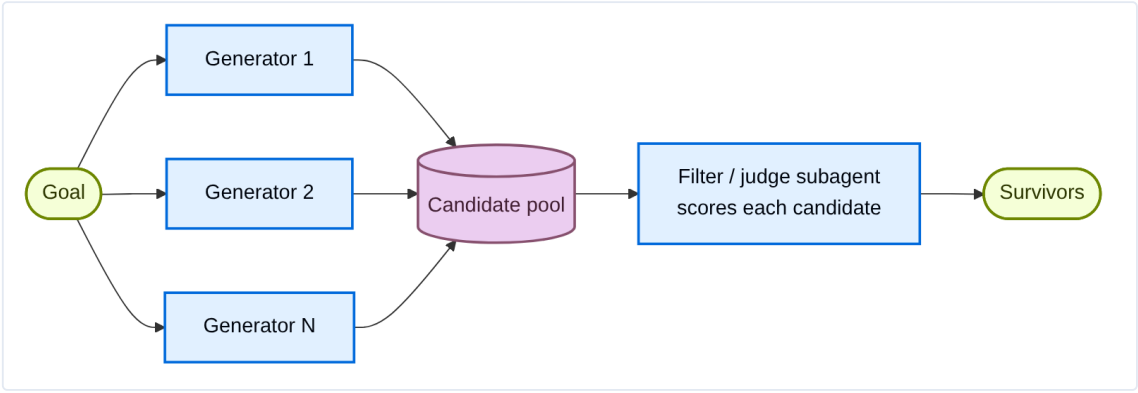


Figure 52. Generate and filter.

```ts
const candidates = await Promise.all(Array.from({ length: 5 }, () =>
  task({ description: "Propose a design", subagentType: "designer" })));
const scores = await Promise.all(candidates.map((c) =>
  task({ description: `Score: ${c}`, subagentType: "judge", responseSchema: scoreSchema })));
const best = candidates[scores.indexOf(Math.max(...scores.map(s => s.value)))];
```

5. Tournament

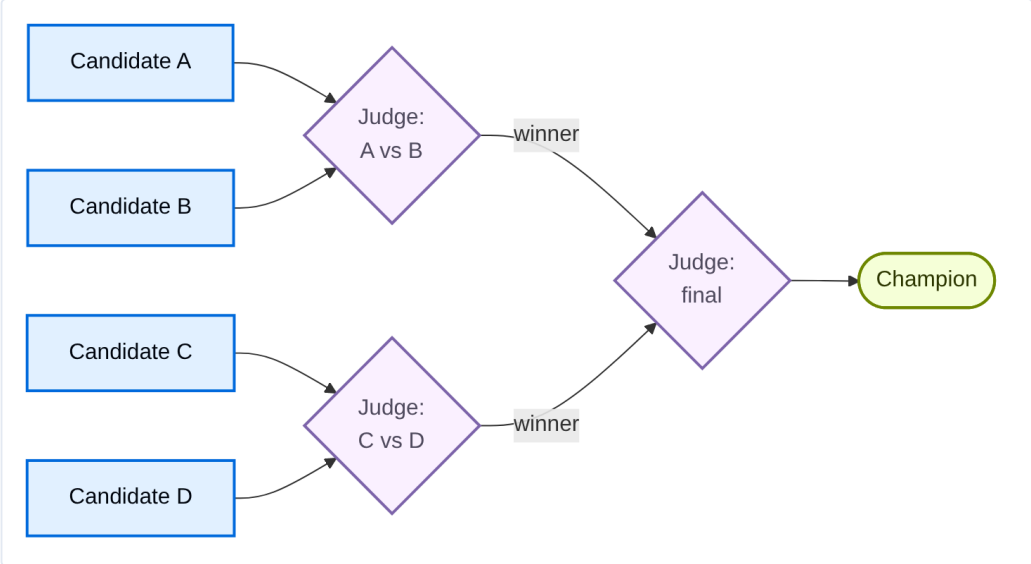


Figure 53. Tournament: pairwise judge rounds.

```ts
let pool = candidates;
while (pool.length > 1) {
  const next = [];
  for (let i = 0; i < pool.length; i += 2) {
    if (i + 1 === pool.length) { next.push(pool[i]); continue; }
    const v = await task({ description: `Which is better?\nA: ${pool[i]}\nB: ${pool[i+1]}`,
                           subagentType: "judge", responseSchema: pickSchema });
    next.push(v.winner === "A" ? pool[i] : pool[i + 1]);
  }
  pool = next;
}
```

6. Loop until done

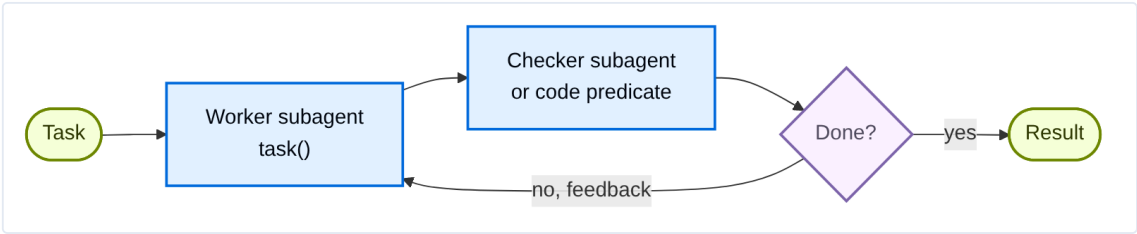


Figure 54. Loop until done.

```ts
const seen = new Set();
while (true) {
  const found = await task({ description: "Find unused exports not in: " + [...seen].join(","),
                             subagentType: "scanner", responseSchema: listSchema });
  const fresh = found.items.filter((x) => !seen.has(x));
  if (fresh.length === 0) break;
  fresh.forEach((x) => seen.add(x));
}
```

Recursive language model (RLM) style. Keep the working set in interpreter variables, select slices, dispatch with `task()`, synthesize.

Advantages, limitations, gotchas

Advantages. Deterministic orchestration; parallelism; typed results (`responseSchema`); minimal context use; reduces false positives with verification.

Limitations. Beta; Python 3.11+; the docs specify **no concurrency caps, timeouts or dispatch quotas**; `mode="thread"` keeps interpreter variables across turns.

GOTCHAS

- ⚠️ **Approval bypass:** `task()` runs inside `eval` and **skips the normal tool path**, so the parent's `interrupt_on` is **not enforced per dispatch**. To require approval, gate the **`eval` tool** itself.
- With `responseSchema`, don't `JSON.parse` the result; parse only when a subagent deliberately returns a JSON *string*.
- File discovery in code requires the tool on the PTC allowlist.

CHAPTER 40

Async Subagents: Background Work

> **At a glance:** Async subagents run as **background tasks on an Agent Protocol server**. The supervisor gets a **task ID immediately** and keeps working; it can later `check`, `update` (steer), `cancel` or `list` them. Needs LangSmith Deployment (v0.5+).

What / why / when

What. Remote or co-deployed graphs launched non-blockingly, each on its own persistent thread.

Why. Long-running work (minutes to hours), parallel work, and **steerable** work: the user can redirect a task mid-flight.

When. Results aren't needed before continuing; you want a responsive supervisor.

	Sync	Async
Supervisor	Blocks until done	Gets a **task ID** immediately and continues
Mid-task steering	❌	✅ `update_async_task`
Cancellation	❌	✅ `cancel_async_task`
State	Stateless	Persistent thread per task
Best for	Results needed now	Long-running, parallel, steerable

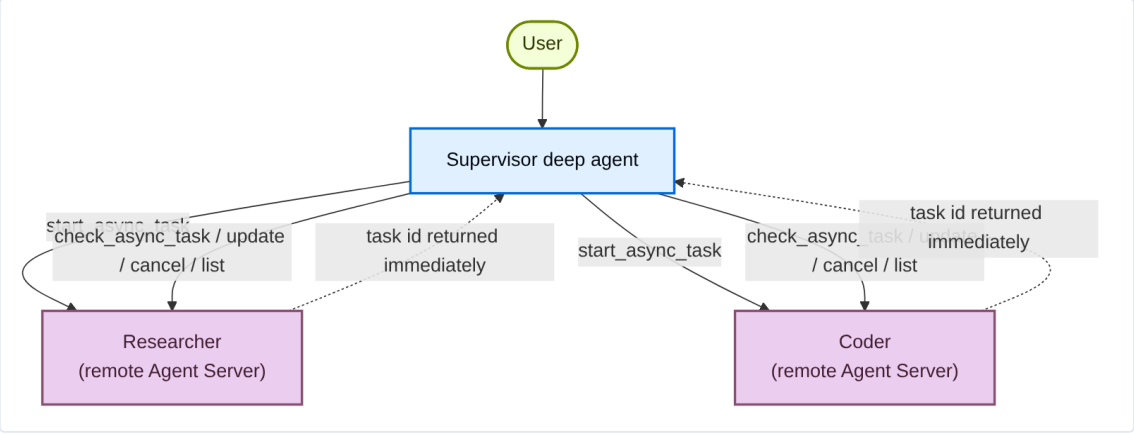


Figure 55. Async topology: supervisor launches a researcher and a coder, then checks on them.

How it works: the supervisor's tools (`AsyncSubAgentMiddleware`)

Tool	Does
`start_async_task`	Launches a task and returns its **ID**
`check_async_task`	Returns status and (if complete) the result
`update_async_task`	Sends new instructions to a running task

Tool	Does
`cancel_async_task`	Stops a running task
`list_async_tasks`	Lists all tasks with live status

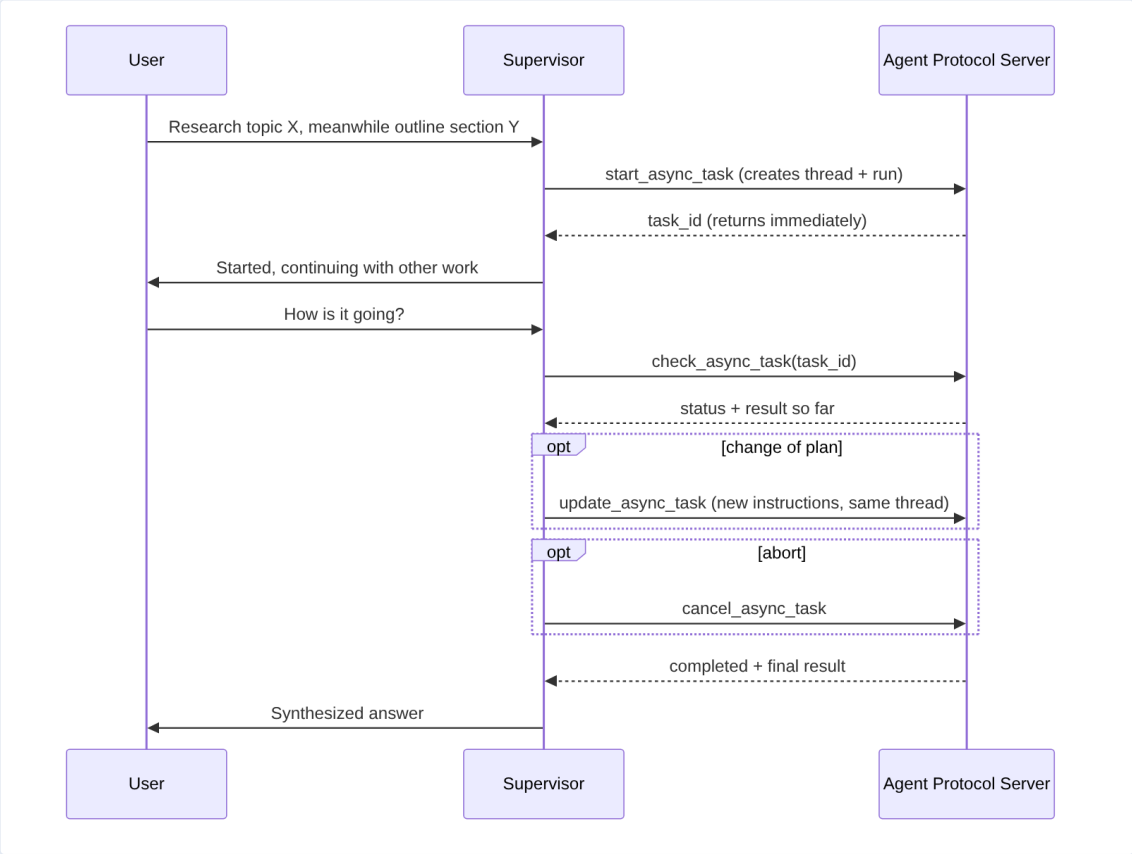


Figure 56. Async lifecycle sequence: launch, continue, check, update/cancel, complete.

LIFECYCLE DETAILS

- **Launch:** creates a server thread and starts a run; **thread ID = task ID**.
- **Update (steer):** starts a new run on the *same* thread, interrupting the previous one; the subagent restarts with full history + the new instructions; the task ID doesn't change.
- **Cancel:** server cancel; task marked `cancelled`.
- **List:** live status for non-terminal tasks; terminal ones (`success`, `error`, `cancelled`) come from cache.

Configuration

PYTHON

```python
from deepagents import AsyncSubAgent, create_deep_agent

async_subagents = [
    AsyncSubAgent(
        name="researcher",
        description="Researches a topic in depth; use for long-running research",   # drives routing: be specific
        graph_id="researcher",       # graph/assistant ID on the server
        # url omitted -> ASGI transport (co-deployed in the same langgraph.json, RECOMMENDED)
    ),
    AsyncSubAgent(
        name="coder", description="Writes and tests code", graph_id="coder",
        url="https://coder.example.com",          # set -> HTTP transport (remote, independent scaling)
```

```
        headers={"Authorization": "Bearer ..."},  # custom auth for remote
        mode="isolated",                           # "fork" is Python-only, experimental
    ),
]
agent = create_deep_agent(model="anthropic:claude-sonnet-4-6", subagents=async_subagents)
```

Topology	Transport	When
Single deployment (recommended)	ASGI (no `url`)	Everything in one `langgraph.json`
Split	HTTP (`url`)	Supervisor and subagents on separate servers / teams
Hybrid	Mix	Some co-deployed, some remote

`mode="fork"` copies the parent conversation at launch; later parent messages aren't forwarded.

Notifications and waiting (beta, `deepagents>=0.7.26`)

- On an Agent Protocol server the supervisor receives a `HumanMessage` **notification** when a subagent succeeds, errors, times out or **waits**.
- Read it from `message.additional_kwargs["deepagents_notification"]` → `task_id`, `agent_name`, `status`, `run_id`.
- A `waiting` subagent (paused on `interrupt()` / HITL) is answered by the **user through the subagent's own thread**, not by the supervisor. Helper signatures from the docs: `answer_waiting_subagent(task_id, graph_id, answer)` and `find_subagent_threads(parent_thread_id)`.

State and threads

Task metadata lives in a dedicated `async_tasks` state channel, separate from messages, so it **survives summarization**. Subagent threads are tagged `metadata={"parent_thread_id": ...}`.

Advantages, limitations, gotchas

Advantages. Non-blocking; steerable; cancellable; durable per-task threads; scale subagents independently.

Limitations. Needs an Agent Protocol server (LangSmith Deployment); notifications/waiting are beta.

GOTCHAS

- **Worker slots:** each active run uses one; 3 subagents + supervisor = 4. Under-provisioning queues launches (`langgraph dev --n-jobs-per-worker 10`).
- **Premature polling:** the supervisor may `check` right after launch; tell it in the system prompt to return control to the user.
- **Stale status** in chat history: always `check` / `list`. **ID truncation:** models sometimes shorten task IDs, breaking `check` / `cancel`.
- `update` on a waiting task drops the pending request; `cancel` on a waiting task only marks it cancelled.
- `parent_thread_id` metadata is editable by anyone with thread access; verify before trusting.
- Fork prompt-cache hits need matching model, system prompt, tools and middleware. Vague descriptions → poor routing.
- Reference implementation: `github.com/langchain-ai/async-deep-agents` (Python and TypeScript).

Delegation cheat sheet

Need	Use
Isolate a heavy subtask, get one answer	**Sync subagent** (isolated)
Continue a half-done conversation in a subagent	**Fork** subagent
Many parallel workers orchestrated in code	**Dynamic subagents** (`task()` in `eval`)
Long-running background work you can steer/cancel	**Async subagent**
Track multi-step progress	**Task planning** (`write_todos`)
Subagent needs its own tools/model/skills	Set them in its spec (they don't all inherit)

PART VI

06

Steering

Steering answers *"how do humans (or graders) stay in control of the agent?"*: pause before risky actions, or loop until quality criteria are met.

IN THIS PART

41 Human-in-the-Loop: Approvals and Interrupts

42 Grading Rubrics (Beta)

CHAPTER 41

Human-in-the-Loop: Approvals and Interrupts

> **At a glance:** Set `interrupt_on` to pause the graph **before** specific tool calls. A human then **approves**, **edits**, **rejects** or **responds**, and the run resumes. **Requires a checkpointer** and the **same** `thread_id` on resume.

What / why / when

What. `HumanInTheLoopMiddleware` (added automatically when you pass `interrupt_on`) interrupts matching tool calls and waits for decisions. Works on the main agent **and** subagents.

Why. Irreversible or sensitive actions (delete, send email, payments, DB writes), compliance workflows, long tasks that need guidance.

When. Whenever a wrong tool call would be costly. Persistence makes pauses indefinite (hours or days).

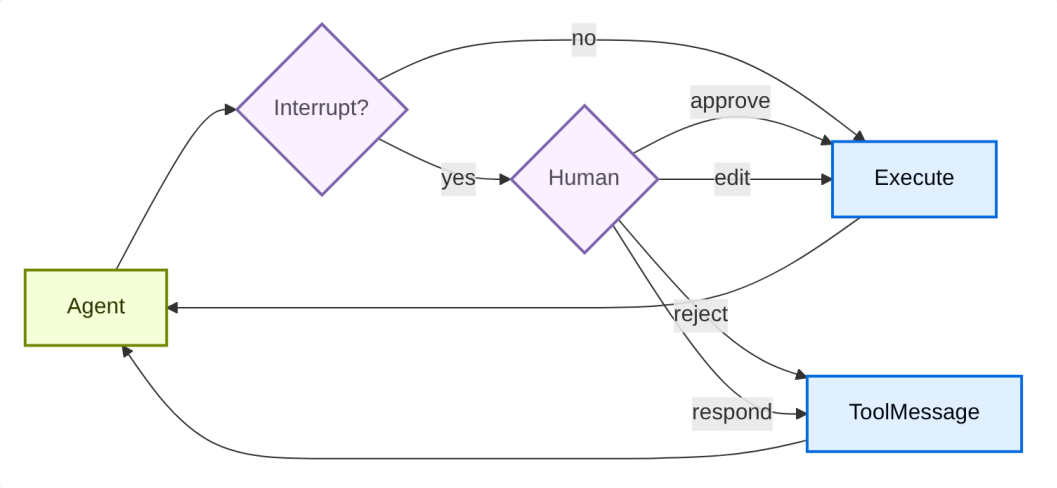


Figure 57. HITL flow: agent → interrupt check → human decision (approve / edit / reject / respond) → execute or ToolMessage → agent.

How to configure `interrupt_on`

PYTHON

```python
from langgraph.checkpoint.memory import MemorySaver
from deepagents import create_deep_agent

agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
    tools=[remove_file, fetch_file, notify_email],
    interrupt_on={
        "remove_file": True,                                    # all four decisions allowed
        "fetch_file": False,                                    # never interrupt
        "notify_email": {"allowed_decisions": ["approve", "reject"]},  # no editing
    },
    checkpointer=MemorySaver(),                                 # REQUIRED
)
```

- `True` = all four decisions. `False` = no interrupt.

- Dict form (`InterruptOnConfig`): `allowed_decisions` and optionally **`when`**, a predicate `(ToolCallRequest) -> bool` (**Python only, `langchain>=1.3.3`**). If `when` returns `False`, the call is **auto-approved** (excluded from the batch).

```python
interrupt_on={
    "execute_sql": {
        "allowed_decisions": ["approve", "reject"],
        "when": lambda request: "drop" in request.tool_call["args"]["query"].lower(),   # only risky queries pause
    },
}
```

- **Subagents** may define their own `interrupt_on` in their spec (inherits the parent's otherwise; needs a checkpointer).

The four decisions

Decision	Behaviour	Typical use
`approve`	Run with the original args	Send the drafted email
`edit`	Run with modified args	Change the recipient
`reject`	Skip the tool; send feedback to the agent	Deny a deletion
`respond`	The human's reply becomes a **synthetic tool result**; nothing executes	`ask_user`-style tools

Guidance. Use `respond` only when the human *is effectively the tool*. **Don't use it to deny** side-effecting tools: the model may read the reply as success. Keep edits minimal; large edits can make the model re-plan and re-run tools.

The interrupt payload and resuming

```python
import uuid
from langgraph.types import Command

config = {"configurable": {"thread_id": str(uuid.uuid4())}}      # keep the SAME thread_id on resume
result = agent.invoke(
    {"messages": [{"role": "user", "content": "Delete temp.txt and email the team"}]}, config=config
)

if result.get("__interrupt__"):
    payload = result["__interrupt__"][0].value
    # {"action_requests": [{"name": str, "args": dict}, ...],
    #  "review_configs":  [{"action_name": str, "allowed_decisions": [...]}, ...]}
    review = {c["action_name"]: c for c in payload["review_configs"]}

    decisions = []
    for req in payload["action_requests"]:                       # ONE decision per request, SAME order
        if req["name"] == "notify_email":
            decisions.append({"type": "reject", "message": "Don't email anyone yet."})
        else:
            decisions.append({"type": "approve"})
        # edit example:
        # {"type": "edit", "edited_action": {"name": "remove_file", "args": {"path": "/tmp/other.txt"}}}

    result = agent.invoke(Command(resume={"decisions": decisions}), config=config)
```

- **Batching:** multiple tools needing approval in one turn → **one batched interrupt**. (`langchain>=1.4.6` adds a **per-call mode**, `interrupt_mode="per_call"` on `HumanInTheLoopMiddleware`, where each gated call pauses separately and is answered by its ID; batched remains the default.)
- **Streaming:** `stream_mode=["messages", "updates"]`; the interrupt arrives as `"__interrupt__"` in `updates`.

- **Parallel pending interrupts** — resume with a map of interrupt ID → value (LangGraph interrupts).

In-tool interrupts (custom, free-form)

```python
from langgraph.types import interrupt, Command
from langchain.tools import tool

@tool
def transfer_funds(amount: float, to: str) -> str:
    """Transfer money."""
    approval = interrupt({"type": "approval_request", "amount": amount, "to": to})   # returns the resume value
    return "sent" if approval.get("approved") else "cancelled"

agent.invoke(Command(resume={"approved": True}), config=config)     # arbitrary payload, NOT {"decisions": ...}
```

Filesystem permission interrupts (`deepagents>=0.6.8`)

```python
from deepagents import FilesystemPermission
FilesystemPermission(operations=["write"], paths=["/secrets/**"], mode="interrupt")
```

Triggers on `write_file` / `edit_file` (and `delete`) matching the path; resume with the same `{"decisions": [...]}` shape; merges with `interrupt_on`. See Part III (Permissions).

Advantages, limitations, gotchas

Advantages. Safe-by-default for risky tools; durable pauses; fine-grained (per tool, per call via `when`); works for subagents.

LIMITATIONS

- **Doesn't protect dynamic/PTC calls.** `task()` and `tools.*` inside the interpreter skip the normal tool path — gate the `eval` tool instead (see Part III, Part V).
- Persistent shell sessions (`ShellToolMiddleware`) don't yet work with HITL interrupts.

GOTCHAS

- Forgetting the checkpointer, or **changing `thread_id` on resume**, breaks resumption.
- Decision order and count must match `action_requests`.
- `reject` without `message` sends a default "not executed, don't retry" note: supply a specific message for side-effecting tools.
- In-tool `interrupt()` resume values are free-form, not the `decisions` schema.
- `respond` is **not** a denial mechanism.

CHAPTER 42

Grading Rubrics (Beta)

> **At a glance:** `RubricMiddleware` runs an **LLM-as-a-judge** against a checklist after the agent finishes. If the verdict is `needs_revision`, per-criterion feedback goes back and the agent runs again, until a terminal verdict or `max_iterations` (default 3). Requires `deepagents>=0.6.5`; API may change.

What / why / when

What. A grader model reviews the output against a **rubric** (a newline-delimited checklist, one **verifiable** criterion per line).

Why. Tasks with a clear definition of "done" that the agent won't reliably hit on the first attempt (format rules, groundedness, tests passing).

When. You can phrase "done" as checks; you'd otherwise hand-roll a retry loop.

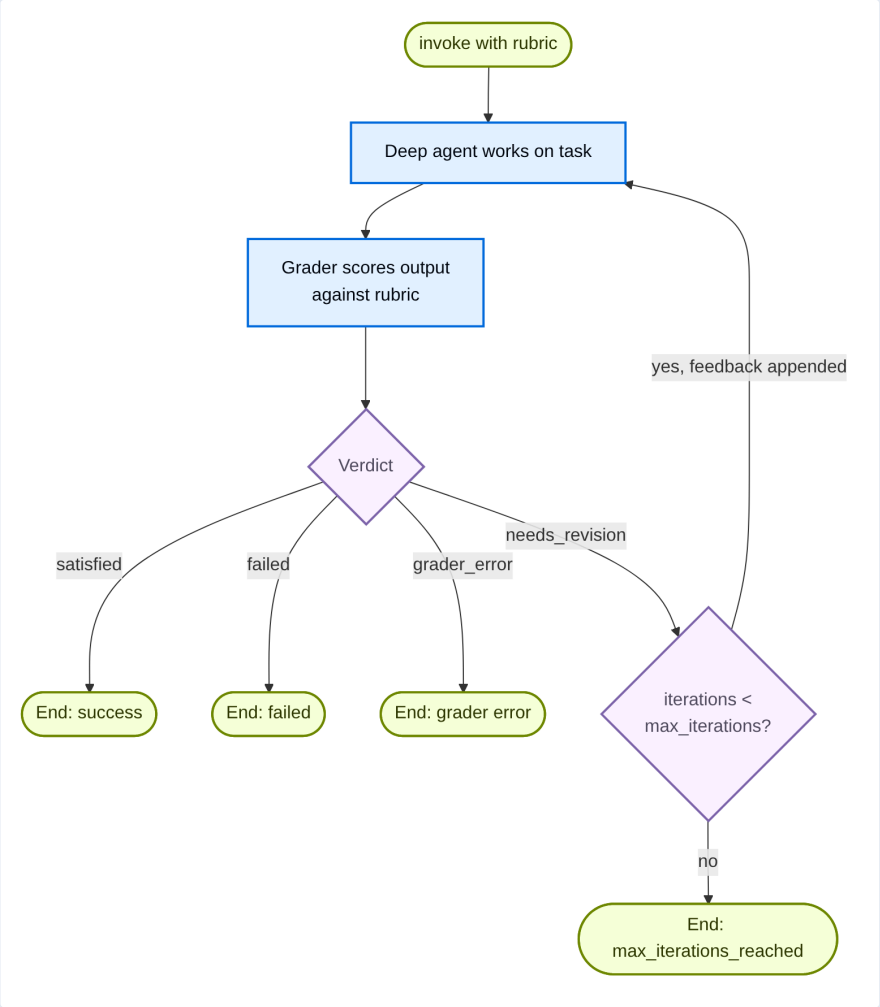


Figure 58. Rubric loop: invoke with a rubric → agent works → grader verdict → loop on needs_revision up to max_iterations.

How to use

```python
from deepagents import RubricMiddleware, create_deep_agent
from langchain.messages import HumanMessage
from langgraph.checkpoint.memory import InMemorySaver

def on_eval(ev):                                    # observability only; errors here are swallowed
    print(ev.iteration, ev.result, ev.explanation)

agent = create_deep_agent(
    model="google_genai:gemini-3.6-flash",
    middleware=[
        RubricMiddleware(
            model="anthropic:claude-haiku-4-5",     # grader: often a smaller/cheaper model
            max_iterations=3,
            tools=None,                             # give the grader tools (e.g. run tests) to gather
evidence
            on_evaluation=on_eval,
        ),
    ],
    checkpointer=InMemorySaver(),
)

rubric = """- The answer is one sentence
- The answer includes a source URL
"""
result = agent.invoke(
    {"messages": [HumanMessage("What is LangGraph?")], "rubric": rubric},
    config={"configurable": {"thread_id": "t1"}},
)
print(result.get("_rubric_status"))                 # e.g. satisfied / max_iterations_reached
```

SIGNATURE

```python
RubricMiddleware(
    model: str | BaseChatModel,         # required grader
    system_prompt: str | None = None,   # default grader prompt if omitted (set it to give the grader a
role)
    tools: list | None = None,          # evidence-gathering tools
    max_iterations: int = 3,            # positive int
    on_evaluation: Callable[[RubricEvaluation], None] | None = None,
)
```

`RubricEvaluation`: `grading_run_id`, `iteration`, `result`, `explanation`, `criteria` (each `{name, passed: True}` or `{name, passed: False, gap}`).

Verdicts

Status	Loops?
`satisfied`	No
`needs_revision`	**Yes** (until the cap)
`max_iterations_reached`	No (terminal; stored on `_rubric_status`)
`failed` (rubric malformed or unevaluable)	No
`grader_error` (grader raised)	No (empty `criteria`; explanation has the exception)

Streaming evaluations

```python
stream = agent.stream_events(input, version="v3", transformers=[CustomTransformer])
# read stream.custom for "rubric_evaluation_start" / "rubric_evaluation_end"
```

Advantages, limitations, gotchas

Advantages. Automatic quality loop; grader can use tools; cheap grader model; observable.

Limitations. Beta; an LLM judge is itself fallible; cost scales with iterations.

GOTCHAS

- On cap exhaustion the last evaluation still reads `needs_revision`; check `_rubric_status == "max_iterations_reached"` after `invoke`.
- Exceptions in `on_evaluation` are **swallowed** (logged): don't use it for control flow.
- **No `rubric` in state → the middleware does nothing.**
- A new `rubric` string starts a new run (so does reusing one after a terminal verdict); persistence needs a **checkpointer + shared `thread_id`**, and a new rubric replaces the old.
- `KeyboardInterrupt` / `CancelledError` propagate; the next call with the same rubric resumes the in-flight run.
- Write criteria that are *checkable against the transcript or a tool result*, not vibes.

Choosing a steering tool

Need	Use
Approve/deny a specific tool call	`interrupt_on` (HITL)
Approve only risky arguments	`interrupt_on` with `when`
Approve writes to certain paths	`FilesystemPermission(mode="interrupt")`
Ask the user a question mid-tool	In-tool `interrupt()` or a `respond`-style tool
Auto-check output quality and retry	`RubricMiddleware`
Stop runaway loops	`ModelCallLimitMiddleware` / `ToolCallLimitMiddleware` (Part VII)

PART VII

07

Middleware

The hook system that implements most of the harness.

IN THIS PART

43 Middleware: Overview

44 Prebuilt Middleware

45 Custom Middleware

CHAPTER 43

Middleware: Overview

At a glance: Middleware wraps the agent loop with **hooks** (before/after/around the model and tools) so you can add observability, transformation, reliability and governance **without changing the loop**. Deep Agents *is* a pre-assembled middleware stack; your middleware slots into it.

What / why

What. LangChain middleware gives "finer control over what happens inside the agent". It exposes hooks before and after each step of the core loop: call the model → let it choose tools → run them → repeat until no more tool calls.

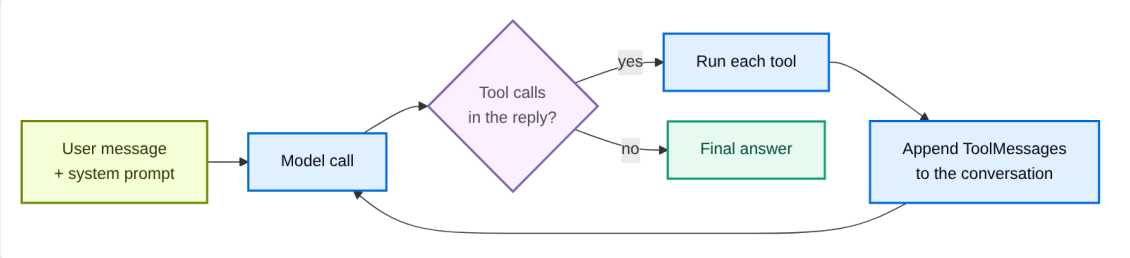


Figure 59. The core agent loop that middleware hooks into.

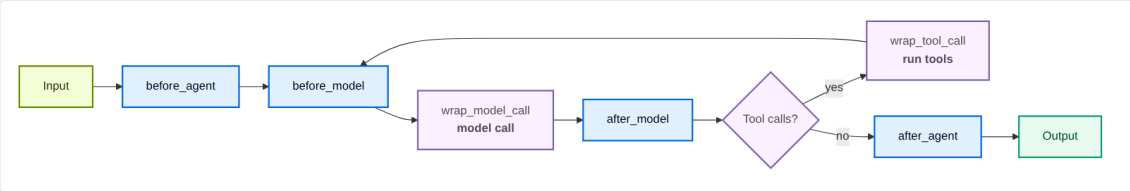


Figure 60. Middleware flow: hooks wrapping the agent loop.

WHY

Concern	Examples
Observability	logging, analytics, debugging
Transformation	change prompts, tool selection, output formatting
Reliability	retries, fallbacks, early termination
Governance	rate limits, guardrails, PII detection

How to add middleware

PYTHON

```python
from langchain.agents import create_agent
from langchain.agents.middleware import SummarizationMiddleware, HumanInTheLoopMiddleware

agent = create_agent(
    model="gpt-5.5",
    tools=[...],
```

```
    middleware=[SummarizationMiddleware(...), HumanInTheLoopMiddleware(...)],
)
# Deep Agents: the same objects go into create_deep_agent(middleware=[...])
```

Middleware inside LangGraph

Middleware is **not a separate runtime**. Its hooks run inside the compiled LangGraph that `create_agent` returns, so the whole agent (middleware included) can be a **node or subgraph** in a larger `StateGraph`: classify before routing, fan out parallel work, mix deterministic and agentic steps.

PYTHON

```python
from langchain.agents import AgentState, create_agent
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.graph import START, StateGraph

email_agent = create_agent(
    model="claude-sonnet-4-6",
    tools=[read_email, send_email],
    middleware=[HumanInTheLoopMiddleware(interrupt_on={"send_email": True})],
)

graph = (
    StateGraph(AgentState)
    .add_node("classify", classify_node)
    .add_node("email_agent", email_agent)        # the agent (and its middleware) is a node
    .add_edge(START, "classify")
    .add_conditional_edges("classify", route)
    .compile()
)
```

`HumanInTheLoopMiddleware` matches tools by `.name` (for `@tool` functions, the function name). HITL interrupts, summarization, PII redaction, retries and custom hooks all move with the agent node.

When to reach for middleware

When behaviour must apply on **every** model or tool call (logging, limits, PII handling, retries, summarization, approval) rather than inside one tool or one prompt; when you want it reusable across agents; or when you need to change the request or response itself.

Advantages, limitations

Advantages. Extend behaviour without touching the loop; composable; reusable; works inside LangGraph; built-in, provider-specific and custom options.

Limitations. Not a separate runtime: behaviour depends on the graph it runs in. **Order matters** (see Custom middleware). In Deep Agents, replacing a built-in by `.name` *replaces* it rather than merging.

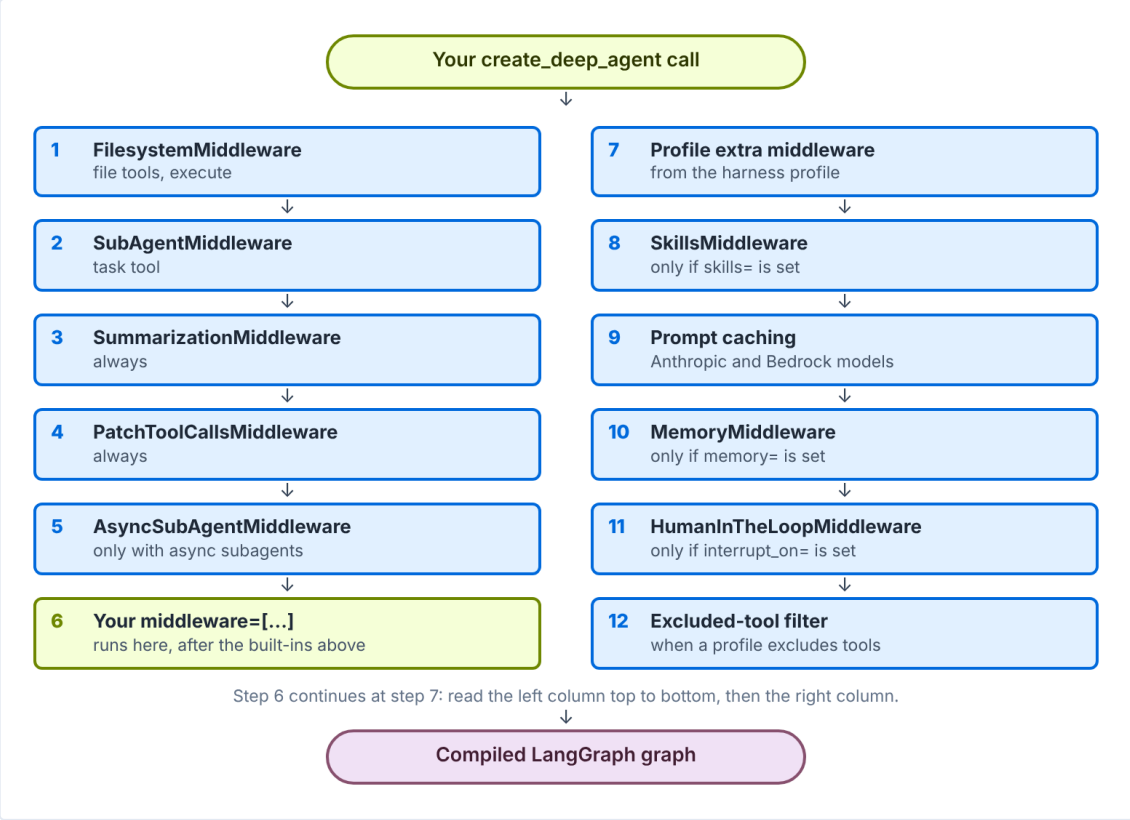


Figure 61. Deep Agents' built-in middleware stack, for orientation.

CHAPTER 44

Prebuilt Middleware

At a glance: 19 ready-made middleware. Pick by problem: **reliability** (tool error, tool retry, model retry, model fallback, call limits), **context** (summarization, context editing, tool selection, provider tool search), **safety** (HITL, PII, rubric), **capabilities** (to-do, shell, filesystem, file search, subagent), **testing** (LLM tool emulator). Provider-specific ones exist for Anthropic, AWS and OpenAI.

Why and how to use prebuilt middleware

Why. Reliability, context control, safety and capability needs recur in every agent; prebuilt middleware solves them with configuration instead of custom loop code.

How. Instantiate and pass them in `middleware=[...]` (to `create_agent` or `create_deep_agent`). Order matters. Deep Agents already includes several (filesystem, subagents, summarization, plus HITL when you set `interrupt_on`).

PYTHON

```python
from deepagents import create_deep_agent
from langchain.agents.middleware import ModelRetryMiddleware, ToolCallLimitMiddleware

agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[ModelRetryMiddleware(max_retries=3), ToolCallLimitMiddleware(run_limit=50)],
)
```

Which one for what

Problem	Middleware
Tool raised an exception	`ToolErrorMiddleware` (convert to a message) · `ToolRetryMiddleware` (retry)
Flaky model API	`ModelRetryMiddleware` → `ModelFallbackMiddleware`
Runaway loop / cost	`ModelCallLimitMiddleware`, `ToolCallLimitMiddleware`
Context too long	`SummarizationMiddleware`, `ContextEditingMiddleware`
Too many tools	`LLMToolSelectorMiddleware`, `ProviderToolSearchMiddleware`
Human approval	`HumanInTheLoopMiddleware`
PII	`PIIMiddleware`
Planning	`TodoListMiddleware`
Quality gate	`RubricMiddleware` (Deep Agents, beta)
Shell/files	`ShellToolMiddleware`, `FilesystemMiddleware`, `FilesystemFileSearchMiddleware`
Delegation	`SubAgentMiddleware` (Deep Agents)
Testing	`LLMToolEmulator`

1. Tool error (`ToolErrorMiddleware`, `langchain>=1.3.14`)

What. Catches exceptions during tool execution and returns them as **error** `ToolMessage`**s** so the model can recover instead of the run halting. It does **not** retry.

Params. `on_error(exc, request) -> content | None` (return content to convert; `None` to propagate) · `aon_error` (async; falls back to `on_error`) · `tools` (default all).

When. Let the model fix bad args; return sanitised errors; keep unexpected exceptions from crashing the agent.

```python
from langchain.agents.middleware import ToolErrorMiddleware

def on_error(exc: Exception, request) -> str | None:
    if isinstance(exc, ValueError):
        return f"`{request.tool_call['name']}` failed with {type(exc).__name__}."
    # return None implicitly -> propagate everything else

agent = create_agent(model="gpt-5.5", tools=[...], middleware=[ToolErrorMiddleware(on_error)])
```

2. Tool retry (`ToolRetryMiddleware`)

What. Retries failed tool calls with **exponential backoff**.

Params (defaults). `max_retries=2` (3 attempts) · `tools=None` · `retry_on` (retryable model errors + unclassified exceptions; accepts an exception tuple or callable) · `on_failure='continue'` (or `'error'`, or a callable) · `backoff_factor=2.0` · `initial_delay=1.0` s · `max_delay=60.0` · `jitter=True` (±25%). Delay = `initial_delay * backoff_factor ** retry_number`.

When. Transient failures in external APIs. **Placement tip:** put it *inside* Tool error with `on_failure="error"` so exceptions reach the error handler.

```python
ToolRetryMiddleware(max_retries=3, backoff_factor=2.0, initial_delay=1.0, tools=["search", "fetch_url"])
```

3. Model retry (`ModelRetryMiddleware`)

What. Same backoff model for failed **model** calls. `on_failure='continue'` returns an `AIMessage` with error details (also `'error'` or a callable). `retry_on` default retries retryable model errors and unclassified exceptions (`langchain>=1.3.16`).

```python
ModelRetryMiddleware(max_retries=3, backoff_factor=2.0, initial_delay=1.0)
```

4. Model fallback (`ModelFallbackMiddleware`)

What. Switches to alternative models when the primary fails. **Params:** `first_model` (required), `*additional_models` tried in order.

When. Provider outage, cost reduction with cheaper fallbacks, cross-provider redundancy. **Retry vs fallback:** retry = same model, transient; fallback = different model, provider down.

```python
ModelFallbackMiddleware("gpt-5.4-mini", "claude-sonnet-4-6")
```

5. Summarization (`SummarizationMiddleware`)

What. Summarises older history at thresholds, keeping recent messages. Summaries are **text-only** (multimodal blocks in summarised messages become text; retained recent messages keep their media).

Params. `model` (required) · `trigger` (none by default; `ContextSize` tuples, `TriggerClause` dicts (AND), or lists (OR); thresholds `fraction` / `tokens` / `messages`) · `keep=('messages', 20)` (exactly one threshold) · `token_counter` (character-based default) · `summary_prompt` (include `{messages}`) · `trim_tokens_to_summarize=4000`. Deprecated: `summary_prefix`, `max_tokens_before_summary`, `messages_to_keep`.

```python
SummarizationMiddleware(model="gpt-5.4-mini", trigger=("tokens", 4000), keep=("messages", 20))
```

(Deep Agents already includes its own: 85% trigger, keep 10%. See Part IV.)

6. Human-in-the-loop (`HumanInTheLoopMiddleware`)

What. Pause so a human can approve/edit/reject tool calls. **Params.** `interrupt_on` (per tool: `allowed_decisions` from `approve` / `edit` / `reject`, or `False`) · `interrupt_mode` (default one pause per model turn; `"per_call"` = one pause per tool call, answered by ID). **Requires a checkpointer.**

```python
HumanInTheLoopMiddleware(interrupt_on={
    "your_send_email_tool": {"allowed_decisions": ["approve", "edit", "reject"]},
    "your_read_email_tool": False,
})
```

Deep Agents wires this via `interrupt_on=` (adds `respond`, `when`). See Part VI.

7. Model call limit (`ModelCallLimitMiddleware`)

What. Cap model calls. **Params.** `thread_limit` (needs a checkpointer) · `run_limit` · `exit_behavior='end'` (graceful) or `'error'`.

```python
ModelCallLimitMiddleware(thread_limit=10, run_limit=5, exit_behavior="end")
```

8. Tool call limit (`ToolCallLimitMiddleware`)

What. Limit tool calls globally or for one tool. **Params.** `tool_name` (omit = all) · `thread_limit` / `run_limit` (at least one; run limits reset each user turn) · `exit_behavior`: `'continue'` (default; block excess calls with error messages), `'error'` (raise `ToolCallLimitExceededError`), `'end'` (single-tool only).

```python
middleware=[
    ToolCallLimitMiddleware(thread_limit=20, run_limit=10),                    # global
    ToolCallLimitMiddleware(tool_name="search", thread_limit=5, run_limit=3),  # per tool
]
```

9. PII detection (`PIIMiddleware`)

What. Detect and handle PII in inputs, outputs and tool results (with `apply_to_output=True` and `langchain>=1.3.2`, also redacts streamed output).

Params. `pii_type` (built-ins: `email`, `credit_card`, `ip`, `mac_address`, `url`, or custom) · `strategy` = `redact` (default) / `block` / `mask` / `hash` · `detector` (regex / compiled regex / function returning `PIIMatch`) · `apply_to_input=True` · `apply_to_output=False` · `apply_to_tool_results=False`.

```python
middleware=[
    PIIMiddleware("email", strategy="redact", apply_to_input=True),
    PIIMiddleware("credit_card", strategy="mask", apply_to_input=True),
]
```

10. To-do list (`TodoListMiddleware`)

What. Gives the agent a `write_todos` tool and prompts for multi-step planning. Params: `system_prompt`, `tool_description` (built-in defaults). **Opt-in in Deep Agents since v0.7.**

```python
agent = create_agent(model="gpt-5.5", tools=[read_file, write_file, run_tests], middleware=
[TodoListMiddleware()])
```

11. LLM tool selector (`LLMToolSelectorMiddleware`)

What. A separate LLM call (structured output) picks relevant tools before the main model runs. **Params.** `model` (default: agent's) · `system_prompt` · `max_tools` (extras dropped) · `always_include` (not counted toward `max_tools`). **When.** 10+ tools, most irrelevant per query.

```python
LLMToolSelectorMiddleware(model="gpt-5.4-mini", max_tools=3, always_include=["search"])
```

12. Provider tool search (`ProviderToolSearchMiddleware`)

What. Defers tool schemas behind the **provider's server-side tool search**, so the model fetches tools on demand. **Params.** `searchable_tools` (names/instances; tools built with `extras={"defer_loading": True}` are deferred regardless). **Requires** Anthropic (Sonnet 4+, Opus 4+, Haiku 4.5+) or OpenAI (gpt-5.5+); others raise `ValueError`.

```python
ProviderToolSearchMiddleware(searchable_tools=["lookup_order"])
```

13. Shell tool (`ShellToolMiddleware`)

What. A persistent shell session (default `/bin/bash`). **Params.** `workspace_root` (temp dir if omitted) · `startup_commands` / `shutdown_commands` · `execution_policy`: `HostExecutionPolicy` (full host access, **default**), `DockerExecutionPolicy` (per-run container), `CodexSandboxExecutionPolicy` · `redaction_rules` (don't stop exfiltration under `HostExecutionPolicy`) · `tool_description`, `shell_command`, `env`.

Limitation. Persistent sessions don't work with HITL interrupts yet.

```python
ShellToolMiddleware(workspace_root="/workspace", execution_policy=HostExecutionPolicy())
```

14. Filesystem (`FilesystemMiddleware`, from `deepagents`)

What. `ls`, `read_file`, `write_file`, `edit_file` (+ `glob`, `grep`, `delete` in Deep Agents) for short- and long-term context. Included by default in `create_deep_agent`. **Params.** `backend` (default `StateBackend`; use `CompositeBackend` to route `/memories/` to a store) · `system_prompt` · `custom_tool_descriptions` · `tools` (allowlist).

```python
from deepagents.middleware.filesystem import FilesystemMiddleware
FilesystemMiddleware(backend=None, tools=["read_file", "ls", "glob", "grep"],
                     custom_tool_descriptions={"ls": "Use ls when..."})
```

15. Subagent (`SubAgentMiddleware`, from `deepagents`)

What. Adds the `task` tool to delegate to subagents. **Params.** `default_model`, `default_tools`, `subagents` (each: `name`, `description`, `system_prompt`, `tools`, optional `model`, `middleware`; or a `CompiledSubAgent`). A `general-purpose` subagent is always available.

```python
from deepagents.middleware.subagents import SubAgentMiddleware
SubAgentMiddleware(default_model="claude-sonnet-4-6", default_tools=[],
    subagents=[{"name": "weather", "description": "Gets weather in cities.",
                "system_prompt": "Use get_weather.", "tools": [get_weather], "model": "gpt-5.5",
"middleware": []}])
```

16. Rubric grading (`RubricMiddleware`, beta)

LLM-judge self-grading loop (`deepagents>=0.6.5`). See Part VI.

```python
RubricMiddleware(model="anthropic:claude-haiku-4-5", max_iterations=3)
```

17. File search (`FilesystemFileSearchMiddleware`)

What. Glob + Grep tools over a root path. **Params.** `root_path` (required) · `use_ripgrep=True` (falls back to Python regex) · `max_file_size_mb=10`. **When.** Code exploration, regex search over big codebases.

```python
FilesystemFileSearchMiddleware(root_path="/workspace", use_ripgrep=True)
```

18. Context editing (`ContextEditingMiddleware`)

What. Clears **older tool outputs** once a token threshold is hit, keeping recent results. **Params.** `edits=[ClearToolUsesEdit()]` · `token_count_method='approximate'|'model'`. `ClearToolUsesEdit`: `trigger=100000`, `clear_at_least=0`, `keep=3`, `clear_tool_inputs=False`, `exclude_tools=()`, `placeholder="[cleared]"`.

```python
ContextEditingMiddleware(edits=[ClearToolUsesEdit(trigger=100000, keep=3)])
```

19. LLM tool emulator (`LLMToolEmulator`)

What. Replaces real tool execution with **LLM-generated responses** for testing/prototyping. **Params.** `tools` (default all; `[]` emulates none) · `model` (default: agent's).

```python
LLMToolEmulator()      # emulate all tools
```

Provider-specific (linked only on the page)

- **Anthropic:** prompt caching, bash tool, text editor, memory, file search.
- **AWS:** prompt caching for Amazon Bedrock.
- **OpenAI:** content moderation.

A typical production reliability stack

```python
middleware=[
    ModelRetryMiddleware(max_retries=3),
    ModelFallbackMiddleware("gpt-5.4-mini", "claude-sonnet-4-6"),
```

```
    ToolRetryMiddleware(max_retries=2, tools=["search"], on_failure="error"),   # inside ToolError
    ToolErrorMiddleware(on_error),
    ModelCallLimitMiddleware(run_limit=50),
    ToolCallLimitMiddleware(run_limit=200),
    PIIMiddleware("email", strategy="redact", apply_to_input=True),
]
```

Docs' principle for error handling: **"Do not catch what you cannot handle."**

Advantages, limitations, gotchas (prebuilt)

Advantages. Battle-tested, configurable, composable. **Gotchas.** Thread limits and HITL need a checkpointer · `ToolCallLimitMiddleware(exit_behavior="end")` is single-tool only (raises `NotImplementedError` otherwise) · `ShellToolMiddleware` default policy is **host access** · provider tool search raises `ValueError` on unsupported providers · version pins differ per feature (`1.3.2`, `1.3.14`, `1.3.16`, `1.4.6`).

CHAPTER 45

Custom Middleware

> **At a glance:** Two hook styles (**node-style** run sequentially at fixed points; **wrap-style** run *around* a call and decide whether/how it runs), two authoring styles (**decorators** for one hook; **`AgentMiddleware` classes** for several hooks, config, sync + async).

What / why / how

What. Your own hooks or classes that run at defined points of the agent loop.

Why. Cross-cutting behaviour that is specific to you: audit logs, dynamic prompts, guardrails, custom routing.

How. Choose a decorator (one hook) or a class (several hooks, config, sync + async); choose node-style (before/after) or wrap-style (around the call); return a state-update dict, a modified request, or `jump_to`.

Hook styles

Node-style hooks (logging, validation, state updates):

Hook	Runs
`before_agent`	Once per invocation, before the agent starts
`before_model`	Before **each** model call
`after_model`	After **each** model response
`after_agent`	Once per invocation, after the agent completes

Wrap-style hooks (retries, caching, transformation): `wrap_model_call` (around each model call) and `wrap_tool_call` (around each tool call). The handler can be called **zero times** (short-circuit), **once** (normal) or **many** (retry).

Convenience: `@dynamic_prompt` generates dynamic system prompts.

Execution order

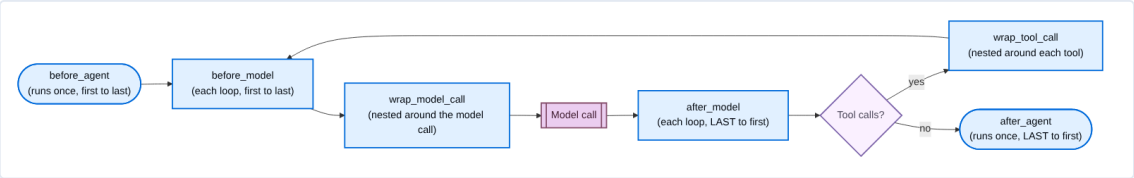


Figure 62. Hook order: before_agent, before_model, wrap_model_call, model, after_model, tools, after_agent.

Given `middleware=[m1, m2, m3]`:

Step	Order
`before_agent`	m1 → m2 → m3 (once)

Step	Order
`before_model`	m1 → m2 → m3 (each loop)
`wrap_model_call`	**nested**: m1 outermost → m2 → m3 → model
`after_model`	**m3 → m2 → m1** (reverse)
`after_agent`	**m3 → m2 → m1** (once, reverse)

Rules: `before_*` first→last · `after_*` last→first · `wrap_*` the first middleware wraps all others. **Put critical middleware first.**

Decorator-based

```python
from typing import Any, Callable
from langchain.agents.middleware import (before_model, wrap_model_call, wrap_tool_call, dynamic_prompt,
                                         AgentState, ModelRequest, ModelResponse)
from langchain.messages import AIMessage
from langgraph.runtime import Runtime

@before_model(can_jump_to=["end"])                      # early exit needs can_jump_to
def check_message_limit(state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
    if len(state["messages"]) >= 50:
        return {"messages": [AIMessage("Conversation limit reached.")], "jump_to": "end"}
    return None

@wrap_model_call
def retry_model(request: ModelRequest, handler: Callable[[ModelRequest], ModelResponse]) -> ModelResponse:
    for attempt in range(3):
        try:
            return handler(request)                     # call through (maybe more than once)
        except Exception as e:
            if attempt == 2:
                raise
            print(f"Retry {attempt + 1}/3 after error: {e}")

agent = create_agent(model="gpt-5.5", tools=[...], middleware=[check_message_limit, retry_model])
```

Decorators: `@before_agent`, `@before_model`, `@after_model`, `@after_agent`, `@wrap_model_call`, `@wrap_tool_call`, `@dynamic_prompt`. Node-style decorators accept `state_schema` and `can_jump_to`.

Use decorators for: single-hook middleware, minimal config, prototyping.

Class-based

```python
from langchain.agents.middleware import AgentMiddleware

class LoggingMiddleware(AgentMiddleware):
    def before_model(self, state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
        print(f"About to call model with {len(state['messages'])} messages")
        return None

    async def abefore_model(self, state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
        return None                                     # async twin: prefix the method with `a`
```

Use classes for: sync **and** async versions, multiple hooks in one middleware, init-time config (thresholds/models), reuse across projects. Class attributes read at compile time: `state_schema` (extend state) · `tools` (ship tools with the middleware) · `transformers` (scope-aware stream transformer factories, `langchain>=1.3.2`; each factory takes a scope tuple and returns a **fresh** transformer to keep subgraphs isolated). Where a jump is needed on class methods use `@hook_config(can_jump_to=[...])`.

Custom state

```python
from typing import NotRequired

class CustomState(AgentState):
    model_call_count: NotRequired[int]
    user_id: NotRequired[str]

@before_model(state_schema=CustomState)
def count_calls(state: CustomState, runtime: Runtime) -> dict[str, Any] | None:
    return {"model_call_count": state.get("model_call_count", 0) + 1}
```

Uses. Counters/flags, sharing data between `before_model` and `after_model`, rate limits, usage tracking, audit logs, conditional routing.

State updates. Node-style hooks **return a dict** merged by the graph's reducers. Wrap-style hooks return `ExtendedModelResponse` with a `Command` (model calls) or a `Command` (tool calls). Messages are additive via reducers.

Composition with several `ExtendedModelResponse`: each command is a separate update; for non-reducer fields **inner commands apply first and outer commands win**; retry-safe (commands from discarded handler calls are dropped).

Agent jumps (`jump_to`)

Return `"jump_to"` in the update dict; the hook must declare targets in `can_jump_to`. Targets: `'end'` (end, or the first `after_agent`), `'tools'` (tools node), `'model'` (model node, or the first `before_model`). Typical: `after_model` sees blocked content and jumps to `'end'` with a replacement message.

Practical patterns

```python
# Dynamic system prompt: rebuild the system message per call
from langchain.messages import SystemMessage

@wrap_model_call
def add_context(request: ModelRequest, handler):
    new = SystemMessage(content=[*request.system_message.content_blocks,
                                 {"type": "text", "text": "Today's focus: billing."}])
    return handler(request.override(system_message=new))

# Dynamic model selection
@wrap_model_call
def pick_model(request, handler):
    model = big_model if len(request.messages) > 10 else small_model
    return handler(request.override(model=model))

# Dynamic tool selection: filter request.tools (all tools must be registered up front)
@wrap_model_call
def only_read_tools(request, handler):
    return handler(request.override(tools=[t for t in request.tools if t.name.startswith("read_")]))

# Tool monitoring
@wrap_tool_call
def monitor(request, handler):
    print("calling", request.tool_call["name"], request.tool_call["args"])
    try:
        out = handler(request); print("ok"); return out
    except Exception as e:
        print("error", e); raise

# Anthropic prompt caching: append {"type": "text", ..., "cache_control": {"type": "ephemeral"}} to the
system message
```

Async hooks, tracing

- Class-based: define `abefore_model`, `aafter_model`, ... Define **both** sync and async when the agent may run either way.
- **Tracing** (`langchain>=1.3.15`): hook spans record inputs/outputs by default. `trace_policy = TracePolicy(process_inputs=omit_payload)` drops the input payload; `configure_trace_policy(...)` sets a global default; a middleware's own `trace_policy` overrides it.

Best practices

1. One focused task per middleware. 2. Handle errors so middleware failures don't crash the agent. 3. Node-style for sequential logic; wrap-style for control flow. 4. Document custom state fields. 5. Unit-test middleware before integration. 6. Order deliberately. 7. Prefer built-ins when they fit.

Deep Agents specifics (don't lose these points)

- Pass your middleware in `create_deep_agent(middleware=[...])`; it is inserted after `PatchToolCallsMiddleware`. A same-`.name` instance **replaces** the built-in in place (`deepagents>=0.7`).
- **Never mutate `self` attributes** in hooks (subagents, parallel tools and threads run concurrently). Use graph state: return `{"x": ...}` from `before_agent`.
- Subagent `middleware` does **not** inherit from the parent.

Advantages, limitations, gotchas (custom)

Advantages. Total control of the loop; testable units; reusable across projects.

Gotchas. The docs' class example uses `SystemMessage` without importing it and has stray `python` / `:::` rendering artifacts; `wrap_*` hooks may call the handler zero times (short-circuit) so don't assume it ran; `after_*` runs in **reverse** order; a `jump_to` without `can_jump_to` is invalid.

PART VIII

08

Frontend

Streaming agent state into a user interface.

> **About the code:** The documentation's frontend samples cover React, Vue, Svelte and Angular. The snippets in this part are **illustrative React/TypeScript** built on the API names those samples use (`useStream` , `stream.subagents` , selector helpers, `stream.values`). Exact imports differ by SDK version; check the frontend package you install.

IN THIS PART

46 Frontend: Overview

47 Frontend Patterns

CHAPTER 46

Frontend: Overview

> **At a glance:** A deep-agent UI should **make delegation visible**: not one opaque chat bubble, but *how* the task is split up and completed. The architecture is **coordinator → workers**; the client uses `useStream` with `apiUrl + assistantId`.

What / why / when

What. Frontend patterns for streaming a deep agent's work: coordinator messages, per-subagent progress, todos, tool-call cards, interrupts, sandbox files.

Why. Deep-agent runs are long and multi-step; users need to see progress, intervene, and trust results.

When. Any UI over `create_deep_agent`, especially with subagents or a sandbox.

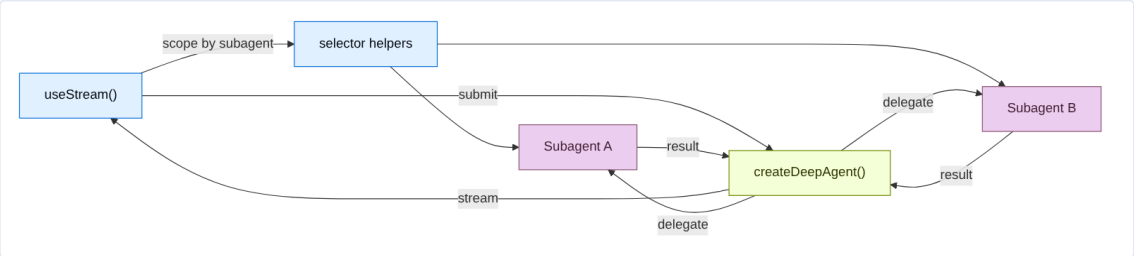


Figure 63. Frontend architecture: useStream scopes data per subagent via selector helpers; backend is createDeepAgent.

Architecture

- **Backend:** `createDeepAgent()` (Python: `create_deep_agent`) with tools, system prompt and subagents.
- **Frontend:** `useStream` connects the same way as with `createAgent` (`apiUrl`, `assistantId`); accepts a type parameter (`useStream<typeof agent>`) for type-safe state.
- The v1 stream handle sends **coordinator messages on the root stream** and provides **subagent discovery snapshots** for scoped views.
- Deep agents run on the same LangGraph runtime, so `useStream`'s core API is unchanged. SDK packages: React, Vue, Svelte, Angular (migration guides exist for earlier versions).

What the SDK exposes

Projection	Use
`stream.messages`	Coordinator conversation and final synthesis
`stream.subagents`	Specialist workers: status, metadata, discovery
`stream.values`	Custom state: todos, plans, report sections, sandbox metadata
Tool-call state	Cards for filesystem/search/browser/domain tools
Interrupts	Pause delegated work for approval or missing input

Projection	Use
Selector helpers (`useMessages(stream, subagent)`, `useToolCalls(stream, subagent)`)	Scope data to one subagent; subscribe only while mounted

```tsx
import { useStream } from "@langchain/react";          // package/import: confirm for your SDK version
import type { agent } from "./agent";

function Chat() {
  const stream = useStream<typeof agent>({
    apiUrl: "http://localhost:2024",                    // LangSmith Deployment URL in production
    assistantId: "agent",
  });

  return (
    <>
      {stream.messages.map((m) => <Message key={m.id} message={m} />)}
      <button onClick={() => stream.submit({ messages: [{ type: "human", content: "Research LangGraph" }]
})}>
        Send
      </button>
    </>
  );
}
```

State rendering principles

- Treat `stream.values` as **shared state** the agent writes and the UI reads.
- Render subagent views by filtering `stream.subagents` and using the selector helpers.
- Use tool-call state to show progress and results as cards.
- General LangChain patterns still apply (markdown messages, tool calling, HITL). For lower-level graph visualisation, LangGraph's frontend patterns map nodes/state keys to components.

Production wiring (from *Going to production*)

Set `apiUrl` / `assistantId`; configure **reconnection** so dropped connections don't lose progress; for deep workflows raise `recursionLimit` (Deep Agents defaults to **10,000**) and use `streamSubgraphs: true`.

Advantages / limitations

Advantages. One hook for messages + subagents + custom state; type-safe; framework-agnostic.

Limitations. Docs' samples are per-framework (syntax differs: Vue `stream.values.value`, Svelte `$derived`, Angular signals); subagent *content* needs selectors, not the snapshot.

CHAPTER 47

Frontend Patterns

Three documented patterns: **Subagent streaming cards**, **Todo list**, **Sandbox IDE**.

Pattern 1: Subagent streaming cards

> **At a glance:** Render coordinator messages separately from each specialist's streaming output. Link each card to the AI message that spawned it via the **tool-call ID**.

What. A coordinator spawns specialists (researcher, analyst, writer). Show each as a card with status and live output; mount cards only when users need them.

HOW THE DATA IS STRUCTURED

- `stream.messages` = **coordinator** messages only.
- `stream.subagents` = **discovery snapshots**: `id`, `name`, namespace, `status`, `output`. **No content.**
- **Selector helpers** (`useMessages`, `useToolCalls`) fetch a subagent's scoped messages/tool calls and subscribe **only while the card is mounted**.
- **Linking key:** the snapshot `id` **equals the tool-call ID that spawned it**. Index subagents by that ID and render each card under the AI message whose `tool_calls` contains it.

```tsx
function SubagentCard({ stream, subagent }) {
  const messages = useMessages(stream, subagent);       // scoped to this subagent; subscribes while
mounted
  const toolCalls = useToolCalls(stream, subagent);
  const lastAi = [...messages].reverse().find((m) => m.type === "ai");
  return (
    <details open={subagent.status !== "completed"}>   // collapsible
      <summary>
        {subagent.name} <Badge status={subagent.status} /> · {toolCalls.length} tool calls
      </summary>
      <p className={subagent.status === "running" ? "pulse" : ""}>
        {lastAi?.text ?? subagent.output}               {/* fallback to snapshot output */}
      </p>
    </details>
  );
}

// In the message list: attach cards to the turn that spawned them
const byToolCallId = Object.fromEntries(stream.subagents.map((s) => [s.id, s]));
{message.tool_calls?.map((tc) => byToolCallId[tc.id] && <SubagentCard stream={stream} subagent=
{byToolCallId[tc.id]} />)}
```

UX details. Name + status badge + tool-call count; latest AI text (fallback `subagent.output`); pulse while running; collapsible; **auto-collapse completed cards when ≥ 5**; overall progress bar (completed / total).

Best practices. Mount selectors only where needed; handle errors **per subagent**; override `recursion_limit` only for unusually deep workflows.

GOTCHAS

- Snapshots lack content: don't look for messages on them.

- **ID mismatches hide cards.**
- The docs' progress bar counts across **all turns**; scope per turn for per-turn progress.
- Example cards start expanded; consider collapsing in busy runs.

Pattern 2: Todo list

> **At a glance:** Opt in with `TodoListMiddleware` on the backend; read `stream.values.todos` on the frontend. Updates are reactive: no polling.

What / why. Show the agent's plan and live progress.

HOW (BACKEND)

PYTHON

```python
from langchain.agents.middleware import TodoListMiddleware
agent = create_deep_agent(model=..., middleware=[TodoListMiddleware()])   # adds write_todos + persists
todos in state
```

HOW (FRONTEND)

TSX

```tsx
const STATUS = {
  pending:     { icon: "○", cls: "text-gray-500" },
  in_progress: { icon: "●", cls: "text-amber-500 animate-pulse" },
  completed:   { icon: "✓", cls: "text-green-600 line-through" },
} as const;

function TodoList({ stream }) {
  const todos = stream.values?.todos ?? [];
  if (todos.length === 0) return null;                    // empty state: hide or show "creating a
plan..."
  const done = todos.filter((t) => t.status === "completed").length;
  const pct = Math.round((done / todos.length) * 100);
  return (
    <section>
      <h4>{done}/{todos.length} · {pct}%</h4><progress value={pct} max={100} />
      {todos.map((t) => <div key={t.id ?? t.content} className={STATUS[t.status].cls}>
{STATUS[t.status].icon} {t.content}</div>)}
    </section>
  );
}
```

Each todo has `content` and `status` ∈ `pending` (hollow, gray) · `in_progress` (filled, amber, pulsing) · `completed` (check, green, strikethrough).

Flow. User submits → agent writes the plan into `todos` → each status change updates state → UI follows. **Layout:** panel above/beside chat; messages below.

GOTCHAS

- ⚠️ **Opt-in is required.** Without `TodoListMiddleware`, `stream.values.todos` is simply absent (examples default to `[]`, hiding the bug).
- Handle empty states; **pulse only one** `in_progress` item; don't poll; use **stable keys** (the docs key by array index, which is fragile if items reorder).
- Framework syntax differs (Vue `stream.values.value`, Svelte `$derived`, Angular signals).

Pattern 3: Sandbox IDE

> **At a glance:** Coding agents need more than chat. Pair a **sandbox-backed deep agent** with a **custom FastAPI file-browsing API** and a **three-panel UI** (file tree · code/diff viewer · chat).

ARCHITECTURE (THREE PARTS)

1. **Deep agent with a sandbox backend** – gets `read_file`, `write_file`, `edit_file`, `delete`, `execute` automatically.
2. **Custom API server:** a FastAPI app registered via the `http.app` field in `langgraph.json`, exposing file-browsing routes.
3. **Frontend:** file tree + code/diff viewer + chat. (No dedicated terminal panel; shell-like behaviour comes from the agent's `execute` tool.)

Sandbox lifecycle. Thread-scoped by default. The sandbox ID lives in **thread metadata**, the *single source of truth* (no in-memory caches); a shared helper (`get_or_create_sandbox_for_thread`) lets the agent and API server resolve the same sandbox. The frontend stores the thread ID in `sessionStorage` so reloads reconnect. Upload starter files before the first run; after `package.json`, run an install command.

API ROUTES (ILLUSTRATIVE)

```
GET /sandbox/{thread_id}/tree?filePath=/app     -> entries (name, type, path, size) + sandbox id   (filter
out node_modules)
GET /sandbox/{thread_id}/file?filePath=...      -> file contents
```

JSON

```
{ "dependencies": ["."], "graphs": {"agent": "./agent.py:agent"}, "http": {"app": "./api.py:app"} }
```

Keeping the UI in sync. Watch stream `ToolMessage`s:

Tool finished	UI action
`write_file` / `edit_file`	Refresh that one file
`execute`	Refresh the **whole tree** (a shell command can change anything)

Change detection. Snapshot file contents *before* a run, compare after → a **changed-files list** (like `git status`) with +/- counts; open the **diff first** on click. Diff libraries: `@pierre/diffs` (React), `@git-diff-view` (Vue/Svelte), `ngx-diff` (Angular).

Best practices. Persist `threadId`; sync files on every relevant tool call, not only at run end; compact one-line results for read-only tools, full output for mutating ones; **keep API keys out of the sandbox** (auth proxy); add rate limits, error handling and privacy middleware before launch.

Advantages / limitations. Real "see what the agent did" UX with cheap diffs; needs a custom API + sandbox lifecycle management; sandbox cost until TTL.

Frontend cheat sheet

Want to show	Use
Coordinator chat	`stream.messages`
Specialist cards	`stream.subagents` + selector helpers; link by tool-call ID

Want to show	Use
Plan/progress	`TodoListMiddleware` → `stream.values.todos`
Files the agent touched	Sandbox backend + custom API + `ToolMessage` watching
Approvals	`stream` interrupts (Part VI)
Protocol-level client	AG-UI (Part IX)

PART IX

09

Protocols

How the agent talks to editors, other agents, tool servers and user interfaces.

> **The question you'll be asked:** *"MCP vs ACP vs A2A vs AG-UI: who talks to whom?"*

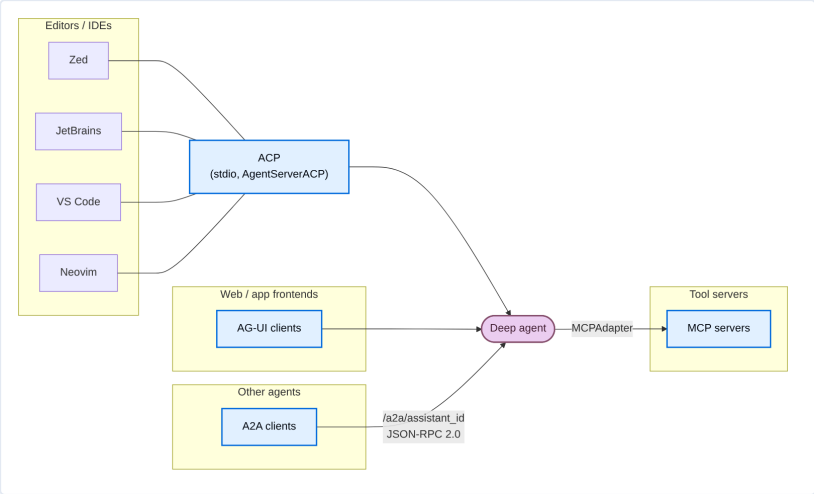


Figure 64. Protocol map: editors use ACP, frontends use AG-UI, other agents use A2A, the agent calls tool servers over MCP.

Protocol	Connects	Use it for	Transport / form
MCP (Model Context Protocol)	Agent ⇄ **tools/data** on external servers	Databases, APIs, filesystems, browsers as tools	stdio / streamable HTTP / in-memory
ACP (Agent Client Protocol)	**Coding agent ⇄ code editor/IDE**	Zed, JetBrains, VS Code (community), Neovim, Toad	stdio (editor launches your script)
A2A (Agent2Agent)	**Agent ⇄ other agents** (cross-framework)	Multi-agent interop (e.g. LangGraph ⇄ Google ADK)	JSON-RPC 2.0 over HTTP (`/a2a/{assistant_id}`)
AG-UI (Agent User Interaction)	**Agent ⇄ user-facing app**	Event-streamed agent UIs	Typed event stream

IN THIS PART

48 ACP: Agent Client Protocol

49 MCP with LangChain

50 A2A with LangSmith

51 AG-UI: Agent User Interaction

CHAPTER 48

ACP: Agent Client Protocol

> **At a glance:** ACP standardises **coding agent ↔ editor/IDE** communication: the editor supplies project context, the agent returns rich updates. Wrap your deep agent in `AgentServerACP` and run it over **stdio**. *Editors → ACP. External tools → MCP.*

What / why / when

What. The Agent Client Protocol: a standard so any ACP editor can host any ACP agent.

Why. One agent implementation works in Zed, JetBrains, VS Code (via a community project) and Neovim plugins without per-editor glue.

When. You want your Deep Agent inside an editor/IDE.

How

```bash
pip install deepagents-acp          # or: uv add deepagents-acp
```

```python
import asyncio
from acp import run_agent
from deepagents import create_deep_agent
from deepagents_acp.server import AgentServerACP
from langgraph.checkpoint.memory import MemorySaver

async def main():
    agent = create_deep_agent(
        model="anthropic:claude-sonnet-4-6",         # provider:model; examples show
Google/OpenAI/Anthropic/OpenRouter/Fireworks/Baseten/Ollama
        system_prompt="You are a coding assistant.",
        # tools=[...], middleware=[...], subagents=[...] all customisable
        checkpointer=MemorySaver(),                  # examples pass a checkpointer
    )
    server = AgentServerACP(agent)
    await run_agent(server)                          # stdio mode: reads stdin, writes stdout

if __name__ == "__main__":
    asyncio.run(main())
```

- **stdio mode:** the *editor launches this script*; you rarely run it by hand.
- The package ships an **example coding agent** with filesystem and shell access (a demo).

Supported clients and config

Client	Notes
Zed	Clone the `deepagents` repo → `uv sync --all-groups` in `libs/acp` → `chmod +x run_demo_agent.sh` → copy `.env.example` to `.env` and set `ANTHROPIC_API_KEY` → add an `agent_servers` entry in Zed `settings.json` with `"type": "custom"` and the script's **absolute** path in `command` → open the Agents panel

Client	Notes
JetBrains IDEs	ACP support
VS Code	Via the community **vscode-acp** project
Neovim	Via ACP-compatible plugins
Toad	Local dev tool: `uv tool install -U batrachian-toad`, then `toad acp "python path/to/your_server.py"`.

```json
{ "agent_servers": { "DeepAgents": { "type": "custom", "command": "/absolute/path/to/run_demo_agent.sh" } }
}
```

Advantages, limitations, gotchas

Advantages. Editor-agnostic; reuses your whole agent (tools, middleware, subagents); checkpointed sessions.

Gotchas. Zed's `command` must be an **absolute path** · the demo needs an API key in `.env` · stdio belongs to the client (no interactive terminal) · ACP is for **editors only**; use MCP for external tool servers.

CHAPTER 49

MCP with LangChain

At a glance: `langchain.mcp` (**beta**, `langchain[mcp]>=1.4.0`, built on **FastMCP**) turns an MCP server's tools into LangChain tools via `MCPAdapter`. The *target you pass* determines the transport. Importing emits a one-time `LangChainBetaWarning`. It replaces the older `langchain-mcp-adapters` (migration guide exists).

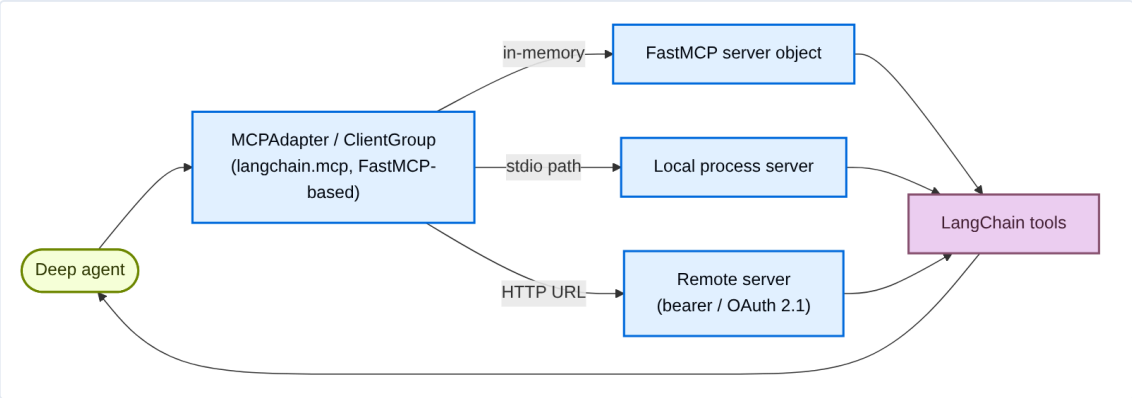


Figure 65. MCPAdapter picks a transport from the target, discovers tools, hands LangChain tools to the agent.

What / why / when

What. MCP is an open standard for giving applications' tools and context to LLMs. `MCPAdapter` discovers a server's tools and converts them into LangChain tools you pass to `create_agent` / `create_deep_agent`.

Why. Reuse existing tool servers (including the public docs server) without custom wrappers; same interface for every transport; FastMCP handles transport inference, protocol negotiation, connection management and auth.

When. In-memory for tests; stdio for local scripts; HTTP for remote services.

How to use it (quickstart)

BASH
```
pip install "langchain[mcp]"      # or: uv add "langchain[mcp]"
```

PYTHON
```
from langchain.agents import create_agent
from langchain.mcp import MCPAdapter

async def main():
    async with MCPAdapter("https://example.com/mcp") as adapter:
        tools = await adapter.list_tools()
        agent = create_agent("claude-sonnet-5", tools)
        return await agent.ainvoke({"messages": [{"role": "user", "content": "..."}]})
```

The adapter is an **async context manager**; tools keep a reference to the client, so the agent still works after the context exits. The public **LangChain docs MCP server** (`https://docs.langchain.com/mcp`, no API key) exposes three tools: docs search, a virtual-filesystem tool for reading/searching docs, and feedback submission.

Transports (inferred from the target)

Target you pass	Transport
In-process `FastMCP` instance	**In memory** (no subprocess/socket); great for tests
`Path` to a script	**stdio** (launches a subprocess, one per adapter)
`str` `http(s)` URL	**Streamable HTTP**

Also accepted: a preconfigured `StreamableTransport`; an `MCPConfig` dict (`{"mcpServers": {...}}`) for several servers; a prebuilt `fastmcp.Client` (control over transport, caching, negotiation).

Security caveat. A `str` must be an http/https URL. FastMCP checks "is it a file path?" *before* "is it a URL?", so a string naming an existing `.py` / `.js` would **launch a subprocess**. `MCPAdapter` rejects non-URL strings: important when targets come from config or model output.

Tools: outputs, errors, metadata

- **Multimodal:** MCP content → LangChain content blocks (`text`, `image`, `file`), readable via `message.content_blocks`.
- **Structured content** goes into the message `artifact` (an `MCPToolArtifact` with `structured_content`), *not* the model-visible text; `None` if absent.
- **Errors:** server-reported `isError=True` → a `ToolMessage` with `status="error"` (the model can retry). **Transport/session failures raise exceptions** (the model can't recover).
- **Metadata** under `metadata["mcp"]`: `tool.annotations` (hints like `read_only_hint`, `destructive_hint`), `tool._meta` (opaque), `server` (name/version). All optional: read defensively with chained `.get()`.

Human-in-the-loop gating from MCP annotations

Gate tools by what the server *declares* instead of hard-coding names: read the destructive hint once at discovery, then pass a `when` predicate to `InterruptOnConfig` (it receives each `ToolCallRequest`, so it can also inspect `request.tool_call["args"]`).

```python
from langgraph.types import Command

# read the server-declared hint once, defensively (every field is optional)
destructive = {
    t.name for t in tools
    if ((t.metadata or {}).get("mcp", {}).get("annotations") or {}).get("destructive_hint")
}
interrupt_on = {
    name: {
        "allowed_decisions": ["approve", "reject"],
        "when": lambda req: req.tool_call["args"].get("dry_run") is not True,   # safe inputs run freely,
risky ones pause
    }
    for name in destructive
}
agent = create_deep_agent(model=..., tools=tools, interrupt_on=interrupt_on, checkpointer=checkpointer)
# approve a paused call
agent.invoke(Command(resume={"decisions": [{"type": "approve"}]}), config)
```

Elicitation (server asks the user for input)

- **On by default.** The adapter advertises the capability and converts the server's request into a LangGraph `interrupt`. Needs a checkpointer and the **same** `thread_id` on resume.

```python
[interrupt] = paused["__interrupt__"]
[question] = interrupt.value["requests"]
answer = {"action": "accept", "content": {"date": "2026-09-14"}}      # or "decline" / "cancel"
result = await agent.ainvoke(Command(resume={"responses": {question["key"]: answer}}), config)
```

- **Caveats:** resuming **re-runs the tool from the start** (make pre-request work idempotent); only servers returning an `InputRequiredResult` can be answered this way (legacy-handshake push elicitation can't); **sampling and roots requests raise errors** (the sessionless protocol has no back-channel).

Connections: lifecycle, multiple servers, scaling

Choose two independent things: server shape and connection lifetime.

Dimension	Option	When
Server shape	Single target	One URL / script / in-process server
	`MCPConfig` dict	Several servers behind **one aggregate connection**
	`ClientGroup`	Several servers, each with its own auth, protocol era or pool
Lifetime	Discover inside `async with`, then exit	Scripts/notebooks/most agents (**default**)
	Hold the adapter open around the agent call	One session across several tool calls
	Per-run discovery + shared pool + cache	Many concurrent runs in a deployment

- **Per-invocation sessions:** by default each tool call opens the client, does the MCP call, releases it (one session per call). Tools are **reentrant** (reuse a held connection). Hold the adapter open to keep a session.
- **`MCPConfig` (aggregate):** FastMCP **prefixes every tool with its config key** (`weather_...`, `calc_...`), mixed transports allowed, **but the whole fleet shares one protocol era**: adding a legacy-only server drops everyone to legacy.

```python
CONFIG = {"mcpServers": {
    "weather": {"command": "python", "args": ["/path/to/weather_server.py"]},
    "calc":    {"command": "python", "args": ["/path/to/calc_server.py"]},
}}
async with MCPAdapter(CONFIG) as adapter:
    tools = await adapter.list_tools()
```

- **`ClientGroup` (independent):** each member keeps its own era, auth, handlers; calls route to the client that advertised the tool; tools namespaced `{server}_{tool}`.

```python
from langchain.mcp import MCPAdapter, ClientGroup
from fastmcp import Client

group = ClientGroup({"weather": Client(legacy_url, mode="legacy"), "calc": Client(modern_url,
mode="auto")})
async with MCPAdapter(group) as adapter:
    tools = await adapter.list_tools()
```

- **Scaling a deployment:** build the agent in a `langgraph dev` **graph factory** (runs once per run → fresh tool catalog); use `cache_mode="use"`; share one HTTP pool via an `httpx_client_factory` whose `aclose()` is a no-op so no client can close it. ⚠️ *Gotcha:* annotation/return types must be importable **at runtime** (not only under `TYPE_CHECKING`): `langgraph-api` uses `typing.get_type_hints()`, and an unresolved annotation makes it inject a config dict instead of the runtime object.

Caching. Opt-in; honours server cache hints (only helps **modern-era** servers). `list_tools(cache_mode=...)`: `use` (default; serve from cache within TTL), `refresh` (fetch fresh + repopulate), `bypass` (skip). Configure the cache on the client (`Client(cache=...)`).

Protocol eras. Legacy: every connection starts with an `initialize` handshake. **Modern:** version `2026-07-28`+; client discovers support by probing `server/discover`. `mode="legacy"` pins the handshake era; `mode="auto"` (default) negotiates the newest. To mix eras, use `ClientGroup` or one adapter per server.

Authentication

`MCPAdapter` delegates to FastMCP: **anything a `fastmcp.Client` accepts as `auth` works** (bearer string, the literal `"oauth"`, any `httpx.Auth`). Build a prebuilt client and pass it in.

Method	Credential source	Interactive step	Best for
Bearer token	Pre-provisioned	None	Simple, known token
OAuth 2.1	Server-issued via flow	Browser (unless persisted)	Servers that issue their own credentials
Per-server	Mixed, per server	Per server	Several servers, separate auth
Per-user	Per caller identity	Depends on deployment	Multi-tenant deployments

PYTHON

```python
# Bearer
async with MCPAdapter(Client(url, auth=token)) as adapter: ...

# OAuth 2.1: discovery + dynamic client registration + browser redirect + token exchange
async with MCPAdapter(Client(url, auth="oauth")) as adapter: ...
# persist tokens so you don't repeat the browser step
oauth = OAuth(mcp_url="https://example.com/mcp", token_storage=...)
client = Client("https://example.com/mcp", auth=oauth)

# Per-server (mix auth types)
group = ClientGroup({"billing": Client(billing_url, auth="oauth"), "docs": Client(docs_url,
auth=docs_token)})

# Per-user (inside a graph factory: authenticate caller at the LangGraph server, then mint/exchange a per-
user token)
user = runtime.user.identity if runtime.user is not None else "anonymous"
auth = BearerAuth(token_for(user))                        # token_for = your exchange (OAuth gateway,
FastMCP provider, ...)
async with MCPAdapter(Client(CONFIG, auth=auth)) as adapter:
    tools = await adapter.list_tools()
```

Gotchas. Default OAuth keeps tokens **in memory** (browser step every run). **Isolate cached responses by verified identity** or one user could see another's cached tool list. FastMCP ships providers for Auth0, WorkOS, Okta and others.

Advantages, limitations

Advantages. One interface across transports; standard protocol; FastMCP handles negotiation/auth; tools usable after the context closes; clean split of model-visible content vs artifacts vs errors.

Limitations. Beta (warning on import); `langchain[mcp]>=1.4.0`; string targets must be http(s); pre-1.4.0 code needs migration; `MCPConfig` forces the oldest era; caching only for modern-era servers; sampling/roots unsupported; a held-open adapter is needed if you want a single session.

CHAPTER 50

A2A with LangSmith

> **At a glance:** **A2A (Agent2Agent)**, a Google-developed protocol for cross-framework agent communication. **LangSmith Agent Server** serves every assistant at `/a2a/{assistant_id}` using **JSON-RPC 2.0** (A2A **v1.0** method names; four v0.3 names still accepted). Each assistant gets an auto-generated **Agent Card**.

What / why / when

What. A standard way for agents from different frameworks to exchange messages and tasks.

Why. Interoperate with other A2A agents without custom glue (e.g. a LangGraph agent talking to a Google ADK agent), with unified tracing.

When. Multi-agent workflows spanning frameworks/teams. (For tools use MCP; for editors ACP; for UIs AG-UI.)

Endpoints and methods

Purpose	Endpoint
JSON-RPC calls	`POST /a2a/{assistant_id}`
Agent Card	`GET /.well-known/agent-card.json?assistant_id={assistant_id}`
Agent Card (per-assistant path)	`GET /a2a/{assistant_id}/.well-known/agent-card.json`

If the card request omits `assistant_id`, `DEFAULT_A2A_ASSISTANT_ID` supplies one. If card URLs show an internal host, set `LANGGRAPH_API_URL` to the public URL. Only JSON-RPC binding is implemented (**no gRPC, no HTTP+JSON**).

v1.0 method	v0.3 name	Status
`SendMessage`	`message/send`	Supported (blocking by default)
`SendStreamingMessage`	`message/stream`	Supported (SSE)
`GetTask`	`tasks/get`	Supported
`CancelTask`	`tasks/cancel`	Supported
`ListTasks`	none	Supported
`GetExtendedAgentCard`	none	Supported (v1.0 name only)
`SubscribeToTask`, `*TaskPushNotificationConfig`	none	**Not yet** (`-32601`)

Any other v0.3 name (incl. `tasks/resubscribe`) → `-32601 Method not found`.

Sending messages

```bash
curl -X POST "https://your-deployment/a2a/{assistant_id}" \
  -H "Content-Type: application/json" -H "Accept: application/json" \
  -d '{
    "jsonrpc": "2.0", "id": 1, "method": "SendMessage",
```

```
    "params": { "message": { "role": "ROLE_USER", "parts": [{"text": "hello"}], "messageId": "test-1" } }
  }'
# response: result.task.id and result.task.contextId  → reuse contextId to continue the conversation

curl "https://your-deployment/a2a/{assistant_id}/.well-known/agent-card.json"     # fetch the Agent Card
```

Return immediately (long tasks): set `returnImmediately: true` in `configuration`; you get the task back as `TASK_STATE_SUBMITTED` / `TASK_STATE_WORKING`, then poll `GetTask`. (v0.3 equivalent: `"blocking": false`; `returnImmediately` wins if both are sent.)

JSON

```
{ "jsonrpc": "2.0", "id": "1", "method": "SendMessage",
  "params": { "message": {"role": "ROLE_USER", "parts": [{"text": "hello"}], "messageId": "message-1"},
              "configuration": {"returnImmediately": true} } }
```

Task-scoped history: `"configuration": {"historyScope": "task"}` (also header `LangGraph-A2A-History-Scope: task`; `GetTask` / `ListTasks` read it from `params`). Streaming ignores `historyScope` and `historyLength`; `historyLength` max 10.

Identifiers, formats, interrupts

- **`contextId`** groups messages into one conversation (session ID; must be a UUID). **`taskId`** identifies one request within a context. On the first message omit both; on later turns send only `contextId`. **Don't resend a `taskId` from a completed task**: a terminal task is rejected with `-32004`.
- **Enum case follows the method family:** v1.0 names return `SCREAMING_SNAKE_CASE` (`TASK_STATE_WORKING`, `ROLE_AGENT`); v0.3 names return lowercase. Use **one family per client**.
- **Envelope:** `SendMessage` → `result.task`; `GetTask` and v0.3 methods → task directly on `result`; `ListTasks` → `result.tasks`.
- **Interrupts → human input:** when the graph calls `interrupt()`, the task enters **`TASK_STATE_INPUT_REQUIRED`** with an artifact named `Interrupt`. Resume by sending a message with the task's `taskId` + `contextId` and a `DataPart` with a **`resume`** key (only one) or plain text.
- **File parts:** inbound and outbound `FilePart` supported. **A2UI v0.9** off by default (`A2A_A2UI_ENABLED=true` or `metadata.a2a.a2ui`; client sends `A2A-Extensions`).
- `A2A_ALLOWED_TOOL_CALL_RESULTS` limits which tool results are published. The graph state **must include a `messages` key**.

Distributed tracing

The server maps A2A **`contextId`** → **LangGraph `thread_id`**, so all turns across agents land in **one LangSmith thread** (use the same LangSmith project across agents). Clients **cannot** set `thread_id` directly. Non-LangGraph agents read `params.message.contextId` and set the `langsmith.metadata.thread_id` span attribute via OpenTelemetry:

PYTHON

```python
thread_id = body["params"]["message"].get("contextId")
with tracer.start_as_current_span("agent") as span:
    if thread_id:
        span.set_attribute("langsmith.metadata.thread_id", thread_id)
```

Agent-to-agent pattern and disabling A2A

1. Build a `message` (`role`, `parts`, `messageId`). 2. On follow-ups add the returned `contextId`. 3. POST `message/send` JSON-RPC to each agent's `/a2a/{assistant_id}`. 4. Read `result.contextId` and pass it to the next agent. Disable the endpoint:

```json
{ "$schema": "https://langgra.ph/schema.json", "http": { "disable_a2a": true } }
```

Version requirements

Feature	Minimum
A2A endpoint	`langgraph-api >= 0.4.21`
Inbound `FilePart`	0.12.0
A2UI, `historyScope`	0.15.0
`response_schema` in interrupts	0.16.0
`returnImmediately` / `blocking`	0.18.0

Advantages, limitations

Advantages. Cross-framework interop; unified tracing per conversation; multi-turn continuity; long-running via `returnImmediately`; HITL via interrupts; discoverable via Agent Card; conformance-testable (A2A TCK).

Limitations (current gaps). Wire shape is still partly **v0.3** (`kind`, `mimeType`) · streaming events are flat with `final` (no `statusUpdate` / `artifactUpdate` wrappers) · `tool_results` snake_case · `+00:00` timestamps not `Z` · `SubscribeToTask` and push-notification config **not implemented** · errors carry no `data` · `A2A-Version` header not read · no agent-card caching headers · `GetExtendedAgentCard` served but not advertised · `historyScope` / `historyLength` ignored in streaming.

CHAPTER 51

AG-UI: Agent User Interaction

> **At a glance:** **AG-UI** (Agent User Interaction Protocol) is an open, lightweight, **event-based** protocol connecting agents to **user-facing apps**. Serve the deep agent from a LangGraph server and wrap it with the TypeScript `@ag-ui/langgraph` adapter; any AG-UI client gets a typed event stream without knowing LangGraph internals.

What / why / when

What. Differs from ACP (editors), MCP (tools), A2A (agents): AG-UI is **agent ⇄ user**.

Why. A standard event vocabulary for text, tools, reasoning, state and interrupts, so frontends are portable.

When. You want a protocol-level UI contract (e.g. with CopilotKit) rather than the bespoke `useStream` path.

How to set it up

BASH
```
pip install deepagents "langgraph-cli[inmem]"
```

PYTHON
```
# agent.py
from deepagents import create_deep_agent
agent = create_deep_agent(model="anthropic:claude-sonnet-4-6", system_prompt="...")
```

JSON
```
{ "dependencies": ["."], "graphs": { "agent": "./agent.py:agent" }, "env": ".env" }
```

BASH
```
langgraph dev          # serves at http://localhost:2024
```

TS
```
import { LangGraphAgent } from "@ag-ui/langgraph";
const agent = new LangGraphAgent({
  graphId: "agent",                          // must match a key in langgraph.json
  deploymentUrl: "http://localhost:2024",    // change for non-local environments
  // emitInterruptOutcome: true,             // structured interrupt results on RUN_FINISHED
});
agent.subscribe({ onTextMessageContentEvent: ({ event }) => console.log(event.delta) });   // on…Event
callbacks (illustrative)
await agent.runAgent({ /* input */ });
```

Event types

Group	Events
Run lifecycle	`RUN_STARTED`, `RUN_FINISHED`, `RUN_ERROR`
Graph progress	`STEP_STARTED`, `STEP_FINISHED`
Assistant text	`TEXT_MESSAGE_START`, `TEXT_MESSAGE_CONTENT`, `TEXT_MESSAGE_END`
Tool calls	`TOOL_CALL_START`, `TOOL_CALL_ARGS`, `TOOL_CALL_END`, `TOOL_CALL_RESULT`

Group	Events
Reasoning	`REASONING_START`, `REASONING_MESSAGE_CONTENT`, `REASONING_END`
Shared state	`STATE_SNAPSHOT` (full baseline) + `STATE_DELTA` (JSON Patch, RFC 6902)
History	`MESSAGES_SNAPSHOT`

Subscribers get these via matching `on…Event` callbacks.

Configuration and methods

- `graphId`, `deploymentUrl`, `initialMessages` (or `addMessage` / `setMessages`).
- **`emitInterruptOutcome: true`** — structured interrupt results on `RUN_FINISHED`; **resume** by sending a `resume` array on the next input, each entry with `interruptId`, `status`, `payload`.
- Methods: **`runAgent`** (stream to a subscriber), **`subscribe`** (persistent subscriber across runs), **`abortRun`**.

Advantages, limitations, gotchas

Advantages. Portable event contract; typed events; efficient state sync (snapshot + deltas); HITL-capable.

Limitations. The adapter is **TypeScript**; Python frontends use CopilotKit or an AG-UI FastAPI bridge.

Gotchas. Default emits a *legacy* `on_interrupt` custom event (set `emitInterruptOutcome`) · `graphId` mismatch fails to load · rely on the initial snapshot + `STATE_DELTA`, not full state each step · examples hard-code port 2024 · resume needs the original `interruptId`.

Protocol decision cheat sheet

I want to…	Use
Give the agent a DB/API/browser	**MCP**
Use my agent inside Zed/JetBrains/Neovim	**ACP**
Let a Google ADK (or any) agent call my agent	**A2A**
Build a web UI with a standard event contract	**AG-UI** (or `useStream`, see Part VIII)

PART X

10

OpenWiki

An open-source CLI that writes and maintains a Markdown wiki of a codebase or of personal knowledge, for coding agents to read.

IN THIS PART

52 OpenWiki: Overview and Quickstart

53 OpenWiki: Code Mode in Depth

54 OpenWiki: Personal Mode

55 OpenWiki: Coding-Agent Integrations, Search and Workspaces

56 OpenWiki: Providers, Configuration and Telemetry

57 OpenWiki: Visualizing and Automating Updates

CHAPTER 52

OpenWiki: Overview and Quickstart

> **At a glance:** OpenWiki is an open-source CLI, built on Deep Agents, that **writes and maintains a Markdown wiki** about a codebase (or about your own knowledge), mainly *for coding agents* to read instead of rediscovering the repository on every task. Humans can read the same files.

What, why and when

What. A single `openwiki` binary that generates architecture, integration, operations and workflow pages as plain Markdown, grounded in versioned source evidence, and keeps them up to date.

Why. Coding agents waste tokens re-reading a repository each session. A durable wiki gives them context first, so they inspect source only when needed. Plain Markdown is diffable, versionable and reviewable.

When. Repositories your agents touch repeatedly; teams who want agent-readable documentation that refreshes in CI; and personal knowledge bases built from your own tools.

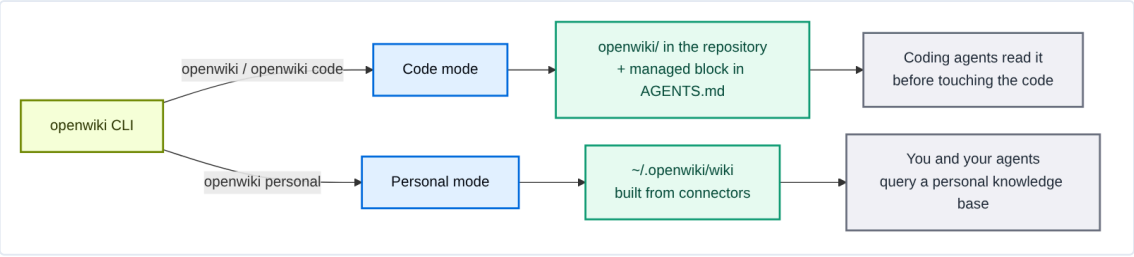


Figure 66. OpenWiki's two modes and where their output lives.

Mode	Command	Output	Use when
Code (default)	`openwiki`, `openwiki code`	`openwiki/` in the current repository	Repo documentation for coding agents
Personal	`openwiki personal`	`~/.openwiki/wiki`	A local knowledge base from configured sources

Bare `openwiki`, `--init` and `--update` all run in code mode.

Quickstart

Prerequisites. Node.js **22.22.0 or later**; model-provider credentials (collected during onboarding); optionally a LangSmith API key for tracing. On Windows prefer `npm` or `pnpm`: Bun can fall back to compiling the native `better-sqlite3` dependency, which may need Visual Studio Build Tools with the "Desktop development with C++" workload.

```bash
npm install -g openwiki
openwiki --init            # run from the repository root
openwiki visualize         # explore the result as a node graph with a Markdown reader
openwiki --update          # refresh after code changes
openwiki personal --init   # optional: personal wiki in ~/.openwiki/wiki
```

On the first interactive `--init`, OpenWiki asks for an inference provider and model, the provider API key (or equivalent credentials), an optional LangSmith API key, and optionally which LangSmith projects should enrich the wiki from runtime traces. Configuration and secrets are saved to `~/.openwiki/.env`.

WAYS TO RUN IT

```bash
openwiki                                  # interactive code-mode session
openwiki "Please generate documentation for this repository"
openwiki -p "Summarize what you can do"    # non-interactive: print the final answer and exit
openwiki "Update openwiki/INSTRUCTIONS.md to focus on the public API"
```

Chat slash commands include `/api-key`, `/langsmith-key` and `/effort`.

Tracing OpenWiki itself (optional) with `LANGSMITH_API_KEY`, `LANGCHAIN_TRACING_V2=true` and `LANGCHAIN_PROJECT=openwiki` in `~/.openwiki/.env` or the environment. This is separate from the *LangSmith connector*, which enriches the wiki using traces from *your* projects; do not confuse the two.

What gets created

Location	Contents
`openwiki/`	Quickstart entry point and topic pages
`openwiki/.claims/`	Claims that ground factual pages
`openwiki/INSTRUCTIONS.md`	Repo-specific scope and priorities (yours to edit)
`openwiki/.run.json`	Run state used to resume interrupted runs
`AGENTS.md` (repo root)	Created or refreshed with a block telling coding agents to consult the wiki
`CLAUDE.md` (if it exists)	A managed block importing `@AGENTS.md` is refreshed
`~/.openwiki/.env`	Saved configuration and secrets

Advantages, limitations and edge cases

Advantages. A reusable context layer; fewer tokens per agent task; plain Markdown; CI-refreshable; works with many providers and coding agents.

Limitations. Wiki quality depends on the model; the wiki goes stale unless you re-run it or automate updates; personal mode depends on each connector's access setup.

> **Edge cases:** re-running `--init` **regenerates the repository wiki and Claims from scratch** but preserves `openwiki/INSTRUCTIONS.md`. `--update` never rewrites `INSTRUCTIONS.md`; to change the brief, edit the file or ask OpenWiki in chat. `--init` and `--update` cannot be combined. An interrupted run on a persistent checkout resumes from `openwiki/.run.json`.

CHAPTER 53

OpenWiki: Code Mode in Depth

> **At a glance:** Code mode treats the wiki as *agent context*, not a human website. Factual pages are backed by **Claims** that cite exact repository evidence, updates reconcile only the Claims whose evidence changed, and runs are resumable page by page.

Commands

```bash
openwiki --init
openwiki --update
openwiki code --update --print          # non-interactive, e.g. for CI
openwiki --init --language ko           # BCP-47 locales such as ko, zh-CN, pt-BR
OPENWIKI_PAGE_CONCURRENCY=4 openwiki --update
```

Directory layout

```
repo/
├── .openwikiignore            # your ignore rules
├── AGENTS.md                  # created if missing; managed OPENWIKI block
├── CLAUDE.md                  # refreshed only if it already exists; imports @AGENTS.md
└── openwiki/
    ├── *.md                   # generated pages (quickstart, architecture, operations, ...)
    ├── index.md, log.md       # reserved OKF scaffolding, not concepts
    ├── INSTRUCTIONS.md        # your brief; preserved across runs
    ├── .claims/               # structured Claim sidecars with evidence
    ├── .page-manifest.json    # per-page progress and source checkpoints
    ├── .last-update.json      # metadata for the last successful check
    ├── .run.json              # transient page queue (deleted on success)
    └── .langsmith.json        # LangSmith workspaces and projects (never keys)
```

How agents find the wiki

`AGENTS.md` carries a managed block, between `<!-- OPENWIKI:START -->` and `<!-- OPENWIKI:END -->`, telling agents when to consult the wiki; content outside the block is untouched. `CLAUDE.md` is updated only if it already exists, and its managed block imports `@AGENTS.md`, so Claude Code reads the same instructions. OpenWiki never creates `CLAUDE.md`. An optional MCP tool, `openwiki_inspect_page_claims`, exposes a page's full Claims to host-driven runs.

Grounded Claims

Claims are the material propositions behind factual pages: behaviour, responsibilities, data flow, invariants, failure semantics, configuration and security boundaries. Each Claim cites exact repository evidence (for example a `repo://` path with a line range) and the evidence version observed. Markdown stays clean; Claim state lives in `openwiki/.claims/`.

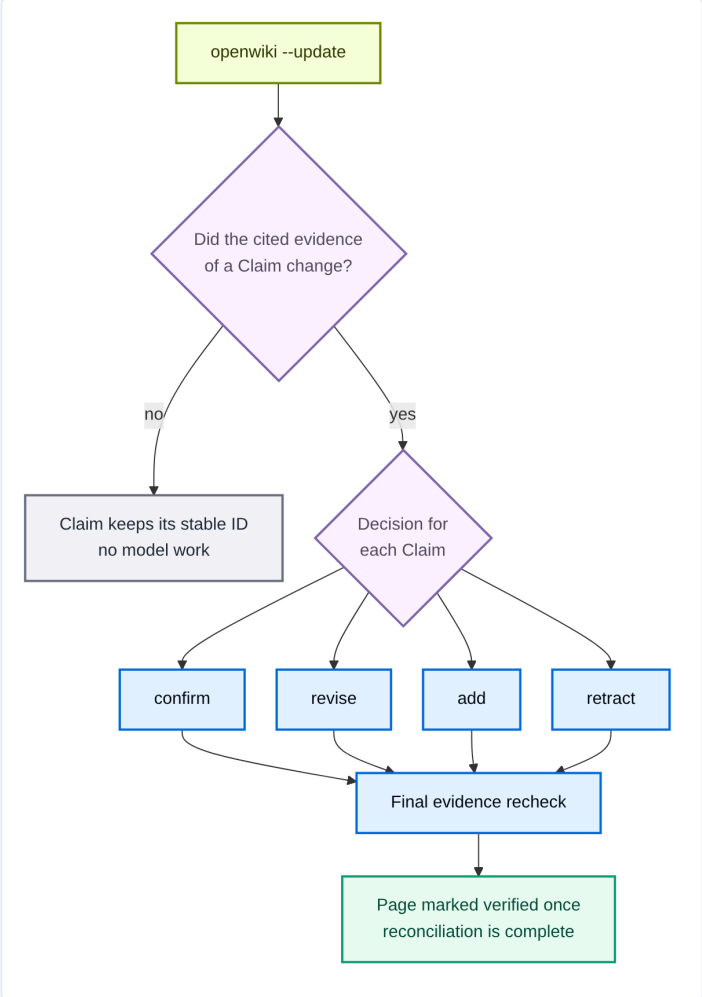


Figure 67. How Claims are reconciled on update.

On `--update`, only Claims that need attention receive a decision: **confirm, revise, add or retract**. Unaffected Claims keep their stable IDs, which keeps updates cheap. Connector-derived facts, including LangSmith-only observations, are excluded from Claims.

Parallel workers, retries and resumability

- Native CLI runs can document pages concurrently. `OPENWIKI_PAGE_CONCURRENCY` ranges from 1 to 8 (default 1); the docs suggest starting at 2 to 4. Each worker owns exactly one page and completed pages are durable resume units. The quickstart page is written **last** so it can link to everything it routes to.
- On a provider rate-limit failure, the run lowers concurrency by one and the affected page is picked up on the next update. Provider retries default to 3, or 5 when concurrency exceeds 1 (`OPENWIKI_PROVIDER_RETRY_ATTEMPTS` overrides).
- `--init` on an existing wiki replaces the generated wiki and Claims but keeps `INSTRUCTIONS.md`. If setup fails before new run state is durable, the previous wiki is restored. Ephemeral CI runners start fresh after a failure unless the workspace is preserved.

Open Knowledge Format (OKF) v0.2

Each concept is a Markdown page with YAML front matter containing a non-empty `type`. New pages get `generated: {by, at}` (body changes advance the stamp; front-matter-only changes do not). Repository pages project Claim evidence into `sources`. `verified` is set only after a page passes a complete, non-empty Claims reconciliation and a final evidence

recheck. Producer-defined extension fields survive updates.

Ignore rules, diagrams and the LangSmith connector

`.openwikiignore` (repo root) supports comments, blank lines, `*` and `**` globs, directory rules and `!` negation. With active rules, discovery is filtered and shell execution is restricted so ignored paths stay out of the run. *It is a read boundary, not a topic guarantee:* the agent may still infer an ignored area from tests, the README, commit messages or the existing wiki.

Diagrams. Mermaid fences are validated after each run. Failures become plain `text` fences with a comment and may be repaired on a later update. Installing `mermaid` and `jsdom` (`npm install mermaid jsdom`) enables validation closer to GitHub's; without them a lightweight fallback check runs.

LangSmith connector. Pulls recent traces (tool calls, outcomes, latency) for chosen projects so documentation reflects runtime behaviour. It is configured during `--init`; the committed `.langsmith.json` stores workspace and project names, **never keys**. The key comes from `OPENWIKI_LANGSMITH_API_KEY` (a repository secret in CI); extra workspaces use `OPENWIKI_LANGSMITH_API_KEY_2`, `_3`, and so on. Keys are workspace- and region-bound (US, EU, APAC).

Customizing

Edit `openwiki/INSTRUCTIONS.md` to steer scope, priorities, tone and terminology. Normal runs read it but never rewrite it. Besides `.openwikiignore` and these brief files, there are no prompt or output templates to configure.

CHAPTER 54

OpenWiki: Personal Mode

> **At a glance:** Personal mode builds a local "personal brain" at `~/.openwiki/wiki` by ingesting configured sources and synthesizing them into wiki pages. Storage is local, but connectors and model providers still receive data during ingestion.

Sources

Source	Credentials	Behaviour
`custom-mcp`	None by default	HTTP or stdio MCP servers; only explicitly safe, **read-only** tools are permitted
`git-repo`	Local paths	Reads configured local repositories and writes compact manifests
`x`	OAuth user-context	Home timeline, user posts, mentions, bookmarks, list posts
`notion`	Notion OAuth (hosted MCP)	OAuth with dynamic client registration
`google`	Gmail OAuth	Recent mail via the Gmail API
`web-search`	`TAVILY_API_KEY`	Tavily search
`hackernews`	None	Public feed and search APIs
`slack`	App client credentials + OAuth	Needs an HTTPS callback

A source type can be configured several times as separate instances (`web-search-1`, `web-search-2`), for example one for AI research and one for sports news.

Commands

```bash
openwiki personal --init
openwiki personal --update "Refresh the wiki from configured connectors"
openwiki auth notion | gmail | x | slack
openwiki auth configure <provider>       # regenerate connector config from saved env vars
openwiki auth tools <provider>           # list live MCP tools for MCP-backed providers
openwiki ingest all                      # or: ingest web-search / web-search-2
openwiki ngrok start [https://your-domain.ngrok.app]   # HTTPS tunnel for Slack OAuth
openwiki cron list | pause <source|all> | resume <source|all> | delete <source|all>
```

Scheduled jobs run `openwiki --update --print`.

Privacy and security

- **Local state** lives under `~/.openwiki`: the wiki, raw connector data and settings. The directory is created with mode `0o700` and `.env` with `0o600`.
- **Secrets** are referenced by environment-variable name in connector configs, with values only in `~/.openwiki/.env`. Connector config files must never contain raw secret values.
- **No host shell:** personal mode does not give the agent shell execution.

- **Data still leaves the machine** during ingestion: connectors call Gmail, Slack, X, Notion and Tavily, and the model provider processes the content.

Layout and configuration

Path	Purpose
`~/.openwiki/wiki/`	The personal wiki
`~/.openwiki/connectors/<connector>/raw/`	Raw data and manifests per connector run
`~/.openwiki/.env`	Secrets and tokens
`~/.openwiki/onboarding.json`	Preferences, template, connected sources, notes and schedule cron expressions
`~/.openwiki/INSTRUCTIONS.md`	Global personal-wiki instructions
`~/.openwiki/logs/`	Logs from scheduled runs

`OPENWIKI_CONFIG_DIR` relocates state (OpenWiki does not move an existing `~/.openwiki`; copy it yourself). `OPENWIKI_OAUTH_CALLBACK_PORT` sets the local callback port. `OPENWIKI_HTTPS_OAUTH_REDIRECT_URI` holds the Slack HTTPS redirect (`openwiki ngrok start` sets it). X and Gmail ignore the HTTPS override and use `http://127.0.0.1:53682/callback` by default.

> **Edge cases:** Gmail and Slack need app client credentials in `~/.openwiki/.env` before authentication. **Scheduling is macOS-only** (LaunchAgent plists in `~/Library/LaunchAgents/`). `cron delete` removes only the saved schedule and unloads its LaunchAgent; auth, connector config, raw data and wiki content remain.

CHAPTER 55

OpenWiki: Coding-Agent Integrations, Search and Workspaces

> **At a glance:** Instead of using its own model, OpenWiki can run **inside** an existing coding agent: the host researches and writes pages with its own tools while OpenWiki manages the page-job lifecycle, Claims validation and finalization. Installed integrations also expose read-only **search** tools over the wiki.

Supported hosts and installation

Codex, Claude Code, GitHub Copilot, OpenCode, Cursor, IBM Bob (Bob Shell), Kiro, Oh My Pi (`omp`) and Antigravity CLI. Installs default to user level, so one install works in any Git repository.

```bash
openwiki integrations install claude        # or codex, copilot, opencode, cursor, bob, kiro, omp, antigravity
openwiki integrations list [--project [path]]
openwiki integrations uninstall <agent> [--project [path]]
```

After installing, restart the coding agent, open the repository and prompt it, for example: *"Initialize this repository's OpenWiki from the current source and tests"* (new wiki) or *"Update this repository's OpenWiki for changes since its last successful run"* (existing wiki). Pi is installed differently: `pi install npm:openwiki` (or `pi install --local npm:openwiki`); its extension starts the CLI through Node.

How host-driven runs work

The host drives research and authoring. For each page it submits sparse Claim decisions (confirm, revise, add, retract); OpenWiki validates each completion, persists Grounded Claims and finalizes metadata through MCP, and blocks completion until the final Claims state is saved.

> **Edge cases:** only **code mode** wikis are supported (not personal mode). The host's authenticated model session is used, so OpenWiki provider credentials are not required. Connector-sourced context, including LangSmith, is not yet supported on this path. Page authoring is **sequential**; parallel workers apply only to the native CLI.

Search and read tools

Tool	Function
`openwiki_search`	Compact, ranked results for a concrete question
`openwiki_read`	Complete sections selected from search results
`openwiki_list_workspaces`, `openwiki_list_wikis`	Discover workspaces and wikis once repositories are linked

These tools are **local, read-only and model-free**. Agents should use them for concrete architecture or dependency questions, read the relevant sections and stop once grounded; retrieval is optional context, not a routine first step.

Workspaces

`openwiki link` groups related repository wikis into a named workspace so an agent can search across a multi-repository service.

```bash
cd ~/dev
openwiki link                       # create a workspace (e.g. "Payments") and pick member repos
openwiki workspace use payments
openwiki workspace current
openwiki workspace clear
```

Members keep their own wikis, a repository can belong to several workspaces, and members need not share a parent directory (the registry lives in `~/.openwiki` or `OPENWIKI_CONFIG_DIR`). Repositories without OpenWiki documentation appear dimmed.

Search scope resolution: no workspace → the wiki itself; one workspace → that workspace; several with an active selection → the active one; several with none → the tool returns `workspace_required` and the agent asks which to use. Workspace results carry a `wiki` identifier that must be passed to `openwiki_read` together with the result's page and section anchors.

CHAPTER 56

OpenWiki: Providers, Configuration and Telemetry

> **At a glance:** OpenWiki supports 14 model-provider identifiers. Credentials and defaults live in `~/.openwiki/.env`; process environment variables take priority. Select the provider with `OPENWIKI_PROVIDER` and `OPENWIKI_MODEL_ID`.

Providers and credentials

Provider ID	Credential
`openai`	`OPENAI_API_KEY`
`openai-chatgpt`	ChatGPT OAuth tokens
`copilot`	GitHub CLI session or `COPILOT_API_KEY`
`openrouter`	`OPENROUTER_API_KEY`
`anthropic`	`ANTHROPIC_API_KEY`
`bob`	`BOB_API_KEY`
`gemini`	`GEMINI_API_KEY` (Google AI Studio)
`gemini-enterprise`	Application Default Credentials + `GOOGLE_CLOUD_PROJECT`
`bedrock`	AWS credentials and region
`baseten`, `fireworks`, `nebius`, `nvidia`	`BASETEN_API_KEY`, `FIREWORKS_API_KEY`, `NEBIUS_API_KEY`, `NVIDIA_API_KEY`
`openai-compatible`	`OPENAI_COMPATIBLE_API_KEY` plus a base URL and model ID

Several providers accept base-URL overrides (`OPENAI_BASE_URL`, `ANTHROPIC_BASE_URL`, `BOB_BASE_URL`). Interactive setup is `openwiki --init`; settings are saved for later `--update` runs.

Global tuning variables

Variable	Effect
`OPENWIKI_PROVIDER_RETRY_ATTEMPTS`	Positive integer; default 3, or 5 when page concurrency > 1
`OPENWIKI_MAX_OUTPUT_TOKENS`	Per-request output budget (maps to `maxOutputTokens` for Gemini, `maxTokens` otherwise). Built-in ceilings: Anthropic Claude 4/5 models 16384; Bedrock 16000 (`OPENWIKI_BEDROCK_MAX_TOKENS` overrides); OpenRouter uses `OPENWIKI_OPENROUTER_MAX_TOKENS` first
`OPENWIKI_REASONING_EFFORT`	Only for models that advertise reasoning; invalid combinations fail before any request
`OPENWIKI_STREAM_IDLE_TIMEOUT`	Bedrock stream idle watchdog in ms (0 disables)
`OPENWIKI_PAGE_CONCURRENCY`	Page workers for native runs (1-8)

Provider-specific notes

- **IBM Bob:** the model ID is fixed to `premium`; streaming is always on.
- **GitHub Copilot:** reuses an active GitHub CLI session or prompts for `gh auth login`; the CLI token stays in the GitHub CLI store. For CI use a GitHub OAuth token in `COPILOT_API_KEY`; classic and fine-grained PATs are rejected.
- **Microsoft Entra ID:** with `openai-compatible` endpoints needing bearer tokens (OpenWiki ≥ 0.7.0): `OPENAI_COMPATIBLE_AUTH=entra-id`, an HTTPS `OPENAI_COMPATIBLE_BASE_URL` without embedded credentials, and optionally `OPENAI_COMPATIBLE_ENTRA_SCOPE`. Workload identity federation uses `AZURE_CLIENT_ID`, `AZURE_TENANT_ID`, `AZURE_FEDERATED_TOKEN_FILE`; otherwise `DefaultAzureCredential`.
- **Gemini Enterprise:** no API key; needs `roles/aiplatform.user` and Model Garden access; set `GOOGLE_CLOUD_LOCATION` explicitly for region-specific models.
- **AWS Bedrock:** explicit `BEDROCK_AWS_*` keys are optional (the SDK default chain applies); some newer models need a cross-region inference profile ID.
- **OpenAI-compatible endpoints** (Ollama at `http://localhost:11434/v1`, LM Studio at `http://localhost:1234/v1`): an API key is still required even if the server ignores it.

> **Critical edge case (blank wiki):** OpenWiki sends **non-streaming** requests internally. Some gateways accept only streaming and may reject the call or return HTTP 200 with empty content, producing a blank wiki with no error. Set `OPENWIKI_OPENAI_COMPATIBLE_STREAMING=true` (token counts then become estimates). Responses API use is opt-in via `OPENWIKI_OPENAI_COMPATIBLE_USE_RESPONSES_API=true`.

Cost and operational caveats

ChatGPT login draws on the usage allowance of a Plus, Pro or Team Codex plan rather than a metered key; treat the stored refresh token like a password. OpenRouter sends no `max_tokens` by default, so credit pre-checks budget for the model's full output ceiling and low balances can fail with 402; setting a limit avoids this but risks truncation. Bedrock without an explicit limit can cap output at 4096 tokens in the Converse API, truncating long pages. Higher page concurrency raises the default retry count.

Configuration directory and telemetry

`~/.openwiki/` holds `.env`, `openwiki.sqlite` (conversation checkpoints), `install-id` (random ID for anonymous telemetry), and personal-mode files. Repository artifacts (`openwiki/`, `.claims/`) live in the project, not here. Set `OPENWIKI_CONFIG_DIR` before starting to relocate state (for example onto a mounted container volume).

Telemetry is on by default and anonymous. Per run it records the command (init/update), the outcome (success/failure/no-op) and, on failure, a coarse error category; on init also the brain mode, provider and connector *names*. It never collects file contents, repository names or data, credentials, prompts, model output, error messages, paths, URLs, model IDs, durations or IP addresses (GeoIP enrichment is disabled). Opt out with `OPENWIKI_TELEMETRY_DISABLED=1` or `DO_NOT_TRACK=1` (permanently via `~/.openwiki/.env`; in CI via the workflow environment). Inspect what would be sent with `--telemetry-file=<path>`.

CHAPTER 57

OpenWiki: Visualizing and Automating Updates

At a glance: `openwiki visualize` explores the wiki as an interactive node graph beside a live Markdown reader. In CI, `openwiki code --update --print` refreshes the wiki on a schedule and opens a pull request only when something changed.

Visualizing

```bash
openwiki visualize                              # serves ./openwiki on 127.0.0.1:4321
openwiki visualize openwiki --port 4400 --no-open
openwiki visualize ~/.openwiki/wiki             # a personal wiki
openwiki visualize <PATH> --export docs/openwiki-visualizer
```

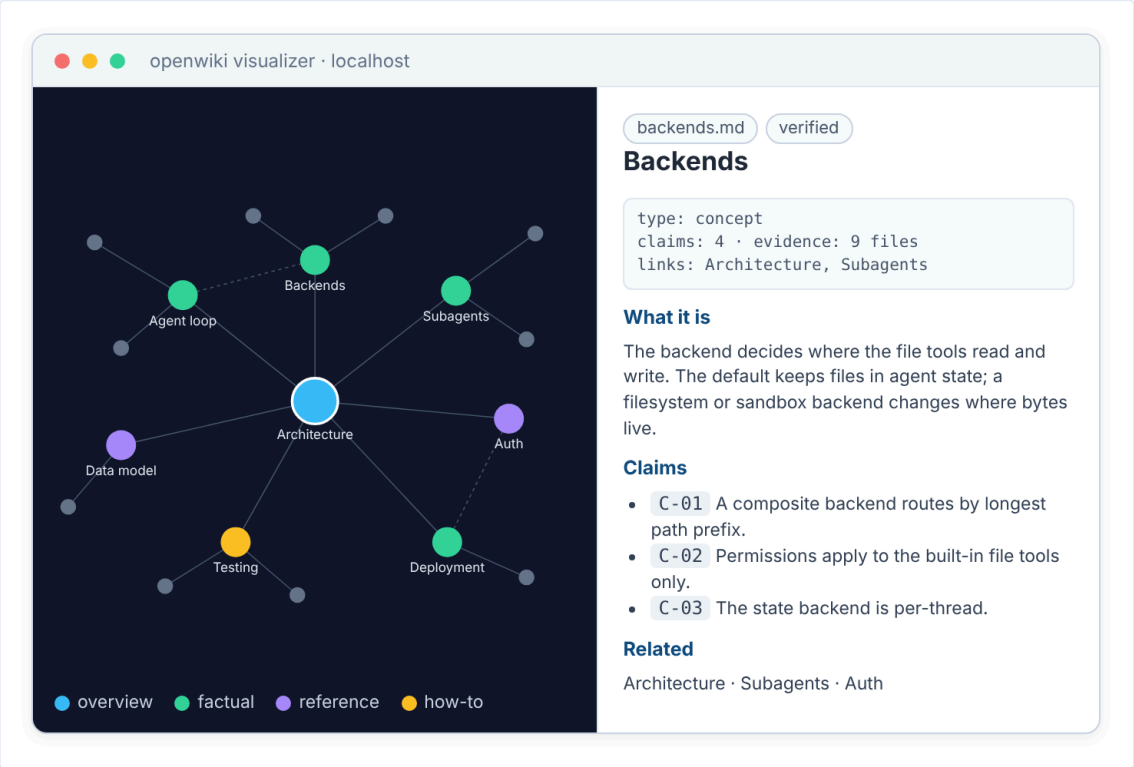


Figure 68. An illustration of the visualizer: wiki concepts as nodes, Markdown links as edges, beside a live reader.

The graph shows concepts as nodes and Markdown links as edges; `INSTRUCTIONS.md` and other scaffolding are omitted. The server live-reloads on edits. If the preferred port is busy it increments to the next one, so read the terminal for the real address. `--export <dir>` writes a **static** site (`index.html` , `client.js` , `client-lib.js` , `styles.css` , `graph.json`) with no live reload that any static host (GitHub Pages, MkDocs) can serve; it cannot be combined with `--port` or `--no-open` .

Edge case: the graph, Markdown and diagram libraries load from a public CDN, so both the local and the exported viewer need an internet connection.

Automating refreshes in CI

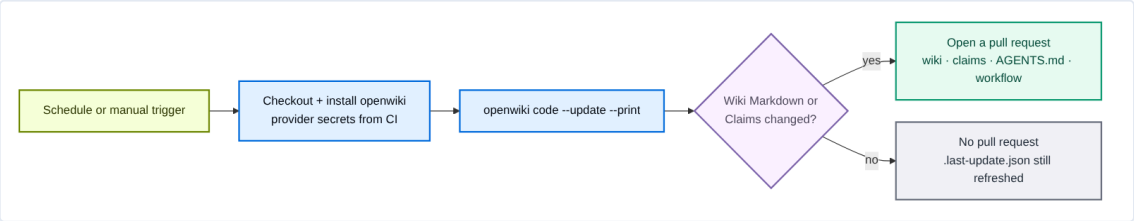


Figure 69. A scheduled OpenWiki refresh in CI.

Example workflows exist for **GitHub Actions** (`.github/workflows/openwiki-update.yml`), **GitLab CI** (`.gitlab-ci.yml` , or included from your pipeline) and **Bitbucket Pipelines** (a custom `openwiki-update` pipeline that you must schedule yourself). The core command is:

```bash
openwiki code --update --print
```

No `--init` is needed: `--update` creates the initial `openwiki/` docs when missing, provided the provider and model environment variables are set. Store secrets as repository secrets or CI variables: a provider key (or a Copilot OAuth token, or cloud credentials), `OPENWIKI_PROVIDER` , `OPENWIKI_MODEL_ID` , and optionally `LANGSMITH_API_KEY` and `OPENWIKI_LANGSMITH_API_KEY` .

The pull/merge request contains the changed wiki files, the Claims under `openwiki/.claims/` , `AGENTS.md` and `CLAUDE.md` , and the workflow file itself. A clean `--update` skips model work and leaves content unchanged, but still refreshes `openwiki/.last-update.json` so freshness checks show the check ran. If neither the wiki Markdown nor the Claims changed, **no PR is opened**. OpenWiki can still publish partial progress if some pages finished before a failure. Scheduled and CI runs send anonymous telemetry under a shared CI identifier; set `OPENWIKI_TELEMETRY_DISABLED=1` to stop it.

> **Not documented, so plan for it yourself:** the cron/trigger syntax in your CI, the token cost of each refresh (and therefore how often to run it), and how a wiki PR conflicts with new commits on the default branch.

PART XI

11

Build-Alongs

Four complete agents built step by step, and a capstone that combines every layer of the harness.

IN THIS PART

58 Build-Along 1: A Deep Research Agent

59 Build-Along 2: A Data Analysis Agent

60 Build-Along 3: A Content Builder Agent

61 Build-Along 4: Retrieval-Augmented Generation with Deep Agents

62 Capstone: A Release-Notes Assistant that Uses Every Layer

CHAPTER 58

Build-Along 1: A Deep Research Agent

> **At a glance:** A coordinator plans with todos, saves the request to a file, delegates web research to a sub-agent that fetches *full pages* (not snippets), consolidates citations and writes a report. One file, `agent.py`. Concepts: subagents for context isolation, a custom search tool, opt-in planning.

Goal. Plan the research, delegate focused tasks to isolated sub-agents, assess results, and synthesize a cited final report.
Prerequisites: an Anthropic or Google key, a Tavily key (free tier suffices) and optionally LangSmith for tracing.
Dependencies: `deepagents tavily-python httpx markdownify` plus `langchain-anthropic` (or `langchain-google-genai` for Gemini).

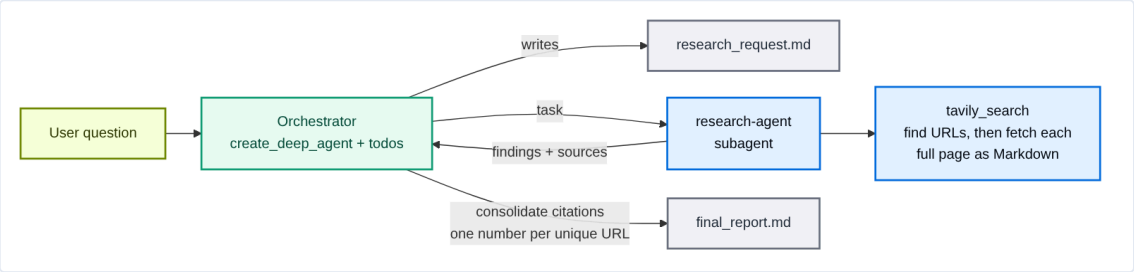


Figure 70. Architecture of the deep research agent.

The tools

Two pieces: a helper that fetches a URL and converts HTML to Markdown, and a search tool that finds URLs with Tavily and then fetches each one.

```python
import os
from typing import Annotated, Literal

import httpx
from langchain.tools import InjectedToolArg, tool
from markdownify import markdownify
from tavily import TavilyClient

tavily_client = TavilyClient(api_key=os.environ["TAVILY_API_KEY"])

def fetch_webpage_content(url: str, timeout: float = 10.0) -> str:
    """Fetch webpage and convert HTML to markdown."""
    headers = {"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36"}
    try:
        response = httpx.get(url, headers=headers, timeout=timeout)
        response.raise_for_status()
        return markdownify(response.text)
    except Exception as e:
        return f"Error fetching {url}: {e!s}"

@tool(parse_docstring=True)
def tavily_search(
    query: str,
    max_results: Annotated[int, InjectedToolArg] = 1,
    topic: Annotated[Literal["general", "news", "finance"], InjectedToolArg] = "general",
) -> str:
    """Search the web for information on a given query.
```

```
    Uses Tavily to discover relevant URLs, then fetches and returns full webpage content as markdown.

    Args:
        query: Search query to execute
        max_results: Maximum number of results to return (default: 1)
        topic: Topic filter - 'general', 'news', or 'finance' (default: 'general')
    """
    search_results = tavily_client.search(query, max_results=max_results, topic=topic)
    result_texts = []
    for result in search_results.get("results", []):
        url, title = result["url"], result["title"]
        content = fetch_webpage_content(url)
        result_texts.append(f"## {title}\n**URL:** {url}\n\n{content}\n---")
    return f"Found {len(result_texts)} result(s) for '{query}':\n\n" + "\n".join(result_texts)
```

Design notes. `max_results` and `topic` are wrapped in `InjectedToolArg`, so the model cannot set them: only `query` is model-controlled and the developer keeps cost under control. Fetching full pages gives the model complete sources instead of snippets. A fetch failure is returned as text (not raised), so one dead link does not kill the run.

The three prompts (abridged)

1. **Orchestrator workflow.** Six steps: *plan* with `write_todos`; *save the request* to `/research_request.md`; *research* by delegating **every** research task through `task()` (the orchestrator never researches directly); *synthesize* so each unique URL gets one citation number; *write* `/final_report.md`; *verify* the report against the saved request. It also fixes the report structure (comparison / list / summary templates), prose-first style, inline `[n]` citations numbered without gaps, and a `### Sources` section.
2. **Researcher.** Frames the sub-agent as a research assistant, injects today's date, and gives it a bounded loop: start broad, assess after each search, narrow, stop when confident. **Budgets:** 2-3 searches for simple queries, up to 5 for complex ones, hard stop at 5. **Stop conditions:** question answered; three or more relevant sources; the last two searches returned similar information.
3. **Delegation rules.** Default to **one** sub-agent. Parallelize only for explicit comparisons (one per entity) or clearly separate aspects. At most `max_concurrent_research_units` parallel calls per iteration, issued in one response, and at most `max_researcher_iterations` rounds (both set to 3).

Wiring the agent

PYTHON

```python
from datetime import datetime
from deepagents import create_deep_agent
from langchain.agents.middleware import TodoListMiddleware
from langchain.chat_models import init_chat_model

max_concurrent_research_units = 3
max_researcher_iterations = 3
current_date = datetime.now().strftime("%Y-%m-%d")

INSTRUCTIONS = (
    RESEARCH_WORKFLOW_INSTRUCTIONS + "\n\n" + "=" * 80 + "\n\n"
    + SUBAGENT_DELEGATION_INSTRUCTIONS.format(
        max_concurrent_research_units=max_concurrent_research_units,
        max_researcher_iterations=max_researcher_iterations)
)

research_sub_agent = {
    "name": "research-agent",
    "description": "Delegate research to the sub-agent. Give one topic at a time.",
    "system_prompt": RESEARCHER_INSTRUCTIONS.format(date=current_date),
    "tools": [tavily_search],
}

model = init_chat_model(model="anthropic:claude-sonnet-4-5-20250929", temperature=0.0)
```

```
agent = create_deep_agent(
    model=model,
    tools=[tavily_search],
    system_prompt=INSTRUCTIONS,
    subagents=[research_sub_agent],
    middleware=[TodoListMiddleware()],        # planning is opt-in
)
```

For Gemini, only the model changes (`ChatGoogleGenerativeAI(model="gemini-3-pro-preview", temperature=0.0)`).

Running it

PYTHON

```python
from langchain.messages import HumanMessage

result = agent.invoke({"messages": [HumanMessage(
    content="What are the main differences between RAG and fine-tuning for LLM applications?")]})

# or stream tokens with event streaming v3
stream = agent.stream_events(
    {"messages": [HumanMessage(content="Compare Python vs JavaScript for web development")]},
    version="v3")
for message in stream.messages:
    for token in message.text:
        print(token, end="", flush=True)
```

The agent writes `/research_request.md` and `/final_report.md` with its file tools. A comparison query normally yields two parallel sub-agents, one per language.

Lessons and edge cases

1. **Separate orchestration from research.** The orchestrator plans and writes; sub-agents search, keeping its context clean.
2. **Bound every loop.** Tool-call budgets and iteration caps prevent runaway cost.
3. **Bias toward one sub-agent.** Premature decomposition wastes tokens; parallelize only for real comparisons.
4. **Fetch full content, not snippets.**
5. **Hide control parameters** from the model with `InjectedToolArg`.
6. **Standardize citations** with a single numbering scheme so sub-agents do not produce duplicate or conflicting references.
7. **Trace it** with LangSmith; multi-step behaviour is hard to debug without traces.

> **Edge cases:** pages that block scrapers return an error string, which the model may mis-cite as a source; instruct the researcher to discard "Error fetching" results. Very long pages blow the researcher's context; consider truncating in `fetch_webpage_content`. Retrieved web content can contain prompt-injection text; the researcher's output is read by the orchestrator, so treat it as data, never instructions.

CHAPTER 59

Build-Along 2: A Data Analysis Agent

> **At a glance:** A deep agent with a **sandbox backend**, a **host-side Slack tool** and opt-in todos analyses a CSV, writes and runs a pandas/matplotlib script, and posts the report and chart to Slack. Concepts: backends for sandboxed execution, custom tools for integrations, secrets kept outside the sandbox.

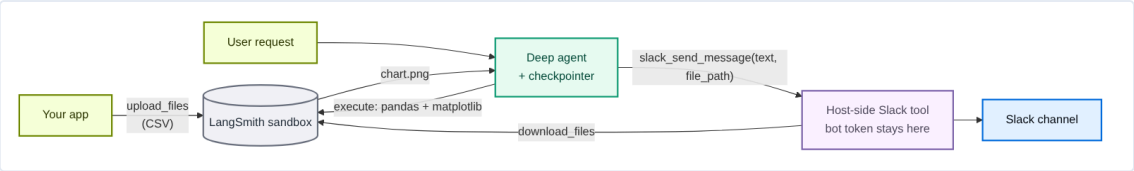


Figure 71. Architecture of the data-analysis agent.

Install: `pip install deepagents`, plus `pip install "langsmith[sandbox]" slack-sdk` for the LangSmith sandbox and Slack. The tutorial strongly recommends a sandbox for production; the local shell backend is for controlled development only. **Tracing:** `LANGSMITH_TRACING=true` and `LANGSMITH_API_KEY`.

Sandbox setup

PYTHON

```python
from deepagents.backends.langsmith import LangSmithSandbox
from langsmith.sandbox import SandboxClient

client = SandboxClient()
ls_sandbox = client.create_sandbox()
backend = LangSmithSandbox(sandbox=ls_sandbox)
```

Other providers follow the same shape: `LocalShellBackend(root_dir=".", virtual_mode=True, env={"PATH": "/usr/bin:/bin"})` (unrestricted shell: dev only), `AgentCoreSandbox`, `DaytonaSandbox`, `E2BSandbox`, `ModalSandbox`, `RunloopSandbox`. A quick readiness check is `backend.execute("echo ready")`.

Upload data. Build a small CSV in memory and upload it:

PYTHON

```python
import csv, io
data = [["Date","Product","Units Sold","Revenue"],
        ["2025-08-01","Widget A",10,250], ["2025-08-02","Widget B",5,125],
        ["2025-08-03","Widget A",7,175],  ["2025-08-04","Widget C",3,90],
        ["2025-08-05","Widget B",8,200]]
buf = io.StringIO(); csv.writer(buf).writerows(data)
backend.upload_files([("/root/data/sales_data.csv", buf.getvalue().encode("utf-8"))])
```

The Slack tool (secrets stay on the host)

PYTHON

```python
import os
from langchain.tools import tool
from slack_sdk import WebClient

slack_client = WebClient(token=os.environ["SLACK_USER_TOKEN"])
channel = "C0123456ABC"

@tool(parse_docstring=True)
```

```python
def slack_send_message(text: str, file_path: str | None = None) -> str:
    """Send message, optionally including attachments such as images.

    Args:
        text: (str) text content of the message
        file_path: (str) file path of attachment in the filesystem.
    """
    if not file_path:
        slack_client.chat_postMessage(channel=channel, text=text)
    else:
        fp = backend.download_files([file_path])
        slack_client.files_upload_v2(channel=channel, content=fp[0].content, initial_comment=text)
    return "Message sent."
```

The tool downloads the artifact from the sandbox and posts it with the Slack SDK. **The token never enters the sandbox.**

Assembly and run

PYTHON

```python
from langchain_core.utils.uuid import uuid7
from deepagents import create_deep_agent
from langchain.agents.middleware import TodoListMiddleware
from langgraph.checkpoint.memory import InMemorySaver

agent = create_deep_agent(
    model="google_genai:gemini-3.6-flash",
    tools=[slack_send_message],
    backend=backend,
    checkpointer=InMemorySaver(),              # multi-turn state
    middleware=[TodoListMiddleware()],
)
config = {"configurable": {"thread_id": str(uuid7())}}

stream = agent.stream_events({"messages": [{"role": "user", "content":
    "Analyze ./data/sales_data.csv in the current dir and generate a beautiful plot. "
    "When finished, send your analysis and the plot to Slack using the tool."}]},
    config, version="v3")
for snapshot in stream.values:
    snapshot["messages"][-1].pretty_print()
```

What the agent does

1. `read_file("./data/sales_data.csv")` **fails**: the sandbox resolves the relative path to `/data/...`, not `/root/data/...`.
2. It recovers by running `pwd` (returns `/root`) and listing `/`, `/root` and `/root/data`, then finds the file.
3. It writes `/root/analyze_sales.py` (pandas, matplotlib, seaborn) that prints a report and saves a 2x3 dashboard at 300 dpi.
4. It runs `cd /root && python analyze_sales.py`.
5. It calls `slack_send_message` with a formatted report and `file_path="/root/sales_analysis_plot.png"`.

The report totals $840.00 revenue and 33 units; Widget A leads with $425 (about 50.6%), Widget B has $325 (38.7%) and Widget C $90 (10.7%); the best day is 2025-08-01 with $250.

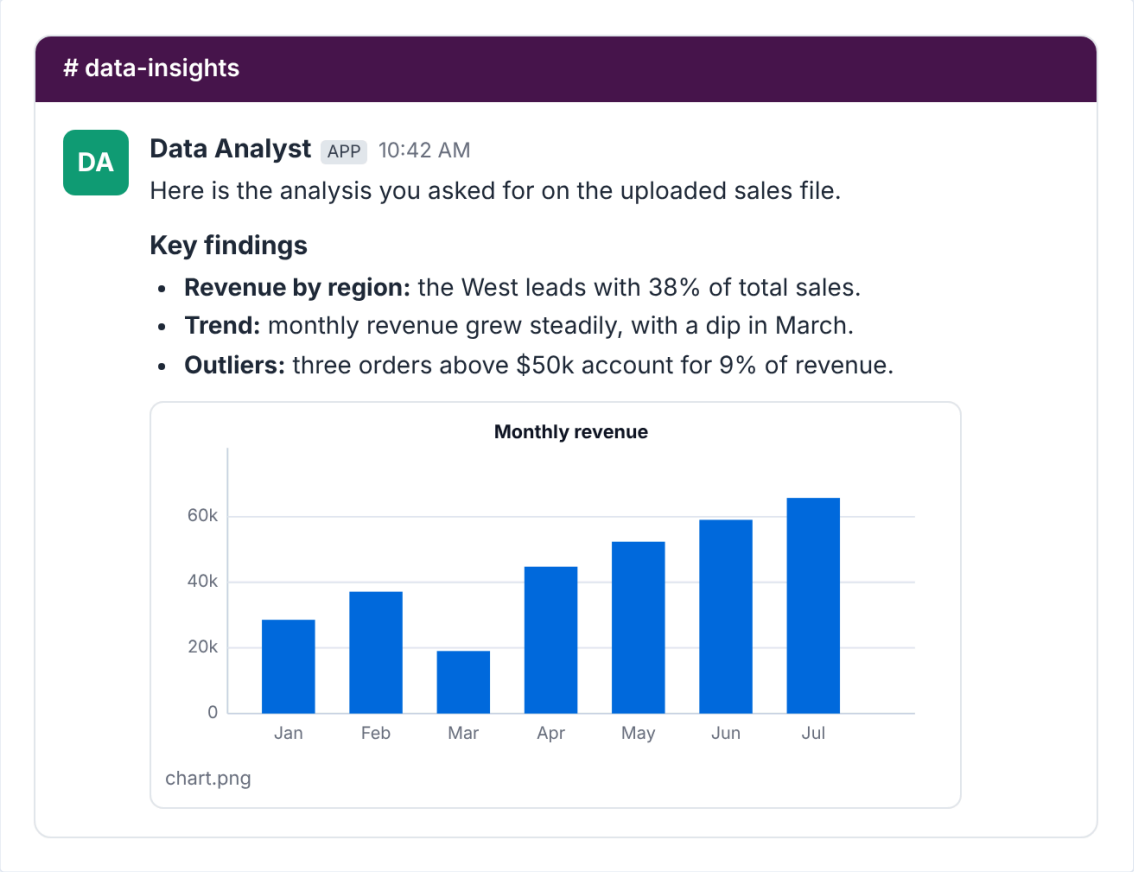


Figure 72. What the agent posts to Slack: a structured report and the dashboard chart.

To retrieve artifacts without Slack, call `backend.download_files(list_of_paths)`.

Lessons and edge cases

- **Path handling matters.** The failed relative read cost several steps. Put **absolute paths** in the prompt or system instructions.
- **Secrets stay in the tool.** Never put the Slack token in the sandbox.
- **Opt into planning** (`TodoListMiddleware`) for multi-step workflows.
- **Use a sandbox in production**, and clean sandboxes up afterwards (idle TTLs).
- **Verify model identifiers** before running; tutorial model names may differ from your provider's current list.

> **Edge cases:** if `download_files` returns an error entry instead of content, the Slack call will fail; check `error` first. Large plots can exceed Slack upload limits. The thread's `InMemorySaver` loses state on restart; use a durable checkpointer in production.

CHAPTER 60

Build-Along 3: A Content Builder Agent

At a glance: Behaviour lives in **files**, not Python: `AGENTS.md` holds brand voice (memory), `skills/*/SKILL.md` hold workflows, a `researcher` subagent is defined in YAML, and two image tools call Gemini. The agent researches, drafts and illustrates blog posts, LinkedIn posts and tweet threads.

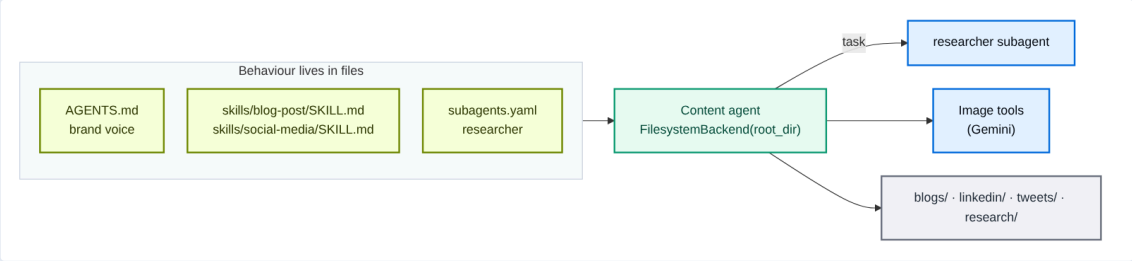


Figure 73. Architecture of the content builder.

```
content-builder-agent/
├── AGENTS.md              # memory: brand voice, writing standards, research rules
├── subagents.yaml         # subagent definitions (researcher)
├── skills/
│   ├── blog-post/SKILL.md
│   └── social-media/SKILL.md
├── content_writer.py      # tools, loader, agent factory, entry point
├── blogs/<slug>/          # output: post.md, hero.png
├── linkedin/<slug>/       # output: post.md, image.png
├── tweets/<slug>/         # output: thread.md, image.png
└── research/<slug>.md     # output: research notes
```

The configuration files (condensed)

`AGENTS.md` is loaded into the system prompt on every run: the agent is a content writer for a technology company covering AI, developer tools and emerging tech; the voice is professional but approachable, clear, confident and engaging; standards are active voice, lead with value, one idea per paragraph, concrete examples and a closing action; format uses H2/H3 headers, syntax-highlighted code, bullets for lists of three or more and a call to action; and it must **research first** using the `researcher` subagent, gathering at least three credible sources.

`subagents.yaml` defines one subagent, `researcher`. Its description tells the main agent to research first and include a save path when delegating; it runs a small, cheap model (Haiku), makes two or three targeted searches, collects statistics, quotes and source URLs, and its only tool is `web_search`. Deep Agents does **not** load subagents from files by default, so a small `load_subagents` helper reads the YAML and maps tool names to functions.

`skills/blog-post/SKILL.md` (`name: blog-post`, description that triggers on articles, tutorials and long-form pieces) requires research via `task` with `subagent_type: "researcher"` before writing, outputs `blogs/<slug>/post.md` **and a required** `hero.png`, prescribes a five-part structure (hook, context, three to five H2 sections, practical application, conclusion with call to action), includes an image-prompt framework (subject, style, composition, palette, lighting, technical details) and SEO guidance (title under 60 characters, meta description of 150-160).

`skills/social-media/SKILL.md` (`name: social-media`) covers LinkedIn posts (1,300-character limit, 3-5 hashtags) and X threads (280 characters per tweet, `1/🧵` format, at most two hashtags per tweet), each with a required `image.png` (one focal point, high contrast, no text in the image, square or 4:5).

Tools

Tool	Purpose	Output
`web_search`	Tavily search (general or news, default five results); returns an error dict if the key is missing	none
`generate_cover`	Gemini image generation (`gemini-2.5-flash-image`)	`blogs/<slug>/hero.png`
`generate_social_image`	Gemini image generation	`<platform>/<slug>/image.png`
`load_subagents`	Reads `subagents.yaml`, maps tool names to functions	none

Creating the agent

Condensed from the tutorial's description; `load_subagents`, the two image tools and `EXAMPLE_DIR` are small helpers you write.

```python
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend

def create_content_writer():
    return create_deep_agent(
        model="anthropic:claude-sonnet-4-6",                # only the model string changes per provider
        memory=["./AGENTS.md"],
        skills=["./skills/"],
        tools=[generate_cover, generate_social_image],
        subagents=load_subagents("./subagents.yaml"),
        backend=FilesystemBackend(root_dir=EXAMPLE_DIR),
    )
```

Run it with `python content_writer.py "Write a blog post about prompt engineering"`. Expected artifacts: `blogs/prompt-engineering/post.md`, `blogs/prompt-engineering/hero.png`, `research/prompt-engineering.md`. The agent should research first, read the findings, write the post and then generate the cover.

Lessons and edge cases

- **Separate memory, skills and subagents:** memory sets standing voice rules, skills load on demand by description, subagents isolate research.
- **Externalize configuration:** editing YAML and Markdown changes behaviour without touching Python.
- **Enforce output contracts in instructions:** "a post is incomplete without its image" makes completeness checkable.
- **Scope filesystem access:** the backend can read, write and delete under `root_dir`, so run it in a dedicated directory and review output before publishing.

> **Known inconsistencies in the source tutorial (verify when you build it):** the blog skill refers to a `generate_image` tool though the registered tools are `generate_cover` and `generate_social_image`; the researcher's prompt mentions `write_file` while the YAML lists only `web_search`; one passage says artifacts go to a system temp directory while the code writes under `EXAMPLE_DIR`; and the text writes `SKILLS.md` where the file is `SKILL.md`. Align tool names between skills and code before relying on the agent.

CHAPTER 61

Build-Along 4: Retrieval-Augmented Generation with Deep Agents

At a glance: Index LangChain's own documentation once; at question time the search tool **writes chunks to files and returns only paths**; a `chunk-analyst` subagent reads **one file** and summarizes it (in parallel); the orchestrator synthesizes an answer with links. The pattern is *retrieve, offload, delegate*. Test question: "How do I stream intermediate tool results from a subagent?"

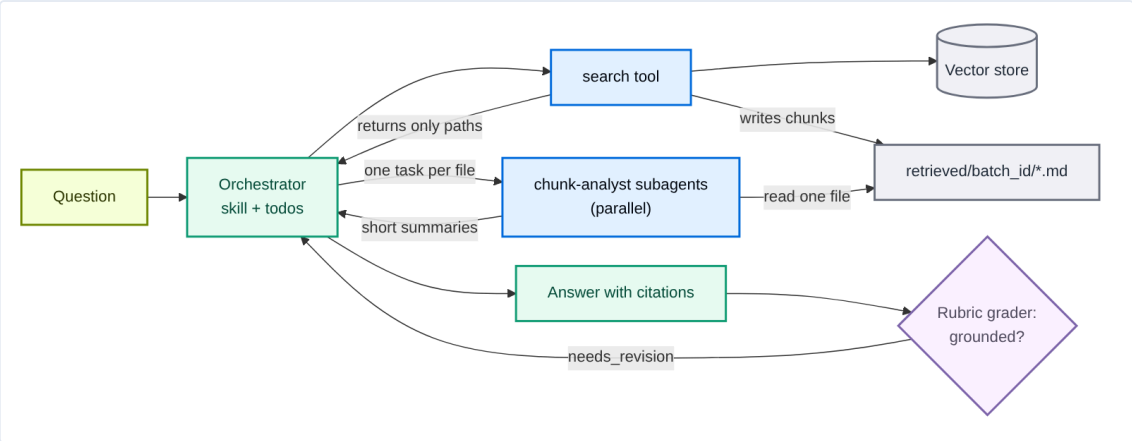


Figure 74. Architecture of the RAG agent.

The four patterns the tutorial combines

Pattern	Summary
Skills-guided retrieval	A skill tells the agent how to search (index, query style, citations)
Rubric-checked grounding	A grader (`RubricMiddleware` , `deepagents>=0.6.5` , beta) checks answers against sources and triggers revisions
Todo-driven investigation	Opt-in planning breaks a question into searches
Retrieve, offload, delegate	Chunks go to the filesystem; subagents analyse them in parallel

(The rubric pattern is only introduced in the tutorial; see the Grading Rubrics chapter for the middleware.)

Indexing: load, split, embed, store

The code in this build-along is condensed from the tutorial's description; the corpus, embeddings model and prompt wording are yours to choose.

PYTHON

```python
import requests
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_core.vectorstores import InMemoryVectorStore
from langchain_openai import OpenAIEmbeddings          # any embeddings model works
```

```
DOC_PATHS = [...]                                          # 14 curated documentation pages

docs = []
for path in DOC_PATHS:
    r = requests.get(f"https://docs.langchain.com/{path}.md", timeout=30)
    if r.ok:
        docs.append(r.text)                                 # failed fetches are skipped

splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
chunks = splitter.create_documents(docs)
vector_store = InMemoryVectorStore.from_documents(chunks, OpenAIEmbeddings(model="text-embedding-3-small"))
```

In the tutorial's run: 14 pages loaded, 589,579 characters, split into **782 chunks**. In production, persist and periodically refresh the store; alternatives to the in-memory store include Chroma, Pinecone and PGVector.

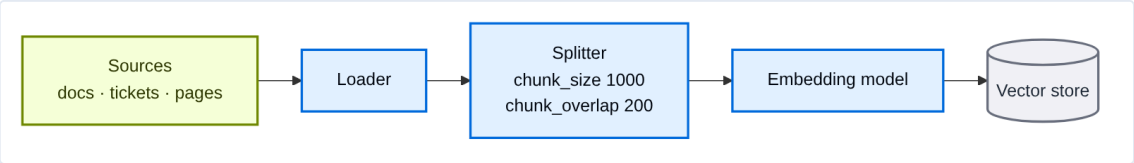


Figure 75. RAG indexing.

The search tool returns paths, not text

PYTHON

```
from deepagents.backends import StateBackend
from langchain.tools import tool
from langchain_core.utils.uuid import uuid7

backend = StateBackend()                                   # the SAME instance must go to create_deep_agent

@tool
def search_documentation(query: str) -> str:
    """Search the LangChain documentation index. Returns file paths of retrieved chunks."""
    hits = vector_store.similarity_search(query, k=4)
    batch_id = str(uuid7())[:8]
    files, paths = [], []
    for n, doc in enumerate(hits, 1):
        path = f"/retrieved/{batch_id}/chunk_{n}.md"
        files.append((path, f"# Source:
{doc.metadata.get('source','unknown')}\n\n{doc.page_content}".encode()))
        paths.append(path)
    backend.upload_files(files)
    return "Saved chunks:\n" + "\n".join(paths)
```

Returning paths keeps the orchestrator's context small, and the shared backend lets built-in tools such as `read_file` see the saved chunks.

The subagent and the orchestrator

PYTHON

```
chunk_analyst = {
    "name": "chunk-analyst",
    "description": "Reads ONE retrieved file and summarizes it",
    "system_prompt": ("Read the file at the path you are given with read_file and return a summary "
                      "under 300 words: key API names, steps, configuration details and the source URL."),
    "tools": [],                                           # built-in file tools only; no vector-store access
}

agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
    tools=[search_documentation],
    backend=backend,
```

```
    system_prompt=INSTRUCTIONS,       # plan -> search -> analyse -> synthesize -> verify
    subagents=[chunk_analyst],
)
```

The orchestrator's prompt answers from documentation rather than memory, treats retrieved content as data only, launches **one `chunk-analyst` task per file path**, runs up to three (`max_concurrent_analysts = 3`) in parallel per iteration, and never pastes chunk contents into its own messages.

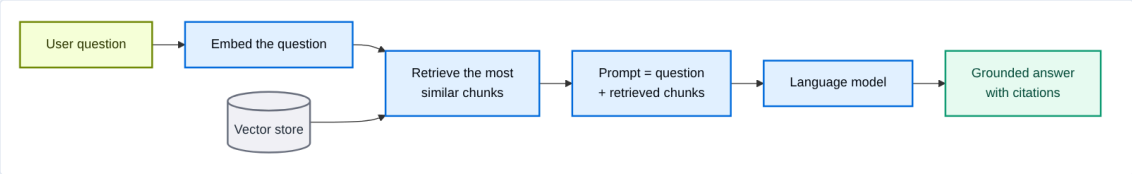


Figure 76. RAG retrieval and generation.

Lessons and edge cases

- Retrieval supplies current, source-grounded content a model's training data cannot.
- **Filesystem offloading** keeps large retrieved text out of the orchestrator's context; **subagents** analyse chunks in parallel, each with a narrow scope.
- **Pass paths, not content**: a deliberate context-management choice.

> **Edge cases:** *indirect prompt injection*: retrieved documents can contain instruction-like text; prompts and a `# Source:` header reduce but do not eliminate the risk, so validate answers against sources. The corpus is only 14 pages and the in-memory index is rebuilt at startup. The tutorial itself is inconsistent about the embedding model (`text-embedding-3-large` in one place, `-small` in the full script), the planning step in the orchestrator prompt, and the Claude model name; choose one and keep it consistent. If the shared `backend` instance is *not* passed to `create_deep_agent`, `read_file` cannot see the chunks the tool saved.

CHAPTER 62

Capstone: A Release-Notes Assistant that Uses Every Layer

At a glance: One agent that ties the book together: a coordinator with two subagents, per-user memory, skills, file permissions, human approval before publishing, a rubric quality gate, fault tolerance, and event streaming. Each requirement below maps to a chapter, so you can build it incrementally and check your understanding.

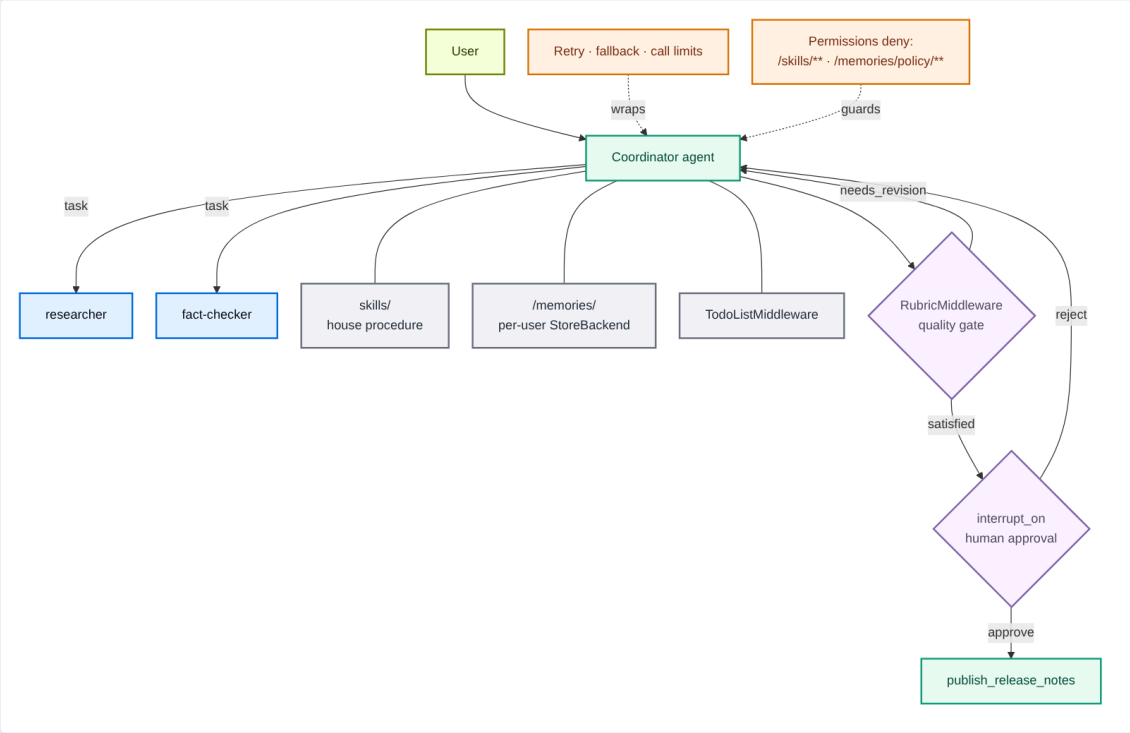


Figure 77. Capstone architecture.

Requirements

#	Requirement	Feature	Chapter
1	Remember each user's tone preferences across conversations	`CompositeBackend` + per-user `StoreBackend` on `/memories/`	Backends, Memory
2	Follow a house release-notes procedure only when relevant	Skills	Skills
3	Never edit its own skills or memory policy	`FilesystemPermission` deny	Permissions
4	Research changes and fact-check claims in isolated contexts	Two subagents	Subagents
5	Ask a human before anything is published	`interrupt_on` + checkpointer	Human-in-the-loop

#	Requirement	Feature	Chapter
6	Reject drafts without a version header and a "Breaking changes" section	`RubricMiddleware`	Grading Rubrics
7	Survive flaky APIs and runaway loops	Retry, fallback, call limits	Fault Tolerance
8	Show progress per subagent in a UI	Event streaming v3	Event Streaming

The code

PYTHON

```python
from deepagents import create_deep_agent, FilesystemPermission, RubricMiddleware
from deepagents.backends import CompositeBackend, StateBackend, StoreBackend
from langchain.agents.middleware import (
    ModelCallLimitMiddleware, ModelFallbackMiddleware, ModelRetryMiddleware,
    TodoListMiddleware, ToolCallLimitMiddleware)
from langchain.tools import tool
from langgraph.checkpoint.memory import InMemorySaver      # use a durable saver in production
from langgraph.types import Command
from langgraph.store.memory import InMemoryStore            # use a durable store in production

# (1) per-user long-term memory; everything else is thread-scoped
backend = CompositeBackend(
    default=StateBackend(),
    routes={"/memories/": StoreBackend(namespace=lambda rt: (rt.server_info.user.identity, "memories"))},
)

@tool
def publish_release_notes(version: str, markdown: str) -> str:
    """Publish approved release notes to the changelog site."""
    ...  # side effect: gated below

researcher = {"name": "researcher", "description": "Collects merged PRs and issues for a version.",
              "system_prompt": "Return a bulleted list of changes with PR links. Be concise.", "tools":
[...]}
fact_checker = {"name": "fact-checker", "description": "Verifies each claim in a draft against its source
link.",
                "system_prompt": "Return only claims you could NOT verify, with reasons.", "tools": [...]}

agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
    tools=[publish_release_notes],
    system_prompt="You draft accurate release notes. Delegate research and fact-checking; publish only
after approval.",
    backend=backend,
    memory=["/memories/AGENTS.md"],                              # (1) always loaded
    skills=["/skills/"],                                         # (2) progressive disclosure
    permissions=[                                                # (3) first match wins
        FilesystemPermission(operations=["write"], paths=["/skills/**", "/memories/policy/**"],
mode="deny"),
    ],
    subagents=[researcher, fact_checker],                        # (4)
    interrupt_on={"publish_release_notes": {"allowed_decisions": ["approve", "edit", "reject"]}},  # (5)
    middleware=[
        TodoListMiddleware(),
        RubricMiddleware(model="anthropic:claude-haiku-4-5", max_iterations=3),                # (6)
        ModelRetryMiddleware(max_retries=3),                                                   # (7)
        ModelFallbackMiddleware("gpt-5.5"),
        ModelCallLimitMiddleware(run_limit=60),
        ToolCallLimitMiddleware(run_limit=200),
    ],
    checkpointer=InMemorySaver(),                                # required for interrupts
    store=InMemoryStore(),
)
```

Running it

```python
rubric = """- The notes start with a version header (## vX.Y.Z)
- The notes contain a 'Breaking changes' section, or say 'None'
- Every claim links to a PR or issue
"""
cfg = {"configurable": {"thread_id": "release-2.4"}}
state = {"messages": [{"role": "user", "content": "Draft and publish notes for v2.4"}], "rubric": rubric}

result = agent.invoke(state, cfg)
if result.get("__interrupt__"):                          # (5) a human decides
    reqs = result["__interrupt__"][0].value["action_requests"]
    decisions = [{"type": "approve"} for _ in reqs]      # one decision per request, same order
    result = agent.invoke(Command(resume={"decisions": decisions}), cfg)
print(result.get("_rubric_status"))                      # (6) satisfied / max_iterations_reached
```

Questions to answer as you build

1. In which order do retry, fallback and the call limits wrap a model call, and what happens when the fallback model also fails?
2. Why must the `StoreBackend` namespace include the user identity? What would a missing namespace expose?
3. The deny rule protects `/skills/**`. What would still let the agent change behaviour (hint: `execute`, custom tools)?
4. A subagent defines its own `permissions`. Does it merge with the parent's?
5. After `max_iterations_reached`, the last evaluation still reads `needs_revision`. How should the calling application treat the draft?
6. What changes if you move the same project to Managed Deep Agents? (See the migration table: backend, store and checkpointer disappear; memory, skills and instructions become files.)

The answers appear in the matching chapters and in the interview section.

PART XII

12

Interview Preparation

A bank of 124 questions in three families, direct, concept and scenario, each answered with a short answer, an elaborated explanation, step-by-step reasoning, advantages, limitations, trade-offs, edge cases and likely follow-ups, followed by system-design walkthroughs, bug-spotting exercises and mock rounds.

IN THIS PART

63 How to Answer Deep Agents Interview Questions

64 Interview Questions: Foundations and Architecture

65 Interview Questions: Tools, Backends and Permissions

66 Interview Questions: Sandboxes, Interpreters and Streaming

67 Interview Questions: Context Management

68 Interview Questions: Delegation and Subagents

69 Interview Questions: Steering with Human-in-the-Loop and Rubrics

70 Interview Questions: Middleware and Reliability

71 Interview Questions: Frontend and Protocols

72 Interview Questions: Deployment and Production

73 Interview Questions: Managed Deep Agents

74 Interview Questions: OpenWiki, Build-Alongs and the Capstone

75 System-Design Walkthroughs

76 Spot the Bug

77 Rapid-Fire Numbers and Defaults

78 Mock Interview Rounds

CHAPTER 63

How to Answer Deep Agents Interview Questions

At a glance: Interviewers rarely want a definition. They want to hear that you know *what the thing is, how it works inside, why it was designed that way, where it breaks, and what you would do instead.* This part trains that habit with three kinds of question (direct, concept and scenario), each answered in full, so that you can reuse the shape of the answers in a live interview.

The three kinds of question

Every question you will be asked falls into one of three families. Each family tests something different, so each needs a different shape of answer.

Kind	Typical wording	What the interviewer is testing	Shape of a strong answer
Direct	"What is X?", "Which parameter does Y?", "What is the default of Z?", "What does `create_deep_agent` return?"	Precise recall: have you actually read and used it?	The exact fact first, then what it means, why it is so, one example and one limit
Concept	"Why does X exist?", "How does Y work inside?", "What is the difference between A and B?", "What are the trade-offs?"	Understanding of the design, not the vocabulary	The idea, the mechanism step by step, the reason for the design, advantages, limitations and a comparison
Scenario	"Your agent does X in production. What do you do?", "Design Y under constraint Z."	Judgement: can you diagnose, choose and defend a decision?	Restate the constraint, diagnose, propose a fix, justify it, then name what could go wrong and the alternatives

The questions in the chapters that follow are grouped by topic. Inside each chapter you will find the three families in the same order: **Direct questions** first, then **Concept questions**, then **Scenario questions**.

How each answer is laid out

Every answer uses the same labels, so you always know where to look.

Label	What it gives you
Short answer	The thirty-second version. If you can only say one thing, say this.
Explanation	The elaborated answer: what the thing is and why it behaves that way.
Step by step	The mechanism as a numbered sequence you could draw on a whiteboard.
Advantages and **Limitations**	What the design buys you and what it costs. Interviewers love a candidate who volunteers both.
Trade-offs and **Compared with**	Why the design is shaped this way, and how it differs from the neighbouring option.
Edge cases and traps	What goes wrong, what the defaults hide, and the behaviours that surprise people. This label separates people who have *read* the documentation from people who have *shipped* something.
Likely follow-ups	The next questions the interviewer will ask, with the answer ready.

You will not use every layer every time. Start with the short answer, then go one layer deeper each time the interviewer leans in. Stopping early is fine. Being unable to go deeper is not.

Answering a direct question

1. **Give the fact first.** "By default it is X." Do not warm up.
2. **Say what it means in practice.** One sentence on the consequence.
3. **Say why.** The reason shows you understand it rather than memorised it.
4. **Add one example or one edge case.** A single unprompted edge case signals real experience.

Answering a concept question

1. **State the idea in one sentence.** Then draw the mechanism as steps.
2. **Say what problem the design solves.** Concepts are answers to problems.
3. **Name the cost.** Every design gives something up. Say what.
4. **Compare with the nearest alternative.** "As opposed to ..." is the phrase that turns a definition into understanding.

Answering a scenario question

1. **Restate the situation and the constraint** in your own words, so the interviewer knows you heard it.
2. **Diagnose before you prescribe.** Name two or three possible causes and how you would tell them apart.
3. **Propose the smallest fix that works**, then say what you would do if it were not enough.
4. **Say what could go wrong with your fix.** This is the part most candidates skip, and the part that earns trust.
5. **Mention how you would verify it:** a trace, a test, a metric.

How the chapters are organised

The question bank holds **124 questions** in eleven topic chapters, numbered Q1 to Q124 so that you can refer to them by number. Each chapter has direct, concept and scenario questions. After the bank come four practice chapters.

Chapter	Questions	Covers
Foundations and Architecture	Q1 to Q12	What Deep Agents is, the three layers, how a turn runs, models, versions
Tools, Backends and Permissions	Q13 to Q24	Built-in tools, backends, routing, namespaces, permission rules
Sandboxes, Interpreters and Streaming	Q25 to Q36	Isolation, secrets, the interpreter, programmatic tool calling, streaming
Context Management	Q37 to Q48	Offloading, summarization, skills, memory, profiles, caching, retrieval
Delegation and Subagents	Q49 to Q60	`task`, inheritance, fork, dynamic and async subagents
Steering with Human-in-the-Loop and Rubrics	Q61 to Q70	Approvals, resume, durability, rubric grading
Middleware and Reliability	Q71 to Q80	Hooks, ordering, prebuilt and custom middleware, fault tolerance
Frontend and Protocols	Q81 to Q90	Streaming UIs, MCP, ACP, A2A and AG-UI
Deployment and Production	Q91 to Q100	Scoping, authentication, durability, secrets, multi-tenancy
Managed Deep Agents	Q101 to Q112	Project files, deploy, memory, sandboxes, channels, schedules

Chapter	Questions	Covers
OpenWiki, Build-Alongs and the Capstone	Q113 to Q124	The wiki generator, the tutorial agents and the capstone review
System-Design Walkthroughs	D1 to D10	Whiteboard problems
Spot the Bug	B1 to B14	Broken snippets
Rapid-Fire Numbers and Defaults		The facts you must recall instantly
Mock Interview Rounds	Rounds 1 to 4	Timed practice

Habits that score well

- **Name the layer.** Before answering, say whether the topic belongs to the runtime (LangGraph), the framework (LangChain) or the harness (Deep Agents). It prevents a whole class of confused answers.
- **Give the default first.** "By default it is X; you change it by doing Y." Interviewers listen for the default because that is what production incidents are made of.
- **Volunteer one edge case.** One unprompted edge case per answer signals real experience.
- **Separate *replace* from *merge*.** A surprising number of Deep Agents rules are replace-not-merge (subagent tools, subagent permissions, same-name middleware). Say the word out loud.
- **Be honest about versions.** Many features are beta and version-gated. Saying "that needs `deepagents>=0.7.13` and is beta" is a strength, not an admission of weakness.
- **Think in failure modes.** For any feature, be ready to say what happens when it is missing, misconfigured or attacked.

Phrases to avoid

Instead of	Say
"Permissions secure the agent."	"Permissions guard the built-in file tools only; `execute`, MCP and custom tools need their own controls."
"Subagents inherit everything."	"Some fields inherit, some replace, and some do not inherit at all. Let me give you the table."
"HITL just works."	"It needs a checkpointer and the same `thread_id` on resume."
"The sandbox keeps secrets safe."	"The sandbox protects the host. Secrets must never enter it."
"Memory is persistent."	"Memory is persistent *only* if the path routes to a store with the right namespace."

A study plan

- **One hour:** read the Foundations questions aloud (Q1 to Q12), then Rapid-Fire.
- **Half a day:** work Tools and Backends, Sandboxes, Delegation and Steering (Q13 to Q36 and Q49 to Q70), since they carry the most trap questions.
- **One day:** do every direct and concept question, then the scenario questions out loud, the ten walkthroughs on a whiteboard, and the bug-spotting set blind.
- **The night before:** reread the four capability categories, the backend model, the subagent inheritance table, the human-in-the-loop requirements, the middleware order and the security boundary. If you can explain those six things, most questions are within reach.

CHAPTER 64

Interview Questions: Foundations and Architecture

At a glance: Twelve questions on what Deep Agents is, how it relates to LangChain and LangGraph, what happens inside one turn, and how to make the first decisions well. Four are direct (recall), four test concepts, and four are scenarios in which you must decide.

Direct Questions

Short, factual questions. Give the fact first, then say what it means in practice.

Q1. What is Deep Agents, and what problem does it solve?

Short answer. Deep Agents is an open-source (MIT) *agent harness*: a pre-assembled, opinionated agent built on LangChain and run by LangGraph. One call to `create_deep_agent` gives you a tool-calling agent that already has a virtual filesystem, subagents, summarization, skills, memory, approvals and sandbox support, so it can finish long, multi-step tasks that break a bare tool-calling loop.

EXPLANATION

A plain agent is a loop. The model is called, any tools it requests are run, the results are appended to the conversation, and the loop repeats until the model stops asking for tools. That loop is fine for a short task. On a long task it fails in four predictable ways:

- **The window fills up.** Every tool result stays in the conversation, so a few large web pages or file reads can use most of the context.
- **There is nowhere to put intermediate work.** Notes, drafts and partial results have to live in the conversation, where they cost tokens on every later call.
- **One context does every subtask.** The researcher, the writer and the reviewer share a single, growing, distracted context.
- **There is no durability or control.** If the process dies, the run is lost, and nothing lets a person approve a risky step.

Deep Agents answers each one with a built-in. Offloading and summarization protect the window. A filesystem behind a pluggable backend gives scratch space and durable memory. Subagents give each subtask a fresh context. Checkpointing, human-in-the-loop approvals and permissions give durability and control.

STEP BY STEP

1. You call `create_deep_agent(model, tools, system_prompt=..., ...)`.
2. The function builds a stack of **middleware** (filesystem, subagents, summarization and so on) around the ordinary tool-calling loop.
3. It compiles the result into a LangGraph graph.

4. You `invoke` or `stream` that graph like any other, and add a checkpointer, a store or a deployment as you need them.

ADVANTAGES

- Batteries included: you do not rebuild planning, offloading and delegation for every project.
- Model-agnostic: any LangChain chat model with tool calling works.
- It is still "just a LangGraph graph", so streaming, checkpointing, interrupts and deployment come with it.

LIMITATIONS

- The model **must** support tool calling.
- The defaults are opinionated. For example, planning (`write_todos`) is **off** unless you add `TodoListMiddleware` (since 0.7), the JavaScript interpreter is off, and `execute` exists only on a sandbox backend.
- There are more moving parts than in a bare agent, so you need to know the defaults to debug it.

EDGE CASES AND TRAPS

- Do not describe it as "a new kind of model" or "a new framework". It is a configuration of existing parts: the loop is unchanged and the new behaviour arrives as middleware.
- A one-tool lookup agent does not need it. `create_agent` is lighter.

LIKELY FOLLOW-UPS

- *Name the four capability categories.* Execution environment, context management, delegation, steering.
- *Name the six headline capabilities.* Take actions, connect to your data, manage growing context, parallelize tasks, stay in the loop, improve over time.

Q2. What does `create_deep_agent` return, and what follows from that?

> **Short answer.** A compiled LangGraph graph (`CompiledStateGraph`). Everything LangGraph offers on a compiled graph therefore applies: `invoke` , `stream` , event streaming, checkpointers, stores, interrupts, `update_state` , and deployment on an Agent Server.

EXPLANATION

Because the return value is a graph rather than a special agent class, you do not learn a second API. The same object you would get from LangGraph can be run, streamed, paused and hosted in the usual ways. It can also be placed inside a larger graph as a node, which is how you combine a free-form agent with a deterministic pipeline.

STEP BY STEP

1. `create_deep_agent(...)` builds the middleware stack and the tool-calling loop, then compiles them.
2. **Persistence is opt-in.** Pass `checkpointer=` to get memory within a thread and resumable interrupts. Pass `store=` for memory that spans threads.
3. **Per-run data** goes in `context=` on `invoke` , validated against `context_schema` . **Conversation identity** goes in `config={"configurable": {"thread_id": ...}}` .
4. To host it, point `langgraph.json` at the variable that holds the agent, or use Managed Deep Agents.

ADVANTAGES

- One mental model for running, streaming, pausing and deploying.
- Free composition: a deep agent can be a node, and a LangGraph subgraph can be a subagent.

LIMITATIONS

- You inherit LangGraph's rules, including that state must be serialisable and that resuming re-runs the interrupted node from the top.

EDGE CASES AND TRAPS

- Without a checkpointer there is **no memory between calls, no resume and no interrupts**. A large share of "human-in-the-loop does not work" bugs are just this.
- `thread_id` and `context` are independent. One names the conversation; the other carries per-run data. Mixing them up is a common production bug.
- If you expose the graph over A2A, or reuse it as a subagent, its state must contain a `messages` key.

LIKELY FOLLOW-UPS

- *How do you add custom state?* Subclass `DeepAgentState` (not a plain `TypedDict`, or you lose the messages reducer) and pass `state_schema=` (needs `deepagents>=0.6.6`).
- *Do subagents get your custom state schema?* Compiled and remote subagents do not inherit the parent `state_schema`.

Q3. Which parameters does `create_deep_agent` take, and which ones do people get wrong?

Short answer. The core ones are `model`, `tools`, `system_prompt`, `middleware`, `subagents`, `skills`, `memory`, `permissions`, `backend`, `interrupt_on`, `response_format`, `state_schema`, `context_schema`, `checkpointer` and `store`. Group them by question: what the agent is, what it knows, what it can do, what it may do, and what persists.

EXPLANATION

Memorising a signature is weak. Grouping it is strong, because each group answers a design question you would ask anyway.

Question	Parameters	What to remember
What *is* the agent?	`model`, `system_prompt`, `name`, `response_format`	`model` is a `provider:model` string or a configured object. `system_prompt` may be a `str` or a `SystemMessage`.
What does it *know*?	`skills`, `memory`	`skills` lists **directories** that contain skill folders, loaded on demand. `memory` lists `AGENTS.md` **file paths**, always loaded into the prompt.
What can it *do*?	`tools`, `subagents`, `middleware`	Built-in tools are added for you. A same-named middleware **replaces** the built-in. The general-purpose subagent is added automatically.
What *may* it do?	`permissions`, `interrupt_on`	Permissions are path rules for the file tools (first match wins, default allow). `interrupt_on` pauses before a tool call and needs a checkpointer.
Where do things *live*?	`backend`, `checkpointer`, `store`	Default backend is `StateBackend()` (files live in the thread's state). The checkpointer holds thread state, the store holds cross-thread data.
What shape is the state?	`state_schema`, `context_schema`	`state_schema` must subclass `DeepAgentState`. `context_schema` types the per-run `context=`.

ADVANTAGES

- Most of the configuration is declarative. You change behaviour by changing a parameter, not by rewriting the loop.

LIMITATIONS

- Several parameters have **replace, not merge** semantics (same-name middleware, a subagent's `tools`, a subagent's `permissions`). It is easy to lose a setting by overriding it.

EDGE CASES AND TRAPS

- `interrupt_on` without a checkpointer cannot pause and resume.
- `permissions` only guard the **built-in file tools**. They do not cover `execute`, MCP tools or your custom tools.
- Subagent `response_format` needs `deepagents>=0.5.3`; custom `state_schema` needs `>=0.6.6`.
- `memory` takes file paths, `skills` takes directories. Passing the wrong kind is a silent failure: nothing loads.

LIKELY FOLLOW-UPS

- *What is the default backend?* `StateBackend()`.
- *What happens if two permission rules match?* The first one in the list wins.

Q4. Name the four capability categories and the six headline capabilities. Where does each live in the product?

Short answer. The categories are **execution environment**, **context management**, **delegation** and **steering**. The six capabilities are: take actions, connect to your data, manage growing context, parallelize tasks, stay in the loop, and improve over time.

EXPLANATION

This is a map question. The interviewer wants to see whether you can place any feature on the map.

Category	What it answers	Where it lives
Execution environment	Where does the agent act, and with what?	Tools, the virtual filesystem, backends, permissions, sandboxes, the interpreter, streaming
Context management	What does it know, and how does that fit in the window?	Skills, memory, offloading, summarization, retrieval, profiles
Delegation	How is work split?	`task` tool, subagents, fork, dynamic and async subagents
Steering	How do humans stay in control?	`interrupt_on`, `Command(resume=...)`, rubric grading, permissions

Headline capability	Delivered mainly by
Take actions	Tools, filesystem, `execute`, interpreter
Connect to your data	Memory, skills, MCP, backends
Manage growing context	Offloading and summarization middleware
Parallelize tasks	Subagents
Stay in the loop	Human-in-the-loop, rubrics
Improve over time	Writing to memory and skills from real usage

ADVANTAGES OF KNOWING THE MAP

- You can answer an unfamiliar question by first naming the category. That alone shows structure.

EDGE CASES AND TRAPS

- Some features straddle categories. Permissions are both *execution* (they limit file access) and *steering* (they enforce rules). Say so rather than forcing a single box.
- "Improve over time" is a pattern, not a switch. The agent writes to files that are later loaded; nothing retrains the model.

LIKELY FOLLOW-UPS

- *Which category does* `interrupt_on` *belong to?* Steering.
- *Which category contains subagent isolation?* Delegation, though the reason for it is context management.

Concept Questions

These test whether you understand the design, not just the vocabulary. Each answer gives the idea, the mechanism, the reason, the cost and a comparison.

Q5. How do LangChain, LangGraph and Deep Agents relate, and how do you choose between them?

> **Short answer.** LangGraph is the **runtime** (state, persistence, interrupts, streaming, deployment). LangChain is the **framework** (models, messages, tools, `create_agent`, middleware). Deep Agents is the **harness**: an opinionated agent assembled from LangChain parts and executed by LangGraph. Choose by how much of the assembly you want to own.

EXPLANATION

Think of the three as a stack rather than as competitors. Deep Agents depends on LangChain's abstractions, and those run on LangGraph. Picking the harness does not remove the lower layers; it hands them to you already wired.

STEP BY STEP

1. **Start from the need.** "Give me a capable agent now" points to the harness. "I want to assemble the agent myself" points to the framework. "I need to control every node and edge" points to the runtime.
2. **Check the dependency direction.** Deep Agents → LangChain → LangGraph. You can always reach down a layer.
3. **Map the features.** Short-term memory is `StateBackend` in Deep Agents and thread state in LangGraph. Long-term memory is `StoreBackend` and the LangGraph store. Subagents correspond to LangGraph subgraphs. Human-in-the-loop corresponds to interrupts, to the HITL middleware and to `interrupt_on`.
4. **Note the altitude.** The harness exposes each concept at the highest level; LangGraph exposes it at the lowest.

Layer	You get	You give up
Deep Agents (harness)	Speed, sensible defaults, long-task features	Fine control over the loop; some opinionated defaults
LangChain (framework)	Models, tools, `create_agent`, middleware	You assemble context, delegation and storage yourself
LangGraph (runtime)	Total control of nodes and edges	You design every behaviour yourself

ADVANTAGES

- Layers compose. Wrap deterministic steps around a deep agent in a `StateGraph`, or use a LangGraph subgraph as a subagent.

LIMITATIONS

- More layers means more places to look when something breaks. Know which layer owns each symptom.

EDGE CASES AND TRAPS

- Middleware is not a separate runtime. It is a set of hooks inside the compiled graph, which is exactly why a whole agent plus its middleware can sit as one node in a larger graph.
- Choosing the harness is not a one-way door. If the control flow turns out to need to be deterministic, you can move that part to LangGraph and keep the agent as a node.

LIKELY FOLLOW-UPS

- *Can you use LangGraph features directly on a deep agent?* Yes: checkpointers, stores, `Command(resume=...)`, `stream`, `update_state`.
- *When would you leave Deep Agents entirely?* When the control flow must be an explicit, auditable graph.

Q6. Walk me through everything that happens in one turn.

Short answer. Load thread state, assemble the system prompt, run the middleware hooks around a model call, execute whichever tools the model requested (offloading large results and delegating through `task`), loop until the model stops asking for tools, then run the after-hooks and checkpoint. This is *the* trace question: answer it slowly and in order.

EXPLANATION

An interviewer asking this is checking whether you have an accurate internal model. The test of a good answer is that you can say, at each step, *which hook fires* and *what could go wrong*.

STEP BY STEP

1. **Invoke.** The caller sends `{"messages": [...]}` plus `config` (with `thread_id`) and optional `context`. With a checkpointer, LangGraph loads the thread's state.
2. **Start-of-run hooks.** `before_agent` hooks run once, first to last. Skill metadata is loaded the first time a thread runs, and is then cached in state and not reloaded on later turns. Memory files named in `memory=` are read so they can be injected into the prompt.
3. **Prompt assembly.** The system prompt is built in a fixed order: your `system_prompt`, the base agent prompt, memory, the skills list, the filesystem prompt (plus `execute` documentation on a sandbox), the `task` prompt, custom middleware prompts, and last the human-in-the-loop prompt if `interrupt_on` is set.
4. **Before each model call.** `before_model` hooks run first to last, and `wrap_model_call` hooks nest with the first middleware outermost. Summarization runs here: if the context has reached its threshold, history is compacted before the model sees it. Prompt caching also applies here for Anthropic and Bedrock models.
5. **Model call.** The model returns either a final answer or an `AIMessage` with tool calls. `after_model` hooks run **last to first**.
6. **Approval.** If a requested tool is listed in `interrupt_on` (or a permission rule says `interrupt`), the graph pauses and returns `__interrupt__`. Nothing executes until a human decides and the caller resumes with the same `thread_id`.

7. **Tool execution.** Built-in file tools go through the backend, after permissions are checked. `task` starts a subagent in a fresh context. Custom and MCP tools run as normal. A tool result over about 20,000 tokens is written to the backend and replaced by a path plus the first 10 lines.
8. **Repair.** `PatchToolCallsMiddleware` fixes dangling or malformed tool calls so that the next model call is valid.
9. **Loop.** Steps 4 to 8 repeat until the model returns a message with no tool calls. Every step is checkpointed.
10. **Finish.** `after_agent` hooks run last to first, and the final state is returned.

ADVANTAGES

- Every behaviour has a hook, so every behaviour can be observed, replaced or extended.

LIMITATIONS

- Many things happen implicitly, so a silent default (a truncated result, a compacted history) can change behaviour without any error.

EDGE CASES AND TRAPS

- Summarization acts only when offloading has nothing left to remove, and it is text-only: media in the compacted range stops reaching the model.
- A resumed tool call **re-executes from the top** of the interrupted node, so side effects that happen before an `interrupt()` run twice.
- `wrap_*` hooks may call the handler zero, one or many times. Never assume a wrapped call happened exactly once.
- Hooks that run "first to last" on the way in run "last to first" on the way out. Forgetting the reversal is the commonest ordering mistake.

LIKELY FOLLOW-UPS

- *Where would you add an audit log?* A `wrap_tool_call` middleware, written as async if you will deploy on Managed Deep Agents.
- *Why does memory come after prompt caching in the stack?* So that edits to memory do not invalidate the cached static prefix.

Q7. Why is the harness delivered as middleware, and what does that buy and cost?

> **Short answer.** Because middleware lets you add or change behaviour at defined points around the model call and the tool call without touching the loop itself. The loop stays standard, every feature is a replaceable unit, and you can add your own. The cost is ordering rules and replace-not-merge surprises.

EXPLANATION

Imagine the alternative: a custom agent class where planning, summarization and permissions are all tangled into one big `run` method. Changing one would risk breaking the others. Middleware instead gives each concern its own object with a small set of hooks (`before_agent` , `before_model` , `wrap_model_call` , `after_model` , `wrap_tool_call` , `after_agent`), and the loop calls them at fixed points.

STEP BY STEP

1. `create_deep_agent` assembles the stack in this order: Filesystem, SubAgent, Summarization, PatchToolCalls, AsyncSubAgent, **your middleware**, profile extras, Skills, prompt caching, Memory, HumanInTheLoop, then an excluded-tool filter.

2. Read it as a story. Capabilities first (files, subagents), then context control (summarization, patching), then your own hooks, then prompt-shaping layers (skills, caching, memory), then gating (approvals), and finally a filter that hides excluded tools.
3. Know the conditions. Filesystem, Summarization and PatchToolCalls are always present. SubAgent is present when at least one synchronous subagent exists (the general-purpose one is added automatically). AsyncSubAgent needs async subagents. Skills needs `skills=`. Memory needs `memory=`. HITL needs `interrupt_on=`.
4. To customise a built-in, pass an instance of the same type. With `deepagents>=0.7` an instance whose `.name` matches **replaces** the built-in in place.

ADVANTAGES

- Features are independent, testable units.
- You can extend with your own hooks (logging, guardrails, retries) at the same points the built-ins use.
- Order is explicit and documented, so behaviour is predictable.

LIMITATIONS

- Order matters. A hook placed on the wrong side of summarization or caching changes what it sees.
- A same-name replacement is **not merged**. A custom `FilesystemMiddleware` needs its own `backend=` and `permissions=`, or the agent's earlier settings are gone.
- Concurrency: subagents, parallel tool calls and threads run at the same time, so a hook must not store per-run data on `self`.

EDGE CASES AND TRAPS

- `FilesystemMiddleware`, `SubAgentMiddleware` and the permission middleware **cannot be excluded**; trying raises `ValueError`.
- Before 0.7 a same-name middleware raised a duplicate error. Remove old workarounds on upgrade.
- A read-only filesystem is expressed through `FilesystemMiddleware(tools=["read_file", "ls", "glob", "grep"])`, and `read_file` must be in that list.

LIKELY FOLLOW-UPS

- *Where does your own middleware land?* After `PatchToolCallsMiddleware`.
- *How would you change the summarization threshold?* Pass a `SummarizationMiddleware(model=..., backend=..., trigger=("tokens", 100_000), keep=("messages", 20))`. The name matches, so it replaces the built-in.

Q8. Deep Agents versus the Claude Agent SDK: why not just use the SDK?

> **Short answer.** Deep Agents is model-agnostic, has pluggable backends, can run the agent either inside *or* outside the sandbox, and ships a multi-tenant deployment path. The Claude Agent SDK is Claude-only, built around the agent running inside a sandbox, and leaves the server layer to you. Neither is "better"; they optimise for different things.

EXPLANATION

A comparison answer earns marks for naming the *axes* of difference and then choosing by requirement. Do not argue that one product wins.

Axis	Deep Agents	Claude Agent SDK
Where the agent runs	Inside the sandbox, or outside calling a remote sandbox as a tool	Inside the sandbox
Execution backend	Pluggable: state, local, remote sandbox, custom	The sandbox's local filesystem
Model provider	Any provider with tool calling	Claude (via Anthropic, Bedrock, Vertex or Azure)
Deployment and tenancy	Managed Deep Agents or `langgraph build`, with scoped threads, per-user sandboxes and role-based access	You write the HTTP, WebSocket or SSE layer, authentication and tenancy

STEP BY STEP

1. List the requirements: which providers, how many tenants, who hosts it, how much of the server you want to write.
2. If you need several providers, multiple tenants or a hosted deployment path, the Deep Agents column fits.
3. If you are all-Anthropic and happy to own the server, the SDK is a reasonable choice.
4. Ask where secrets must live. This decides the "inside or outside the sandbox" pattern.

ADVANTAGES OF RUNNING THE AGENT OUTSIDE THE SANDBOX

- API keys stay on your server rather than inside the sandbox.
- Agent state survives if the sandbox dies.
- You can swap or restart sandboxes without losing the conversation.

LIMITATIONS OF RUNNING IT OUTSIDE

- Every sandbox call crosses a network, so there is added latency per call.

EDGE CASES AND TRAPS

- Running the agent *inside* the sandbox puts its API keys inside the sandbox, which is exactly where untrusted code runs. Know both sides of this trade-off.
- Do not claim a winner. Say what each one optimises for and how you would decide.

LIKELY FOLLOW-UPS

- *Which pattern do Deep Agents' own examples use?* The sandbox-as-a-tool pattern (agent outside, sandbox as a tool).
- *Can you start with one and move?* Yes, provided you keep tool contracts and prompts model-neutral.

Scenario Questions

Scenarios test judgement. Restate the situation, diagnose before prescribing, propose the smallest fix that works, then say what could go wrong with it.

Q9. A teammate proposes Deep Agents for an FAQ bot that calls one lookup API. Do you agree?

Scenario. The bot receives a question, calls one internal search API, and replies in two or three sentences. Conversations are short. A teammate says, "Let's use Deep Agents, it has everything."

Short answer. No. Use `create_agent`. The harness earns its weight on long, multi-step, context-heavy work, and this task is short, single-purpose and never approaches the context limit.

STEP BY STEP

1. **Restate the constraint.** Few tool calls, short answers, low latency expected, low cost per request.
2. **Check how long the task runs.** If it ends in a handful of calls and never approaches the context limit, offloading, summarization and subagents add weight without benefit.
3. **Check how rigid the workflow is.** If the steps are known, a graph or a plain agent is clearer than an agent that plans its own work.
4. **Check cost and latency.** The harness adds prompt text and tool schemas. Version 0.7 cut that sharply, but a minimal agent is still lighter.
5. **Check governance.** If you must prove the order of operations, an agent that decides its own plan is a harder fit than a graph.

ADVANTAGES OF CREATE_AGENT HERE

- Fewer tokens per call, simpler traces, easier evaluation.

LIMITATIONS OF THE LIGHTER CHOICE

- If the requirements grow (long documents, multi-step research, approvals), you will migrate. That is a normal evolution and cheap to do, because the two share LangChain's building blocks.

EDGE CASES AND TRAPS

- "Use Deep Agents for everything" is as weak an answer as "never". Show the decision rule instead.
- The two compose. A deep agent can be a node in a LangGraph pipeline, and a LangGraph subgraph can be a `CompiledSubAgent`.

LIKELY FOLLOW-UPS

- *What would make you switch?* Tool results that no longer fit in context, a need for parallel work, or a need for human approvals.
- *How would you combine them?* Wrap deterministic pre- and post-processing in a `StateGraph`, with the deep agent as the middle node.

Q10. You upgrade from 0.6 to 0.7 and three things break. What do you check?

Scenario. After a dependency bump, the planning panel in your UI is empty, a log parser that read `read_file` output fails, and a team member reports that the memory path they relied on no longer isolates users.

Short answer. Each symptom matches a 0.7 change. Planning is opt-in now (add `TodoListMiddleware`), `read_file` no longer returns a line-number gutter and empty `ls` results say "No files found", and `StoreBackend` needs an explicit `namespace` with concrete backend instances instead of the old factories. Do not plan to roll back across 0.6, because 0.6 introduced delta checkpoints.

STEP BY STEP

1. **Planning.** `create_deep_agent` no longer adds `TodoListMiddleware`. If your UI or prompts expect `write_todos`, add the middleware back explicitly.

2. **Parsers.** `read_file` output has no line-number gutter, and an empty `ls` or `glob` result returns the text "No files found" rather than `[]`. Update every parser and test fixture.
3. **Backends.** `BackendFactory`, `BACKEND_TYPES`, `FileFormat` and `Unset` are gone. Pass real backend instances, and give `StoreBackend` an explicit `namespace` (this is also the tenant-isolation lever).
4. **Middleware.** Delete any workaround for duplicate-middleware errors and use name replacement.
5. **Checkpoints.** Verify before deploying: threads persisted with delta checkpoints (`DeltaChannel`, beta, introduced in 0.6) cannot be read by older versions.
6. **Evaluate.** The default prompts are much shorter (the changelog's benchmark shows input tokens falling from 5,395 to 1,895, about 65%). Re-run your evals, because a custom prompt that leaned on built-in prose may behave differently.

ADVANTAGES OF THE NEW DEFAULTS

- Lower token cost per call, and less surprising prompt prose.

LIMITATIONS

- Behaviour that was implicit (planning on by default) is now something you must ask for.

EDGE CASES AND TRAPS

- The delta-checkpoint warning is a classic gotcha: once threads have been saved with deltas you cannot simply downgrade.
- Dates and numbers here come from the changelog as captured on 22 October 2026. Confirm them against the live changelog.

LIKELY FOLLOW-UPS

- *What arrived in 0.4, 0.5 and 0.6?* 0.4: sandbox integrations and summarization in the model node. 0.5: async subagents and multimodal `read_file`. 0.6: the interpreter, event streaming v3, harness profiles, `ContextHubBackend` and delta checkpoints.
- *How would you make the upgrade safe?* Pin the version, run the evals before and after, and migrate in a branch with a copy of production threads.

Q11. Your agent is correct but expensive. How would you control model cost without losing quality?

Scenario. A research agent uses a top-tier model for everything. Quality is good, but most tokens are spent by bulk subtasks such as reading and summarising pages.

Short answer. Keep a strong model as the coordinator and give bulk subtasks to subagents that use a cheaper model. Where the model must change per request, switch it with a `wrap_model_call` middleware that reads a model name from `context`. Set shared construction parameters once with a provider profile.

STEP BY STEP

1. **Find where the tokens go.** Trace a few runs. Bulk reading and summarising usually dominates.
2. **Split by role.** Give a research subagent a smaller `model` field and keep the main agent on the strong model, so judgement stays strong and volume gets cheaper.
3. **Switch per request if needed.** Add a `context_schema` with a `model` field and a `@wrap_model_call` that calls `request.override(model=init_chat_model(request.runtime.context.model))`.

4. **Share defaults.** `register_provider_profile("openai", ProviderProfile(init_kwargs={"temperature": 0}))` applies to every `openai:` string. A model-level key such as `"openai:gpt-5.5"` inherits the provider-level fields and overrides some.
5. **Measure again.** Compare cost and answer quality on the same test set before and after.

ADVANTAGES

- Large cost reduction with little code, and no change to the loop.

LIMITATIONS

- A cheaper model may follow instructions less reliably. Tool-calling reliability and context size come first when choosing.
- Different providers have different tool-call quirks, so test each pairing.

EDGE CASES AND TRAPS

- `openai:` **strings use the Responses API by default.** To use Chat Completions pass a pre-built model with `use_responses_api=False`. To keep Responses but avoid retention, pass `store=False` and `include=["reasoning.encrypted_content"]`.
- Provider profiles apply **only to strings**, never to pre-built model objects.
- Register profiles **before** creating agents.
- Everything after the **first** colon in a model string is the provider's model ID, so IDs may themselves contain colons or slashes.
- The docs' benchmark scores are "necessary but not sufficient" and shift between runs. Treat them as relative guidance.

LIKELY FOLLOW-UPS

- *Where do API keys go?* In the environment (`.env` locally, workspace secrets when deployed), never in code.
- *How are transient errors handled?* Chat models retry with backoff, controlled by `max_retries` and `timeout`; add `ModelRetryMiddleware` for more control.

Q12. Users say the deployed agent "forgets everything" between messages. What do you do?

Scenario. You wrapped the agent in an HTTP endpoint. Each request calls `agent.invoke({"messages": [new_message]})`. In the second request the agent has no idea what was said in the first.

Short answer. The graph has no checkpointer, or the caller does not send a stable `thread_id`. Add a checkpointer, pass the same `thread_id` each time, and use a store plus a memory route if facts must survive across threads.

STEP BY STEP

1. **Diagnose.** Two causes produce this symptom: no checkpointer (nothing is saved) or a new `thread_id` per request (a new conversation every time). Inspect the config sent on each call.
2. **Fix thread memory.** Create the agent with `checkpointer=` (an in-memory saver locally, a durable one in production) and send `config={"configurable": {"thread_id": user_conversation_id}}` on every call.
3. **Separate the two kinds of memory.** The checkpointer is *short-term*: it holds one thread's state. If preferences or facts should persist across threads, route a path such as `/memories/` to a `StoreBackend` with a per-user namespace, and point `memory=` at an `AGENTS.md` there.

4. **Send only the new message.** With a checkpointer, you pass just the new user message. Re-sending the whole history duplicates it.
5. **Verify.** Run two calls on the same `thread_id` and one on a different thread, and check what each remembers.

ADVANTAGES

- Thread state also gives you resumable interrupts and recovery after a crash.

LIMITATIONS

- State grows with the conversation, so summarization and offloading still matter. A durable checkpointer must be sized and cleaned up.

EDGE CASES AND TRAPS

- An in-memory saver works on a laptop and fails in production behind several processes, because each process has its own memory. Use a durable store.
- Putting a user ID in the `store` namespace is what isolates users. A shared namespace makes everyone share memory.
- `thread_id` names the conversation and `context` carries per-run data. Do not use one for the other.

LIKELY FOLLOW-UPS

- *Where does Managed Deep Agents help?* It provides the thread and durable store plumbing for you.
- *How do you reset a conversation?* Start a new `thread_id`.

CHAPTER 65

Interview Questions: Tools, Backends and Permissions

At a glance: Twelve questions on what the agent can touch and what it may touch. Four direct questions on tools, backends and permission rules; four concept questions on why they are designed the way they are; four scenarios with leaks, ordering mistakes and storage design.

Direct Questions

Short factual questions. State the fact, say what it means, then add one edge case.

Q13. Which tools does a deep agent get by default, and when is each one available?

Short answer. The file tools (`ls` , `read_file` , `write_file` , `edit_file` , `glob` , `grep`) are always present, `delete` since 0.7 when the backend can delete, and `task` when at least one synchronous subagent exists. `execute` needs a sandbox or local-shell backend. `write_todos` and `eval` are opt-in through middleware.

EXPLANATION

The harness adds its own tools next to the ones you pass in `tools=` . Each is gated by something, and the gating is the interesting part.

Tool	Purpose	Available when
`ls`	List files with size and modified time	Always
`read_file`	Read with line offset and limit; returns image, video, audio and PDF blocks for multimodal models	Always
`write_file`	Create a file or overwrite	Always
`edit_file`	Exact string replacement (`replace_all` for every occurrence)	Always
`glob`	Find files by pattern such as `**/*.py`	Always
`grep`	Search contents (files only, content with context, or counts)	Always
`delete`	Delete a file or a directory recursively	`deepagents>=0.7` ; hidden if the backend cannot delete
`execute`	Run a shell command	Sandbox backends and `LocalShellBackend` only
`task`	Start a subagent	At least one synchronous subagent (the general-purpose one is added by default)
`write_todos`	Maintain a structured task list	Opt-in with `TodoListMiddleware` (since 0.7)

Tool	Purpose	Available when
`eval`	Run JavaScript in the QuickJS interpreter	Opt-in with `CodeInterpreterMiddleware` (beta)

STEP BY STEP

1. `create_deep_agent` builds the middleware stack. Filesystem middleware contributes the file tools, SubAgent middleware contributes `task`.
2. The backend decides which optional tools survive. A backend without the sandbox protocol has `execute` **filtered out**; the model never sees it. A backend that cannot delete hides `delete`.
3. Your own tools, MCP tools and provider tool dictionaries are added to the list.
4. The model sees the combined list as tool schemas on every call.

EDGE CASES AND TRAPS

- `write_todos` is not there by default. People upgrading from 0.6 often assume the planning panel will keep working.
- The tool schema of a custom function is inferred from its **type hints and docstring**. A vague docstring degrades tool selection more than most prompt problems.
- The model can only use `execute` if the backend implements it. "Why can the agent not run `pytest`?" usually means the backend is a state backend.

LIKELY FOLLOW-UPS

- *How do MCP tools arrive?* Through an adapter that lists the server's tools (async only), which you pass in `tools=`.
- *Can a tool return an image?* A custom tool may return a list of content blocks, provided the model supports multimodal tool results.

Q14. What is a backend, which ones exist, and what is the default?

Short answer. A backend is the pluggable layer behind the file tools. The agent sees one filesystem; you decide where the bytes live. The default is `StateBackend`: files stored in LangGraph state, scoped to the thread, with no access to the host.

EXPLANATION

The backend also stores things that are not your files: skills, memory, offloaded tool results and summarised history. That is why choosing a backend matters even for agents that never write a file on purpose.

Backend	Storage and scope	`execute`?	Typical use
`StateBackend` (default)	LangGraph state, per thread, persisted by the checkpointer	No	Scratch space, offloaded results
`FilesystemBackend`	Real local disk under `root_dir`	No	Local development, CI, mounted volumes
`LocalShellBackend`	Local disk plus the host shell	Yes	Trusted development only, no sandboxing
`StoreBackend`	LangGraph store, across threads	No	Durable memory
`ContextHubBackend`	LangSmith Hub repository, with commit history	No	Versioned persistence without a separate store
`CompositeBackend`	Routes by path prefix to other backends	Depends on the routes	Mixing scratch and durable storage

Backend	Storage and scope	`execute`?	Typical use
Sandbox backends	Isolated remote environment	Yes	Production code execution
Custom	Anything: a database, S3	Optional	Integrations

STEP BY STEP (HOW TO CHOOSE)

1. Ask whether anything must outlive the thread. If so, you need a store, a hub or a composite.
2. Ask whether the agent must run code. If so, you need a sandbox.
3. Ask whether this is local development. If so, a filesystem or local-shell backend is acceptable.
4. Ask who the tenants are, and where their data must be separated.

EDGE CASES AND TRAPS

- `StateBackend` is checkpointed every step, so **large files add up** in every checkpoint. It also has no effect for calls made outside a graph run.
- `FilesystemBackend(virtual_mode=False)` (the default) has **no path restriction**, even with `root_dir` set. `virtual_mode=True` confines paths, but gives no protection once a shell exists.
- A bare `FilesystemBackend` writes internal artifacts (offloaded results, history) to **real disk**. Wrap it in a `CompositeBackend`.
- `FilesystemBackend` and `LocalShellBackend` must never be used in deployed agents.
- Skill and memory files must be **seeded** into a `StateBackend` with `create_file_data`; raw strings are rejected.

LIKELY FOLLOW-UPS

- *What is `ContextHubBackend` good for?* Version history for free. Every write is a Hub commit with optimistic concurrency, so stale writes fail and you must re-pull and retry.
- *What if you write a custom backend?* Implement `BackendProtocol`, return errors in the result object rather than raising, and remember `delete` is optional.

Q15. How are permission rules evaluated?

Short answer. Rules are checked **in declaration order**. The **first rule whose operations and paths both match decides**. If no rule matches, access is **allowed**. Permissions apply only to the built-in file tools.

EXPLANATION

A rule has three parts: the operations it covers (`read` means `ls`, `read_file`, `glob` and `grep`; `write` means `write_file`, `edit_file` and `delete`), the path patterns (globs, where `**` is recursive and `{a,b}` gives alternatives), and a mode (`allow`, `deny` or `interrupt`).

```python
permissions = [
    FilesystemPermission(operations=["read", "write"], paths=["/workspace/.env"], mode="deny"),
    FilesystemPermission(operations=["read", "write"], paths=["/workspace/**"], mode="allow"),
    FilesystemPermission(operations=["read", "write"], paths=["/**"], mode="deny"),
]
```

STEP BY STEP

1. A tool call arrives, for example `read_file("/workspace/.env")`. The operation is `read`.
2. The rules are tried from the top. Rule 1 matches both the operation and the path, and says deny. Evaluation stops.

3. For `read_file("/workspace/notes.md")`, rule 1 does not match, rule 2 does, and the result is allow.
4. For `read_file("/etc/passwd")`, only rule 3 matches, and the result is deny.
5. Remove rule 3 and the last call is **allowed**, because nothing matched.

ADVANTAGES

- Declarative and auditable: a list, not code.
- Evaluated **before** the backend is called, so it works with any backend.
- Three outcomes, including human approval.
- Subagents inherit the rules.

LIMITATIONS

- Order matters, and the failure is silent: a broad allow above a specific deny makes the deny unreachable.
- The default is permissive. Without a catch-all deny, unlisted paths are open.
- Only the built-in file tools are covered.

EDGE CASES AND TRAPS

- `mode="interrupt"` pauses for approval on matching writes and deletes, needs a checkpointer and `deepagents>=0.6.8`.
- Directory deletion is **all-or-nothing**: it checks `write` on the target and every descendant, and denies the whole operation if any is denied, even empty directories. A single file uses ordinary first-match (an earlier narrow allow beats a later catch-all deny, from `deepagents>=0.7.3`).
- If the default backend of a `CompositeBackend` is a sandbox, every permission path must fall under a route prefix. A path outside any route, including `/**`, raises `NotImplementedError`.

LIKELY FOLLOW-UPS

- *How do you make memory read-only?* `FilesystemPermission(operations=["write"], paths=["/memories/**"], mode="deny")`.
- *How do you make an agent read-only overall?* Deny `write` on `/**`.

Q16. How do you add your own tools, and what makes one good?

Short answer. Pass a plain function, a LangChain `@tool` or a tool dictionary in `tools=[...]`. The schema is inferred from the function's type hints and docstring, so the docstring is the tool's interface. A good tool has one clear job, typed arguments, and a result that is compact, predictable and safe to feed back to the model.

EXPLANATION

The model never sees your code. It sees a name, a description and a parameter schema, and chooses a tool by reading them. Everything you want the model to know about when to call a tool must therefore be in those three places.

STEP BY STEP

1. Write a function with type hints. Use `Literal[...]` for closed sets so the model cannot invent values.
2. Write a docstring that says what the tool does, when to use it and what it returns.
3. Return a string or structured text that is as short as it can be. Large results are offloaded by the harness, but compact results are cheaper and more accurate.
4. For third-party servers, load MCP tools through the adapter (async) and pass them in `tools=`.

5. For built-in provider capabilities (such as web search), pass the provider's tool dictionary rather than hosting the tool yourself.

ADVANTAGES

- No separate schema to maintain; works with any provider.

LIMITATIONS

- Vague docstrings degrade tool selection.
- Tools bypass filesystem permissions. A custom tool that reads files is not stopped by a `FilesystemPermission`.

EDGE CASES AND TRAPS

- Return errors as readable text. An unhandled exception or a cryptic code leaves the model guessing.
- A tool that mutates state (sends an email, posts to Slack) should be gated by `interrupt_on`, since permissions do not cover it.
- MCP tools through the adapter are async only, so an app that calls the agent synchronously must be adapted.

LIKELY FOLLOW-UPS

- *How does a tool return an image?* A list of content blocks, with a model that accepts multimodal tool results.
- *Where do secrets go?* Inside the tool (host-side), never in the prompt.

Concept Questions

These test whether you understand why the design is shaped the way it is.

Q17. Why does Deep Agents use a pluggable backend, and what are the trade-offs of each choice?

> **Short answer.** Because the same agent has to run in very different storage realities: scratch space in a thread, files on a laptop, durable per-user memory, an isolated sandbox, your own bucket. A backend keeps the tool surface identical while the storage changes, so you can swap storage without touching the prompt.

EXPLANATION

Without the abstraction you would write a different set of file tools for each environment, and each would need its own prompt text. With it, the model sees one filesystem, and you decide the reality behind it.

STEP BY STEP

1. The model calls `read_file("/notes/plan.md")`.
2. Permissions are checked (if configured).
3. The call is passed to the backend's `read` method.
4. The backend returns a result object, including an `error` field when something fails.
5. The harness formats the result for the model.

ADVANTAGES

- One tool surface (`ls`, `read_file`, `write_file`, `edit_file`, `glob`, `grep`, `delete`) regardless of where the bytes live.

- Swap storage without changing the prompt.
- Route by path with `CompositeBackend`.
- Write your own backend for your own storage.

LIMITATIONS

- `StateBackend` is thread-scoped and checkpointed, so large files add up.
- `FilesystemBackend` and `LocalShellBackend` are not safe in deployed agents.
- Durable storage needs an explicit store and namespace.
- Every path you permission-gate must fall under a route when the default backend is a sandbox.

TRADE-OFFS

Choice	You gain	You pay
`StateBackend`	No setup, thread isolation, free checkpointing	Not shared across threads; large files bloat checkpoints
`StoreBackend`	Durable and cross-thread	Needs a store and a namespace factory; a missing namespace shares data
`FilesystemBackend`	Real files, real tools	No isolation by default; writes internal artifacts to disk
Sandbox	A real shell and isolation	Cost until shut down, latency, secrets caution
Custom	Your storage	You implement the protocol and own the edge cases

EDGE CASES AND TRAPS

- A custom backend must **return errors in the result object rather than raise**. A raised exception breaks the loop instead of informing the model.
- `grep` in the protocol is a **literal-string** search, not a regular expression. A custom backend that interprets it as a regex will surprise the model.
- `write` is create-only in the protocol: a conflict is an error result, and overwrite is a separate decision.
- `files_update` and callable `backend=` are deprecated. Pass instances.

LIKELY FOLLOW-UPS

- *How would you enforce a policy such as "never write under `/prod/`"?* Declarative permissions for simple path rules; a delegating backend wrapper for logic such as rate limits or audit logs.
- *How are binary files handled?* With `offload_binary_content=True` (`deepagents>=0.7.21`, opt-in) blobs go to `/blobs/<sha256>`, message history keeps references, and media is restored before each model call.

Q18. How does `CompositeBackend` route, and what is the canonical memory pattern?

Short answer. It routes by **path prefix**, and the **longest matching prefix wins**. Unmatched paths go to the `default` backend. The canonical pattern is `default=StateBackend()` for thread scratch and `routes={"/memories/": StoreBackend(namespace=...)}` for durable per-user memory.

EXPLANATION

Think of it as a mount table. The agent sees one tree. Under `/memories/` the files are durable and per user. Everywhere else they are scratch files that disappear with the thread.

```python
backend = CompositeBackend(
    default=StateBackend(),
    routes={"/memories/": StoreBackend(
        namespace=lambda rt: (rt.server_info.user.identity,)
    )},
)
agent = create_deep_agent(model=..., backend=backend, store=store)  # store goes here
```

STEP BY STEP

1. Create the composite with a default and a dictionary of prefix to backend.
2. A path is resolved by checking prefixes from longest to shortest, so `/memories/projects/` beats `/memories/`.
3. Listing operations (`ls`, `glob`, `grep`) fan out across backends and keep the original prefixes in their results.
4. Pass the **store** to `create_deep_agent(store=...)`, not to the composite.
5. Add a namespace factory so that each user has a private partition.

ADVANTAGES

- Durable and ephemeral data coexist with one set of tools.
- Using `StateBackend` as the default keeps internal data (offloads, summarised history) ephemeral, which is usually what you want.

LIMITATIONS

- If the **default backend is a sandbox**, every permission path must fall under a route prefix, or `NotImplementedError` is raised.
- Routing is by prefix only; there is no routing by file type or size.

EDGE CASES AND TRAPS

- Without a route to a store, `/memories/` is not durable at all. It is just a folder in thread state, and it vanishes with the thread. This is the most common "memory does not persist" bug.
- The store is passed to `create_deep_agent`, not the composite. Passing it in the wrong place leaves the store backend without a store.
- A trailing slash in the prefix matters for how paths are matched, so use `"/memories/"` consistently.

LIKELY FOLLOW-UPS

- *How do you pre-seed shared knowledge?* Write it to the store from application code: `store.put(namespace, path, create_file_data(text))`.
- *How do you make part of the tree read-only?* Add a permission rule that denies `write` on that prefix.

Q19. How do namespace factories isolate tenants, and what do they not do?

Short answer. A `StoreBackend` takes a **namespace factory**, a function from the runtime to a tuple of strings. The tuple is the partition key for the data. It isolates *storage*. It does not authenticate anyone. From 0.7 you must give `StoreBackend` an explicit namespace.

EXPLANATION

`NamespaceFactory = Callable[[Runtime], tuple[str, ...]]`. The runtime exposes your own `context`, the `server_info` (assistant id, graph, verified user identity) and `execution_info` (thread id, run id, checkpoint id).

Scope	Factory
Per user	`lambda rt: (rt.server_info.user.identity,)`
Per assistant	`lambda rt: (rt.server_info.assistant_id,)`
Per thread	`lambda rt: (rt.execution_info.thread_id,)`
User and assistant	`lambda rt: (rt.server_info.assistant_id, rt.server_info.user.identity)`
Organization	`lambda rt: (rt.context.org_id,)`

STEP BY STEP

1. Decide the scope: user, assistant, thread, user plus assistant, or organization.
2. Express it as a lambda over `Runtime`, using the **verified** identity rather than a value the user typed.
3. Combine parts for tighter scoping.
4. Verify with two users: write as A, read as B, and confirm B sees nothing.

ADVANTAGES

- Tenant boundaries are expressed in one place and enforced in storage.

LIMITATIONS

- It gives you partitions, not permissions. A user who can claim another's identity can read that partition.

EDGE CASES AND TRAPS

- **Missing namespace means shared data.** In older defaults the legacy key was the assistant, so every user of that assistant saw the same files. This is the most-cited multi-tenant bug.
- Allowed characters in namespace parts are alphanumerics and `- _ . @ + : ~`. Wildcards `*` and `?` are rejected.
- `rt.server_info` needs `deepagents>=0.5.0`.
- Shared writable scopes (assistant or organization) are a **prompt-injection** channel: one user's malicious content becomes every user's instruction. Make shared scopes read-only and write them from application code.

LIKELY FOLLOW-UPS

- *Per-user plus per-org?* Use both parts in a tuple, or use two routes with two namespaces.
- *Where does authentication happen?* In the deployment layer (authentication and authorization handlers), not in the backend.

Q20. Why don't permissions protect `execute`, MCP tools and custom tools, and how do you secure those?

Short answer. Because permissions are path rules on the **built-in file tools**. A shell can run any command, and a custom or MCP tool can do anything its code does, so none of them go through the path check. You secure them with other layers: approvals, narrow tools, sandbox isolation and credential handling.

EXPLANATION

Permissions work because every built-in file tool takes a path and an operation, which can be checked against a rule. A shell command is just a string. `cat /workspace/.env` and `python -c "print(open('/workspace/.env').read())"` are the same intent in different clothes, and no path rule can see through that.

STEP BY STEP (DEFENCE IN DEPTH)

1. **Narrow the tools.** Prefer a specific tool (`get_invoice(id)`) over a general one (`run_sql(query)`).
2. **Gate by approval.** Put sensitive tools and `execute` in `interrupt_on` .
3. **Isolate execution.** Use a sandbox rather than a local shell, and restrict its network.
4. **Keep secrets out of reach.** Use host-side tools or an auth proxy so the agent never holds credentials.
5. **Treat results as untrusted.** Output from tools and sandboxes is input to the model and may carry injected instructions.
6. **Log.** A `wrap_tool_call` middleware gives you an audit trail.

ADVANTAGES OF THE LAYERED APPROACH

- No single layer has to be perfect.

LIMITATIONS

- Approvals add friction and can be rubber-stamped by tired reviewers. Reserve them for actions that matter.

EDGE CASES AND TRAPS

- The interpreter's programmatic tool calling (PTC) and `task()` inside code **skip the normal tool path**, so `interrupt_on` is not enforced per call. Gate the `eval` tool instead and keep its allowlist narrow.
- A `mode="interrupt"` rule covers only the built-in file tools too.
- Saying "permissions secure the agent" in an interview is a red flag. Say what they cover and what they do not.

LIKELY FOLLOW-UPS

- *How would you stop the agent reading `.env` with a shell?* Do not put `.env` where the sandbox can see it. Inject nothing secret into the sandbox at all.

Scenario Questions

Restate, diagnose, fix, then say what could go wrong with the fix.

Q21. A permission list protects `.env` , but the agent can still read it. Why?

Scenario. You wrote three rules: allow read and write on `/workspace/**` , deny read and write on `/workspace/.env` , and deny everything on `/**` . A test shows the agent reading `/workspace/.env` .

Short answer. The broad allow comes first, so it decides and evaluation stops. The deny for `.env` is unreachable. Put specific denies before the broad allow and keep the catch-all deny last.

STEP BY STEP

1. **Trace the call.** `read_file("/workspace/.env")` has operation `read` .
2. **Apply first-match.** Rule 1 matches (`read` is listed and the path is under `/workspace/**`) and says allow. Evaluation stops.
3. **Conclude.** Rules 2 and 3 never run for this call.
4. **Fix the order.** Deny `/workspace/.env` first, then allow `/workspace/**` , then deny `/**` .

5. **Test both sides.** Assert that `.env` is denied, that a normal workspace file is allowed, and that `/etc/passwd` is denied.

```python
permissions = [
    FilesystemPermission(operations=["read", "write"], paths=["/workspace/.env"], mode="deny"),
    FilesystemPermission(operations=["read", "write"], paths=["/workspace/**"], mode="allow"),
    FilesystemPermission(operations=["read", "write"], paths=["/**"], mode="deny"),
]
```

TRADE-OFFS

- Putting specifics first makes rules read like exceptions, which is how people reason about them anyway.

EDGE CASES AND TRAPS

- If the agent also has `execute`, it can read `.env` with a shell command and ignore every rule. The real fix may be to keep the file out of the environment.
- Subagents inherit the rules unless a subagent sets its own `permissions`, which **replaces** the parent's list.
- Add a regression test that asserts the order, since a later edit can reintroduce the bug without any error.

LIKELY FOLLOW-UPS

- *How would you require approval instead of denying?* Use `mode="interrupt"` with an anchored pattern such as `/secrets/**` (needs a checkpointer and `deepagents>=0.6.8`).

Q22. Two users see each other's memories. Find and fix it.

Scenario. Your assistant remembers preferences under `/memories/`. In production a user sees notes that clearly belong to someone else.

Short answer. The `StoreBackend` has no per-user namespace (or the route is missing and everything lives in a shared place). Add a namespace factory that uses the verified user identity, route `/memories/` to it, and test with two users.

STEP BY STEP

1. **Reproduce.** Write a memory as user A, read it as user B.
2. **Inspect the backend.** Is `/memories/` routed to a `StoreBackend`? If it is not, files sit in thread state, which is isolated per thread, so that setup would lose memory rather than leak it. A cross-user leak points to a store with a shared or default namespace.
3. **Inspect the namespace.** A missing namespace used a legacy default keyed by assistant, which every user of the assistant shares.
4. **Fix it.** `StoreBackend(namespace=lambda rt: (rt.server_info.user.identity,))`, routed under `/memories/`.
5. **Check authentication.** The identity must come from verified authentication, not from a value in the request body.
6. **Migrate.** Existing shared notes must be inspected and split or deleted, because a namespace change does not move old data.
7. **Test.** Add a two-user test to CI.

ADVANTAGES OF FIXING IT AT THE STORAGE LAYER

- The agent cannot ask for another user's data, whatever the prompt says.

LIMITATIONS

- Namespaces do not protect against a compromised identity or against a prompt that gets the agent to write secrets into a shared scope.

EDGE CASES AND TRAPS

- Shared scopes (assistant, organization) are a prompt-injection channel. Keep them read-only and write them from application code.
- Namespace parts have an allowed character set. An identity with unusual characters may be rejected.
- In 0.7 an explicit namespace is required, so the bug is more likely in an older deployment than a new one.

LIKELY FOLLOW-UPS

- *Would you also add a permission rule?* A deny on `write` to a shared prefix, yes.
- *How would you notify affected users?* Treat it as a data incident: scope, notify, rotate, document.

Q23. Design the storage layout for an agent with scratch work, durable memory and a shared read-only reference library.

Scenario. A support agent needs scratch space for each conversation, per-user preferences that persist, and a company policy library that every conversation can read but no conversation can change.

Short answer. Use a `CompositeBackend` with `StateBackend` as default for scratch, a per-user `StoreBackend` at `/memories/`, and a read-only policy route at `/policies/` seeded from application code. Add permissions that deny `write` on `/policies/**`.

STEP BY STEP

1. **Choose the default.** `StateBackend()`: scratch work and internal artifacts stay in the thread.
2. **Route memory.** `/memories/` to `StoreBackend(namespace=lambda rt: (rt.server_info.user.identity,))`.
3. **Route policies.** `/policies/` to a `StoreBackend` with a fixed, shared namespace, such as `("org", "policies")`.
4. **Seed policies from code.** Use `store.put(namespace, path, create_file_data(text))` at deploy time, not through the agent.
5. **Lock them.** `FilesystemPermission(operations=["write"], paths=["/policies/**"], mode="deny")`.
6. **Load what is always needed.** Point `memory=` at the user's `AGENTS.md` under `/memories/`, and put long reference material in `skills`, so it loads on demand rather than all at once.
7. **Test.** Check that scratch disappears on a new thread, memory survives, policies are readable and unwritable.

ADVANTAGES

- Each kind of data has the lifetime and visibility it should have.

LIMITATIONS

- More routes mean more rules to keep consistent, and permission paths must fall under a route if the default backend is a sandbox.

EDGE CASES AND TRAPS

- The longest prefix wins, so a nested route such as `/policies/drafts/` can override its parent. Do not create overlapping routes by accident.

- Policies are read by the model, which means they can also be quoted to users. Do not put internal-only data in a library the agent may paraphrase.
- Updating policies means writing to the store. Plan how new versions roll out and whether running threads see them.

LIKELY FOLLOW-UPS

- *How would you handle per-organization policies?* Namespace the policy route by `rt.context.org_id`.
- *What if the agent also needs a shell?* Add a sandbox backend and keep policies outside it; the shell should not be able to alter the library.

Q24. You are building a backend on top of your company's object store. What do you need to get right?

Scenario. Files must live in a bucket, with versioning and audit. The team wants a `BackendProtocol` implementation.

Short answer. Implement every required method with the documented semantics, return errors in result objects instead of raising, keep `grep` literal, make `write` create-only, make `delete` optional, and put policy checks in a wrapper or in declarative permissions.

STEP BY STEP

1. **Map paths to keys.** Virtual paths such as `/notes/plan.md` become `prefix/notes/plan.md`.
2. **Implement the methods.** `ls`, `read`, `write`, `edit`, `glob`, `grep` and (optionally) `delete`.
3. **Follow the rules.** `read` supports `offset` and `limit` and returns an error result for a missing file. `write` is create-only and reports a conflict as an error. `edit` requires `old_string` to be unique unless `replace_all`, and reports the occurrence count. `glob` returns an empty list when nothing matches. `grep` is a literal search.
4. **Return, do not raise.** Convert storage exceptions into the `error` field.
5. **Sort and describe.** Sort `ls` output by path and include `is_dir`, `size` and `modified_at` where you can.
6. **Add policy.** Use declarative permissions for path rules and a delegating wrapper for rate limits and audit logs.
7. **Test with the real tools.** Run an agent against it, not just unit tests.

ADVANTAGES

- The agent keeps the same tools and prompts while your storage provides durability, versioning and audit.

LIMITATIONS

- Object stores are not filesystems: no atomic rename, eventual consistency in some systems, and listing cost. `edit` becomes read-modify-write, which can race.

EDGE CASES AND TRAPS

- Concurrent subagents may edit the same key. Decide whether to use conditional writes or accept last-writer-wins.
- Binary content: use base64 encoding in `FileData`, or opt in to binary offloading.
- Check the import paths and result types in your installed version, since the protocol evolved (for example `BackendContext` became `Runtime` in 0.5.2, and several accessors were removed in 0.7).

LIKELY FOLLOW-UPS

- *If you do not implement `delete`?* The tool is hidden from the model.
- *How would you test the permission interaction?* Assert denied paths never reach your backend's methods.

CHAPTER 66

Interview Questions: Sandboxes, Interpreters and Streaming

At a glance: Twelve questions on running code safely, orchestrating tools from code, handling media, and streaming what the agent is doing. Direct questions cover the facts and limits, concept questions cover the security and design reasoning, and scenarios cover cost, secrets, a tool-heavy workload and a streaming UI.

Direct Questions

Facts first, then what they mean in practice.

Q25. When do you need a sandbox, and how does it differ from an interpreter or a local shell?

Short answer. You need a sandbox when the agent must write files, install packages, run tests or call command-line tools. A sandbox is an isolated remote environment exposed as a backend that adds the `execute` tool. An interpreter is an in-process JavaScript workspace for orchestration. `LocalShellBackend` is a host shell with no isolation, for trusted development only.

EXPLANATION

Three things can look alike from the model's side, because each lets it "run code". They differ in where the code runs, what it can touch and what it costs.

	Sandbox	Interpreter	`LocalShellBackend`
Isolation	OS-level, remote	WebAssembly engine, one per thread	None
Shell and package installs	Yes	No	Yes, on your host
Network and filesystem	Yes	None by default	Yes
Best for	Code, tests, command-line tools	Orchestration and data shaping	Trusted local development

STEP BY STEP

1. If the task is only reading and writing files, a filesystem backend is enough.
2. If it needs an operating system (a shell, `pip install`, test runners), choose a sandbox provider: LangSmith, Daytona, E2B, Modal, Runloop, Vercel, AgentCore or NVIDIA OpenShell.
3. If it needs to compose tool calls, hold state, loop and batch without paying tokens for every intermediate value, add the **interpreter**.
4. If you are prototyping on your own machine and trust everything, `LocalShellBackend` exists, with a 120-second timeout and a 100,000-byte output cap.

ADVANTAGES OF A SANDBOX

- Keeps agent-run code away from your host.
- Provides a real shell for installs, tests and command-line tools.
- Has several provider integrations, so you can pick on cost, speed and location.

LIMITATIONS

- It costs money until shut down, teardown differs by provider, and permissions do not cover `execute`.

EDGE CASES AND TRAPS

- Sandboxes **cost money until shut down**. Always tear down in a `finally` block (`delete_sandbox`, `stop()`, `kill()`, `terminate()` or `shutdown()`, depending on the provider) and set an **idle TTL**.
- If the backend does not implement the sandbox protocol, the `execute` tool is **filtered out** and the agent never sees it.
- Very large `execute` output is written to a file that the agent reads incrementally, which protects the context window.
- The documentation mentions both `SandboxBackendProtocol` and `SandboxBackendProtocolV2`. Check which one your version uses.

LIKELY FOLLOW-UPS

- *Thread-scoped or assistant-scoped sandbox?* See Q30.
- *Can the interpreter replace a sandbox?* No. It has no shell, network, filesystem or package manager.

Q26. What are the "two planes" of file access in a sandbox?

> **Short answer.** The agent's file tools work *through* `execute()` inside the sandbox. Your application code uses the provider's native file-transfer API (`upload_files` and `download_files`) from outside. Two planes, two trust levels.

EXPLANATION

`BaseSandbox` builds the agent's file tools on top of `execute()`, so every agent file operation is really a command run inside the sandbox. Your application is different: it talks to the provider directly and never passes through the agent.

STEP BY STEP

1. **Before the run**, seed inputs: `backend.upload_files([("/data/sales.csv", csv_bytes)])`. This bypasses the agent entirely.
2. **During the run**, the agent reads and writes with its own tools.
3. **After the run**, collect artefacts: `backend.download_files(["/out/report.png"])`. Each result carries `path`, `content` (bytes or `None`) and `error`.
4. Middleware can automate the cycle: upload in `abefore_agent`, download in `aafter_agent`.

ADVANTAGES

- The transfer API works for binary files and is safer than asking the agent to `cat` them.
- Your application controls what goes in and what comes out.

LIMITATIONS

- The transfer API is provider-specific, so portable code needs a small adapter.

EDGE CASES AND TRAPS

- Always **check `error`** per path. One failed download does not raise an exception.
- Use **absolute paths** in your instructions, so the agent does not waste calls on `pwd` and `ls`.
- Never trust a downloaded file. It was produced by an agent that may have been steered, so scan or validate it.

LIKELY FOLLOW-UPS

- *Why not give the agent a download tool?* It adds an agent-controlled path to your application, which is exactly the boundary the second plane keeps closed.

Q27. What does the interpreter provide, and what are its limits?

Short answer. `CodeInterpreterMiddleware` (package `langchain-quickjs`, beta) adds an `eval` tool that runs JavaScript in a QuickJS WebAssembly workspace, one instance per thread. Allowlisted tools appear as `tools.camelCaseName(...)` and subagents as `task()`. Defaults: 64 MB heap, 5 seconds per `eval`, 4,000 returned characters, 256 tool calls per `eval`.

EXPLANATION

The interpreter exists for one reason: so that orchestration logic can be deterministic code rather than many model turns, and so that large intermediate values never enter the model's context.

Parameter	Default	Meaning
`memory_limit`	64 MB	Heap cap per thread
`timeout`	5.0 s	Per `eval` call
`max_result_chars`	4000	Truncates the text returned to the model
`ptc`	`None`	Allowlist of tools exposed as `tools.*`
`max_ptc_calls`	256	`tools.*` calls per `eval`
`subagents`	`True`	Exposes the `task()` global
`mode`	`"thread"`	Persistence: `thread`, `turn` or `call`
`capture_console`	`True`	Captures `console.log`, `warn` and `error`

STEP BY STEP

1. Add the middleware with an allowlist: `CodeInterpreterMiddleware(ptc=["web_search"])`.
2. The model writes code. Python's `web_search` becomes `tools.webSearch`.
3. The code runs in the Wasm workspace. Only its returned value, truncated to 4,000 characters, goes back to the model.
4. State persists according to `mode`: across turns (`thread`, the default), across `eval` calls within one turn (`turn`), or not at all (`call`).

ADVANTAGES

- Intermediate values never enter the context, which is cheaper and less noisy.
- Loops, batching and retries live in code.
- Parallel tool and subagent calls with `Promise.all`.
- No host access by default.

LIMITATIONS

- It does not provide agent tools, filesystem, network, datetime, shell or package installs unless you grant a tool through the allowlist.
- Beta: the API may change. It needs `langchain-quickjs` 0.3.0 or later and Python 3.11 or later.

EDGE CASES AND TRAPS

- Snapshots keep only **serialisable** data. Functions are gone after a restore, and a snapshot restores memory, not external effects.
- The required `langchain-quickjs` version is inconsistent in the documentation (0.2.0 on one page, 0.3.0 on another). Install the newer one.
- It is **not** a place for shell commands, installs or tests.

LIKELY FOLLOW-UPS

- *Does it need a checkpointer?* Cross-turn persistence does not, but adding one also stores snapshots in checkpoint history.

Q28. How does multimodality interact with context compression?

> **Short answer.** `read_file` returns media as content blocks that the model can see, if the model supports that modality. **Offloading keeps non-text blocks; summarization drops media into text.** After a summarization, earlier images no longer reach the model.

EXPLANATION

Media can arrive in three ways: as blocks in the user's message, from `read_file` on supported types (images, video, audio, `.pdf`, `.ppt`, `.pptx`), or as a list of content blocks returned by a tool. Whether the model can use it depends on the model, not on the harness.

STEP BY STEP

1. Input media arrives as content blocks.
2. **Offloading** counts text tokens only. Images are kept as they are.
3. When **summarization** fires, the compacted range becomes a text-only summary. Media inside it is no longer visible to the model, although the full conversation is saved as text on the filesystem.
4. The agent can re-read a media file from the backend to look at it again, if it still has the path.

ADVANTAGES

- Standard LangChain content blocks for input and tool output; `read_file` handles many media types directly.

LIMITATIONS

- Depends on the model's modalities and the provider's MIME support.
- Base64 inflates token use quickly.
- Summarization silently drops media.

EDGE CASES AND TRAPS

- Save large media to a backend and return a short description plus a path, rather than embedding base64 in a long conversation.

- Delegate image-heavy inspection to a subagent that returns compact text.
- Supply a custom token counter or tune thresholds if image tokens are expensive.
- Checklist: the model accepts the modality as input, accepts it in tool results, and the provider supports your MIME types.

LIKELY FOLLOW-UPS

- *How do you keep an important screenshot visible after summarization?* Keep its path in the summary or in a file, and have the agent call `read_file` on it when needed.

Concept Questions

These test whether you understand the security model and the design reasoning.

Q29. What does a sandbox *not* protect against, and how do you handle secrets?

Short answer. A sandbox protects the **host**. It does not stop context injection (steering the agent into running commands) or network exfiltration. So **never put secrets inside a sandbox**. Use host-side tools or an authentication proxy that injects credentials from outside.

EXPLANATION

Anything inside the sandbox is readable by the agent, and therefore readable by anyone who has managed to inject instructions into the agent's context through a web page, a document or a tool result. The sandbox's job is to stop the damage reaching your machine. It was never meant to keep a secret from the thing running in it.

STEP BY STEP

1. **Threat model.** The agent can run arbitrary commands inside the sandbox, and the sandbox may have network access.
2. **Keep secrets out.** No environment variables, no mounted files, no provider `secrets` options, even short-lived ones.
3. **Use a host-side tool.** The agent calls it by name, the tool holds the credential, and the agent never sees it. The data-analysis tutorial posts to Slack this way.
4. **Or use an authentication proxy.** It sits outside the sandbox, matches outbound requests by host, and injects headers such as `Authorization: Bearer ...` from workspace secrets.
5. **If you absolutely must inject secrets:** require approval on all tool calls, restrict or block the network, use narrow short-lived credentials, and monitor traffic.
6. **Treat all sandbox output as untrusted input.** Redact sensitive patterns with middleware, and block the network when it is not needed.

ADVANTAGES

- A compromised agent can run commands but has nothing worth stealing.

LIMITATIONS

- Host-side tools must be written and maintained for each integration, and a proxy adds infrastructure.

EDGE CASES AND TRAPS

- **Thread-scoped** sandboxes (one per conversation, named `thread-{id}`) are clean and safer. **Assistant-scoped** sandboxes are shared across conversations, so files and packages accumulate and can leak between users unless you control scope.
- The sandbox ID belongs in thread metadata or assistant configuration as the single source of truth.
- In Managed Deep Agents an idle sandbox is deleted after `idle_ttl_seconds` (default 600) **together with its contents**, which cannot be recovered.

LIKELY FOLLOW-UPS

- *Is a sandbox enough to run untrusted code from the internet?* It protects the host. You still restrict network egress and credentials.

Q30. Should the agent run inside the sandbox, or call it as a tool? What about sandbox scope?

Short answer. The agent can run **inside** the sandbox (it mirrors local development, but keys live in the sandbox and you build the communication layer) or **outside** it, calling the sandbox as a tool (keys stay on your server and agent state survives sandbox failure, at the cost of per-call latency). Deep Agents' own examples use the second pattern. Separately, sandboxes can be thread-scoped or assistant-scoped.

EXPLANATION

Two independent decisions are often confused. *Where does the agent run?* and *how long does a sandbox live?*

Pattern	Pros	Cons
Agent runs *in* the sandbox	Mirrors local development	API keys live in the sandbox; you build the communication layer
Sandbox as a tool	Keys stay outside; failures do not lose agent state	Per-call latency

Scope	Behaviour
Thread-scoped (default)	One sandbox per conversation (`thread-{thread_id}`)
Assistant-scoped	One shared sandbox per assistant; files and packages persist

STEP BY STEP

1. List where secrets must live. If the agent holds provider keys, keep it outside the sandbox.
2. Decide how stateful the sandbox must be. A coding agent that builds up a project across a conversation fits thread scope. A long-lived shared toolbox fits assistant scope.
3. If you choose assistant scope, plan cleanup: TTLs, snapshots and a reset procedure.
4. Store the sandbox ID as metadata, so a resumed thread finds the same sandbox.

ADVANTAGES OF SANDBOX-AS-A-TOOL

- Restarting a sandbox does not lose the conversation, and keys never enter the untrusted environment.

LIMITATIONS

- Each call crosses a network boundary. A chatty agent feels the latency.

- Assistant-scoped sandboxes accumulate state, which may include one user's files.

EDGE CASES AND TRAPS

- Do not claim that one pattern is "correct". Say what each optimises for.
- Always set an **idle TTL**, in either scope.

LIKELY FOLLOW-UPS

- *What if the sandbox dies mid-run?* With the agent outside, the thread's state survives and the agent can create a fresh sandbox; with the agent inside, the run is lost.

Q31. What is programmatic tool calling, and what is its security model?

> **Short answer.** Programmatic tool calling (PTC) lets the model write code that calls allowlisted tools as functions (`tools.webSearch({ query })`) inside the interpreter. Loops, branches, retries and parallel calls run as code, and only the final value returns to the model. The cost is that PTC and `task()` **skip the normal tool-calling path**, so per-call approvals are not enforced and the allowlist becomes a permission boundary.

EXPLANATION

Normally, ten searches mean ten model turns, and each result passes through the model's context. With PTC the model writes a small program once, and the program makes the ten calls and returns a short summary.

```ts
const results = await Promise.all(
  topics.map((t) => tools.webSearch({ query: `${t} best practices` })),
);
return results.map((r) => r.title);
```

STEP BY STEP

1. You allowlist tools with `ptc=[...]`. Python names become camelCase.
2. The model calls the `eval` tool with code.
3. The code runs in a Wasm workspace with no filesystem, network, shell or clock unless granted.
4. Tool calls made by the code are executed by the host, with the allowlisted tool's real implementation.
5. The return value (up to 4,000 characters) goes back to the model.

ADVANTAGES

- Cheaper and faster: fewer model turns, fewer tokens.
- Deterministic control flow for loops, retries and batching.
- Parallelism with `Promise.all`, including parallel subagents through `task()`.

LIMITATIONS

- Bypasses per-call human approval.
- Debugging is harder when failures happen inside generated code.
- The 5-second timeout and 256-call limit bound each `eval`.

EDGE CASES AND TRAPS

- Gate the `eval` tool with `interrupt_on` rather than individual tools, and keep the allowlist narrow.

- The Wasm sandbox isolates the *engine*, not the *capabilities you grant*. A powerful allowlisted tool is as dangerous inside `eval` as outside.
- For hostile multi-tenant workloads, isolate in separate processes or containers as well.
- It is a scoped interpreter, not a full production sandbox.

LIKELY FOLLOW-UPS

- *Why not just allow the model to call tools in parallel?* It can, but each result still enters the context. PTC keeps intermediate values out.

Q32. Compare classic streaming with event streaming. How does a UI know what each subagent is doing?

Short answer. Classic streaming uses `stream(..., subgraphs=True, version="v2")` and you route by namespace: a segment starting with `tools:` means a subagent. Event streaming (`stream_events(..., version="v3")`, beta, `deepagents` 0.6 and later) gives typed projections and a first-class `stream.subagents`, and is recommended for new apps.

EXPLANATION

Classic streaming gives you one mixed stream of chunks and you parse it. Event streaming gives you separate iterators per projection (`messages`, `tool_calls`, `values`, `output`), on the parent and on each subagent handle.

	Classic	Event streaming
Call	`agent.stream(..., subgraphs=True, version="v2")`	`agent.stream_events(..., version="v3")`
Subagents	You detect `tools:<id>` namespaces	`stream.subagents` handles
Chunk handling	You parse shapes and modes	Typed iterators
Status	Stable	Beta

STEP BY STEP (CLASSIC)

1. Pass `subgraphs=True`. Without it you get **no** subagent events.
2. Namespace `()` is the main agent. `("tools:<id>",)` is a subagent spawned by `task` call `<id>`.
3. Choose modes: `updates` for lifecycle and interrupts, `messages` for tokens and tool-call chunks, `custom` for `get_stream_writer()` progress, or a list to combine.
4. Track lifecycle: *pending* is the `task` call appearing in the main model node, *running* is the first event from a `tools:` namespace, *complete* is the matching `ToolMessage` with `name == "task"`.

STEP BY STEP (EVENT STREAMING)

1. Iterate `stream.subagents`. Each handle has `name`, `status` (`started`, `completed`, `failed`, `interrupted`), `messages`, `tool_calls`, nested `subagents` and `output`.
2. Discovery is lazy: message and tool streams open only when accessed.
3. For exact arrival order, iterate the raw events and filter on the method.

ADVANTAGES OF EVENT STREAMING

- No namespace parsing, nested subagents are explicit, and statuses are first-class.

LIMITATIONS

- Beta, and a different mental model from classic code.

EDGE CASES AND TRAPS

- Chunk shapes vary with `version` and mode. With `subgraphs=True`, a single mode yields `(namespace, data)` and several modes yield `(namespace, mode, data)`. Confirm what your installed version returns.
- Reading `subagent.output` on a failed subagent can raise. Wrap it.
- Summarization tokens appear in streams. Filter with `metadata.get("lc_source") == "summarization"`.
- `subagents` reflects product-level delegation; `subgraphs` exposes graph internals. Prefer `subagents` for user-facing UI.

LIKELY FOLLOW-UPS

- *How do you link a UI card to the message that spawned it?* The subagent's snapshot `id` equals the spawning `tool_call_id`.
- *How are approvals shown?* Interrupts appear as `__interrupt__` in `updates`; resume with the same `thread_id`.

Scenario Questions

Restate, diagnose, fix, then say what could go wrong with the fix.

Q33. The sandbox bill keeps growing and nobody can explain why.

Scenario. Your coding assistant runs in a cloud sandbox. At the end of the month the sandbox provider's bill is three times what you expected, and the dashboard shows many sandboxes in a running state with no active conversations.

Short answer. Sandboxes were created and never torn down. Create them in a `try` / `finally` with provider-specific teardown, set an idle TTL, store the sandbox ID with the thread so that you reuse rather than re-create, and add a sweeper for orphans.

STEP BY STEP

1. **Count.** List sandboxes by age and by owner thread. Find which creation path leaks.
2. **Find the leak.** Common causes: an exception before teardown, a server restart mid-run, a new sandbox created on every request instead of looked up, and assistant-scoped sandboxes that nobody ever cleans.
3. **Fix creation.** Look up the sandbox by thread metadata first and create only if none exists.
4. **Fix teardown.** Wrap each run in `try` / `finally`. Use the provider's call (`delete_sandbox`, `stop()`, `kill()`, `terminate()` or `shutdown()`).
5. **Set an idle TTL** so that forgotten sandboxes stop on their own.
6. **Sweep.** A scheduled job deletes sandboxes with no thread, or older than a limit.
7. **Alert.** A budget or count alarm catches the next leak early.

ADVANTAGES

- Cost becomes proportional to usage.

LIMITATIONS

- An idle TTL that is too short deletes work in progress, and in Managed Deep Agents the contents are deleted with the sandbox (`idle_ttl_seconds` , default 600) and cannot be recovered.

EDGE CASES AND TRAPS

- A `finally` block does not run if the process is killed. That is why TTL and sweeper both matter.
- If a user returns after the TTL, the thread's files are gone. Decide whether to snapshot, or to tell the agent to re-create its environment.
- Do not delete a sandbox that another thread still uses (assistant scope).

LIKELY FOLLOW-UPS

- *How do you protect user work?* Save artefacts out through the transfer plane before teardown.

Q34. The analysis agent must post charts to Slack, but you do not want the Slack token near the sandbox. Design it.

Scenario. A data-analysis agent runs code in a sandbox to build charts, then must post them to a Slack channel. A security reviewer says, "If the model is tricked into leaking the token, we are done."

Short answer. Keep the token on the host. Download the chart through the file-transfer plane, and expose a narrow `post_to_slack(channel, file_path, message)` tool that runs on your server with the token. The agent calls it by name and never sees the secret. Require approval for the first use if the channel is sensitive.

STEP BY STEP

1. **Do not inject the token** into the sandbox: no environment variable, no file, no `secrets` option.
2. **Produce the chart in the sandbox**, writing to a fixed output path.
3. **Write a host-side tool.** It takes a channel from an allowlist, a sandbox path under the output directory, and a message. It downloads the file with `backend.download_files([...])` , checks the error field, checks the type and size, and posts to Slack with the token it holds.
4. **Constrain the tool.** Validate the channel, the path prefix and the file type. Rate-limit it.
5. **Gate it.** Put the tool in `interrupt_on` if posting to a public channel is sensitive.
6. **Log.** Record who, which thread, which file, which channel.
7. **Alternative.** An authentication proxy outside the sandbox injects the header for outbound requests to `slack.com` , if the agent must call the API itself.

ADVANTAGES

- A steered agent can misuse the tool within its narrow limits, but cannot exfiltrate the credential.

LIMITATIONS

- You write and maintain the tool, and it expands the host's attack surface, so it must validate its inputs.

EDGE CASES AND TRAPS

- The agent could still post sensitive data to an allowed channel. The token is safe; the data is not. Limit channels and require approval where it matters.
- Check `error` on every downloaded path. A failed download does not raise.

- Treat the generated chart as untrusted: scan it before posting.

LIKELY FOLLOW-UPS

- *Where do you store the token?* In workspace secrets or your secret manager, on the host.
- *What if the sandbox needs to call an internal API?* Use the proxy pattern, scoped by host.

Q35. Your agent makes 40 searches one after another and costs a fortune. What do you change?

Scenario. A market-scan agent searches 40 topics, one per model turn. Each result is large, the context fills, and the run is slow and expensive.

Short answer. The work is a loop with no judgement in the middle. Move it into code with the interpreter and programmatic tool calling, so the 40 searches run in parallel and only a compact result returns. If the results themselves need reasoning, delegate groups of topics to subagents instead.

STEP BY STEP

1. **Diagnose.** Are the 40 calls independent? Does the model need each full result, or only a distilled value?
2. **If independent and mechanical:** add `CodeInterpreterMiddleware(ptc=["web_search"])` and let the model write one `Promise.all` over the topics, extracting only titles and snippets.
3. **If each result needs judgement:** use subagents, for example one per group of topics. They search and read in their own contexts and return short summaries.
4. **Control size.** Return at most a few thousand characters from `eval`. Write bulk results to a file and return the path.
5. **Gate.** Keep the PTC allowlist narrow, and decide if `eval` needs approval.
6. **Measure.** Compare tokens, time and answer quality against the old version.

ADVANTAGES

- Fewer model turns, far fewer tokens, parallel speed.

LIMITATIONS

- The code can fail in ways that are harder to debug than a turn-by-turn trace.
- The 4,000-character return limit, 5-second timeout and 256-call cap all apply.
- Beta feature.

EDGE CASES AND TRAPS

- Rate limits on the search provider apply to the parallel burst. Add concurrency control.
- PTC calls skip per-call approvals. Do not allowlist tools with side effects unless that is acceptable.
- If you need a shell, packages or file edits, you need a sandbox, not the interpreter.

LIKELY FOLLOW-UPS

- *Why not just let the model emit 40 parallel tool calls?* Possible, but all 40 results enter the context.
- *Could subagents run from code?* Yes, `task()` is available in the interpreter.

Q36. Design the streaming for a UI that shows parallel subagents, tokens and approvals.

Scenario. Product wants a screen with one card per subagent (name, status, current tool), a live transcript for the coordinator, and an approval dialog for risky actions. Users should see nested delegation too.

Short answer. Use event streaming (`stream_events(..., version="v3")`) for a first-class `stream.subagents` with status, tool calls and nested handles. Use `stream.messages` for the coordinator transcript. Handle interrupts and resume with the same `thread_id` .

STEP BY STEP

1. **Start the stream.** `stream = agent.stream_events(input, version="v3")` (or `astream_events` in async servers).
2. **Create cards.** Iterate `stream.subagents` . Each handle gives `name` , `status` (`started` , `completed` , `failed` , `interrupted`) and a snapshot `id` that equals the spawning `tool_call_id` .
3. **Show tools.** For each handle, read `tool_calls` and display the current `tool_name` and `input` .
4. **Recurse.** For nested delegation, iterate `subagent.subagents` .
5. **Run consumers concurrently.** In async code use `asyncio.gather` over the coordinator and each subagent consumer. In sync code, use `stream.interleave("messages", "subagents")` .
6. **Show approvals.** When a subagent's status becomes `interrupted` , or an `__interrupt__` appears, render the dialog, collect decisions, and resume with the same `thread_id` .
7. **Handle failure.** Wrap reads of `output` on a failed subagent.
8. **Filter noise.** Hide summarization tokens using `metadata.get("lc_source") == "summarization"` .

ADVANTAGES

- Little parsing, accurate lifecycle, explicit nesting.

LIMITATIONS

- Beta, requires `deepagents` 0.6 or later, and the model is different from classic `stream_mode` code.

EDGE CASES AND TRAPS

- Use `subagents` , not `subgraphs` , for user-facing UI: `subgraphs` shows internal graph nodes.
- Streams are lazy. A consumer that never reads a projection never opens it.
- For exact arrival order across coordinator and subagents, iterate raw events and filter by method.
- An empty namespace in raw events is the coordinator.
- Reconnection: if the browser disconnects, you need a way to reattach to the thread; see the frontend and deployment chapters.

LIKELY FOLLOW-UPS

- *If you must support the classic stream?* Use `subgraphs=True` , `version="v2"` , and route by `tools:` namespaces.
- *How do you show progress inside a long tool?* Emit `get_stream_writer()` updates and listen to the `custom` mode.

CHAPTER 67

Interview Questions: Context Management

At a glance: Twelve questions on what the agent knows and how that fits in the window. Direct questions give the numbers and the vocabulary. Concept questions explain the design of skills, memory, offloading, summarization and caching. Scenarios cover memory leaks between users, stale skills, a run that loses its place, and retrieval.

Direct Questions

State the fact, say what it means in practice, then add one edge case.

Q37. At what sizes do offloading and summarization trigger?

Short answer. A single tool result over **20,000 tokens** is saved to the backend and replaced by a file path plus its **first 10 lines**. Summarization triggers at **85% of the model's** `max_input_tokens` and keeps the most recent **10%** of tokens verbatim. With no model profile it falls back to 170,000 tokens and keeps 6 messages.

EXPLANATION

Context management is a set of levers that act in a fixed order, from cheapest to most lossy. Knowing the numbers lets you say what the agent will do before it does it.

Lever	Trigger	What happens	What is lost
Offloading of tool results	A result over 20,000 tokens	Saved to the backend; the model sees the path and the first 10 lines and can re-read or `grep` it	Nothing, the full text is on disk
Offloading of tool inputs	Large `write_file` or `edit_file` arguments over 20,000 tokens, once the context passes 85%	Replaced by a pointer	Nothing, the file holds the content
Summarization	Context reaches 85% of `max_input_tokens` and offloading has nothing left to remove	Older history becomes a structured summary (intent, artefacts, next steps); the most recent 10% stays	Exact wording, figures, and media
Overflow recovery	A `ContextOverflowError`	Forces summarization and a retry	As above
Isolation	You delegate	A subagent keeps its tool calls out of the parent window and returns one report	Nothing from the parent, but the parent only sees the report

STEP BY STEP

1. The system prompt, memory and skills catalogue are present from the first call, so they are paid for on every call.
2. As tools return, large results are written to files and replaced by short stubs.
3. When the window nears 85%, large tool inputs are replaced by pointers.

4. If that is not enough, the older history is summarised and the full original text is written to the filesystem.
5. If the provider still rejects the request as too large, the middleware summarises and retries.

EDGE CASES AND TRAPS

- Summarization is **text-only**. Media in the compacted range no longer reaches the model, while offloading keeps media.
- The thresholds can be tuned: `SummarizationMiddleware(model=..., backend=..., trigger=("tokens", 100_000), keep=("messages", 20))`. `trigger` accepts `fraction`, `tokens` or `messages`, and a list means OR.
- `create_summarization_tool_middleware(model, backend)` adds a `compact_conversation` tool so the agent can compact *between tasks*. It does not disable automatic summarization.
- `excluded_tools` shrinks tool schemas on every turn, but that is configuration, not compression.

LIKELY FOLLOW-UPS

- *Offloading versus context editing?* `ContextEditingMiddleware` clears older tool outputs after a threshold (default trigger 100,000 tokens, keep the last 3, placeholder `[cleared]`). Offloading saves the full result to a file.
- *Where does the offloaded text go?* To the backend, which is why a bare `FilesystemBackend` puts it on real disk.

Q38. What is a skill, and how is one laid out and loaded?

Short answer. A skill is a folder with a `SKILL.md` file: YAML frontmatter (`name`, `description`) plus instructions, and optionally scripts, references and assets. You pass the **parent directory** in `skills=["/skills/"]`. At startup only each skill's name and description enter the prompt; the full file is read on demand.

EXPLANATION

A skill packages a procedure ("how we write a release note", "how to analyse a CSV") so that it costs almost nothing until it is needed. The rules for the frontmatter are strict, and a violation usually fails silently.

STEP BY STEP

1. Create `skills/<name>/SKILL.md` with frontmatter. `name` is 1 to 64 lowercase letters, digits and hyphens and **must match the directory name**. `description` is at most 1,024 characters and should say what the skill does *and when to use it*.
2. Pass `skills=["/skills/"]`. Each entry is a directory that **contains** skill directories.
3. For a `StateBackend`, seed the files at invoke time with `files={"/skills/x/SKILL.md": create_file_data(text)}`. For a store or filesystem backend, upload them first.
4. At run time the catalogue is injected into the prompt. The agent chooses a skill by its description and reads it with `read_file`.
5. Supporting files are referenced with relative paths, one level deep.

ADVANTAGES

- Cheap: only a line per skill is always in the prompt.
- Developer-owned and versionable.

LIMITATIONS

- Selection depends entirely on the description. Vague or overlapping descriptions cause wrong or missed skills.
- Skill content is invisible until read, so rules that must apply on every turn do not belong in a skill.

EDGE CASES AND TRAPS

- Files over 10 MB are skipped.
- The general-purpose subagent inherits skills. Custom subagents do not; they need their own `"skills": [...]`.
- Scripts are readable anywhere, but execute only on a sandbox backend, and the sandbox must have the files synced in.

LIKELY FOLLOW-UPS

- *Can a skill ship tools?* Yes, via `include_tools` and `SkillsMiddleware(tools=...)` (`deepagents>=0.7.22`), but those tools are gated until the agent has read the skill.
- *How big should the body be?* Keep it under roughly 5,000 tokens and put detail in referenced files.

Q39. State, runtime context, store and config: what is each, and when do you use which?

Short answer. **State** is mutable and checkpointed per thread. **Runtime context** is immutable per run and is not shown to the model unless you expose it. The **store** is cross-thread key-value memory. **Config** (`configurable`) carries identifiers such as `thread_id`.

EXPLANATION

These four are confused constantly, and mixing them is a source of real bugs, usually by putting a thing in the wrong one.

	Lifetime	Mutable	In the prompt?	Example
State	Thread (checkpointed)	Yes	Messages yes, other fields no	`files`, `todos`, custom fields
Runtime context	One run	No	No, unless you surface it	`user_id`, role, feature flags
Store	Across threads	Yes	No, unless read	`/memories/` files
Config	One call	No	No	`thread_id`

STEP BY STEP

1. Define a context dataclass and pass `context_schema=Context`. Invoke with `context=Context(...)`.
2. Read it in a tool through the `ToolRuntime[Context]` parameter (`runtime.context`), or in middleware through the runtime.
3. Add custom state by subclassing `DeepAgentState` and passing `state_schema=` (`deepagents>=0.6.6`).
4. Use dynamic prompts (`@dynamic_prompt`) for per-run prompt variation, because `system_prompt=` is static.

EDGE CASES AND TRAPS

- Context **propagates to all subagents** and their tools.
- Compiled and remote subagents do not inherit the parent `state_schema`.
- Context is not verified identity. Authorise against the verified caller, not against a field that the client could set.
- `thread_id` and `context` are independent. One names the conversation; the other carries per-run data.

LIKELY FOLLOW-UPS

- *Where does a user's role belong?* In runtime context (immutable per run), read by tools and middleware, derived from the verified identity.
- *Where do long-lived facts belong?* In the store, under a namespace.

Q40. Provider profiles versus harness profiles: what is the difference, and how do they merge?

Short answer. A **provider profile** controls how the model object is *constructed* (init keyword arguments). A **harness profile** controls how the *agent is shaped*: prompt text, tool descriptions, excluded tools and middleware, extra middleware, and the general-purpose subagent. Both are registered before agents are created, keyed by provider or by `provider:model`.

EXPLANATION

Think of the provider profile as "how do I talk to this vendor" and the harness profile as "how should the agent behave on this model". They exist so that you can tune per model without forking the harness.

STEP BY STEP

1. Register: `register_provider_profile(key, ProviderProfile(init_kwargs={...}))` and `register_harness_profile(key, HarnessProfile(...))`.
2. Keys are provider-level (`"openai"`) or model-level (`"openai:gpt-5.5"`). Only the first colon separates provider from model ID. A model-level profile inherits unset fields from the provider-level one.
3. For model objects, lookup is: exact `provider:identifier`, then the identifier alone (if it contains a colon), then provider defaults.
4. Re-registering under an existing key **merges** onto the previous profile. Prompts: the new one wins. `excluded_tools` and `excluded_middleware`: set union. `init_kwargs`: key by key, new wins. `pre_init`: chained. `extra_middleware`: the same type replaces in place, new types are appended.
5. Distribute defaults as plugins through the entry points `deepagents.harness_profiles` and `deepagents.provider_profiles`. Load order: built-ins, plugins, then direct calls.

ADVANTAGES

- Model-specific tuning lives in one place and applies to every agent, including subagents (which re-resolve against their own model).

LIMITATIONS

- There is **no wildcard key**.
- Order of registration matters, and so does registering before agent creation.

EDGE CASES AND TRAPS

- Provider profiles apply only to `provider:model` **strings**, never to pre-built model objects.
- `FilesystemMiddleware`, `SubAgentMiddleware` and the permission middleware cannot be excluded (`ValueError`).
- To run with no subagents: `GeneralPurposeSubagentProfile(enabled=False)` **and** no synchronous subagents. Async subagents are unaffected.
- The caller's `system_prompt` always comes first and `system_prompt_suffix` always last.

LIKELY FOLLOW-UPS

- *Which profile do you use to set the temperature?* The provider profile (`init_kwargs`).
- *Which one removes a tool from the model's view?* The harness profile (`excluded_tools`).

Concept Questions

These test whether you understand why context is managed the way it is.

Q41. Skills, memory, tools and instructions: how do you decide where something belongs?

Short answer. *Tools* are actions. *Instructions* (the system prompt) are always-on behaviour. *Memory* is small, always-loaded, agent-updatable knowledge. *Skills* are procedures loaded only when relevant. Put each thing where its loading cost matches how often it is needed.

EXPLANATION

The decision is an economics question. Everything in the always-loaded set is paid for on every call, in tokens and in distraction. Everything on demand is paid for only when used, but must be discoverable.

	Loaded	Writable by the agent	Size discipline
Instructions	Every run	No	Short and stable
Memory (`AGENTS.md`)	Every run	Often	Very small
Skills	On demand (progressive disclosure)	Usually not	Body under about 5,000 tokens
Tools	Schema on every call	n/a	Few and well described

STEP BY STEP

1. Ask **"does it do something?"** If yes, it is a tool (or an MCP tool).
2. Ask **"is it needed on every single run?"** If yes and it is short: fixed behaviour goes in the instructions, and a fact or preference that changes over time goes in memory.
3. Ask **"is it a multi-step procedure needed only sometimes?"** That is a skill. Only its name and description sit in the prompt.
4. Ask **"who should be able to change it?"** Skills and instructions are usually developer-owned and read-only to the agent. Memory is often writable by the agent.
5. Ask **"how big is it?"** Push detail into skills or cold files.

ADVANTAGES OF THE SPLIT

- The always-on prompt stays small, so every run is cheaper and more focused.
- Different change rates have different homes: policy changes by deployment, memory by use, skills by editing a file.

LIMITATIONS

- On-demand things can be missed. If the description does not match the task, the skill is never read.

EDGE CASES AND TRAPS

- Bloated memory degrades every run, not just the ones that need it.
- Two skills with overlapping descriptions cause the wrong one to be chosen.
- Do not put rules that must apply on every turn in a skill, because it is invisible until read.

LIKELY FOLLOW-UPS

- *Where do per-user facts go?* Memory in a user-scoped namespace.
- *Where do organisation-wide policies go?* Read-only memory seeded from application code, or the instructions.

Q42. Explain progressive disclosure for skills. What are the three levels, and who handles each?

Short answer. Level 1: at startup only each skill's `name` and `description` go into the prompt. Level 2: when the task matches, the agent reads the full `SKILL.md` with `read_file`. Level 3: scripts, references and assets load only if the instructions call for them. `SkillsMiddleware` handles levels 1 and 2; the model handles level 3.

EXPLANATION

Progressive disclosure is the idea that an agent should be told *that* something exists cheaply and told *how* to use it only when needed. It is how a library with a hundred skills costs a hundred lines of prompt rather than a hundred documents.

STEP BY STEP

1. **Catalogue (level 1).** `SkillsMiddleware` reads each skill's frontmatter once per thread and injects the list.
2. **Selection.** The model compares the task to the descriptions and decides whether a skill applies.
3. **Load (level 2).** It calls `read_file` on the `SKILL.md`. The body is now in context.
4. **Resources (level 3).** The body may refer to `references/guide.md` or `scripts/convert.py`. The model reads or runs them when needed.
5. **Cache.** The catalogue is cached per thread under `skills_metadata`.

ADVANTAGES

- Large libraries at low cost, with detail pulled in only when relevant.

LIMITATIONS

- The catalogue entry is the only thing the model sees when choosing. A poor description makes a good skill unreachable.
- Because the body loads through an ordinary tool call, it can later be summarised away.

TRADE-OFFS

- A skill saves tokens but costs one extra tool call and relies on the model noticing it should load.
- Pinning (below) trades tokens for certainty.

EDGE CASES AND TRAPS

- **Precedence.** With several sources, a later source wins on duplicate names, which can silently shadow a base skill.
- If summarization drops the read of a `SKILL.md`, its gated tools are gated again, and the model needs to read the file again.

LIKELY FOLLOW-UPS

- *How do you force a skill into context?* **Pin** it: pass `"pinned_skills": ["name"]` in the run input (`deepagents>=0.7.23`). Its `SKILL.md` is injected as a frozen `HumanMessage`; re-pin to pick up edits.

Q43. Walk through context engineering as the window fills. What is lost in summarization, and what survives?

Short answer. Four levers: shape the input context, **offload** large tool input and output to files, **summarize** old history, and **isolate** work in subagents. Detail is lost from the model's view (though kept on disk), media is dropped to text, and anything that depended on a specific message being present can break. Durable facts should live in files, state or memory, not only in the conversation.

EXPLANATION

The levers are ordered by cost. Offloading loses nothing, because the full text is on disk. Summarization is lossy by nature, since a summary is shorter than the thing it summarises. Isolation avoids the problem by keeping bulk work out of the parent window in the first place.

STEP BY STEP

1. **Shape the input.** Keep the system prompt, memory and skill catalogue small. They are present from the first call.
2. **Offload.** Large tool results and inputs become file references.
3. **Summarise.** Old history becomes a structured summary. The full original is saved to the filesystem.
4. **Isolate.** Delegate bulk reading and exploration to subagents that return one report.
5. **Make state external.** Todos in state, findings in files, decisions in memory. These survive summarization because they are not only in the messages.

WHAT THE SUMMARY CAN AND CANNOT CARRY

Carries	Does not carry
Intent, key artefacts, next steps	Exact figures, long quotes, images and other media

ADVANTAGES

- Long runs become possible without a human managing the window.

LIMITATIONS

- The summary is itself a model output and can be wrong or omit what later turns out to matter.
- Behaviour after summarization can differ from behaviour before it, which makes bugs hard to reproduce.

EDGE CASES AND TRAPS

- Skill tools are gated on having read `SKILL.md`. If summarization drops that read, calls fail until the agent reads it again.
- The `async_tasks` state channel is separate from messages, so async subagent metadata **survives summarization**.
- Summarization tokens are streamed. Filter them in UIs with `lc_source == "summarization"`.
- Mitigate by pinning critical skills, keeping memory current, and writing intermediate findings to files.

LIKELY FOLLOW-UPS

- *How would you test summarization?* Lower the trigger (for example 5,000 tokens) in a test and assert that the agent still completes the task correctly.

Q44. How does prompt caching work in Deep Agents, and why is memory placed after it?

Short answer. For Anthropic and Amazon Bedrock models, `create_deep_agent` automatically caches the static prompt prefix (base instructions, skills catalogue). `MemoryMiddleware` sits *after* the caching layer so that edits to memory do not invalidate the cached prefix.

EXPLANATION

Provider-side prompt caching stores a prefix of the request so that later calls with the same prefix cost less and respond faster. Any change in the prefix, even one character, invalidates it. So the design goal is: put things that never change first, things that change often last.

STEP BY STEP

1. The first call writes the static prefix to the provider's cache.
2. Later calls within the time-to-live reuse it, lowering cost and latency. The default TTL is five minutes; `AnthropicPromptCachingMiddleware(ttl="1h")` overrides it.
3. Because memory can change during a run, placing it after the caching boundary keeps the cached portion stable.
4. Fork subagents benefit when they share the parent's prompt prefix. Leave a fork's `system_prompt` unset to keep the cache shared.

ADVANTAGES

- Lower cost and latency for long runs with a big static prefix.

LIMITATIONS

- Caching is a no-op on other providers.
- It rewards stability. Anything that changes the early prompt on every request defeats it.

EDGE CASES AND TRAPS

- A fork `system_prompt` *appends* and reduces cache reuse. For async forks, cache hits need a matching model, system prompt, tools and middleware.
- Changing the tool list between requests changes the prefix.
- Dynamic prompts that inject a timestamp at the top silently disable caching. Put volatile text at the end.

LIKELY FOLLOW-UPS

- *How would you check that caching works?* Look at the provider's usage fields for cache read and cache creation tokens across two consecutive calls.

Scenario Questions

Restate, diagnose, fix, then say what could go wrong with the fix.

Q45. Design the memory for a multi-user assistant. What are the risks?

Scenario. An assistant should remember each user's preferences, learn from conversations, and also follow a company style guide. It will serve thousands of users.

Short answer. Route `/memories/` to a `StoreBackend` through a `CompositeBackend`, scope it **per user** by default, keep any shared scope read-only and seeded by application code, and treat everything in memory as untrusted input. Consolidate in the background on a schedule whose interval matches its lookback window.

STEP BY STEP

1. **Scope.** Thread (checkpoint only), user `(user_id,)` as the recommended default, assistant `(assistant_id,)` for shared learning across users, organisation `(org_id,)` for read-only policy.
2. **Wire it.** `memory=["/memories/AGENTS.md"]` makes the file always loaded. The composite routes the path to the store, and the namespace factory partitions it.
3. **Pre-seed.** The style guide is written from application code with `store.put(...)`, not by the agent.
4. **Update patterns.** Either hot-path writes during the conversation (adds latency), or background consolidation ("sleep-time compute"): a scheduled agent reads recent conversations and merges facts into memory.
5. **Guardrail.** Deny writes to shared prefixes with `permissions`, and add policy hooks or approvals for sensitive writes.

ADVANTAGES

- The assistant improves over time without retraining, and each user's data stays separate.

LIMITATIONS

- Memory is paid for on every run, so it must stay small. Without pruning it grows and degrades quality.

EDGE CASES AND TRAPS

- **Prompt injection.** Shared writable memory lets one user plant instructions that execute in another user's conversation. This is the headline risk.
- **Consolidation cadence.** The schedule interval must equal the lookback window (a six-hour job searches six hours back), or you reprocess or drop memories. Cron expressions are **UTC**.
- **Concurrent writes** to one file are last-write-wins. Serialise consolidation, or use per-topic files.
- `InMemoryStore` is for development only.
- Writes show up as tool calls in LangSmith traces, which is how you audit them.
- **Episodic memory.** Past conversations are checkpointed threads. Expose search over them as a tool rather than copying them into memory.

LIKELY FOLLOW-UPS

- *Short-term versus long-term memory?* Short-term is thread-scoped through checkpoints; long-term persists across threads through the store.

- *How do users see and delete what is remembered?* Expose the memory file in your UI, and treat deletion as a first-class requirement (privacy law may demand it).

Q46. You edited a skill, but the agent still behaves as before. What do you check?

Scenario. The team improved `SKILL.md` for the release-notes skill and redeployed. In a new conversation it works. In an existing conversation, and in one test, the agent ignores the change or behaves as if the skill did not exist.

Short answer. Skills are loaded once per thread and cached under `skills_metadata`. Existing threads keep the old catalogue until it is reset with `{"skills_metadata": None}` (`deepagents>=0.7.16`). If a skill never loads, check the path level, the name, the description and the subagent's own `skills` list.

STEP BY STEP (THE "WHY DID NOTHING LOAD" CHECKLIST)

1. **Is it a new or an existing thread?** With a checkpointer, an existing thread keeps its cached metadata. Reset with `{"skills_metadata": None}`. Use `None`, **not** `[]`: an empty list means "no skills" for the rest of the thread.
2. **Is the path at the right level?** Passing the skill folder (`/skills/blog-post/`) instead of its parent (`/skills/`) silently loads nothing.
3. **Do `name` and directory match?** `name` must equal the directory name and use only lowercase letters, digits and hyphens, up to 64 characters.
4. **Is the description good enough?** Up to 1,024 characters, saying what and *when*. If it is vague the model never selects it.
5. **Is the file really in the backend?** With `StateBackend` the file must be seeded with `create_file_data` at invoke time. With other backends, it must already be uploaded.
6. **Is a later source shadowing it?** A later source wins on duplicate names.
7. **Is it a subagent?** The general-purpose subagent inherits skills. Custom subagents need their own `skills`.
8. **Is the file over 10 MB?** It is skipped.
9. **Does it need to be certain?** Pin it with `"pinned_skills": ["name"]` (`deepagents>=0.7.23`). Re-pin to pick up edits.

ADVANTAGES OF THIS CHECKLIST

- Each item takes a minute to test, and together they cover nearly every silent skill failure.

LIMITATIONS

- Many failures are silent, so you need a test that asserts a skill is selected, not just that the agent answers.

EDGE CASES AND TRAPS

- Resetting `skills_metadata` to `None` reloads from the backend; resetting to `[]` disables skills.
- Pinned text is a frozen message. Editing the skill does not update an existing pin.

LIKELY FOLLOW-UPS

- *How would you test skill selection?* A small eval that sends a task per skill and asserts that the corresponding `SKILL.md` was read.

Q47. After two hours the agent forgets a decision it made earlier and repeats work. Why, and what do you change?

Scenario. A long-running research agent made a key decision in hour one (excluded a data source for a stated reason). In hour two it re-includes that source and redoes the analysis.

Short answer. The decision lived only in the early conversation, which was summarised away. Make important state external: write decisions to a file or memory, keep a todo list, pin critical instructions, and delegate bulk work to subagents so that the parent's window fills more slowly.

STEP BY STEP

1. **Confirm.** Look at the trace for a summarization event before the mistake, and check the summary text for the decision.
2. **Understand the loss.** The summary carries intent, artefacts and next steps. A rationale for excluding one source may not make it in.
3. **Externalise.** Instruct the agent to record decisions in `/notes/decisions.md` as it makes them, and to read that file at the start of each phase.
4. **Track progress.** Add `TodoListMiddleware` (opt-in since 0.7) for a visible, structured list in state.
5. **Pin rules.** If a skill must stay in force, pin it so it is injected as a frozen message.
6. **Slow the fill rate.** Delegate bulk reading to subagents that return compact reports.
7. **Tune.** Raise or lower the trigger and the amount kept so that summarization happens at natural boundaries.
8. **Test.** Reproduce with a low trigger and assert the decision is still respected.

ADVANTAGES

- State in files is exact, durable, inspectable and shared with subagents.

LIMITATIONS

- It relies on the model actually writing and reading the file. Say so in the instructions and test it.

EDGE CASES AND TRAPS

- Subagents do not see the parent's history. A decision must be handed over in the task description or via a file path.
- Files in `StateBackend` are checkpointed on every step. Keep them small.
- Do not rely on the summary for exact numbers.

LIKELY FOLLOW-UPS

- *Could you use memory instead of a notes file?* Memory is for knowledge that should survive across threads. Per-run decisions fit a thread-scoped file better.

Q48. Design retrieval-augmented generation with Deep Agents.

Scenario. You must answer questions over a large document collection, with citations, without filling the window with retrieved text.

> **Short answer.** Expose retrieval as a tool, **write the retrieved chunks to files and pass paths** instead of text, let a subagent summarise each file in parallel, synthesise with citations in the parent, use a skill to encode the retrieval strategy, and optionally grade groundedness with a rubric.

STEP BY STEP

1. **Index once.** Load documents, split (the tutorial uses `RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)`), embed, and store in a vector store.
2. **Search tool.** Query the store, write each result batch to `/retrieved/{batch_id}/...` on the *same backend instance* the agent uses, and return the **paths**.
3. **Fan out.** Spawn one `chunk-analyst` subagent per file. Each reads a single file and returns a short summary.
4. **Synthesise.** The coordinator writes the answer with citations.
5. **Check.** A rubric such as "every claim cites a source URL" gates the output.
6. **Pick the RAG shape.** 2-step RAG always retrieves first (predictable, good for FAQs). Agentic RAG lets the model decide when to retrieve (flexible, variable latency). Hybrid adds query enhancement and validation loops (best quality, most iteration).

Shape	Strength	Weakness
2-step	Predictable, cheap	Retrieves even when unnecessary
Agentic	Flexible	Variable latency and cost
Hybrid	Best quality	Most moving parts

ADVANTAGES

- Paths keep the coordinator small, and subagents work in parallel.

LIMITATIONS

- More moving parts than a single prompt, and retrieval quality still sets the ceiling on answer quality.

EDGE CASES AND TRAPS

- **Indirect prompt injection** through retrieved documents is a real risk. Prompts reduce but do not eliminate it. Validate answers and check citations.
- The search tool and `create_deep_agent` must share the **same backend instance**, or `read_file` cannot see the saved chunks.
- Allowlist domains for URL fetching, and report failed fetches rather than silently falling back to model knowledge.
- Persist and refresh the vector store in production. `InMemoryVectorStore` is for tutorials.

LIKELY FOLLOW-UPS

- *Why files instead of putting chunks in the prompt?* Chunks would fill the window; paths keep the coordinator small and let subagents work in parallel.
- *How do you evaluate it?* Hold out questions with known source passages, and check both retrieval (was the right chunk found) and generation (is every claim supported).

CHAPTER 68

Interview Questions: Delegation and Subagents

At a glance: Twelve questions on splitting work between agents. Direct questions cover `task`, the subagent spec, inheritance and the async tools. Concept questions explain why delegation helps and what it costs, and how to choose between isolated, fork, dynamic and async subagents. Scenarios cover lost tools, parallel review, steerable long jobs and poor delegation.

Direct Questions

State the fact, say what it means, then add one edge case.

Q49. What does the `task` tool do, and what does a subagent specification contain?

Short answer. `task` spawns an ephemeral child agent in a **fresh context**, runs it to completion, and returns **one final report** as a tool result. The parent never sees the child's intermediate tool calls. That is *context quarantine*, plus specialisation and parallelism. A subagent is defined by a spec: `name`, `description`, `system_prompt`, and optionally `tools`, `model`, `middleware`, `interrupt_on`, `skills`, `response_format`, `permissions` and `mode`.

EXPLANATION

Think of `task` as a function call whose implementation is another agent. The parent supplies a description of the job. The child works autonomously, using as many tool calls as it needs, and returns a single message. The parent pays for the report, not for the work.

```python
research_subagent = {
    "name": "research-agent",                # required; used in task() calls
    "description": "Researches deep questions",  # required; drives delegation decisions
    "system_prompt": "You are a careful researcher.",  # required for isolated mode
    "tools": [internet_search],              # if set, REPLACES the parent's tools
    "model": "openai:gpt-5.5",               # inherits if omitted
}
agent = create_deep_agent(model="google_genai:gemini-3.6-flash", subagents=[research_subagent])
```

STEP BY STEP

1. The parent model decides to delegate and calls `task` with a subagent name and a task description.
2. A subagent instance starts with its own prompt, tools, model and (optionally) skills. By default it is **isolated**: it sees only the description.
3. It works autonomously until done and returns a single message. It cannot send several messages back, and the parent is blocked meanwhile (synchronous subagents).
4. Several `task` calls in one model turn run in **parallel**.
5. The final report becomes the `ToolMessage` that the parent reads.

ADVANTAGES

- Keeps the parent's window clean, enables specialisation, and allows parallel work.

LIMITATIONS

- One report back and no mid-task messages (use async subagents to steer). A synchronous subagent blocks the supervisor.

EDGE CASES AND TRAPS

- Ask subagents for **concise** reports and keep raw data in files, passing paths.
- A general-purpose subagent exists by default. Passing your own spec named `general-purpose` **replaces** it.
- A UI should link a subagent to its parent turn by the spawning tool-call ID.
- LangSmith traces carry `lc_agent_name` metadata, so you can filter by agent.
- The documentation is inconsistent about the argument names: one page shows `task(name=..., task=...)`, another uses `subagent_type` and `description`. Check your installed tool schema.

LIKELY FOLLOW-UPS

- *What is the general-purpose subagent?* A default subagent with the filesystem tools that inherits the main agent's skills. Disable it with `GeneralPurposeSubagentProfile(enabled=False)` plus no other synchronous subagents.

Q50. State the subagent inheritance rules precisely.

Short answer. `tools` and `permissions` **replace** the parent's if set. `model` and `interrupt_on` inherit. `middleware` and `skills` do **not** inherit. `response_format` is per subagent. Isolated subagents do not inherit the parent's system prompt either.

EXPLANATION

This is the table interviewers love, because every row hides a real bug.

Field	Inherits?	Rule
`tools`	Yes	If set, **replaces** (does not add to) the parent's tools
`model`	Yes	Defaults to the main agent's model
`middleware`	No	A matching `.name` replaces a default instance
`interrupt_on`	Yes	The subagent's value wins; needs a checkpointer
`skills`	No	Must be set explicitly
`permissions`	Yes	If set, **replaces** the parent's rules (no merge)
`system_prompt`	No (isolated)	Required for isolated mode; for fork, an appended addendum

STEP BY STEP (APPLY IT TO A CASE)

1. The parent has tools `[search, send_email]` and a rule that denies writes to `/memories/**`.
2. A subagent declares `tools=[fetch_url]` and `permissions=[allow read /workspace/**]`.
3. The subagent now has only `fetch_url`, not `search` or `send_email`.
4. Its permissions no longer deny `/memories/**`, because the list **replaced** the parent's.

5. If the intent was "add a tool", you must list the whole set.

EDGE CASES AND TRAPS

- `interrupt_on` requires a checkpointer.
- Runtime context flows to all subagents regardless of the table above.
- Compiled and remote subagents do not inherit the parent `state_schema`.

LIKELY FOLLOW-UPS

- *Which row is the biggest security risk?* `permissions`. A subagent that sets its own list silently drops the parent's deny rules.

Q51. What are the supervisor's tools for async subagents, and what does each do?

Short answer. `start_async_task` launches a task and returns its ID. `check_async_task` returns status and, when complete, the result. `update_async_task` sends new instructions to a running task. `cancel_async_task` stops it. `list_async_tasks` lists all tasks with live status. Each task has its own persistent thread, and the thread ID is the task ID.

EXPLANATION

Async subagents run as background tasks on an Agent Protocol server (LangSmith Deployment, `deepagents` 0.5 and later). The supervisor does not wait. It gets an ID immediately and carries on talking to the user.

	Sync	Async
Supervisor	Blocks until done	Gets a **task ID** immediately and continues
Mid-task steering	No	Yes, `update_async_task`
Cancellation	No	Yes, `cancel_async_task`
State	Stateless	Persistent thread per task
Best for	Results needed now	Long-running, parallel, steerable work

STEP BY STEP

1. **Configure** `AsyncSubAgent(name, description, graph_id, url=None, headers=None, mode="isolated")`. No `url` means **ASGI** transport (co-deployed in one `langgraph.json`, recommended). A `url` means **HTTP** transport (remote, independent scaling).
2. **Launch.** `start_async_task` creates a server thread, starts a run, and returns the ID.
3. **Continue.** The supervisor keeps working or talking to the user.
4. **Check.** `check_async_task` returns status and, when complete, the result.
5. **Steer.** `update_async_task` starts a new run on the *same* thread, interrupting the previous one. The subagent restarts with its full history plus the new instructions, and the task ID does not change.
6. **Cancel or list.** `cancel_async_task` stops it; `list_async_tasks` gives live status for running tasks, and terminal ones (success, error, cancelled) come from a cache.

EDGE CASES AND TRAPS

- Task metadata lives in the `async_tasks` state channel, so it **survives summarization**.

- `mode="fork"` copies the parent conversation at launch; later parent messages are not forwarded. Fork is Python-only and experimental for async.
- Subagent threads are tagged `parent_thread_id`, which is editable by anyone with thread access. Verify before trusting it.

LIKELY FOLLOW-UPS

- *Can you use async subagents in a plain local script?* No, they need an Agent Protocol server.
- *How does the supervisor learn that a task finished?* Beta notifications (`deepagents>=0.7.26`) arrive as a `HumanMessage` with `additional_kwargs["deepagents_notification"]`.

Q52. What is a `CompiledSubAgent`, and how does a subagent return structured output?

Short answer. A `CompiledSubAgent` wraps any compiled LangGraph graph as a subagent, provided its state has a `messages` key. `response_format` makes a subagent return validated JSON to the parent (`deepagents>=0.5.3`).

EXPLANATION

Sometimes the specialist is not a deep agent at all. It might be a plain `create_agent` graph or a hand-built `StateGraph`. `CompiledSubAgent` lets the parent delegate to it through the same `task` tool.

```python
custom = CompiledSubAgent(
    name="data-analyzer",
    description="Specialized agent for data analysis",
    runnable=create_agent(model="anthropic:claude-sonnet-4-6", tools=[run_sql], system_prompt="..."),
    mode="isolated",
)
```

STEP BY STEP

1. Build the graph and wrap it with a name, a description and the runnable.
2. For typed output, give a normal subagent a `response_format` (a Pydantic model, `ToolStrategy`, `ProviderStrategy` or a raw type).
3. The parent receives the JSON as the `ToolMessage` content.
4. Remember that compiled and remote subagents do not inherit the parent `state_schema`.

ADVANTAGES

- Reuse existing graphs, and get machine-readable results.

LIMITATIONS

- A compiled subagent keeps its **own** system prompt, even in fork mode.

EDGE CASES AND TRAPS

- A missing `messages` key means the wrapper cannot exchange results.
- The description drives routing. Vague descriptions cause poor delegation.
- In dynamic subagents, use `responseSchema` on `task()` and do **not** `JSON.parse` the result.

LIKELY FOLLOW-UPS

- *Why use structured output?* So the parent can branch on fields without parsing prose.

Concept Questions

These test why delegation is designed the way it is.

Q53. Why use subagents at all? Why not give the parent more tools?

Short answer. Because tool output floods the parent's context, and a single context cannot work on independent subtasks at once. A subagent keeps the heavy work in its own window and returns one compact report, so the parent stays focused and can run several subagents in parallel. The cost is extra model calls, a single handoff and the risk of a poor brief.

EXPLANATION

Imagine the parent researching five companies. Each needs ten searches and five page reads. If the parent does everything, fifty tool results fill its window, early details are summarised away, and the five jobs are done one after another. With five subagents, each does its fifty calls privately and returns a paragraph. The parent reads five paragraphs.

STEP BY STEP

1. The parent names the subtasks and writes a clear brief for each.
2. It issues several `task` calls in one turn, which run in parallel.
3. Each subagent works in a clean context with only what it needs.
4. Each returns a concise report.
5. The parent synthesises.

ADVANTAGES

- **Context isolation:** bulk data never enters the parent's window.
- **Parallelism:** independent subtasks run together.
- **Specialisation:** each subagent can have its own prompt, tools, model and skills, such as a cheaper model for bulk reading.
- **Structured results** when needed.

LIMITATIONS

- Every subagent is at least one more model run, so total tokens can rise even while the parent's window shrinks.
- A subagent only knows what the brief says. A thin brief gives a thin result.
- One report back. If the subtask goes wrong halfway, the parent finds out at the end (unless it is async).

TRADE-OFFS

- Delegating small, quick tasks costs more than doing them. Delegate when the subtask is large, noisy or independent.

EDGE CASES AND TRAPS

- Without a good description the model will not delegate, or will pick the wrong specialist.
- Ask for concise reports and keep raw data in files, passing paths.
- Isolated subagents do not see the parent's conversation. Put everything they need in the task description.

LIKELY FOLLOW-UPS

- *Is it always cheaper?* No. It protects the parent's context, which matters most on long tasks.

Q54. Isolated versus fork subagents: when do you use each?

Short answer. Isolated (the default) starts from only the task description and your system prompt: good for focused, low-context work. Fork (beta, `deepagents>=0.7.13`) inherits the parent's conversation and state: good for continuing work the parent has already started. A fork **cannot call `task`**.

EXPLANATION

The difference is what the child already knows. An isolated subagent is a new colleague you hand a written brief. A fork is a clone of yourself who remembers everything you just learned.

Dimension	Isolated	Fork
Context	Task description only	Parent's full history and state
System prompt	Yours (required)	Parent's, your addendum appended
Skills	Via `skills`	Not supported; inherits the parent's
Can call `task`	Yes	No
Cache reuse	Own prefix	Shares the parent's prefix if you leave `system_prompt` unset

STEP BY STEP

1. Use isolated when the subtask is self-contained ("research topic X") and you want a clean window.
2. Use fork when the subagent needs what the parent just learned ("draft review comments for the issues you found").
3. Mechanically, a fork's trailing `task` call is replaced by a short continuation preamble, and the fork's final answer returns as a normal tool result.
4. Pass the full set of `tools` you want for a fork, because `tools` replaces.

ADVANTAGES OF FORK

- No need to restate context, and it can share the parent's prompt cache.

LIMITATIONS OF FORK

- It carries the whole history, so it is expensive, and it cannot delegate further.

EDGE CASES AND TRAPS

- A `CompiledSubAgent` keeps its **own** system prompt even in fork mode.
- A fork's `system_prompt` *appends* and reduces cache reuse.
- Forks cost tokens because they carry the whole history. Use them deliberately.

LIKELY FOLLOW-UPS

- *How does a fork keep the cache?* Leave its `system_prompt` unset so that the prefix is identical.

Q55. What are dynamic subagents, which pattern solves which problem, and what is

the security catch?

Short answer. Instead of one `task` call per model turn, the agent writes JavaScript for the interpreter that calls `task()` many times with loops and `Promise.all`. Orchestration becomes deterministic code, and intermediate results never enter the context. The catch: `task()` inside `eval` **bypasses the normal tool path**, so per-call approvals do not fire.

EXPLANATION

Normal delegation costs a model turn per decision. Dynamic subagents let one turn describe a whole workflow in code.

Pattern	Shape	Use for
Classify and act	Route each item to a specialist	Ticket triage
Fan-out and synthesise	One `task()` per item, then merge	Per-file code review
Adversarial verification	Find issues, then sceptical verifiers confirm	Security audits, low false positives
Generate and filter	Parallel proposals, score, pick best	Design alternatives
Tournament	Pairwise judging until one winner	Subjective optimisation
Loop until done	`while` rounds, dedupe with a `Set`, stop when nothing new	Exhaustive sweeps

STEP BY STEP (ADVERSARIAL VERIFICATION)

1. A scanner subagent produces candidate findings, each with file and line.
2. For each finding, dispatch a separate verifier with a **sceptical system prompt** and a structured verdict schema.
3. Keep only findings the verifiers confirm. The documentation's diagram shows three verifiers voting by majority.
4. Synthesise the confirmed list.

ADVANTAGES

- Cheap orchestration, high parallelism, and patterns that improve precision.

LIMITATIONS

- Beta. Needs `langchain-quickjs` and Python 3.11 or later.
- The documentation specifies **no concurrency caps, timeouts or dispatch quotas**. Cost can run away.

EDGE CASES AND TRAPS

- **Approval bypass.** `interrupt_on` does not guard `task()` or `tools.*` inside `eval`. Gate the **`eval` tool itself**.
- Set your own limits (for instance with call-limit middleware) and be aware of cost.
- Say **"workflow"** in the prompt to encourage code-based orchestration. Plain phrasing yields one delegation.
- File discovery in code needs the tool on the PTC allowlist, for example `ptc=["glob"]`.

LIKELY FOLLOW-UPS

- *How do you stop a runaway loop?* Cap rounds in the code, rely on the 5-second per-`eval` timeout, and use model and tool call limits.

Q56. Sync, dynamic or async: how do you choose, and what are the async gotchas?

Short answer. Sync when you need the answer now and the work is isolated. Dynamic when many parallel workers need deterministic orchestration inside one turn. Async when the work is long-running and you want a responsive supervisor that can steer or cancel. The async gotchas are worker slots, premature polling, stale status, truncated IDs and waiting subagents.

EXPLANATION

Need	Choice	Why
Isolate one heavy subtask	Sync subagent	Simple, one report
Continue half-done work	Fork subagent	Inherits context
200 items, verify each	Dynamic subagents	Loops and voting in code
Hours-long research the user may redirect	Async subagent	Non-blocking, steerable
Track visible progress	`TodoListMiddleware`	`write_todos` in state

STEP BY STEP (DECISION)

1. Does the supervisor need the result before its next step? Yes: sync or dynamic. No: async.
2. Is the workload many similar items with shared logic? Yes: dynamic.
3. Will a human want to steer mid-flight? Yes: async.
4. Does it need approval per dispatch? Then avoid dynamic, or gate `eval`.

ADVANTAGES AND LIMITATIONS IN ONE LINE EACH

- Sync is simple but blocks. Dynamic is powerful but bypasses per-call approval. Async is non-blocking and steerable but needs infrastructure.

EDGE CASES AND TRAPS (ASYNC)

- **Worker slots.** Each active run uses one. Three subagents plus the supervisor means four. Under-provisioning queues launches (`langgraph dev --n-jobs-per-worker 10`).
- **Premature polling.** The supervisor may `check` right after launch. Tell it in the system prompt to return control to the user.
- **Stale status** in chat history, and **truncated task IDs**, both break `check` and `cancel`. Always call `list` or `check`.
- A **waiting** subagent (paused on `interrupt()`) is answered by the **user through the subagent's own thread**, not by the supervisor. `update` on a waiting task drops the pending request; `cancel` only marks it cancelled.

LIKELY FOLLOW-UPS

- *Can you mix them?* Yes. Hybrid topologies mix co-deployed and remote subagents, and a dynamic workflow can call sync subagents.

Scenario Questions

Restate, diagnose, fix, then say what could go wrong with the fix.

Q57. After adding a subagent, the parent's rules and tools seem to have vanished. Why?

Scenario. You added a `fetcher` subagent with its own `tools` and `permissions`. Afterwards, the subagent can no longer search, and a security test shows it writing to `/memories/`, which the parent forbids.

Short answer. `tools` and `permissions` **replace** the parent's values; they do not extend them. List the full tool set you want, and repeat the deny rules in the subagent's permission list.

STEP BY STEP

1. **Compare to the table.** The subagent set `tools=[fetch_url]`, so it lost `search`. It set `permissions=[...]`, so the parent's deny on `/memories/**` no longer applies.
2. **Fix tools.** List `[search, fetch_url]` explicitly, or share a variable that both lists use.
3. **Fix permissions.** Start the subagent's list from the parent's rules and add to it: define `BASE_RULES` once and use `BASE_RULES + extra`.
4. **Check other fields.** The subagent has no `skills` and no `middleware` unless set. If it also needs a skill or a retry middleware, add them.
5. **Test.** Write a test that runs the subagent and asserts both that it can call `search` and that it cannot write to `/memories/`.

ADVANTAGES OF REPLACE SEMANTICS

- A subagent can be given a *narrower* surface than the parent, which is the safer direction.

LIMITATIONS

- The default is easy to misread, and a "small" override can widen access.

EDGE CASES AND TRAPS

- Permissions guard only the built-in file tools. A subagent with `execute` can bypass them.
- Runtime context still flows to the subagent, so tools there see the same `context`.

LIKELY FOLLOW-UPS

- *How would you prevent it in code review?* A helper that builds subagent specs from a shared base and a lint test that asserts the permission list always contains the mandatory denies.

Q58. Review 200 documents in parallel with high precision. Design it.

Scenario. A compliance team needs every document in a set of 200 checked against a policy. False positives waste reviewers' time, and the agent must not flood its context.

Short answer. Use dynamic subagents. Fan out one reviewer per document in code, then apply adversarial verification: a separate sceptical verifier confirms each finding. Return only confirmed findings and write bulk output to files.

STEP BY STEP

1. **Enable the tools.** Add `CodeInterpreterMiddleware(ptc=["glob"])` so that code can discover files, with subagents available through `task()`.
2. **Fan out.** In one `eval`, use `Promise.all` to run a `reviewer` subagent per document, each with a structured `responseSchema` listing findings (file, line, rule, evidence).
3. **Verify.** For each finding, dispatch a `verifier` with a sceptical system prompt and a verdict schema. Optionally use three verifiers and a majority vote.
4. **Filter.** Keep only confirmed findings.
5. **Control cost.** Add concurrency control and a limit on total dispatches, because none is built in. Use a cheaper model for first-pass reviewers.
6. **Report.** Write the confirmed list to a file and return a short summary and the path.
7. **Gate.** The workflow bypasses per-call approvals, so approve the `eval` call itself if the cost matters.

ADVANTAGES

- Parallel speed, high precision from verification, and a small parent context.

LIMITATIONS

- Beta features, no built-in quotas, and more model calls than a single pass.
- Sceptical verifiers can reject true findings, so measure recall as well as precision.

EDGE CASES AND TRAPS

- Do not `JSON.parse` the result of a call with `responseSchema`; it is already structured.
- The 5-second timeout and 256-call cap apply per `eval`. For 200 documents plus verification you may need several `eval` calls, or batches.
- Provider rate limits will hit a burst of 200 parallel calls.

LIKELY FOLLOW-UPS

- *How would you measure quality?* Build a labelled set, measure precision and recall of findings before and after verification.
- *What if a document is enormous?* Have the reviewer read it in sections from a file, not in one call.

Q59. Design an hours-long research job that the user can steer.

Scenario. A user asks for a market study that takes hours. They want to keep chatting, add a new competitor midway, and cancel if it goes wrong.

Short answer. Use an async subagent on an Agent Protocol server. The supervisor starts the task, returns control to the user, and uses `check`, `update` and `cancel` on request. Provision worker slots, instruct the supervisor not to poll, and verify task IDs with `list`.

STEP BY STEP

1. **Deploy** the supervisor and a `researcher` graph in one `langgraph.json` (ASGI, recommended).
2. **Configure** `AsyncSubAgent(name="researcher", description=..., graph_id="researcher")` with a specific description.
3. **Instruct the supervisor:** after `start_async_task`, report the task ID, and return control to the user. Do not poll in a loop.
4. **Steer.** When the user adds a competitor, call `update_async_task` with the new instruction. It restarts the run on the same thread with full history, and the task ID does not change.
5. **Cancel.** `cancel_async_task` on request.
6. **Notify.** Use notifications (beta, `deepagents>=0.7.26`) so the supervisor learns of success, error, timeout or waiting.
7. **Provision.** Each active run uses a worker slot: supervisor plus N subagents.

ADVANTAGES

- Non-blocking, steerable and cancellable, with a durable thread per task.

LIMITATIONS

- Needs LangSmith Deployment or another Agent Protocol server, and notification and waiting features are beta.

EDGE CASES AND TRAPS

- **Stale status** in the chat history: always call `check` or `list` for the truth.
- **Truncated IDs:** models sometimes shorten task IDs. Verify with `list`.
- A **waiting** subagent needs an answer from the user through *its own thread*, not through the supervisor.
- `update` on a waiting task drops the pending request.
- Summarization does not lose task metadata, because the `async_tasks` channel is separate from messages.

LIKELY FOLLOW-UPS

- *What if the user closes the browser?* The task runs on the server. When they return, `list` shows the state.
- *How do you bound cost?* A timeout per task, a maximum number of concurrent tasks, and a budget check in the subagent.

Q60. The coordinator never delegates, or delegates everything and returns raw data. How do you fix delegation quality?

Scenario. In testing, one version of the agent does all the work itself and fills its window. Another delegates every small step, and each subagent returns pages of raw text that the parent then has to read.

Short answer. Delegation is driven by descriptions and briefs. Write specific descriptions that say when to use each subagent, tell the parent when to delegate and when not to, require concise structured reports, and move bulk data into files passed by path.

STEP BY STEP

1. **Inspect the traces.** Does the parent see the subagents in its tool list, and how do their descriptions read?
2. **Sharpen descriptions.** "Researches a topic in depth; use when the question needs more than three searches" routes better than "helps with research".

3. **Instruct the parent.** In the system prompt: delegate large, noisy or independent subtasks; do small lookups yourself; give each subagent a complete brief including the goal, constraints, and the expected output format.
4. **Instruct the subagent.** Require a short report with the key facts and file paths. Raw data goes in files.
5. **Use structured output** (`response_format`) when the parent needs to branch on fields.
6. **Set the model.** A cheaper model for bulk subagents reduces the cost of over-delegation.
7. **Evaluate.** Count tool calls in the parent versus children and the size of reports across a test set.

ADVANTAGES

- Better routing, smaller reports and lower total cost, with no code changes beyond prompts.

LIMITATIONS

- Delegation quality depends on the model, so test on the model you ship.

EDGE CASES AND TRAPS

- Isolated subagents see only the description. If the brief omits a constraint, the subagent will not know it.
- A custom subagent named `general-purpose` replaces the default one, which may remove a useful catch-all.
- Parallel `task` calls in one turn run concurrently. Ask the parent to batch independent calls in a single turn.

LIKELY FOLLOW-UPS

- *How would you detect over-delegation automatically?* Track the ratio of subagent runs to parent tool calls and the average report size in LangSmith, filtering by `lc_agent_name`.

CHAPTER 69

Interview Questions: Steering with Human-in-the-Loop and Rubrics

At a glance: Ten questions on keeping humans and graders in control. Direct questions cover the configuration, the four decisions and the rubric loop. Concept questions explain durability, the different kinds of interrupt, and how to choose between approval, grading and limits. Scenarios cover an approval that never fires, a double side effect, conditional approval and a rubric that never passes.

Direct Questions

State the fact, say what it means, then add one edge case.

Q61. How do you configure human-in-the-loop, and what are the four decisions?

Short answer. Pass `interrupt_on` to `create_deep_agent`, mapping tool names to `True` (all four decisions), `False` (never interrupt) or a config with `allowed_decisions` and an optional `when` predicate. The decisions are `approve`, `edit`, `reject` and `respond`. It **requires a checkpointer**.

EXPLANATION

`HumanInTheLoopMiddleware` is added for you when you pass `interrupt_on`. It pauses the graph *before* a matching tool call runs, and it works on the main agent and on subagents.

```python
agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
    tools=[remove_file, fetch_file, notify_email],
    interrupt_on={
        "remove_file": True,                                          # all four decisions
        "fetch_file": False,                                          # never interrupt
        "notify_email": {"allowed_decisions": ["approve", "reject"]},  # no editing
    },
    checkpointer=MemorySaver(),                                       # required
)
```

Decision	Behaviour	Typical use
`approve`	Run the tool with its original arguments	Send the drafted email
`edit`	Run the tool with modified arguments	Change the recipient
`reject`	Skip the tool and send feedback to the agent	Deny a deletion
`respond`	The human's reply becomes a **synthetic tool result**; nothing executes	`ask_user`-style tools

STEP BY STEP

1. List the tools whose mistakes would be costly: deletes, emails, payments, database writes.

2. Set the allowed decisions per tool. Remove `edit` where editing makes no sense.
3. Add a checkpointer, so the pause can last for hours or days.
4. Optionally narrow the pause with `when`, a predicate `(ToolCallRequest) -> bool` (Python only, `langchain>=1.3.3`). If it returns `False`, the call is **auto-approved**.
5. A subagent may define its own `interrupt_on`; it inherits the parent's otherwise, and also needs a checkpointer.

ADVANTAGES

- Safe by default for risky tools, with durable pauses and per-call precision.

LIMITATIONS

- It does not protect `task()` or `tools.*` calls made from inside the interpreter, because those skip the normal tool path.
- Persistent shell sessions (`ShellToolMiddleware`) do not yet work with interrupts.

EDGE CASES AND TRAPS

- Use `respond` only when the human *is effectively the tool*. Do not use it to deny a side-effecting tool, because the model may read the reply as success.
- Keep edits minimal. Large edits can make the model re-plan and re-run tools.

LIKELY FOLLOW-UPS

- *What if no checkpointer is set?* The run cannot pause and resume, so the interrupt cannot work.

Q62. Walk through the interrupt payload and how you resume.

Short answer. When a gated tool is requested, the result contains `__interrupt__`. Its value has `action_requests` (name and arguments) and `review_configs` (allowed decisions per action). You build **one decision per request, in the same order**, and resume with `Command(resume={"decisions": decisions})` on the **same `thread_id`**.

EXPLANATION

PYTHON

```python
config = {"configurable": {"thread_id": str(uuid.uuid4())}}   # keep the SAME thread_id
result = agent.invoke({"messages": [...]}, config=config)

if result.get("__interrupt__"):
    payload = result["__interrupt__"][0].value
    decisions = []
    for req in payload["action_requests"]:                    # one decision per request, same order
        if req["name"] == "notify_email":
            decisions.append({"type": "reject", "message": "Do not email anyone yet."})
        else:
            decisions.append({"type": "approve"})
    result = agent.invoke(Command(resume={"decisions": decisions}), config=config)
```

STEP BY STEP

1. Invoke with a `thread_id`. The graph runs until a gated call is requested, then stops and checkpoints.
2. Read `result["__interrupt__"]`. In a streaming app, the interrupt arrives as `__interrupt__` in the `updates` mode.
3. Show each request to a human along with its allowed decisions.
4. Build the decision list, matching the **count and the order** of `action_requests`.
5. Resume with `Command(resume={"decisions": [...]})` and the same configuration.

6. `approve` runs the tool with the original arguments. `edit` runs it with `edited_action`. `reject` skips it and sends your message to the agent. `respond` makes the reply a synthetic tool result.

ADVANTAGES

- Resumption can happen in a different process, on a different day.

LIMITATIONS

- The caller must keep the `thread_id` and a durable checkpointer.

EDGE CASES AND TRAPS

- **No checkpointer, or a changed `thread_id` on resume, breaks the resume.**
- A `reject` without a message sends a default "not executed, do not retry" note. Supply a specific message for side-effecting tools.
- Several tools that need approval in one turn produce **one batched interrupt** by default. `langchain>=1.4.6` adds a per-call mode.
- Parallel pending interrupts resume with a map of interrupt ID to value.

LIKELY FOLLOW-UPS

- *Why does the order matter?* Decisions are matched to requests by position.
- *How do you stream it?* Use `stream_mode=["messages", "updates"]` and watch for `__interrupt__`.

Q63. How does `RubricMiddleware` work, and what are the verdicts?

Short answer. After the agent finishes, a grader model checks the output against a newline-delimited checklist in `state["rubric"]`. On `needs_revision`, per-criterion feedback goes back and the agent runs again, up to `max_iterations` (default 3). It needs `deepagents>=0.6.5` and is beta.

EXPLANATION

A rubric is an LLM-as-a-judge loop that you do not have to hand-write. You say what "done" means as checkable criteria, and the middleware keeps the agent working until the criteria pass or the cap is reached.

Status	Loops?
`satisfied`	No
`needs_revision`	Yes, until the cap
`max_iterations_reached`	No (terminal; stored on `_rubric_status`)
`failed` (rubric malformed or unevaluable)	No
`grader_error` (the grader raised)	No (empty `criteria`; the explanation has the exception)

STEP BY STEP

1. Add `RubricMiddleware(model="anthropic:claude-haiku-4-5", max_iterations=3, tools=None, on_evaluation=...)` and a checkpointer.
2. Invoke with `{"messages": [...], "rubric": "- The answer is one sentence\n- The answer includes a source URL\n"}`.
3. The grader returns a verdict. `satisfied` ends the loop. `needs_revision` loops with feedback.

4. After `invoke`, read `result.get("_rubric_status")`.
5. Give the grader tools (for example a test runner) when criteria need evidence.

ADVANTAGES

- An automatic quality loop with a cheap grader, and observability through `on_evaluation`.

LIMITATIONS

- Beta. An LLM judge is itself fallible, and cost scales with iterations.

EDGE CASES AND TRAPS

- On cap exhaustion the last evaluation still reads `needs_revision`. **Check `_rubric_status == "max_iterations_reached"`.**
- Errors inside `on_evaluation` are **swallowed**, so never use it for control flow.
- **No `rubric` in state means the middleware does nothing.**
- A new rubric string starts a new run. Persistence needs a checkpointer and a shared `thread_id`.

LIKELY FOLLOW-UPS

- *Which model should grade?* Often a smaller, cheaper one, but test whether it catches the failures you care about.

Concept Questions

These test whether you understand how steering is designed and what it can and cannot guarantee.

Q64. Compare `when` predicates, per-call mode, in-tool interrupts and permission interrupts.

Short answer. All four pause a run. They differ in *who decides when* and *what the resume value looks like*.

EXPLANATION

Mechanism	Decided by	Resume value	Notes
`interrupt_on` with `when`	A predicate on the tool call	`{"decisions": [...]}`	`when` returning `False` means auto-approved; Python only, `langchain>=1.3.3`
Per-call mode	`interrupt_mode="per_call"`	Answered by call ID	`langchain>=1.4.6`; batched stays the default
In-tool `interrupt()`	Your tool code	Free-form payload	Not the `decisions` schema
`FilesystemPermission(mode="interrupt")`	A path rule	`{"decisions": [...]}`	`deepagents>=0.6.8`; merges with `interrupt_on`

STEP BY STEP (CONDITIONAL APPROVAL)

1. Write `"when": lambda request: "drop" in request.tool_call["args"]["query"].lower()` for an `execute_sql` tool.
2. Safe queries run freely. Risky ones appear in the batched interrupt.
3. For MCP tools, derive the set of destructive tools from server-declared annotations (`destructive_hint`) at discovery, and pass a `when` that lets `dry_run` calls through.

ADVANTAGES

- You can match the friction to the risk: no pause for safe calls, a pause for dangerous ones.

LIMITATIONS

- A `when` predicate is code you must maintain. A naive keyword check can be bypassed by creative arguments.

TRADE-OFFS

- Batched interrupts are efficient for a human who reviews several calls together. Per-call mode suits workflows where each call is answered independently.
- In-tool interrupts give maximum freedom (any payload) but lose the standard decision schema.

EDGE CASES AND TRAPS

- **An in-tool interrupt re-executes the tool from the top on resume.** Anything before `interrupt()` runs twice, so make it idempotent.
- Parallel pending interrupts resume with a map of interrupt ID to value.
- MCP elicitation uses the same mechanism and the same re-run rule.
- An unanchored permission interrupt such as `/**/secrets` over-fires on `ls` , `glob` , `grep` and directory deletes. Anchor it: `/secrets/**` .

LIKELY FOLLOW-UPS

- *How do you ask the user a question mid-tool?* An in-tool `interrupt()` or a `respond` -style tool.

Q65. Why can an interrupt last for days, and what does that imply for your tools?

> **Short answer.** Because LangGraph checkpoints the state at the interrupt and the run simply stops. Nothing is held in memory, so the pause can last as long as the checkpointer retains the thread. When resumed, the interrupted node **re-executes from the top**, so side effects before the interrupt can repeat.

EXPLANATION

Durable execution is what turns approval from a blocking prompt into a workflow step. A worker can crash, a server can be redeployed, and the human can answer next week. The checkpoint holds the *state*. It does not hold the *effects* of anything the node already did.

STEP BY STEP

1. The model requests a gated tool. The graph checkpoints and returns `__interrupt__` .
2. The process that was running can end. Nothing is waiting.
3. The human decides, through any client, possibly on another device.
4. A worker loads the checkpoint for the `thread_id` and resumes.

5. The interrupted node starts again from the top, with the decision available.

ADVANTAGES

- Indefinite pauses, crash recovery, time travel to earlier checkpoints, and an audit point before irreversible actions.

LIMITATIONS

- You must keep the checkpointer and retention policy in mind. A deleted thread cannot be resumed.
- Large `StateBackend` files are checkpointed every step. Delta checkpoints (v0.6, beta) reduce that, but rolling back to an older version is unsupported once threads persisted with deltas.

EDGE CASES AND TRAPS

- Anything before an in-tool `interrupt()` runs twice. Put the interrupt first, or make earlier steps idempotent.
- Retries handle a failed *call*; checkpoints handle a failed *process*. Both are needed.
- Stale approvals: a decision made days later may refer to data that has changed. Show the current context at review time.

LIKELY FOLLOW-UPS

- *How do you avoid double emails?* Use an idempotency key, and place the interrupt before the send.

Q66. When do you use approvals, rubrics or call limits?

> **Short answer.** Approvals gate *actions* before they happen. Rubrics check *outputs* after the agent finishes and loop until they pass. Call limits stop *runaway behaviour*. They solve different problems and are often combined.

EXPLANATION

Need	Use
Approve or deny a specific tool call	`interrupt_on`
Approve only risky arguments	`interrupt_on` with `when`
Approve writes to certain paths	`FilesystemPermission(mode="interrupt")`
Ask the user a question mid-tool	In-tool `interrupt()` or a `respond`-style tool
Check output quality and retry automatically	`RubricMiddleware`
Stop runaway loops	`ModelCallLimitMiddleware`, `ToolCallLimitMiddleware`

STEP BY STEP

1. Ask whether the harm is in the *action* (approval) or in the *result* (rubric).
2. Ask whether a human must decide (approval) or a checkable criterion can (rubric).
3. Ask what happens if the agent loops (limits).
4. Combine: approvals on the risky tools, a rubric on the final output, and limits as a seat belt.

ADVANTAGES OF COMBINING

- Each layer catches a different failure, so no single layer needs to be perfect.

LIMITATIONS

- Approvals cost human time, rubrics cost model calls, and limits can cut off legitimate long work.

EDGE CASES AND TRAPS

- Approvals do not cover tools called through the interpreter. Gate the `eval` tool.
- A rubric cannot catch what the grader cannot see. If the check needs evidence, give the grader tools.
- Limits that are too low cause false failures. Set them from observed distributions.

LIKELY FOLLOW-UPS

- *Which would you add first for a new agent?* Limits and a narrow set of approvals on irreversible tools.

Scenario Questions

Restate, diagnose, fix, then say what could go wrong with the fix.

Q67. The approval never pauses. The agent just deletes the file.

Scenario. You set `interrupt_on={"delete_record": True}` and, in staging, the agent deleted a record without any pause.

Short answer. Check, in order: is there a checkpointer, does the tool name match exactly, is the call made through the interpreter (which skips interrupts), and is it a built-in file tool covered instead by a permission interrupt?

STEP BY STEP

1. **Checkpointer.** `interrupt_on` needs one. Without it the run cannot pause and resume.
2. **Name.** The key must equal the tool's registered name. `delete_record` and `deleteRecord` differ.
3. **Value.** `False`, or a `when` predicate that returned `False`, means "no pause".
4. **Path.** If the call runs inside `eval` as `tools.deleteRecord(...)` or `task()`, interrupts are not enforced. Gate the `eval` tool, or remove the tool from the allowlist.
5. **Built-in tool.** For `delete` or `write_file`, use `FilesystemPermission(mode="interrupt")` (`deepagents>=0.6.8`) or list those tool names in `interrupt_on`.
6. **Subagent.** A subagent that sets its own `interrupt_on` wins over the parent's. A subagent without it inherits.
7. **Test.** Write a test that asserts `__interrupt__` appears for each gated tool.

ADVANTAGES OF THE TEST

- It turns a silent safety gap into a failing build.

LIMITATIONS

- Tests cover the tools you remembered to list. New tools must be added deliberately.

EDGE CASES AND TRAPS

- Streaming consumers may ignore `__interrupt__`, so the UI looks as if the run ended.

- A middleware ordering problem could hide a call. Keep the human-in-the-loop layer where `create_deep_agent` puts it.

LIKELY FOLLOW-UPS

- *How would you make the safe path the default?* A wrapper that lists every side-effecting tool in `interrupt_on` unless it is explicitly marked read-only.

Q68. A user approves a transfer, and the customer is charged twice. What happened?

Scenario. A `transfer_funds` tool calls the payment API, then calls `interrupt()` to ask for approval, then returns. After the user approved, two transfers appeared.

Short answer. On resume, the tool re-executed from the top, so the payment call before the interrupt ran again. Put the `interrupt()` first, and make the payment idempotent with a key.

STEP BY STEP

1. **Read the code order.** The charge happens before `interrupt()`.
2. **Understand resume.** The interrupted node re-runs from the start. The first run charged, paused, then the resumed run charged again before reaching the point where `interrupt()` returns the answer.
3. **Fix the order.** Call `interrupt()` first, and charge only after approval.
4. **Add an idempotency key.** Derive it from the thread and the tool call ID, so a repeat is recognised by the payment provider.
5. **Prefer declarative gating.** Where possible, use `interrupt_on` for the tool instead of a hand-written in-tool interrupt, so the pause happens before the tool body runs.
6. **Test.** Assert exactly one charge per approved request, including after a simulated crash and resume.

PYTHON

```python
@tool
def transfer_funds(amount: float, to: str) -> str:
    """Transfer money."""
    approval = interrupt({"type": "approval_request", "amount": amount, "to": to})  # first
    if not approval.get("approved"):
        return "cancelled"
    return pay(amount, to, idempotency_key=...)      # side effect only after approval
```

ADVANTAGES

- A single, well-placed interrupt plus an idempotency key is robust even against crashes.

LIMITATIONS

- Not every external API supports idempotency keys. Then you need your own deduplication record.

EDGE CASES AND TRAPS

- Any work before `interrupt()` that has side effects, such as sending a notification, will repeat.
- A resumed run may happen on a different worker. Do not rely on in-memory state.
- Elicitation from MCP tools uses the same mechanism and has the same re-run rule.

LIKELY FOLLOW-UPS

- *How do retries interact?* A retry layer around the tool can also repeat the call, so idempotency matters twice over.

Q69. Only dangerous SQL should need approval, and MCP tools should be gated by their own hints. Design it.

Scenario. The agent has a read-write SQL tool and many MCP tools from third parties. Reviewers are overloaded when every call pauses, but destructive actions must not slip through.

Short answer. Use `interrupt_on` with a `when` predicate that pauses only risky SQL, and build the gating list for MCP tools from the server's `destructive_hint` annotations at discovery, letting `dry_run` calls through.

STEP BY STEP

1. **SQL.** `interrupt_on={"execute_sql": {"allowed_decisions": ["approve", "reject"], "when": is_risky}}`. Write `is_risky` to parse the statement and treat anything other than a read as risky, rather than searching for a keyword such as `drop`.
2. **Prefer least privilege.** Give the tool a database role that cannot do the dangerous thing at all. Approval is the second line, not the first.
3. **MCP discovery.** After listing tools, read each tool's annotations once and collect the names with `destructive_hint` set.
4. **Build entries.** For each such tool, add an `interrupt_on` entry with a `when` that returns `False` for dry runs.
5. **Allowed decisions.** Offer `approve` and `reject` for destructive tools. Skip `edit` unless the reviewer can reasonably change the arguments.
6. **Log.** Record every decision with the reviewer, the arguments and the time.
7. **Review the metric.** Track the approval rate. A near-100% approval rate means the gate is noise; a high rejection rate means the prompt needs work.

ADVANTAGES

- Reviewers see only calls that matter.

LIMITATIONS

- Annotations are self-declared by the server and may be missing or wrong. Treat unannotated tools from untrusted servers as risky by default.
- A predicate can be bypassed by unusual syntax. Pair it with database permissions.

EDGE CASES AND TRAPS

- `when` is Python-only (`langchain>=1.3.3`).
- Tools called via the interpreter skip the gate.
- If a predicate raises an exception, decide whether the safe behaviour is to pause. Write it to fail closed.

LIKELY FOLLOW-UPS

- *What if the reviewer wants to change the query?* Allow `edit` , and keep the edit small to avoid the model re-planning.

Q70. The rubric loop ends after three tries and the output is still wrong. What do you do?

Scenario. A report generator uses `RubricMiddleware` with criteria such as "the report is high quality" and "all claims are supported". After three iterations it stops, the explanation still says `needs_revision`, and users receive a poor report.

Short answer. Check `_rubric_status` for `max_iterations_reached` and treat that as a failure, rewrite the criteria so they are checkable against the transcript or a tool result, give the grader tools for evidence, and decide what the application does when the cap is hit.

STEP BY STEP

1. **Detect the cap.** After `invoke`, read `result.get("_rubric_status")`. On `max_iterations_reached` the last evaluation still says `needs_revision`, so the status, not the text, is the signal.
2. **Rewrite criteria.** "High quality" is not checkable. Use: "every numbered claim has a source URL", "the report has sections for A, B and C", "no section is under 100 words".
3. **Give evidence tools.** If a criterion needs facts, such as "the URLs return 200", provide the grader a tool that checks them.
4. **Read the per-criterion feedback.** The `criteria` list tells you which criterion fails each iteration. If the same one fails every time, the agent cannot satisfy it as written.
5. **Tune the budget.** Raise `max_iterations` only if iterations are making progress, and watch the cost.
6. **Handle failure.** On the cap, return the best draft with a visible warning, hand off to a human, or retry with a stronger model.
7. **Check the grader.** A weak grader may pass bad output or fail good output. Test it on labelled examples.

ADVANTAGES

- The loop is observable through `on_evaluation`, so you can see why it fails.

LIMITATIONS

- Cost scales with iterations, and a rubric cannot make a task possible that the agent cannot do.

EDGE CASES AND TRAPS

- **No `rubric` in state means nothing happens.** A silently missing rubric looks like a pass.
- Exceptions in `on_evaluation` are swallowed. Do not use it for control flow.
- A new rubric string starts a new run, and the old one is replaced.
- `grader_error` and `failed` are terminal and do not loop. Alert on them.

LIKELY FOLLOW-UPS

- *Would you combine rubrics with approvals?* Yes. The rubric improves the draft, and an approval gates the irreversible step such as publishing.

CHAPTER 70

Interview Questions: Middleware and Reliability

At a glance: Ten questions on the hooks that wrap the agent loop. Direct questions cover the hook types, the prebuilt middleware and their defaults. Concept questions explain execution order, how to write your own, and how to layer reliability. Scenarios cover a production fault-tolerance design, tool overload, PII and a middleware that breaks under concurrency.

Direct Questions

State the fact, say what it means, then add one edge case.

Q71. What is middleware, and which hooks does it offer?

Short answer. Middleware wraps the agent loop with hooks so that you can add observability, transformation, reliability and governance without changing the loop. There are **node-style** hooks (`before_agent` , `before_model` , `after_model` , `after_agent`) that run sequentially at fixed points, and **wrap-style** hooks (`wrap_model_call` , `wrap_tool_call`) that run *around* a call and decide whether and how often it executes.

EXPLANATION

The core loop is: call the model, let it choose tools, run them, repeat until there are no more tool calls. Middleware exposes a hook before and after each step. Deep Agents *is* a pre-assembled stack of middleware, and yours slots into it.

Hook	Runs	Typical use
`before_agent`	Once per invocation, before the agent starts	Load per-run data
`before_model`	Before **each** model call	Validate, trim, update state
`wrap_model_call`	Around each model call	Retry, cache, switch model, rewrite the request
`after_model`	After **each** model response	Inspect or block output
`wrap_tool_call`	Around each tool call	Monitor, retry, short-circuit
`after_agent`	Once per invocation, after the agent completes	Final logging, cleanup

Authoring styles: **decorators** (`@before_model` , `@wrap_model_call` , `@wrap_tool_call` , `@dynamic_prompt`) for a single hook, and **`AgentMiddleware` subclasses** for several hooks, configuration, sync plus async versions, extra state or bundled tools.

STEP BY STEP

1. Decide the concern: observability, transformation, reliability or governance.
2. Choose node-style for sequential checks and state updates, and wrap-style for control flow.
3. Choose a decorator for one hook, a class for anything more.

4. Return a state-update dict, a modified request (`request.override(...)`), or `jump_to`.

ADVANTAGES

- Extend behaviour without touching the loop. Composable and reusable across agents.
- Works inside LangGraph. The whole agent, middleware included, can be a node in a larger graph.

LIMITATIONS

- Middleware is **not a separate runtime**, so its behaviour depends on the graph it runs in.
- **Order matters.** In Deep Agents, a same-name replacement *replaces*; it does not merge.

EDGE CASES AND TRAPS

- `HumanInTheLoopMiddleware` matches tools by `.name` (for `@tool` functions, the function name).
- Wrap hooks may call the handler **zero** times (short-circuit), **once** or **many** times (retry).

LIKELY FOLLOW-UPS

- *When do you reach for middleware?* When behaviour must apply on every model or tool call, when you want it reusable, or when you must change the request or response itself.

Q72. Which prebuilt middleware solves which problem?

Short answer. Reliability: tool error, tool retry, model retry, model fallback, call limits. Context: summarization, context editing, tool selector, provider tool search. Safety: human-in-the-loop, PII, rubric. Capabilities: to-do list, shell, filesystem, file search, subagent. Testing: LLM tool emulator.

EXPLANATION

There are nineteen ready-made middleware. The skill is matching the symptom to the middleware, not memorising constructors.

Problem	Middleware
A tool raised an exception	`ToolErrorMiddleware` (convert to a message), `ToolRetryMiddleware` (retry)
Flaky model API	`ModelRetryMiddleware`, then `ModelFallbackMiddleware`
Runaway loop or cost	`ModelCallLimitMiddleware`, `ToolCallLimitMiddleware`
Context too long	`SummarizationMiddleware`, `ContextEditingMiddleware`
Too many tools	`LLMToolSelectorMiddleware`, `ProviderToolSearchMiddleware`
Human approval	`HumanInTheLoopMiddleware`
Personal data	`PIIMiddleware`
Planning	`TodoListMiddleware`
Quality gate	`RubricMiddleware` (Deep Agents, beta)
Shell and files	`ShellToolMiddleware`, `FilesystemMiddleware`, `FilesystemFileSearchMiddleware`
Delegation	`SubAgentMiddleware` (Deep Agents)
Testing without real tools	`LLMToolEmulator`

Provider-specific ones also exist: Anthropic (prompt caching, bash tool, text editor, memory, file search), AWS (prompt caching for Bedrock) and OpenAI (content moderation).

STEP BY STEP

1. Name the symptom.
2. Pick the middleware from the table.
3. Pass it in `middleware=[...]` to `create_agent` or `create_deep_agent`.
4. Decide its position, since order matters.

ADVANTAGES

- Battle-tested, configurable, and composable. Configuration replaces custom loop code.

LIMITATIONS

- Version pins differ per feature (`1.3.2`, `1.3.14`, `1.3.16`, `1.4.6`), so check what you have installed.

EDGE CASES AND TRAPS

- Thread limits and human-in-the-loop need a checkpointer.
- `ShellToolMiddleware` defaults to **host** access (`HostExecutionPolicy`).
- `ProviderToolSearchMiddleware` raises `ValueError` on unsupported providers.
- Deep Agents already includes filesystem, subagent and summarization middleware, plus human-in-the-loop when you set `interrupt_on`.

LIKELY FOLLOW-UPS

- *Which would you add to every production agent?* Model retry, a call limit and, where relevant, PII handling.

Q73. What are the defaults of the reliability middleware?

Short answer. Retries: 2 retries (3 attempts), backoff factor 2.0, initial delay 1 second, maximum delay 60 seconds, plus or minus 25% jitter. Tool-call limits default to `exit_behavior='continue'`. Summarization in Deep Agents triggers at 85% and keeps 10%. Context editing triggers at 100,000 tokens, keeps 3 results and leaves `[cleared]`.

EXPLANATION

Middleware	Defaults worth knowing
`ToolRetryMiddleware`	`max_retries=2`, `tools=None` (all), `on_failure='continue'`, `backoff_factor=2.0`, `initial_delay=1.0`, `max_delay=60.0`, `jitter=True`
`ModelRetryMiddleware`	Same backoff model. `on_failure='continue'` returns an `AIMessage` with error details
`ModelFallbackMiddleware`	`first_model` required; additional models are tried in order
`ModelCallLimitMiddleware`	`thread_limit` (needs a checkpointer), `run_limit`, `exit_behavior='end'` or `'error'`
`ToolCallLimitMiddleware`	`tool_name` (omit for all), `thread_limit`, `run_limit`, `exit_behavior='continue'` (default), `'error'` or `'end'` (single tool only)
`PIIMiddleware`	`strategy='redact'`, `apply_to_input=True`, `apply_to_output=False`, `apply_to_tool_results=False`
`ContextEditingMiddleware`	`ClearToolUsesEdit(trigger=100000, keep=3, placeholder="[cleared]")`

Middleware	Defaults worth knowing
`SummarizationMiddleware`	No trigger by default in LangChain; Deep Agents sets 85% and keeps 10%

Delay between retries is `initial_delay * backoff_factor ** retry_number`, with jitter.

STEP BY STEP

1. Retry delay for the first retry is about 1 second, the second about 2 seconds, and so on, never above 60 seconds.
2. A run limit resets on each user turn. A thread limit accumulates across the conversation.
3. For PII, `redact` replaces with `[REDACTED_EMAIL]`, `mask` shows the last digits, `hash` is deterministic, and `block` raises.

ADVANTAGES

- Sensible defaults mean a one-line addition gives real protection.

LIMITATIONS

- Defaults are generic. A retry that is right for a rate limit is wasteful for a malformed request.

EDGE CASES AND TRAPS

- `ToolCallLimitMiddleware(exit_behavior="end")` works only for a single named tool; otherwise it raises `NotImplementedError`.
- `apply_to_output` and `apply_to_tool_results` default to false. With `apply_to_output=True` and `langchain>=1.3.2`, streamed output is redacted too.
- Integration exceptions such as `ModelRateLimitError` and `ModelTimeoutError` carry an `is_retryable` flag.

LIKELY FOLLOW-UPS

- *Retry or fallback?* Retry is the same model for a transient problem. Fallback is a different model when the provider is down.

Concept Questions

These test whether you understand how the hooks compose.

Q74. Explain the hook execution order with a concrete trace.

Short answer. With `middleware=[m1, m2, m3]`, `before_*` hooks run m1 to m3, `after_*` hooks run **m3 to m1**, and `wrap_*` hooks nest with m1 outermost. Put critical middleware first, because the first one wraps everything after it.

EXPLANATION

Picture an onion. The first middleware is the outermost layer. On the way in you pass m1, m2, m3 before reaching the model. On the way out you pass m3, m2, m1.

STEP BY STEP (TRACE ONE MODEL CALL)

1. `before_agent`: m1, m2, m3 (once per invocation).
2. `before_model`: m1, m2, m3 (before every model call).

3. `wrap_model_call`: m1 starts and calls its handler, which enters m2, which enters m3, which calls the model. The response unwinds m3, m2, m1.
4. `after_model`: **m3, m2, m1**.
5. Tools run, with `wrap_tool_call` nested the same way.
6. Loop back to step 2 while the model keeps requesting tools.
7. `after_agent`: **m3, m2, m1** (once).

ADVANTAGES

- Predictable composition. An outer retry wraps inner caching, for example, by position alone.

LIMITATIONS

- The reversal of `after_*` hooks is easy to forget, and some interactions (retry versus error conversion) depend on placement.

TRADE-OFFS

- Outer placement sees everything but also sees retries and fallbacks as one call. Inner placement sees each attempt.

EDGE CASES AND TRAPS

- A wrap hook may call its handler **zero times** (a cache hit), **once**, or **many** times (retry). Never assume the wrapped call ran exactly once.
- Early exit is done by returning `jump_to` (`'end'`, `'tools'` or `'model'`) from a hook that declared `can_jump_to`. A `jump_to` without it is invalid.
- Several `ExtendedModelResponse` commands compose: inner commands apply first and outer commands win for non-reducer fields. Commands from discarded handler calls are dropped.
- Deep Agents inserts your middleware **after** `PatchToolCallsMiddleware`.

LIKELY FOLLOW-UPS

- *Where would you put a logger that must see every call?* First in the list, so that it wraps the others.

Q75. You are asked to write custom middleware. What are the patterns and pitfalls?

> **Short answer.** Use decorators for a single hook and classes for several hooks, configuration, sync plus async, extra state or bundled tools. Return state-update dicts from node hooks, use `request.override(...)` in wrap hooks, never mutate `self`, and write async hooks if you will deploy on Managed Deep Agents.

EXPLANATION

Good middleware does one job, changes the request rather than the world, and keeps its state in the graph.

STEP BY STEP

1. **Pick the style.** `@before_model`, `@wrap_model_call`, `@wrap_tool_call` or `@dynamic_prompt` for one-offs. A subclass of `AgentMiddleware` for everything else.
2. **Choose node or wrap.** Node-style for sequential checks and state updates (limit a conversation to 50 messages). Wrap-style for control flow (retry, cache, rewrite the request).
3. **Change the request.** `handler(request.override(model=...))`, `override(tools=[...])` or `override(system_message=...)`. All tools must be registered up front. You filter; you do not add.

4. **Add state.** Extend `AgentState` with `NotRequired` fields and pass `state_schema=`. Node hooks return a dict that the reducers merge.
5. **Write both sync and async** (`before_model` and `abefore_model`) when the agent may run either way.
6. **Test it in isolation** before wiring it in.

```python
@wrap_model_call
def pick_model(request, handler):
    model = big_model if len(request.messages) > 10 else small_model
    return handler(request.override(model=model))
```

ADVANTAGES

- Total control of the loop at the points you choose, in testable, reusable units.

LIMITATIONS

- You can break the agent if a hook throws. Handle errors so that middleware failures do not crash the run.

EDGE CASES AND TRAPS

- **Never mutate `self` attributes inside hooks.** Subagents, parallel tool calls and threads run concurrently. Use graph state, for example by returning `{"x": ...}` from `before_agent`.
- Subagent `middleware` does **not** inherit from the parent.
- Class attributes `state_schema`, `tools` and `transformers` are read at compile time. A stream-transformer factory must return a **fresh** transformer per scope.
- In Managed Deep Agents, hooks must be **async**. Synchronous hooks work locally but fail after moving.
- Tracing (`langchain>=1.3.15`): hook spans record inputs and outputs by default. Use `TracePolicy(process_inputs=omit_payload)` to keep payloads out of traces.
- The documentation's class example uses `SystemMessage` without importing it. Import it yourself.

LIKELY FOLLOW-UPS

- *How do you add a dynamic system prompt?* `@dynamic_prompt`, or a `wrap_model_call` that builds a new `SystemMessage` and passes `request.override(system_message=...)`.

Q76. How do you layer reliability, and what does "do not catch what you cannot handle" mean?

Short answer. Route each error to whoever can fix it: transient errors to retries, provider outages to fallback, LLM-recoverable tool errors to `ToolErrorMiddleware`, user-fixable gaps to `interrupt_on`, runaway loops to call limits, and everything unexpected to a loud exception.

EXPLANATION

Reliability middleware fails when it hides problems. An exception swallowed and turned into a vague message makes the model guess. An exception allowed to propagate makes the failure visible. The principle is to catch only what you can meaningfully handle.

Error type	Handler
Transient (network, rate limit)	`ModelRetryMiddleware`, `ToolRetryMiddleware`
LLM-recoverable (bad arguments, parse failure)	`ToolErrorMiddleware`

Error type	Handler
User-fixable (missing information)	`interrupt_on`
Provider outage	`ModelFallbackMiddleware`
Runaway loops	`ModelCallLimitMiddleware`, `ToolCallLimitMiddleware`
Unexpected	Let the exception propagate

STEP BY STEP

1. **Rate limit** the provider with an `InMemoryRateLimiter` passed to `init_chat_model(..., rate_limiter=...)`.
2. **Retry** model calls. Scope `ToolRetryMiddleware` to tools that call *external* services.
3. **Fallback** to another model only after retries are exhausted.
4. **Convert** recoverable tool exceptions to messages with `ToolErrorMiddleware(on_error)`, returning a string for the model or `None` to propagate.
5. **Cap** loops with run and thread limits.
6. **Recover from crashes** with checkpointing. A process failure is not a call failure.

ADVANTAGES

- A clear taxonomy makes failures diagnosable and the system predictable.

LIMITATIONS

- More layers mean more interactions to test.

EDGE CASES AND TRAPS

- **Placement.** Retries must exhaust before an error is turned into a message, otherwise the conversion hides the exception from the retry layer. Test the layering rather than trusting list order. The documentation suggests putting tool retry *inside* tool error, with `on_failure="error"`, so exceptions reach the error handler.
- **Retries multiply cost.** `max_retries=3` on a non-transient failure burns four calls. Retry only retryable errors and pair with call limits.
- **Non-idempotent tools.** Retrying `send_email` or `charge_card` can duplicate the effect. Use idempotency keys, exclude them from `ToolRetryMiddleware(tools=[...])`, or gate with `interrupt_on`.
- Retries handle a failed *call*; checkpoints handle a failed *process*.

LIKELY FOLLOW-UPS

- *Why not catch every exception and tell the model?* The model cannot fix a bug in your code or an expired credential, and the real error would be hidden from your monitoring.

Scenario Questions

Restate, diagnose, fix, then say what could go wrong with the fix.

Q77. Design the fault-tolerance layer for a production agent.

Scenario. Your agent calls a model provider and three external APIs. Last month it suffered a provider outage, a burst of rate limits, a bad-argument loop that burned budget, and a double-sent email after a retry.

Short answer. Rate-limit the client, retry transient errors with backoff, fall back to a second model after retries are exhausted, convert only LLM-recoverable tool errors, cap loops, gate or exclude non-idempotent tools, checkpoint for crashes, and let the unexpected propagate to your alerting.

STEP BY STEP

1. **Rate limit.** `InMemoryRateLimiter` on the model, matched to the provider's quota.
2. **Model retry.** `ModelRetryMiddleware(max_retries=3, backoff_factor=2.0, initial_delay=1.0)`.
3. **Fallback.** `ModelFallbackMiddleware("gpt-5.4-mini", "claude-sonnet-4-6")`, tried only after retries are exhausted.
4. **Tool retry.** `ToolRetryMiddleware(max_retries=2, tools=["search"], on_failure="error")` for idempotent, external, read-only tools only.
5. **Tool errors.** `ToolErrorMiddleware(on_error)` returning a sanitised message for `ValueError`-type argument mistakes, and `None` to propagate everything else.
6. **Loop caps.** `ModelCallLimitMiddleware(run_limit=50)` and `ToolCallLimitMiddleware(run_limit=200)`, plus a tighter per-tool limit on expensive tools.
7. **Non-idempotent tools.** Put `send_email` in `interrupt_on`, give it an idempotency key, and exclude it from retries.
8. **Crashes.** A durable checkpointer, so that a failed worker resumes from the last step.
9. **Observe.** Alert on fallbacks, limit hits and propagated exceptions.

ADVANTAGES

- Each failure from last month has a named owner in the stack.

LIMITATIONS

- Fallback models behave differently. Test prompts and tools on every model in the chain.
- Limits may cut off valid long tasks. Choose them from real distributions.

EDGE CASES AND TRAPS

- Layer order: errors must reach the retry layer before conversion.
- `thread_limit` needs a checkpointer.
- A fallback to a model without the needed tool-calling quality can fail more subtly than an outage.

LIKELY FOLLOW-UPS

- *How do you test it?* Inject failures with fake models or the tool emulator and assert on the outcome of each layer.

Q78. The agent has 40 tools. It picks the wrong ones, and every call is slow and expensive. What do you do?

Scenario. An internal assistant exposes 40 tools. Most are irrelevant to any one query. Tool schemas dominate the prompt, and the model sometimes calls a similar-sounding tool.

Short answer. Reduce what the model sees. Use `LLMToolSelectorMiddleware` to pick a few relevant tools per query, or `ProviderToolSearchMiddleware` to defer schemas to the provider's tool search. Also sharpen tool descriptions, and split tools between subagents.

STEP BY STEP

1. **Measure.** How many tokens do the tool schemas take? How often is the wrong tool chosen?
2. **Selector.** `LLMToolSelectorMiddleware(model="gpt-5.4-mini", max_tools=3, always_include=["search"])` makes a separate structured-output call to choose tools first. Extras are dropped; `always_include` tools do not count toward the maximum.
3. **Provider search.** `ProviderToolSearchMiddleware(searchable_tools=[...])` defers schemas behind the provider's server-side tool search. It needs Anthropic (Sonnet 4 and later, Opus 4 and later, Haiku 4.5 and later) or OpenAI (gpt-5.5 and later); others raise `ValueError`.
4. **Descriptions.** Make each tool's docstring say what it does, when to use it, and when *not* to.
5. **Specialise.** Give each subagent only its own tools. This also isolates context.
6. **Re-evaluate.** Compare accuracy, tokens and latency.

ADVANTAGES

- Smaller prompts and fewer wrong calls.

LIMITATIONS

- The selector is an extra model call, and a poor choice removes the tool the agent needed. The provider approach is provider-specific.

EDGE CASES AND TRAPS

- Tools must be registered up front. A selector filters; it does not create tools.
- Deferred tools can also be marked with `extras={"defer_loading": True}`.
- Keep a small `always_include` set for tools that are needed on nearly every request.

LIKELY FOLLOW-UPS

- *How do you know the selector is choosing well?* Log chosen tools against the tools the agent later tries to use, and look for misses.

Q79. Personal data must not reach the model or your traces. What do you do?

Scenario. Support tickets contain emails and card numbers. Compliance says the model provider must never receive card numbers and traces must not store raw payloads.

> **Short answer.** Use `PIIMiddleware` with `redact`, `mask`, `hash` or `block` strategies on input, output and tool results as needed, configure trace policies to omit payloads, and remember that redaction does not protect against a tool or shell that can exfiltrate data.

STEP BY STEP

1. **Inventory.** Where does PII enter: user input, tool results, files, memory?
2. **Choose strategies.** `PIIMiddleware("email", strategy="redact", apply_to_input=True)` and `PIIMiddleware("credit_card", strategy="mask", apply_to_input=True)`. Use `block` where the presence of the data should stop the run.
3. **Cover tool results.** Set `apply_to_tool_results=True`, because it defaults to false.
4. **Cover output.** Set `apply_to_output=True` (with `langchain>=1.3.2`, streamed output is redacted too).
5. **Custom types.** Provide a `detector` (regex, compiled regex or function returning `PIIMatch`) for identifiers the built-ins do not know.
6. **Traces.** Use `TracePolicy(process_inputs=omit_payload)` (`langchain>=1.3.15`) so hook spans do not store raw payloads, and `configure_trace_policy(...)` for a global default.
7. **Test.** Feed known sample data through the agent and search the provider logs and traces for it.

ADVANTAGES

- Declarative, reusable, and applied at one choke point.

LIMITATIONS

- Detectors miss unusual formats and give false positives. Redaction can remove information the model needed.

EDGE CASES AND TRAPS

- `ShellToolMiddleware` defaults to **host** access. Redaction rules do not stop exfiltration under it.
- Memory and files may already contain PII. Redaction on the way in does not clean what is stored.
- `hash` is deterministic, which is useful for matching but reveals equality.

LIKELY FOLLOW-UPS

- *How do you handle PII in a sandbox?* Keep it out where possible, and treat anything inside as readable by the agent.

Q80. Your middleware gives wrong counts under parallel subagents, and fails after you move to Managed Deep Agents. Why?

> **Scenario.** A custom middleware counts model calls using `self.count += 1`. In tests with parallel subagents, totals are wrong. After deployment to Managed Deep Agents, its hooks raise errors.

> **Short answer.** Mutating `self` is unsafe because threads, subagents and parallel tool calls run concurrently and share the instance. Use graph state with a reducer instead. Managed Deep Agents always uses async entry points, so write async hooks.

STEP BY STEP

1. **Diagnose the count.** One middleware instance is shared by every run, so concurrent runs race on `self.count`. It also leaks counts between threads.

2. **Move the state.** Extend `AgentState` with a `NotRequired` counter, pass `state_schema=`, and return `{"model_call_count": state.get("model_call_count", 0) + 1}` from a node hook.
3. **Scope.** State is per thread and checkpointed, so counts are correct and survive restarts. For per-run counts, reset in `before_agent`.
4. **Subagents.** Subagent `middleware` does not inherit. Add the middleware to each subagent that needs it, and remember each has its own state.
5. **Async.** Define `abefore_model` (and other `a*` hooks) alongside the sync ones. Managed Deep Agents always calls `ainvoke` and `astream`.
6. **Test.** Run the agent with parallel `task` calls and assert on counts, and run the same tests through the async path.

ADVANTAGES

- State in the graph is thread-safe, observable, checkpointed and visible to other hooks.

LIMITATIONS

- Custom state adds fields to every checkpoint. Keep it small.

EDGE CASES AND TRAPS

- A module-level counter has the same problem as `self`.
- Hooks may run in different threads or event loops. Do not hold non-thread-safe clients on the instance without protection.
- Reducers matter: fields with no reducer are overwritten by the last writer.

LIKELY FOLLOW-UPS

- *When is mutable instance state acceptable?* For immutable configuration set at construction, such as thresholds or a model.

CHAPTER 71

Interview Questions: Frontend and Protocols

At a glance: Ten questions on showing a deep agent's work to people and connecting it to other systems. Direct questions cover the frontend projections and the four protocols. Concept questions explain how MCP, A2A and AG-UI work and where they bite. Scenarios cover empty cards, a coding IDE, a third-party MCP server and agent-to-agent integration.

Direct Questions

State the fact, say what it means, then add one edge case.

Q81. What does the frontend SDK expose for a deep agent, and what must you configure for production?

Short answer. One `useStream` hook with `apiUrl` and `assistantId`. It exposes `stream.messages` (coordinator conversation), `stream.subagents` (specialist workers: status, metadata, discovery), `stream.values` (custom state such as todos), tool-call state and interrupts, plus selector helpers such as `useMessages(stream, subagent)` and `useToolCalls(stream, subagent)` for scoped content. For production, configure reconnection, raise `recursionLimit` (Deep Agents defaults to 10,000) and set `streamSubgraphs: true`.

EXPLANATION

A deep-agent UI should make delegation visible: not one opaque chat bubble, but how the task is split up and completed. The architecture is coordinator to workers, and the hook is the same one used for any LangGraph agent, because deep agents run on the same runtime.

Projection	Use
`stream.messages`	Coordinator conversation and final synthesis
`stream.subagents`	Specialist workers: status, metadata, discovery
`stream.values`	Custom state: todos, plans, report sections, sandbox metadata
Tool-call state	Cards for filesystem, search, browser and domain tools
Interrupts	Pause delegated work for approval or missing input
Selector helpers	Scope data to one subagent; subscribe only while mounted

STEP BY STEP

1. Call `useStream<typeof agent>({ apiUrl, assistantId })`. The type parameter gives type-safe state.
2. Render `stream.messages` as the main conversation.
3. For specialists, filter `stream.subagents` and use the selector helpers inside each card.
4. Read `stream.values` for shared state the agent writes.

5. Configure reconnection so that dropped connections do not lose progress.

ADVANTAGES

- One hook for messages, subagents and custom state, with type safety and framework-agnostic SDKs (React, Vue, Svelte, Angular).

LIMITATIONS

- Subagent *content* needs selectors; the snapshot does not carry it.
- Syntax differs by framework (for example Vue `stream.values.value`, Svelte `$derived`, Angular signals).

EDGE CASES AND TRAPS

- The documentation's frontend samples are per framework. Confirm imports for the SDK version you install.
- `recursionLimit` is a safety net for deep workflows. Override it only when you have a reason.

LIKELY FOLLOW-UPS

- *Where do you put approvals?* Interrupts arrive on the stream; render a dialog and resume on the same thread.

Q82. MCP, ACP, A2A and AG-UI: who talks to whom?

Short answer. MCP connects an agent to tools and data. ACP connects a coding agent to a code editor. A2A connects an agent to other agents. AG-UI connects an agent to a user-facing application.

EXPLANATION

Protocol	Connects	Transport	Typical use
MCP	Agent and tool servers	stdio, streamable HTTP, in-memory	Databases, APIs, browsers as tools
ACP	Agent and editor or IDE	stdio (the editor launches your script)	Zed, JetBrains, VS Code (community), Neovim, Toad
A2A	Agent and agent	JSON-RPC 2.0 over HTTP	Cross-framework agents
AG-UI	Agent and frontend	Typed event stream	Portable agent UIs

STEP BY STEP (CHOOSE)

1. The agent needs a database or API: **MCP**.
2. You want it inside an editor: **ACP**, by wrapping the agent in `AgentServerACP` and calling `run_agent(server)`.
3. Another team's agent (say a Google ADK agent) must call yours: **A2A**.
4. You want a standard event contract for a web UI: **AG-UI**, or the LangGraph-native `useStream` path.

ADVANTAGES

- Standards avoid per-integration glue code.

LIMITATIONS

- Each is at a different maturity level, and several are beta or partial on the platform side.

EDGE CASES AND TRAPS

- ACP is for editors only. Use MCP for external tool servers.

- In ACP, stdio belongs to the client (no interactive terminal), and Zed's `command` must be an **absolute path**.
- Agents can sit on both sides: a Managed Deep Agents deployment is an MCP *server* (its `/mcp` endpoint) and can also use MCP *connectors* as a client.

LIKELY FOLLOW-UPS

- *Can one agent use all four?* Yes: tools over MCP, an editor over ACP, peers over A2A, a UI over AG-UI.

Q83. How do A2A's identifiers, endpoints and Agent Card work on LangSmith?

Short answer. LangSmith Agent Server exposes each assistant at `POST /a2a/{assistant_id}` using JSON-RPC 2.0, with an auto-generated **Agent Card** at `/.well-known/agent-card.json`. `contextId` groups a conversation and maps to the LangGraph `thread_id`. `taskId` identifies a single request.

EXPLANATION

A2A is a contract for agents built on different frameworks to call each other without knowing each other's internals. The Agent Card is the discovery document: it says who the agent is and how to call it.

STEP BY STEP

1. Send `SendMessage` with `message.role = "ROLE_USER"`, `parts` and a `messageId`. The response contains `result.task.id` and `result.task.contextId`.
2. For follow-ups, send only the `contextId`.
3. For long tasks set `configuration.returnImmediately: true` and poll `GetTask`.
4. If the graph calls `interrupt()`, the task enters `TASK_STATE_INPUT_REQUIRED` with an artifact named `Interrupt`.
5. Resume by sending a message with the task's `taskId` and `contextId` and a `DataPart` with a single `resume` key, or plain text.

ADVANTAGES

- Cross-framework interoperability, a discoverable card, and distributed tracing: the server maps `contextId` to `thread_id`, so turns across agents land in one LangSmith thread.

LIMITATIONS

- Only the JSON-RPC binding exists. `SubscribeToTask` and push-notification configuration are not implemented (`-32601`).

EDGE CASES AND TRAPS

- **Do not resend a terminal task's `taskId`.** It is rejected with `-32004`.
- Enum case follows the method family: v1.0 names return `SCREAMING_SNAKE_CASE`, v0.3 names lowercase. Use one family per client.
- Clients cannot set `thread_id` directly, and the graph state must include a `messages` key.
- Version floors: endpoint `langgraph-api >= 0.4.21`; `returnImmediately` 0.18.0; `historyScope` 0.15.0.
- Disable it with `"http": {"disable_a2a": true}` in `langgraph.json`.

LIKELY FOLLOW-UPS

- *How do non-LangGraph agents get traced into the same thread?* They set the `langsmith.metadata.thread_id` span attribute.

Concept Questions

These test how the protocols work underneath and where they surprise people.

Q84. How does MCP work in LangChain, and what are its security caveats and authentication options?

Short answer. `langchain.mcp.MCPAdapter` (beta, `langchain[mcp]>=1.4.0`, built on FastMCP) turns an MCP server's tools into LangChain tools. The **target you pass determines the transport**: an in-process `FastMCP` instance uses memory, a `Path` uses stdio, and an `http(s)` URL uses streamable HTTP.

EXPLANATION

MCP lets you reuse existing tool servers without writing wrappers. FastMCP handles transport inference, protocol negotiation, connection management and authentication, so the same interface works for tests, local scripts and remote services.

STEP BY STEP

1. `async with MCPAdapter("https://example.com/mcp") as adapter: tools = await adapter.list_tools()`. Pass the tools to the agent. The tools keep a reference to the client, so they still work after the context exits.
2. **Multiple servers.** An `MCPConfig` (`{"mcpServers": {...}}`) gives one aggregate connection with tools **prefixed by their config key**, but the whole fleet shares **one protocol era**, so adding a legacy-only server drops everyone to legacy. A `ClientGroup` keeps each server's own era, authentication and handlers, with tools namespaced `{server}_{tool}`.
3. **Outputs.** MCP content becomes content blocks. Structured content goes into the message `artifact`, not the model-visible text. A server-reported `isError=True` becomes a `ToolMessage` with `status="error"` (the model can retry). Transport and session failures **raise**.
4. **Authentication.** Anything a `fastmcp.Client` accepts works: a bearer string, `auth="oauth"` (OAuth 2.1 with discovery, dynamic registration and a browser redirect), per-server auth through `ClientGroup`, or per-user tokens minted inside a graph factory.
5. **Human gating.** Read `destructive_hint` annotations once at discovery and build `interrupt_on` entries with a `when` predicate.
6. **Elicitation.** Server requests for user input become LangGraph interrupts (on by default). Resume with `{"responses": {key: answer}}`.

ADVANTAGES

- Reuse the ecosystem of tool servers, with one interface across transports.

LIMITATIONS

- Beta. Async only. Stdio servers run code on your machine.

EDGE CASES AND TRAPS

- **Subprocess risk.** FastMCP checks "is it a file path?" *before* "is it a URL?", so a string naming an existing script would **launch a subprocess**. `MCPAdapter` rejects non-URL strings, which matters when targets come from configuration or model output.
- Default OAuth keeps tokens in memory, so the browser step repeats every run. Persist them.
- **Isolate cached responses by verified identity**, or one user could see another's cached tool list.

- Elicitation resume **re-runs the tool from the start**. Sampling and roots requests raise errors.
- Caching helps only modern-era servers. `list_tools(cache_mode=...)` takes `use`, `refresh` or `bypass`.
- In deployments, build the agent in a graph factory and make sure type annotations are importable at runtime. An unresolved annotation makes `langgraph-api` inject a configuration dictionary instead of the runtime object.

LIKELY FOLLOW-UPS

- *Which transport for tests?* In-memory, by passing a `FastMCP` instance.
- *How do you stop destructive MCP tools running unattended?* Gate them with `interrupt_on` built from `destructive_hint`.

Q85. What happens when an A2A task needs human input, and what are the wire-format traps?

Short answer. If the graph calls `interrupt()`, the A2A task enters `TASK_STATE_INPUT_REQUIRED` and exposes an artifact named `Interrupt`. The client resumes by sending a new message that carries the existing `taskId` and `contextId` plus the answer. The wire format is still partly v0.3, which causes most integration problems.

EXPLANATION

A2A models a unit of work as a *task* with a lifecycle. Human-in-the-loop fits naturally: the task pauses, an external caller supplies input, and it continues. LangSmith maps this onto LangGraph interrupts.

STEP BY STEP

1. The remote agent runs and reaches an `interrupt()`.
2. The server returns the task with state `TASK_STATE_INPUT_REQUIRED` and an `Interrupt` artifact describing what is needed.
3. The caller shows this to a human or another system.
4. The caller sends a message with the task's `taskId` and `contextId` and a `DataPart` with a single `resume` key, or plain text.
5. The graph resumes on the same thread, because `contextId` is the `thread_id`.

ADVANTAGES

- Interrupts across agent boundaries use the same durable mechanism as local approvals.

LIMITATIONS

- Streaming events are flat with a `final` flag, and push notifications are not implemented.

EDGE CASES AND TRAPS

- Wire shape is still partly v0.3 (`kind`, `mimeType`), and timestamps use `+00:00` rather than `Z`.
- A terminal task cannot be re-used. Sending its `taskId` again is rejected.
- Resume with the **same** `contextId`. A new one starts a new thread.
- The graph state must have `messages`. Without it, the A2A layer cannot map the conversation.
- Resume re-executes the interrupted node from the top, as with any interrupt.

LIKELY FOLLOW-UPS

- *How do you poll a long task?* `returnImmediately: true`, then `GetTask` until the state is terminal.

Q86. AG-UI or `useStream`: how do you decide, and what are the AG-UI gotchas?

Short answer. `useStream` is the first-class LangGraph frontend path (messages, subagents, custom state). AG-UI is a protocol-level contract that makes the frontend portable to any AG-UI client such as CopilotKit. Choose AG-UI when you want the standard event vocabulary, and `useStream` when you want the richest LangGraph-specific features.

EXPLANATION

The question is "how much do I want my UI tied to LangGraph?". `useStream` knows about subagents, interrupts and your custom state. AG-UI knows a standard set of events, so any compliant client works, at the cost of LangGraph-specific richness.

STEP BY STEP

1. Serve the agent with `langgraph dev` (default `http://localhost:2024`) and point `langgraph.json` at it.
2. Wrap it with the TypeScript `@ag-ui/langgraph` adapter: `new LangGraphAgent({graphId: "agent", deploymentUrl: "http://localhost:2024"})`.
3. Subscribe to events: run lifecycle (`RUN_STARTED`, `RUN_FINISHED`, `RUN_ERROR`), steps, text messages, tool calls, reasoning, and state (`STATE_SNAPSHOT` plus `STATE_DELTA` as RFC 6902 JSON Patch).
4. Use `runAgent`, `subscribe` (persistent) and `abortRun`.

ADVANTAGES

- Portable across clients and frameworks, with efficient state deltas.

LIMITATIONS

- The adapter is TypeScript. Python frontends use CopilotKit or an AG-UI FastAPI bridge.
- Less LangGraph-specific detail, such as first-class subagent handles.

EDGE CASES AND TRAPS

- Set `emitInterruptOutcome: true`. Otherwise the adapter emits a legacy `on_interrupt` custom event. With it, resume by sending a `resume` array with `interruptId`, `status` and `payload`.
- `graphId` must match a key in `langgraph.json` or the graph fails to load.
- Rely on the initial snapshot plus deltas, not on full state at each step.

LIKELY FOLLOW-UPS

- *Which would you pick for an internal tool built on LangGraph?* `useStream`.
- *Which for a product with a third-party agent UI?* AG-UI.

Scenario Questions

Restate, diagnose, fix, then say what could go wrong with the fix.

Q87. The subagent cards never appear and the plan panel is empty. What do you check?

Scenario. Your UI shows the coordinator's messages, but no specialist cards, and the progress panel is blank, although the traces show subagents ran and a plan was written.

Short answer. Cards hide when the snapshot `id` does not match the spawning tool-call ID, or when you look for content on a snapshot (snapshots carry none). The plan is empty if `TodoListMiddleware` was never added, because `stream.values.todos` is then simply absent.

STEP BY STEP

1. **Check the linking key.** The snapshot `id` equals the tool-call ID that spawned it. Index subagents by that ID and render each card under the AI message whose `tool_calls` contains it.
2. **Check what you read.** `stream.subagents` gives discovery snapshots (`id`, `name`, namespace, `status`, `output`), with **no content**. Fetch content with `useMessages(stream, subagent)` and `useToolCalls(stream, subagent)`, which subscribe only while the card is mounted.
3. **Check the backend.** Is `TodoListMiddleware()` in `middleware`? Since 0.7 planning is opt-in. The samples that default to `stream.values?.todos ?? []` hide the bug.
4. **Check streaming options.** `streamSubgraphs: true` and reconnection settings.
5. **Check scoping.** A progress bar that counts across all turns will look wrong in later turns. Scope per turn.
6. **Handle states.** Show empty and error states per subagent. Pulse only one `in_progress` item. Use stable keys, not array indices.
7. **Reduce clutter.** Auto-collapse completed cards when there are five or more.

ADVANTAGES

- Mounting selectors only where needed keeps the UI cheap.

LIMITATIONS

- Three independent causes produce similar symptoms, so test each.

EDGE CASES AND TRAPS

- A `completed` subagent may still hold useful output. Fall back to `subagent.output` when no AI message exists yet.
- A failed subagent needs an error state, and reading its `output` can raise.

LIKELY FOLLOW-UPS

- *How would you test this?* A component test with a fixture stream that includes a subagent, a failure and a plan update.

Q88. Design the interface for a coding agent that edits files in a sandbox.

Scenario. Users want to watch a coding agent change a project: see the file tree, open files and diffs, and chat with the agent.

Short answer. Pair a sandbox-backed deep agent with a custom FastAPI file-browsing API registered through `http.app` in `langgraph.json`, and a three-panel UI (file tree, code or diff viewer, chat). Keep the sandbox ID in thread metadata and refresh the UI by watching tool messages.

STEP BY STEP

1. **Agent.** A deep agent with a sandbox backend, which gets `read_file`, `write_file`, `edit_file`, `delete` and `execute` automatically.
2. **API.** A FastAPI app, registered via `"http": {"app": "./api.py:app"}`, with routes such as `GET /sandbox/{thread_id}/tree?filePath=...` and `GET /sandbox/{thread_id}/file?filePath=...`. Filter out `node_modules`.
3. **Sandbox identity.** Thread-scoped by default. Store the sandbox ID in **thread metadata**, the single source of truth, and use a shared helper (`get_or_create_sandbox_for_thread`) in both the agent and the API server.
4. **Frontend.** File tree, code or diff viewer, chat. There is no terminal panel; shell-like behaviour comes from the agent's `execute` tool.
5. **Keep it in sync.** Watch stream `ToolMessage`s. On `write_file` or `edit_file`, refresh that one file. On `execute`, refresh the whole tree, because a shell command can change anything.
6. **Show changes.** Snapshot file contents before a run and compare afterwards to build a changed-files list with plus and minus counts. Open the diff first.
7. **Reconnect.** Persist the thread ID (for example in `sessionStorage`).
8. **Harden.** Keep API keys out of the sandbox (use an auth proxy), and add rate limits, error handling and privacy middleware.

ADVANTAGES

- A real "see what the agent did" experience with cheap diffs.

LIMITATIONS

- You must build and operate a custom API and manage sandbox lifecycle and cost.

EDGE CASES AND TRAPS

- Do not cache sandbox IDs in memory in the API. The agent and API may run in different processes.
- Upload starter files before the first run, and run an install command after `package.json` is written.
- Return compact one-line results for read-only tools and full output for mutating ones.

LIKELY FOLLOW-UPS

- *How do you stop it running up cost?* An idle TTL on the sandbox and cleanup on thread deletion.

Q89. A team wants to plug a third-party MCP server into the agent. How do you do it safely?

Scenario. An external vendor offers an MCP server with tools that read and modify customer records. Targets come from a configuration file edited by several teams.

Short answer. Connect by `http(s)` URL through `MCPAdapter`, which rejects non-URL strings (avoiding accidental subprocess launches), use per-server authentication through a `ClientGroup`, gate destructive tools through `destructive_hint`, isolate caches by verified identity, and persist OAuth tokens.

STEP BY STEP

1. **Pin the target.** Use an allowlist of URLs. A string that happens to name a local script could launch a subprocess under FastMCP's own rules, which is why `MCPAdapter` rejects non-URL strings.
2. **Separate servers.** Use a `ClientGroup`, so each server keeps its own protocol era, authentication and handlers. Tools are namespaced `{server}_{tool}`.
3. **Authenticate.** Prefer OAuth 2.1 or per-user tokens minted in a graph factory over a shared bearer string. Persist tokens, because the default keeps them in memory and repeats the browser step every run.
4. **Gate.** Read `destructive_hint` once at discovery and build `interrupt_on` entries with a `when` that lets `dry_run` calls through. Treat unannotated tools as risky.
5. **Handle errors.** A server-reported `isError=True` becomes an error tool message the model can see. Transport failures raise, so wrap them.
6. **Isolate caches.** Cache tool lists and responses per verified identity.
7. **Deal with elicitation.** Server requests for input become interrupts. The tool re-runs from the start on resume.
8. **Test.** Run against an in-memory `FastMCP` fake with the same tool names.

ADVANTAGES

- Reuse the vendor's tools without writing wrappers.

LIMITATIONS

- You inherit the vendor's tool quality and security. Treat tool descriptions and results as untrusted input.

EDGE CASES AND TRAPS

- A tool description can contain instructions aimed at the model (indirect prompt injection). Review descriptions at integration time.
- One legacy-only server in an `MCPConfig` drops the whole fleet to legacy. Use a `ClientGroup`.
- Sampling and roots requests raise errors.

LIKELY FOLLOW-UPS

- *How would you log tool calls?* A `wrap_tool_call` middleware, written async if you will deploy on Managed Deep Agents.

Q90. Another team's agent, built with a different framework, must call yours. What do you set up?

Scenario. A partner team runs a Google ADK agent. It needs to ask your deep agent to analyse documents and wait for a result, sometimes after a human approval.

Short answer. Use A2A. Deploy your agent on LangSmith Agent Server, share its Agent Card, have the partner call `POST /a2a/{assistant_id}` with a stable `contextId`, and use `returnImmediately` plus `GetTask` for long jobs. Handle `INPUT_REQUIRED` for approvals.

STEP BY STEP

1. **Prepare the graph.** Its state must contain a `messages` key. A2A maps `contextId` to `thread_id`, and clients cannot set `thread_id` directly.
2. **Check versions.** The endpoint needs `langgraph-api >= 0.4.21`; `returnImmediately` needs 0.18.0.
3. **Publish discovery.** The Agent Card is auto-generated at `/.well-known/agent-card.json`.
4. **Call pattern.** The partner sends `SendMessage` with `ROLE_USER`, `parts` and a `messageId`, and keeps the `contextId` for follow-ups.
5. **Long tasks.** `configuration.returnImmediately: true`, then poll `GetTask`.
6. **Approvals.** If the graph calls `interrupt()`, the task enters `TASK_STATE_INPUT_REQUIRED` with an `Interrupt` artifact. The partner resumes with the same `taskId` and `contextId`.
7. **Auth.** Protect the endpoint with your deployment's authentication, and authorise per caller.
8. **Tracing.** Because `contextId` is the `thread_id`, both sides' turns land in one thread. Non-LangGraph agents can set `langsmith.metadata.thread_id`.

ADVANTAGES

- Neither team needs to know the other's framework.

LIMITATIONS

- Only JSON-RPC is supported. No gRPC, no HTTP+JSON binding, and no push notifications.

EDGE CASES AND TRAPS

- **Do not resend a terminal task's `taskId`** (rejected with `-32004`).
- Use one enum-case family (v1.0 or v0.3) per client.
- Resume re-runs the interrupted node from the top.
- Disable the endpoint with `"http": {"disable_a2a": true}` if you do not want it.

LIKELY FOLLOW-UPS

- *Why not MCP?* MCP exposes tools. A2A exposes an agent with its own lifecycle, which suits delegating a whole job.

CHAPTER 72

Interview Questions: Deployment and Production

At a glance: Ten questions on taking an agent from a laptop to a service that many people use. Direct questions cover scoping, deployment options and how to invoke a deployed agent. Concept questions explain durable execution, the three authentication layers and secrets. Scenarios cover a multi-tenant design, a crash mid-run, choosing where to host and a pre-launch review.

Direct Questions

State the fact, say what it means, then add one edge case.

Q91. What are threads, users and assistants, and why do most production bugs come from scoping?

Short answer. A **thread** is one conversation, a **user** is a person (identity from your authentication layer), and an **assistant** is a configured agent instance. Almost every production bug is a scoping bug: "whose memory or files are these?"

EXPLANATION

Primitive	Meaning	Scope rule
Thread	One conversation	Messages and scratch files are thread-scoped by default
User	Someone using the agent	Memory and files can be private (per user) or shared. Identity comes from your authentication layer
Assistant	A configured agent instance	Memory and files can be tied to one assistant or shared across all

Deciding scope up front drives backend namespaces, sandbox lifecycle and authentication.

STEP BY STEP

1. Decide, for each kind of data, which scope it has: messages and scratch files default to thread, memory defaults to user (recommended), and shared instructions can be assistant-scoped.
2. Express the scope in code: a namespace factory for stores (`user.identity`, `assistant_id`, `thread_id`, or combinations) and the sandbox lifecycle (`thread-{id}` versus `assistant-{id}`).
3. Carry identity on each call: `thread_id` in `configurable`, and `context={"user_id": ...}` for per-run data.
4. Combine with authorization handlers so that a user cannot name another user's thread.

ADVANTAGES

- Naming scope explicitly turns vague "leaks" into testable properties.

LIMITATIONS

- Every new feature (memory, files, sandboxes) adds another place where a scope has to be chosen.

EDGE CASES AND TRAPS

- An omitted `StoreBackend` namespace shares data across users.
- Assistant-scoped sandboxes accumulate state and can leak across users.
- `thread_id` and `context` are independent. Generate a fresh `thread_id` per new conversation (for example a UUID).
- A graph factory receives a `RunnableConfig`, **not** a full `Runtime`.

LIKELY FOLLOW-UPS

- *What is the recommended default for memory?* User-scoped.

Q92. Which deployment option would you choose, and what does `langgraph.json` contain?

Short answer. Managed Deep Agents when you want a hosted harness and runtime, with the agent as a project of files. LangSmith Deployment (custom code) when you need your own application code, routes or advanced authentication. Self-host (`langgraph build` to a Docker image) when you must run everything yourself.

EXPLANATION

Option	You own	Choose when
Managed Deep Agents	Business logic as files	No infrastructure wanted; channels and schedules wanted
LangSmith Deployment	Application code, routes, authentication	Custom endpoints; MCP or A2A exposure
Self-host	Checkpointer, store, authentication, servers	Strict data or infrastructure control

LangSmith Deployment provisions threads, runs, a **store** and a **checkpointer**, and gives you authentication, webhooks, cron and observability.

```json
{
  "dependencies": ["."],
  "graphs": { "agent": "./agent.py:agent" },
  "env": ".env"
}
```

`dependencies` lists packages (`["."]` is the current directory), `graphs` maps an ID to `"./<file>:<variable>"`, and `env` is the path to a `.env` file.

STEP BY STEP

1. Ask whether you need custom routes, your own backend, store or checkpointer. If yes, not Managed Deep Agents.
2. Ask whether you want Slack, schedules and OAuth connections out of the box. If yes, Managed Deep Agents.
3. Ask where data may live. If nowhere managed, self-host.
4. Write `langgraph.json` so that the platform can find the graph.

ADVANTAGES

- Durability, scoping and authentication are largely provided by the platform, and the same code runs locally and deployed.

LIMITATIONS

- Platform features such as Agent Auth, the auth proxy and custom RBAC roles are LangSmith-specific. Self-hosting means you rebuild them.

EDGE CASES AND TRAPS

- Async subagents need an Agent Protocol server.
- Managed Deep Agents is in public beta on LangSmith Cloud, and wording differs across its pages.

LIKELY FOLLOW-UPS

- *What does a graph in* `langgraph.json` *point at?* A variable holding a compiled graph, or a factory.

Q93. How do you invoke a deployed agent, and why does production code use async?

Short answer. Every production invocation carries two independent things: `thread_id` (in `config={"configurable": {...}}`), which names the conversation and lets the checkpointer persist and resume, and `context`, which carries per-run data such as `user_id` and flags and is read by tools and middleware through `runtime.context`. LLM apps are I/O-bound, so use async tools, async middleware hooks and async graph factories.

EXPLANATION

```python
client = get_client(url="https://my-deployment", api_key="...")
thread = await client.threads.create()      # new conversation; reuse thread_id for follow-ups

async for chunk in client.runs.stream(
    thread["thread_id"], "agent",
    input={"messages": [{"role": "user", "content": "hi"}]},
    context={"user_id": "u1"},
    stream_mode="updates",
):
    print(chunk)
```

STEP BY STEP

1. Create a thread for a new conversation and keep its ID. Reuse it for follow-ups.
2. Send only the new message as input. The checkpointer supplies the history.
3. Send per-run data in `context`.
4. Stream the output, and handle `__interrupt__` events.
5. Use async end to end: native async tools avoid thread overhead, `abefore_agent` and friends cover middleware, and `async` graph factories manage resources such as sandboxes and MCP connections.

ADVANTAGES

- Better throughput per worker for I/O-bound work.

LIMITATIONS

- Mixing sync and async code can block the event loop or fail outright.

EDGE CASES AND TRAPS

- Convention is `a`-prefixed methods (`ainvoke`, `astream`, `abefore_agent`). Managed Deep Agents always calls the async ones, so synchronous-only hooks fail there.
- `thread_id` is not `context`. Using one in place of the other is a common production bug.
- Do not put secrets in `context` that a tool might echo back to the model.

LIKELY FOLLOW-UPS

- *How does a new conversation start?* A fresh `thread_id`.

Concept Questions

These test whether you understand why the platform is built this way.

Q94. Why does durable execution matter, and what are its limits?

> **Short answer.** LangGraph checkpoints every step, so a run interrupted by failure, timeout or an approval request resumes from its last state without reprocessing. That gives crash recovery, indefinite pauses, time travel and an audit point before irreversible actions. Its limit: checkpoints store **state**, not external effects.

EXPLANATION

Long agent runs are fragile. Providers time out, workers restart, humans take days to approve. Without checkpointing, every interruption would mean starting over, repeating cost and, worse, repeating side effects. With it, the run is a sequence of saved steps.

STEP BY STEP

1. A worker crashes mid-run.
2. Another worker picks up the thread and resumes from the latest checkpoint.
3. For human approval, the run simply *stops* at the interrupt. Days later `Command(resume=...)` continues it.
4. Time travel lets you rewind to an earlier checkpoint and replay.
5. LangSmith Deployments configure a persistent checkpointer for you. Self-hosting means you own it.

ADVANTAGES

- Resilience without custom recovery code, and a natural place to insert human decisions.

LIMITATIONS

- Storage grows with the conversation and with large files in `StateBackend`.

EDGE CASES AND TRAPS

- A resumed node re-executes from the top, so side effects before an `interrupt()` can repeat. Use idempotency keys.
- Large `StateBackend` files are checkpointed every step. v0.6's delta checkpoints reduce that, but **rolling back to an older version is unsupported** once threads persisted with deltas.
- Retries handle a failed *call*; checkpoints handle a failed *process*. Both are needed.
- `InMemorySaver` is not durable. A restart loses everything.

LIKELY FOLLOW-UPS

- *How do you test crash recovery?* Kill the worker mid-run in staging and confirm exactly one completion of each side effect.

Q95. Describe the three authentication layers and keep them distinct.

Short answer. (1) **End-user identity and access control**: custom authentication verifies users, and authorization handlers tag ownership and filter resources. (2) **Team RBAC**: who on *your* team can touch the deployment (Admin, Editor, Viewer, custom roles on Enterprise). (3) **End-user credentials for the agent**: Agent Auth (managed OAuth), the sandbox auth proxy and workspace secrets.

EXPLANATION

#	Concern	Mechanism
1	User identity and access control	Custom auth verifies users; authorization handlers run after auth to tag ownership (`owner: user_id`), filter resources so users see only their own, or return HTTP 403
2	Team RBAC	Workspace Admin (full), Editor (create and modify; cannot delete runs or manage members), Viewer (read-only), custom roles on Enterprise
3	End-user credentials	Agent Auth (managed OAuth 2.0: first use interrupts with a consent URL, then resumes with a stored and refreshed token), sandbox auth proxy, workspace secrets for org-wide keys

STEP BY STEP

1. A user calls your agent. Custom authentication verifies who they are.
2. An authorization handler tags the new thread with `owner: user_id`, filters list queries to that owner, or returns 403.
3. The agent needs to act as the user on a third-party service. Agent Auth interrupts with a consent URL on first use, then stores and refreshes the token.
4. Code in a sandbox needs an API. The auth proxy injects headers outside the sandbox.
5. Separately, a teammate opens the LangSmith workspace. RBAC decides what they may do.

ADVANTAGES

- Each layer has one job, so each can be reasoned about and audited separately.

LIMITATIONS

- Features such as Agent Auth and custom RBAC roles are platform-specific.

EDGE CASES AND TRAPS

- RBAC and end-user authentication are different things. Interviewers love asking you to confuse them.
- Workspace secrets are for keys the *server* uses. Never use environment variables or uploaded files for secrets that a sandbox can read.
- Authorise against the verified identity, not a field that the client could set.

LIKELY FOLLOW-UPS

- *Where does a user's OAuth token live?* With Agent Auth, stored and refreshed by the platform; the agent code does not see the token.

Q96. How do you manage secrets in production?

Short answer. Keep secrets on the server side. Use workspace secrets for keys the server uses, host-side tools or the sandbox auth proxy for anything the agent must reach, and never environment variables or uploaded files that a sandbox can read.

EXPLANATION

A sandbox protects the host but not against prompt injection or exfiltration, and anything inside it can be read by the agent. So the design rule is that secrets should be applied *outside* the place where untrusted code runs.

STEP BY STEP

1. **Server keys** (model provider, database): workspace secrets or your secret manager, read by server code.
2. **Agent-callable APIs:** write a host-side tool. The agent names it; the tool holds the credential.
3. **Calls made by code inside a sandbox:** the LangSmith auth proxy intercepts outbound requests and injects headers per destination host (for example `api.openai.com` gets `Authorization: Bearer ${OPENAI_API_KEY}`), resolved from workspace settings. Keys never appear in sandbox code, environment variables or logs.
4. **User-owned credentials:** Agent Auth for OAuth, so that the user consents and the platform stores the token.
5. **Prevent leakage:** redact with middleware and restrict network egress.

ADVANTAGES

- A steered agent cannot print a key it never held.

LIMITATIONS

- More plumbing than an environment variable, and the proxy is LangSmith-specific.

EDGE CASES AND TRAPS

- If you must inject secrets anyway: approval on all tool calls, a restricted or blocked network, narrow short-lived credentials, and traffic monitoring.
- Do not put secrets in prompts, memory files, skills or tool results.
- Logs and traces can capture secrets. Use trace policies to omit payloads.

LIKELY FOLLOW-UPS

- *How do you rotate a key?* Update the workspace secret. Nothing in code or images should embed it.

Scenario Questions

Restate, diagnose, fix, then say what could go wrong with the fix.

Q97. Design a multi-tenant SaaS in which organisations are strictly isolated.

Scenario. Your product serves many companies. Each has users, shared company knowledge and private user preferences. A leak between companies would be a contract-ending incident.

Short answer. Authenticate users, derive tenant identity from the verified token, scope every store namespace by organisation (and user where private), use per-thread sandboxes, tag threads with owners through authorization handlers, keep shared company knowledge read-only, and test isolation with two tenants.

STEP BY STEP

1. **Authenticate.** Custom auth verifies the user and attaches `org_id` from the token, not from request input.
2. **Authorise.** Authorization handlers tag each thread with the owner, filter list queries to that owner and org, and return 403 for others' threads.
3. **Scope storage.** `StoreBackend` namespaces use `(org_id, user_id)` for private memory and `(org_id, "shared")` for company knowledge. Never omit the namespace.
4. **Make shared data read-only.** Seed from application code and deny agent writes with a permission rule, because shared writable memory is a prompt-injection channel across users.
5. **Isolate execution.** Thread-scoped sandboxes (`thread-{id}`), with TTLs. Do not use assistant-scoped sandboxes across tenants.
6. **Keep secrets out.** Host-side tools and the auth proxy.
7. **Guardrails.** PII middleware, call limits, and approvals on irreversible tools.
8. **Observe.** Tag traces with `org_id`, and alert on cross-tenant access attempts.
9. **Test.** Two-tenant tests for memory, files, threads and tool access, run in CI.

ADVANTAGES

- Isolation is enforced in storage and authorization, not by asking the model to behave.

LIMITATIONS

- More configuration, and a mistake in any single layer (a missing namespace) breaks the guarantee.

EDGE CASES AND TRAPS

- `context` is not verified identity. Derive tenancy from the authenticated principal.
- Shared tools (a search index, a database) need per-tenant filters of their own.
- Caches (including MCP tool lists) must be keyed by verified identity.

LIKELY FOLLOW-UPS

- *How would you prove isolation to a customer?* Automated cross-tenant tests, access logs and a documented threat model.

Q98. A worker dies halfway through a two-hour run. What happens, and what should you have prepared?

Scenario. A deployment is restarted for a patch while several long runs are in progress. Some runs resume correctly. In one, a customer receives two emails.

Short answer. With a durable checkpointer, another worker resumes from the latest checkpoint. The duplicate email came from a node that re-executed from the top and repeated a side effect. Prepare idempotency keys, gate non-idempotent tools, and keep retries, fallbacks and checkpoints in their lanes.

STEP BY STEP

1. **Confirm the mechanism.** Checkpoints store state at each step. After a crash, the run resumes at the last completed step.
2. **Find the duplicate.** The email tool ran, then the process died before the step was checkpointed, so the step ran again. Or an `interrupt()` followed the send inside one tool.
3. **Fix idempotency.** Give `send_email` an idempotency key derived from the thread and tool call ID, and have the email system deduplicate.
4. **Gate.** Put the tool in `interrupt_on`, so the send happens only after an approval, and exclude it from `ToolRetryMiddleware`.
5. **Order effects.** Place any `interrupt()` first within a tool and do side effects after.
6. **Check infrastructure.** A persistent checkpointer, not an in-memory one, and enough worker slots.
7. **Test.** Chaos test in staging: kill workers mid-run and assert one effect per approved action.

ADVANTAGES

- Deployments no longer cancel work in progress.

LIMITATIONS

- At-least-once execution is the realistic guarantee. Exactly-once effects require your own idempotency.

EDGE CASES AND TRAPS

- Delta checkpoints (v0.6, beta) mean you cannot roll back to an older version after threads are persisted with them.
- A restart during a streaming response can drop the client's connection. Configure reconnection in the frontend.
- Sandbox state is outside the checkpoint. A resumed run may find its sandbox gone after a TTL.

LIKELY FOLLOW-UPS

- *How do retries and checkpoints differ?* Retries handle a failed call. Checkpoints handle a failed process.

Q99. A startup and a bank both ask for an agent. Where do you host each?

Scenario. The startup has three engineers and wants Slack and scheduled reports in weeks. The bank has strict data-residency rules, an existing identity platform and its own security review.

Short answer. The startup uses Managed Deep Agents. The bank self-hosts, or uses LangSmith Deployment only if its data rules allow, and rebuilds platform features (authentication, secrets, RBAC) on its own stack. The decision follows who must own the checkpointer, store, authentication and servers.

STEP BY STEP

1. **List constraints.** Residency, existing identity, compliance reviews, team size, time to launch.
2. **Startup.** Managed Deep Agents gives channels, schedules, sandboxes and OAuth connections declared in files, with no servers to run. Accept the beta status and the loss of control over the backend, store and checkpointer.
3. **Bank.** Self-host with `langgraph build` into a Docker image. Provide your own persistent checkpointer, store, authentication integration, sandbox provider and observability.
4. **Rebuild what is platform-specific.** Agent Auth, the sandbox auth proxy and custom RBAC roles are LangSmith features. Plan equivalents: an OAuth broker, an egress proxy and your own role model.

5. **Middle path.** LangSmith Deployment with custom code, if the bank's rules allow a managed control plane.
6. **Plan migration.** Keep agent logic in `create_deep_agent` form so that it can move between options.

ADVANTAGES

- Each choice matches the team's capacity to operate.

LIMITATIONS

- Self-hosting means owning upgrades, scaling and incident response. Managed means accepting vendor limits and beta risk.

EDGE CASES AND TRAPS

- Async subagents need an Agent Protocol server.
- Managed Deep Agents does not support custom backends, stores or checkpointers, and has a 200 MB build limit.
- Check pricing, quotas and regions with the vendor before committing. They change.

LIKELY FOLLOW-UPS

- *Can you start managed and leave later?* Yes, if you keep logic portable and avoid platform-only features where you can.

Q100. You are asked to review a deep agent before launch. What is on your checklist?

Scenario. A team says the agent is ready. You have a day to review it. What do you look at, and in what order?

Short answer. Scoping and isolation first, then execution safety, guardrails, durability, secrets and finally the frontend. Each item is a known failure mode with a concrete check.

STEP BY STEP

1. **Scoping.** Does every `StoreBackend` have a namespace? Is identity taken from verified authentication? Is `thread_id` fresh per conversation? Run a two-user test.
2. **Backends.** No `FilesystemBackend` or `LocalShellBackend` in deployed code. `/memories/` routed to a store, not just a folder in state.
3. **Execution.** A sandbox if code runs, with thread scope and an idle TTL. Teardown in `finally`. No secrets inside.
4. **Permissions.** Order of rules (specific denies first, catch-all deny last). Remember that `execute`, MCP and custom tools are not covered.
5. **Approvals.** `interrupt_on` for irreversible tools, with a checkpointer. Interpreter calls gated through `eval`.
6. **Guardrails.** Call limits, retries scoped to idempotent tools, fallback, PII handling, trace policy.
7. **Durability.** A persistent checkpointer, a crash test, idempotency for side effects.
8. **Prompt injection.** Shared writable memory? Untrusted documents? Is output validated?
9. **Secrets.** Workspace secrets, auth proxy, host-side tools. Nothing in prompts or traces.
10. **Frontend.** Reconnection, `recursionLimit`, `streamSubgraphs`, per-subagent error handling, and stable keys.
11. **Evals and tracing.** A test set, regression tracking and filtering by `lc_agent_name`.
12. **Versions.** Pinned dependencies, and a note of beta features used.

ADVANTAGES

- A checklist turns a vague worry into a bounded review.

LIMITATIONS

- It catches known problems. Novel failures need monitoring after launch.

EDGE CASES AND TRAPS

- The most common launch bugs are: missing namespace, missing checkpointer, in-memory saver in production, no TTL on sandboxes, and permission rules in the wrong order.
- Docs and package versions disagree in places. Test against the versions you will ship.

LIKELY FOLLOW-UPS

- *What would block the launch?* A cross-user leak, an ungated irreversible tool, or secrets reachable by the agent.

CHAPTER 73

Interview Questions: Managed Deep Agents

At a glance: Twelve questions on the hosted way to run a deep agent (public beta). Direct questions cover the project layout, the deploy pipeline, memory and sandboxes. Concept questions explain what ships where, the trust model, channels and graph factories. Scenarios cover memory that works in a DM but not a channel, a migration, an exposed webhook and a schedule that misbehaves.

Direct Questions

State the fact, say what it means, then add one edge case.

Q101. What is Managed Deep Agents, and how does `define_deep_agent` differ from `create_deep_agent`?

Short answer. Managed Deep Agents (MDA) is a LangSmith service. You write the agent as a **project of files** and the `mda` CLI uploads it. LangSmith runs it on managed infrastructure: the open-source harness plus the Agent Server runtime. `define_deep_agent` returns a **definition**; at deploy time the runtime compiles it with `create_deep_agent` and supplies the backend, store and checkpointer.

EXPLANATION

You provide business logic. MDA provides the harness (agent loop, filesystem, subagents) and the managed runtime (API, threads, runs, streaming, an MCP endpoint, sandboxes, schedules).

	`create_deep_agent`	`define_deep_agent`
Returns	A compiled agent	A **definition**
Who runs it	You	The managed runtime
Backend, store, checkpointer	You own them	Supplied by the runtime
Extras required	None	A static `name`

Each capability maps to a file or directory. Only `agent.py` is required, and it must export a variable named `agent`.

Capability	Path
Model and core configuration	`agent.py` (required)
Instructions (system prompt)	`instructions.md`
Skills	`skills/`
Tools	`tools/`
MCP connectors	`tools/mcp.py`

Capability	Path
Middleware	`middleware/`
Sandbox	`sandbox/`
Memory	`memory.py`
Identity	`identity.py`
Channels	`channels/`
Schedules	`schedules/`
Evals	`evals/`

STEP BY STEP

1. Scaffold with `uvx --from managed-deepagents mda init research-assistant`.
2. Define the agent in `agent.py` with a static `name` and model.
3. Add files for the capabilities you want. A capability is enabled by the *presence* of its file.
4. Run `mda dev` locally (with Studio), then `mda deploy`.

ADVANTAGES

- No servers to run, a readable file layout, the same harness (no new API), and a deployed MCP endpoint.

LIMITATIONS

- You give up control of the backend, store and checkpointer. It is in beta, so APIs may move. Check quotas, regions and pricing.

EDGE CASES AND TRAPS

- The agent `name` must be a static string starting with a letter, because it is the graph ID.
- `state_schema` has no Python equivalent, and custom backends, stores and checkpointers are not supported.

LIKELY FOLLOW-UPS

- *One-liner?* MDA is Deep Agents plus LangSmith Agent Server, declared as files.

Q102. What does `mda deploy` do, and what are its limits?

Short answer. It validates the project, collects secrets from `.env`, checks that a model key exists, syncs instructions and skills to Context Hub, compiles into `.mda/build`, uploads the source, triggers a remote build, polls until `DEPLOYED`, reconciles cron schedules and provisions Slack. The result is an Agent Server API plus an MCP endpoint.

EXPLANATION

STEP BY STEP

1. Validate the project and load the agent entry.
2. Resolve the API key and optional workspace.
3. Collect non-reserved `.env` values as secrets.
4. Verify the model-provider key exists. Fail *before upload* if not.

5. Sync deploy-owned context to Context Hub.
6. Compile into `.mda/build` and extract `schedules/` and `channels/`.
7. Create or find the hosted deployment by name.
8. Archive, upload and trigger a remote build.
9. Poll until the revision is `DEPLOYED` (unless `--no-wait`).
10. Reconcile managed cron jobs.
11. Provision the Slack channel.

Command	Purpose
`mda init <name>`	Scaffold a Python project
`mda build [path]`	Compile without deploying
`mda dev [path]`	Run locally with Studio
`mda deploy [path]`	Compile, sync Context Hub, upload and deploy
`mda connections ...`	Manage credentials
`mda channels init slack`	Add a Slack declaration
`mda evals init`	Set up a Harbor eval workspace
`mda logs [path]`	Tail deployed logs
`mda delete` or `mda destroy`	Delete the deployment and its resources

ADVANTAGES

- One command from project to running service.

LIMITATIONS

- A build may not exceed **200 MB**.

EDGE CASES AND TRAPS

- A project with a Slack channel **cannot use `--no-wait`**, because provisioning needs the deployed server URL.
- `--no-wait` also skips schedule reconciliation.
- `--context-strategy overwrite|keep-hub` is mandatory in non-interactive shells when a Context Hub conflict occurs.
- Reserved variables (`LANGSMITH_API_KEY`, `LANGGRAPH_HOST_API_KEY`, `LANGCHAIN_API_KEY`, `LANGSMITH_WORKSPACE_ID`) authenticate the deploy but are never uploaded as user secrets. `.env` files are excluded from the build archive.
- `LANGSMITH_ENDPOINT` selects the region. Set it before the first deploy, and check current regional availability.

LIKELY FOLLOW-UPS

- *How do you debug a failed build?* Open the dashboard URL and read the revision logs (`BUILD_FAILED` or `DEPLOY_FAILED`), or tail with `mda logs`.

Q103. How does durable memory work in MDA, and how is access decided?

Short answer. `memory.py` declares up to two layers: **agent** (`/memories/agent/` , shared by all callers) and **user** (`/memories/user/` , one authenticated person). `AGENTS.md` at a mount root is **hot** (loaded into every model call); other files are **cold** (read on demand). Each run decides per layer whether to mount it.

EXPLANATION

By default a managed agent remembers only within a thread. Durable memory persists across threads, sessions and deployments, backed by Context Hub. It needs `managed-deepagents>=0.8.0`.

```python
memory = define_memory(agent=MemoryLayer(), user=MemoryLayer())
```

STEP BY STEP (ACCESS DECISION PER RUN)

1. **Resolve the caller.** User memory stops here unless the caller is authenticated as a *person*.
2. **Evaluate the layer's policy:** your `allow(context)` function if supplied, otherwise the default.
3. **Mount and load.** Layers that pass mount at their paths and their hot memory loads.

Run source	Agent memory	User memory
Slack one-to-one DM	Allowed	Allowed
Slack channel or group DM	Allowed	Denied
Direct API run	Allowed	Denied
Studio, verified user	Allowed	Allowed (policy not called)

ADVANTAGES

- Learning that persists, with privacy defaults chosen by context.

LIMITATIONS

- The agent layer is shared and influenceable by every caller.

EDGE CASES AND TRAPS

- A policy can only **widen access within the caller the runtime authenticated**. It cannot enable user memory for a service principal such as a plain LangSmith API key, and it cannot select someone else's memory.
- Returning false removes the mount for that run. Stored memory is not deleted.
- Writes outside the mounted locations are **not durable**.
- Local (`mda dev`) and deployed stores are separate.
- Deploys never overwrite durable content already in Context Hub.

LIKELY FOLLOW-UPS

- *What goes in agent memory?* Only what all callers may read and modify. Never personal data or secrets.

Q104. How do sandboxes work in MDA?

Short answer. Declare `sandbox/__init__.py` with `define_sandbox`. Each durable thread gets its own LangSmith sandbox, cloned from a snapshot built once from an optional `setup.sh`. Defaults: `idle_ttl_seconds=600` (the sandbox **and its contents** are deleted after that much inactivity, unrecoverably) and `default_timeout=600` seconds per command.

EXPLANATION

The sandbox is enabled **only while `sandbox/` exists**. The built-in tools do the work: `ls`, `read_file`, `write_file`, `edit_file`, `delete`, `glob`, `grep` and `execute`.

STEP BY STEP

1. `mda deploy` and `mda dev` run `sandbox/setup.sh` once, under `bash -e`, and save the result as a snapshot. A non-zero exit fails the deploy.
2. New threads **clone** the snapshot rather than rerun the script. The snapshot rebuilds only when `setup.sh` changes.
3. The **sandbox proxy** runs outside the sandbox. It injects headers into matching outbound requests and controls reachable destinations, so code can call authenticated APIs without holding credentials. A rule has `name`, `match_hosts` and `headers`, and `access_control.allow_list` restricts destinations.
4. Authored tools and middleware reach the sandbox through `runtime.backend` (`managed-deepagents>=0.8.0`), bound to the thread's sandbox and resolved lazily.

ADVANTAGES

- Isolated execution per thread, fast start from a snapshot, and secrets kept outside.

LIMITATIONS

- LangSmith Sandboxes are the documented backend. Idle sandboxes are deleted with their contents.

EDGE CASES AND TRAPS

- Deploy-forwarded `.env` values are available *during* `setup.sh`, but thread sandboxes do not inherit them. Anything written to disk becomes part of every thread's image, so **never write secrets in `setup.sh`**.
- If a snapshot rebuild fails, the live deployment is not moved. The last successful snapshot keeps serving.
- Declaring a sandbox **clears `permissions`**, because a permission rule would disable the sandbox `execute` tool.
- `runtime.backend` covers only the sandbox, not Context Hub. Without a sandbox it is `None`, so guard every call, and always check `error` on upload and download results.
- `mda dev` falls back to a temporary local folder when the sandbox provider fails, so test on a development deployment.

LIKELY FOLLOW-UPS

- *What is deleted by `mda delete`?* The deployment, its managed sandboxes, `{deployment}--setup-*` snapshots and any registry.

Concept Questions

These test whether you understand how the pieces fit.

Q105. What ships in the build, what syncs to Context Hub, and what happens on conflict?

Short answer. Four kinds of file are treated differently. **Managed context** (instructions, skills) syncs to Context Hub. **Application code** (tools, middleware) ships in the build. **Managed configuration** (channels, schedules, sandbox, identity, memory, MCP declarations) enables features by existing. **Dependencies and secrets** come from `.env` and the project metadata.

EXPLANATION

Content	Source	Synced to Context Hub	Agent-writable
System prompt	`instructions.md`	Yes	No
Skills	`skills/`	Yes (every UTF-8 file)	No
Durable memory	`memory` declaration	Enables the tree only	Yes, under `/memories/`

Not synced: tools, middleware, MCP connectors, channels, schedules, sandboxes and the agent definition. They ship with the compiled deployment.

STEP BY STEP (CONFLICT HANDLING)

1. A conflict arises when `instructions.md` or `skills/` changed in the Hub since the last deploy.
2. The CLI halts before syncing.
3. In an interactive shell it asks whether to overwrite the Hub copy. The default (No) keeps the Hub version and skips that section.
4. In CI it does not prompt and the deploy errors until you pass `--context-strategy overwrite` or `keep-hub`.
5. If the repository changes *during* a sync, the deploy fails with a commit conflict. Re-run it.

ADVANTAGES

- Instructions and skills can change without a redeploy, through the Hub.

LIMITATIONS

- Hub edits last only until a later deploy overwrites them, unless you pick `keep-hub`.

TRADE-OFFS

- Keeping project files authoritative is predictable and reviewable. Editing in the Hub is fast but invisible to code review.

EDGE CASES AND TRAPS

- Deploys remove deployed skill files that no longer exist locally.
- `keep-hub` means later project edits to that section do not reach the deployment until a future deploy resolves the conflict.
- Local `mda dev` uses a mock Hub, so it never updates the hosted repository.

LIKELY FOLLOW-UPS

- *Where are tools versioned?* In the build, with your code, not in the Hub.

Q106. Explain connections, identity and the three trust sources in the runtime.

Short answer. *Connections* keep credentials in the LangSmith workspace, not the project. *Identity* decides who may call the agent. The runtime keeps **context** (what the caller asked for), **server info** (who the caller verifiably is) and **channel** (where the message came from) separate, and you authorise only against server info.

EXPLANATION

Connections. A connection links the agent to an external service. Tools resolve it at runtime with `connections.get(slug, {"type": "agent" | "user"})`.

Owner	Resolves to	Typical use
`agent`	One credential belonging to the deployment, shared by all callers	Web search, one team channel
`user`	The calling person's own credential	Acting as the requester

Identity. Declared in `identity.py`:

Provider	Who verifies	Per-user private threads
LangSmith API key (default)	LangSmith Cloud	No
Supabase	MDA, via a Supabase JWT	Yes
Your own backend	Your API, asserting the user ID with a shared secret	Yes

STEP BY STEP

1. Create a connection with the CLI: an opaque secret (agent-owned), general OAuth (bring your own app) or MCP OAuth.
2. Resolve it in code. Your code never passes a user ID; the runtime knows the caller.
3. For user-owned connections, the runtime raises one `credential_authorization_required` interrupt listing the missing slugs. The frontend opens each `connect_url`, polls the Agent Auth session, then resumes with `credential_authorization_completed`. The UI never sees the token.
4. Choose identity, then in code authorise against `runtime.server_info.principal`.

THREE TRUST SOURCES

- *Context* comes from the API call, is validated against your schema, and selects behaviour but **grants no access**.
- *Server info* comes from identity resolution of a verified token. MDA strips identity keys that clients send through `configurable`.
- *Channel* comes from managed channel ingress. A `channel` key placed in a direct API call's context does **not** populate `runtime.channel`.

ADVANTAGES

- Credentials stay out of the project, OAuth is handled by the platform, and trust sources cannot be confused.

LIMITATIONS

- User-owned connections hold OAuth grants only, and are managed from the CLI.

EDGE CASES AND TRAPS

- `connections.get` *selects* an owner. It never creates one. A credential under the same slug but a different owner does not satisfy the lookup.
- Holders of `MDA_INGRESS_SECRET` can impersonate any user. Never ship it to browsers or mobile apps.
- Workspace access bypasses per-user privacy: Studio can read every thread.
- Adding Supabase later does not assign owners to existing threads.
- Prefer a **dedicated team-owned account** over a personal one for agent-owned OAuth grants.
- HTTP statuses: 401 for a missing or wrong credential header, 403 for a caller reaching another user's threads.

LIKELY FOLLOW-UPS

- *Which identity mode for a consumer app?* Supabase, or your own backend with the ingress secret held server-side.

Q107. How do channels and agent-owned interrupts work?

Short answer. A channel connects a deployed agent to a messaging service: inbound events start runs, and replies go back to the same conversation. Built-ins are Slack and HTTP. An agent-owned interrupt lets your own tool post a custom Slack form and pause until a person answers.

EXPLANATION

A channel has three jobs: verify and normalise inbound events and start a run, send the agent's reply to the originating conversation, and provision the provider resources. Declarations live in `channels/<name>.py`, each exporting a module-level `channel`.

STEP BY STEP (SLACK)

1. Add it with `mda channels init slack` and declare `channels.slack(...)`.
2. On the first deploy the CLI prints an HTTPS authorization link. Open it, choose the workspace, approve, and press Enter.
3. A DM always starts a run. A channel message starts one only if it mentions the agent (unless `trigger_on_all_messages=True`). A reply in an active thread starts one. Another bot's message needs `allow_bot_triggers=True`.
4. **Each deployment supports one Slack channel.**

STEP BY STEP (HTTP)

1. You supply `verify` and `parse`, and optionally `post`.
2. `verify` runs first. True continues, false returns 401, raising returns 500.
3. `parse` returns a message (`user_id`, `thread_id` as a **UUID**, `target`, `content`) or `ignore`.
4. The endpoint is `POST https://<deployment-url>/channels/<name>/events`. A `202` means *accepted*, not *completed*.

STEP BY STEP (AGENT-OWNED INTERRUPT)

1. The tool posts a Block Kit message into the thread with a **correlation ID**.
2. It calls `interrupt()`.
3. The user presses a button or submits a modal.
4. The platform forwards the answer as a `user_prompt_response` event and `interrupt()` returns the response.

ADVANTAGES

- Agents live where people already work, with custom approval forms.

LIMITATIONS

- Slack human-in-the-loop supports only `approve` and `reject`. The built-in Slack channel sends text only.

EDGE CASES AND TRAPS

- `verify` **is the only barrier** between the public internet and an agent run. Verify the signature over the **raw bytes**, with a secret from the environment.
- Use `uuid.uuid5` to map external IDs to a UUID, or the request fails with 400.
- A tool that pauses re-executes **from the top** on resume. Guard the posting step so you do not post the form twice.
- A dismissed modal leaves the interrupt parked. Never set `private_metadata` on modals. Two people pressing the same button resume once, then start a fresh turn.
- Reactions never block a reply. A missing emoji is not evidence of a failed run.

LIKELY FOLLOW-UPS

- *Does file exchange need anything?* A sandbox and `managed-deepagents>=0.8.0` (200 MiB per file each way). Attachment is never automatic.

Q108. When and how do you use a graph factory, and what rules must it obey?

Short answer. Export a function named `agent` that receives a `ManagedServerRuntime` and returns a definition. It lets callers choose behaviour through `context` and is the only way to select managed resources (instructions, skills, MCP, sandbox) per run. It must have no side effects, keep interrupts stable, use an inline `name`, install every model integration it can return, and never treat context as verified identity.

EXPLANATION

A static definition fixes the agent at deploy time. A factory lets one deployment serve several variants.

Field	Omitted	Supplied	Cleared
`instructions`	Uses `instructions.md`	Uses the supplied prompt	`""` removes authored instructions
`skills`	Exposes all project skills	Exposes only the selected skills	`[]` exposes none
`mcp`	Uses the declaration	Uses only the selected definitions	`[]` exposes no MCP tools
`sandbox`	Uses the declaration	Uses the supplied sandbox	`None` disables the sandbox

STEP BY STEP (THE RULES)

1. **No side effects.** The factory runs at the start of each run, on resume and retry, and when Agent Server reads schemas. For the same context it must return the same definition.
2. **Stable interrupts.** Middleware that can interrupt must be present in *every* configuration, so resumed runs still have it.
3. **Inline `name`.** `mda build` reads the name without calling the factory.
4. **Install every model integration** the factory can return. The build does not detect them.
5. **Run context is not verified identity.** Authorise against the runtime's `server_info`.
6. For per-model-call changes *within* a run, write custom middleware instead.

ADVANTAGES

- One deployment, many behaviours, selected by the caller.

LIMITATIONS

- Memory is not selectable per run. Declare it in `memory.py` and control access with an `allow` policy.

EDGE CASES AND TRAPS

- On channel runs `context` is `None`. `execution_runtime` is `None` when the server loads the agent only to inspect its schema.
- A skill path must end in `skills/<name>` and point to a skill in the project. Generated skill names cannot duplicate project skill names.
- The factory must return the same graph structure every time.

LIKELY FOLLOW-UPS

- *Why must interrupts be stable?* A run paused under one configuration must resume under a graph that still contains the interrupting node.

Scenario Questions

Restate, diagnose, fix, then say what could go wrong with the fix.

Q109. The agent remembers users' preferences in DMs but not in Slack channels. Is that a bug?

Scenario. Users complain that the managed agent knows their preferences when they message it directly, but in a team channel it behaves as if it had never met them.

Short answer. It is the default access policy working as designed. User memory is mounted by default only for Slack one-to-one DMs, because the runtime cannot tell from outside whether a channel is private. Decide whether you really want personal memory in a shared channel before overriding.

STEP BY STEP

1. **Check the defaults.** Slack DM: agent and user memory allowed. Slack channel or group DM: agent memory only. Direct API: agent only.
2. **Understand why.** Personal memory in a public channel could leak one person's preferences to everyone present.
3. **If you still want it,** supply `allow` for the user layer. A policy can only **widen access within the caller the runtime authenticated**, so it works for Slack users resolved to a person, but not for a service principal.
4. **Consider alternatives.** Put team-wide conventions in the agent layer, and tell users to DM the agent for personal requests.
5. **Update the instructions.** Add a *Memory* section: keep compact knowledge in `AGENTS.md`, never store personal data in the agent layer, treat memory as untrusted notes, and do not claim to remember something if a write failed.
6. **Test in Studio,** where verified users receive every declared layer, then on a deployed development agent.

ADVANTAGES

- Safe-by-default behaviour prevents accidental disclosure.

LIMITATIONS

- The behaviour surprises users who assume memory is global.

EDGE CASES AND TRAPS

- Agent memory is shared and influenceable by every caller. Store nothing personal there.
- Returning false from a policy removes the mount for that run. Stored memory is not deleted.
- Locally taught facts do not appear after deployment, because the stores are separate.

LIKELY FOLLOW-UPS

- *How would you let users see what is remembered?* Expose the memory file through a tool or UI, since it is just a file under `/memories/user/`.

Q110. You are migrating a working `create_deep_agent` project to MDA. What breaks?

Scenario. The project uses a custom `StoreBackend` setup, a `state_schema`, synchronous middleware, a permission list and a `system_prompt` string. The team wants it hosted.

Short answer. Parameters describing infrastructure are removed (backend, store, checkpointer), `state_schema` has no Python equivalent, synchronous middleware must become async, permissions are cleared if a sandbox is declared, and the prompt, skills and memory move into files.

STEP BY STEP

1. **Map parameters.** `model`, `tools`, `middleware`, `subagents`, `interrupt_on`, `response_format`, `context_schema`, `cache` and `debug` are unchanged. A new required static `name`. `system_prompt` moves to `instructions.md`, `skills` to `skills/<name>/SKILL.md`, `memory` to `memory.py` with `define_memory(...)`, and `backend` to `sandbox/` with `define_sandbox(...)`. `store` and `checkpointer` are removed.
2. **Handle what does not carry over.** Custom backends (the managed sandbox is the only filesystem and shell option), custom stores and checkpointers, custom application routes (use LangSmith Deployment instead), and Python custom state schemas.
3. **Make middleware async.** MDA always calls `ainvoke` and `astream`.
4. **Reconsider permissions.** They stay, but are cleared when a sandbox is declared.
5. **Mechanics.** Add `managed-deepagents` to the existing manifest, and do not run `mda init`, which creates a new directory. Export `agent` from a root `agent.py` (re-exporting from another module fails with a misleading error about `name`). Put credentials in the project `.env`. Add `.env`, `.env.*` and `.mda/` to `.gitignore`.
6. **Move memory content.** `define_memory` mounts an empty tree. Move needed knowledge into instructions or skills before deleting old memory files.
7. **Verify.** `mda build`, then `mda dev`, then `mda deploy` to a development deployment.

ADVANTAGES

- The harness is identical, so building first on Deep Agents and moving later is a normal path.

LIMITATIONS

- Loss of control over storage and persistence, and any custom state schema design must be rethought.

EDGE CASES AND TRAPS

- `mda build` does not validate the definition, so leftover managed parameters fail only when the graph loads under `mda dev`.
- Everything beside the entry ships in the deployment. Watch the 200 MB limit.
- Shell environment variables are not read. Credentials must be in the project `.env`.

LIKELY FOLLOW-UPS

- *When would you decide not to migrate?* When you need a custom backend, store, checkpointer or routes.

Q111. An HTTP channel webhook is live. How do you make sure only the real provider can start runs?

Scenario. You exposed an HTTP channel so that an orders system can post comment events. Security asks what stops anyone on the internet from triggering the agent.

Short answer. Nothing but your `verify` callback, because the endpoint has no platform authentication. Verify an HMAC signature over the raw request bytes with a secret from the environment, return false on any mismatch, map IDs to UUIDs safely, and ignore events you do not recognise.

STEP BY STEP

1. **Signature.** Compute `hmac.new(secret, context.raw_body, sha256)` and compare with the provider's header using `hmac.compare_digest`. Use the **raw bytes**, not a re-serialised body.
2. **Secret handling.** Read it from the environment (not `connections.get`, which needs a run). Rotate it by updating `.env` and redeploying.
3. **Return values.** `True` continues, `False` returns 401, and raising returns 500. Prefer returning false for bad input.
4. **Parse defensively.** Return `{"type": "ignore"}` for events you do not handle. Required fields are `user_id` (verified string), `thread_id` (a **UUID**, via `uuid.uuid5`), `target` and `content`.
5. **Provider namespacing.** Use the same `provider` value for channels serving the same service, so a user maps to one principal.
6. **Prevent replay.** Check a timestamp in the signed payload and reject old ones.
7. **Limit blast radius.** Keep tools narrow, gate irreversible ones with approvals, and treat message content as untrusted input.
8. **Test.** Send valid, invalid, replayed and malformed requests.

ADVANTAGES

- A single, small function is the entire perimeter, so it is easy to review and test.

LIMITATIONS

- If it is wrong, the agent is open to the internet.

EDGE CASES AND TRAPS

- A `202` means accepted, not completed. Do not report success to the provider on that basis.

- `user_id` must be verified. An untrusted field here becomes a principal.
- No reply is posted when a run pauses on an interrupt or when the agent already delivered a final message. `on_error` replaces the default error reply.

LIKELY FOLLOW-UPS

- *How do you answer a provider's verification challenge?* `parse` may return a Starlette `response` (needs `starlette>=1.6.0`).

Q112. The Monday-to-Friday digest did not run after you changed the schedule. Diagnose.

Scenario. You edited `schedules/daily_digest.py` to change the time and deployed with `--no-wait`. The next morning nothing arrived, and a colleague says an old cron is still firing.

Short answer. `--no-wait` exits before the deployment reaches `DEPLOYED`, so schedules were not reconciled. Redeploy without `--no-wait`. Also check that the declaration is static, that the cron has five fields, and that exactly one of `prompt` or `input` is set.

STEP BY STEP

1. **Reconciliation.** When a deployment reaches `DEPLOYED`, `mda deploy` deletes cron jobs created by earlier declarations and creates jobs for the current ones. Removing a file and redeploying removes its cron. With `--no-wait` this step is skipped.
2. **Validate the file.** It must export a top-level `schedule`. `cron` must be a standard **five-field** expression. Exactly one of `prompt` or `input`. Literals only: no environment reads, function calls or computed values.
3. **Timezone.** Omit it for UTC, or set an IANA zone. A wrong zone makes it run at the wrong hour.
4. **First fire.** A deployed schedule does not run immediately. The first fire waits for the next slot.
5. **Delivery.** `deliver_to` posts through a Slack channel using a **channel ID** (`managed-deepagents>=0.4.0`), and the bot must have access.
6. **Threads.** The default is an ephemeral thread per run. For state that accumulates, use a persistent thread that you create yourself in Studio and copy the UUID. Persistent threads require durable memory. A bad ID gives HTTP 422.
7. **Runtime schedules.** Created by tools through `runtime.schedules` (`>=0.9.0`), are untouched by reconciliation, and are limited to 20 per owner per agent.

ADVANTAGES

- Declared schedules are code-reviewed and reconciled automatically.

LIMITATIONS

- Declarations must be static, so dynamic logic belongs in the agent.

EDGE CASES AND TRAPS

- `mda dev` creates local crons (needs `managed-deepagents>=0.9.3` to execute). `--no-schedules` stops them.
- Paused runtime schedules count toward the limit.
- Cron is UTC unless a timezone is given.

LIKELY FOLLOW-UPS

- *How would you let a user schedule their own reminder?* Write a tool that calls `runtime.schedules.create(...)` with `owner={"type": "user"}`, so it runs as that person.

CHAPTER 74

Interview Questions: OpenWiki, Build-Alongs and the Capstone

At a glance: Twelve questions on the practical end of the book. OpenWiki questions cover what it writes, how Claims keep pages honest, providers, privacy and host-driven runs. Build-along questions cover the design lessons behind the research, data-analysis, content-builder and RAG agents. The scenarios finish with the capstone review questions posed in the book, answered in full.

Direct Questions

State the fact, say what it means, then add one edge case.

Q113. What is OpenWiki, what does it create, and how do you run it?

Short answer. OpenWiki is an open-source CLI, built on Deep Agents, that writes and maintains a **Markdown wiki** about a codebase (code mode) or about your own knowledge (personal mode). Its main reader is a **coding agent**, which consults the wiki first instead of rediscovering the repository every session. Humans can read the same files.

EXPLANATION

Coding agents waste tokens re-reading a repository on every task. A durable wiki is a context layer: architecture, integrations, operations and workflows as plain Markdown, grounded in versioned evidence, and refreshed as the code changes. Plain Markdown is diffable, reviewable and works with any agent.

Mode	Command	Output	Use when
Code (default)	`openwiki`, `openwiki code`	`openwiki/` in the repository	Documentation for coding agents
Personal	`openwiki personal`	`~/.openwiki/wiki`	A local knowledge base from your sources

Bare `openwiki`, `--init` and `--update` all run in code mode.

STEP BY STEP

1. Install: `npm install -g openwiki` (Node.js **22.22.0 or later**).
2. From the repository root, run `openwiki --init`. The first interactive run asks for a provider and model, the provider key, an optional LangSmith key and optional LangSmith projects. Settings are saved in `~/.openwiki/.env`.
3. Run `openwiki visualize` to explore the result as a node graph beside a Markdown reader.
4. After code changes, run `openwiki --update`.
5. For scripts and CI use `openwiki code --update --print`, which prints the final answer and exits.

Created	Contents
`openwiki/`	Quickstart entry point and topic pages
`openwiki/.claims/`	Claims that ground factual pages
`openwiki/INSTRUCTIONS.md`	Scope and priorities (yours to edit)
`openwiki/.run.json`	Run state used to resume an interrupted run
`AGENTS.md` (repo root)	A managed block telling coding agents to consult the wiki
`CLAUDE.md`	Refreshed only if it already exists; imports `@AGENTS.md`

ADVANTAGES

- Fewer tokens per agent task, a reusable context layer, plain Markdown, CI-refreshable, and many supported providers and agents.

LIMITATIONS

- Quality depends on the model. The wiki goes stale unless you re-run it or automate updates. Personal mode depends on each connector's access setup.

EDGE CASES AND TRAPS

- Re-running `--init` **regenerates the wiki and Claims from scratch** but preserves `INSTRUCTIONS.md`. `--update` never rewrites `INSTRUCTIONS.md`. To change the brief, edit the file or ask OpenWiki in chat.
- `--init` and `--update` cannot be combined.
- OpenWiki never creates `CLAUDE.md`. It only refreshes one that exists.
- Tracing OpenWiki itself (`LANGCHAIN_TRACING_V2` and friends) is separate from the **LangSmith connector**, which enriches the wiki from *your* projects' traces.

LIKELY FOLLOW-UPS

- *Why not just keep a hand-written `AGENTS.md`?* It does not scale to architecture detail and it drifts. OpenWiki's `AGENTS.md` block simply points agents at the wiki.

Q114. What are Claims, and how does an update stay cheap and honest?

> **Short answer.** A **Claim** is a material proposition behind a factual page (behaviour, data flow, invariants, failure semantics, configuration, security boundaries) stored with the **exact repository evidence** that supports it. On `--update`, only Claims whose evidence changed get a decision: **confirm, revise, add or retract**. Unaffected Claims keep their stable IDs.

EXPLANATION

Markdown stays clean because Claim state lives in `openwiki/.claims/`, not in the page text. Each Claim cites evidence, for example a `repo://` path with a line range, and the version of that evidence that was observed. That makes a page checkable: you can see what source a statement rests on and whether the source moved.

STEP BY STEP (AN UPDATE)

1. OpenWiki finds the Claims whose evidence changed.
2. For each one, the model decides to confirm it, revise it, add a new Claim or retract it.

3. Pages are rewritten from the reconciled Claims. Unaffected Claims are untouched.
4. A page earns `verified` in its front matter only after a **complete, non-empty** Claims reconciliation and a final evidence recheck.
5. A clean `--update` skips model work and leaves content unchanged, but still refreshes `.last-update.json` so freshness checks show that the check ran.

RELATED MECHANICS

- **Parallel workers.** `OPENWIKI_PAGE_CONCURRENCY` runs 1 to 8 page workers (default 1; start at 2 to 4). Each worker owns exactly one page, and the quickstart page is written **last** so it can link to everything.
- **Retries.** On a rate-limit failure the run lowers concurrency by one. Retries default to 3, or 5 when concurrency exceeds 1.
- **Ignore rules.** `.openwikiignore` supports comments, `*` and `**` globs, directories and `!` negation. It restricts discovery and shell access.
- **Format.** Pages follow the Open Knowledge Format (OKF) v0.2: YAML front matter with a non-empty `type`.

ADVANTAGES

- Cheap updates, traceable statements, and resumable runs (completed pages are durable units).

LIMITATIONS

- Claims ground *repository* facts only. Connector-derived facts, including LangSmith-only observations, are excluded from Claims.

EDGE CASES AND TRAPS

- `.openwikiignore` is a **read boundary, not a topic guarantee**. The agent may still infer an ignored area from tests, the README, commit messages or the existing wiki. It is not a secrets control.
- Mermaid diagrams are validated after each run. Failures become plain `text` fences with a comment and may be repaired later. Installing `mermaid` and `jsdom` gives validation closer to GitHub's.
- `--init` on an existing wiki replaces the generated wiki and Claims. If setup fails before new run state is durable, the previous wiki is restored. Ephemeral CI runners start fresh after a failure.

LIKELY FOLLOW-UPS

- *How would you tell whether a page is trustworthy?* Look for `verified` in its front matter and read the Claims' evidence.

Q115. How are providers, configuration, privacy and telemetry handled in OpenWiki?

Short answer. OpenWiki supports 14 provider identifiers. Choose one with `OPENWIKI_PROVIDER` and `OPENWIKI_MODEL_ID`. Configuration lives in `~/.openwiki/.env`, and process environment variables take priority. Telemetry is anonymous and **on by default**. Personal mode stores data locally, but connectors and the model provider still receive it during ingestion.

EXPLANATION

Family	Provider IDs
Direct APIs	`openai`, `anthropic`, `gemini`, `openrouter`, `bob`

Family	Provider IDs
Hosted ecosystems	`bedrock`, `gemini-enterprise`, `copilot`, `openai-chatgpt`
Inference platforms	`baseten`, `fireworks`, `nebius`, `nvidia`
Any compatible endpoint	`openai-compatible` (needs a key, base URL and model ID)

STEP BY STEP (A SAFE SETUP)

1. Run `openwiki --init` interactively once, or set `OPENWIKI_PROVIDER`, `OPENWIKI_MODEL_ID` and the matching key in the environment.
2. Tune only if needed: `OPENWIKI_MAX_OUTPUT_TOKENS`, `OPENWIKI_REASONING_EFFORT` (invalid combinations fail before any request), `OPENWIKI_PAGE_CONCURRENCY`.
3. Decide on telemetry. Per run it records the command, outcome, a coarse error category and, on init, the brain mode, provider and connector **names**. It never collects file contents, repository names, credentials, prompts, output, paths, URLs, model IDs, durations or IP addresses. Opt out with `OPENWIKI_TELEMETRY_DISABLED=1` or `DO_NOT_TRACK=1`. Inspect it with `--telemetry-file=<path>`.
4. Keep state where you want it with `OPENWIKI_CONFIG_DIR` (for example a mounted container volume).

PERSONAL MODE PRIVACY

- State lives under `~/.openwiki` (directory mode `0o700`, `.env` mode `0o600`). Secrets are referenced by variable name in connector configs, never as raw values.
- The agent gets **no host shell** in personal mode.
- Ingestion sends data out: connectors call Gmail, Slack, X, Notion and Tavily, and the provider processes the content.

ADVANTAGES

- Works with many providers, including local servers and enterprise clouds.

LIMITATIONS

- Scheduled personal updates are **macOS-only** (LaunchAgents). `cron delete` removes only the schedule. Auth, connector config, raw data and wiki content remain.

EDGE CASES AND TRAPS

- **Blank wiki.** OpenWiki sends **non-streaming** requests internally. Some gateways accept only streaming and may reject the call or return HTTP 200 with empty content, producing a blank wiki without an error. Set `OPENWIKI_OPENAI_COMPATIBLE_STREAMING=true` (token counts then become estimates).
- OpenAI-compatible servers such as Ollama or LM Studio still need an API key set, even if the server ignores it.
- OpenRouter sends no `max_tokens` by default, so credit pre-checks can fail with a 402 on low balances. Bedrock without an explicit limit can truncate long pages at 4,096 tokens.
- Copilot in CI needs a GitHub OAuth token in `COPILOT_API_KEY`. Classic and fine-grained PATs are rejected.
- OpenWiki does not move an existing `~/.openwiki` when you set `OPENWIKI_CONFIG_DIR`. Copy it yourself.

LIKELY FOLLOW-UPS

- *Is it safe to run on a private repository?* The code is sent to your chosen model provider. Check that provider's terms, and use `.openwikiignore` as a read boundary, not a guarantee.

Concept Questions

These test whether you understand the design, not the commands.

Q116. How do host-driven runs, search tools and workspaces work?

Short answer. OpenWiki can run **inside** a coding agent (Claude Code, Codex, Copilot, OpenCode, Cursor, Bob, Kiro, Oh My Pi, Antigravity). The host researches and writes; OpenWiki manages the page-job lifecycle, validates Claims and finalizes. Installed integrations also expose **local, read-only, model-free** search tools, and `openwiki link` groups several repositories into a **workspace**.

EXPLANATION

Native runs use OpenWiki's own configured model. Host-driven runs reuse the model session you already have, so no OpenWiki provider credentials are needed.

STEP BY STEP

1. Install an integration: `openwiki integrations install claude` (user level by default, so it works in any Git repository).
2. Restart the coding agent and prompt it: *"Initialize this repository's OpenWiki from the current source and tests"* (new) or *"Update this repository's OpenWiki for changes since its last successful run"* (existing).
3. For each page the host submits sparse Claim decisions. OpenWiki validates each completion through MCP, persists the Claims and blocks completion until the final Claims state is saved.
4. Later, agents use `openwiki_search` (compact ranked results) and `openwiki_read` (complete sections). `openwiki_list_workspaces` and `openwiki_list_wikis` discover scope.
5. In a multi-repo service, `openwiki link` creates a workspace, and `openwiki workspace use <name>` selects it.

SEARCH SCOPE RESOLUTION

Situation	Scope
No workspace	The wiki itself
One workspace	That workspace
Several, one active	The active one
Several, none active	`workspace_required`; the agent must ask which

ADVANTAGES

- No second model bill or key, and one wiki shared across repositories.

LIMITATIONS

- Only **code mode** wikis are supported on this path. Connector context (including LangSmith) is not supported yet. Page authoring is **sequential**, because parallel workers apply only to native runs.

TRADE-OFFS

- Native runs are faster (parallel pages) and independent of any host. Host-driven runs reuse your session and subscription but are slower.

EDGE CASES AND TRAPS

- Workspace results carry a `wiki` identifier that must be passed to `openwiki_read` together with the page and section anchors.
- Retrieval is optional context. Agents should search for concrete architecture questions, read the relevant sections and stop once grounded, not search by reflex.
- A repository without OpenWiki documentation appears dimmed in a workspace.

LIKELY FOLLOW-UPS

- *Can a repository be in two workspaces?* Yes, and members need not share a parent directory.

Q117. Explain "retrieve, offload, delegate" in the RAG build-along, and why the search tool returns paths.

Short answer. The search tool writes retrieved chunks to the agent's **filesystem** and returns only their **paths**. The orchestrator then launches one `chunk-analyst` subagent per file, in parallel, each reading one file and returning a short summary. Big text never enters the orchestrator's context, only the summaries do.

EXPLANATION

Normal RAG pastes the top chunks into the prompt. With a deep agent that wastes the scarcest resource: the coordinator's context. The pattern combines three ideas from earlier chapters: offloading to files, context quarantine in subagents, and parallelism.

STEP BY STEP

1. **Index once.** Load 14 curated pages (589,579 characters), split with `RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)` into 782 chunks, embed and store.
2. **Search.** `search_documentation` runs `similarity_search(query, k=4)`, writes each hit to `/retrieved/<batch>/chunk_n.md` with a `# Source:` header and returns the paths.
3. **Delegate.** The orchestrator starts one `chunk-analyst` per path, up to three at a time. The analyst has `tools=[]` (built-in file tools only) and returns under 300 words: API names, steps, configuration and the source URL.
4. **Synthesize.** The orchestrator answers from the summaries with links, and verifies.

ADVANTAGES

- The orchestrator stays small, analysis is parallel, and each subagent has a narrow scope and no vector-store access.

LIMITATIONS

- Extra model calls and latency compared with stuffing four chunks into a prompt. For a simple FAQ that is overkill.

TRADE-OFFS

- Summaries lose detail. If an exact snippet matters, have the analyst quote it or let the orchestrator read the file itself.

EDGE CASES AND TRAPS

- The tool and the agent must share **the same backend instance**. If the instance given to `create_deep_agent` is a different one, `read_file` cannot see the saved chunks.
- **Indirect prompt injection.** Retrieved documents can contain instruction-like text. Prompts treat retrieved content as data and the `# Source:` header helps, but this reduces risk, not eliminates it. Validate answers against sources.

- The in-memory vector store is rebuilt at startup. Production needs a persistent store (Chroma, Pinecone, PGVector) and a refresh policy.
- Keep the embeddings model consistent between indexing and querying. The tutorial itself mentions two different ones, so choose one.

LIKELY FOLLOW-UPS

- *Where does the rubric pattern fit?* A `RubricMiddleware` grader can check the final answer against the sources and trigger a revision, within an iteration cap.

Q118. What do the research and data-analysis build-alongs teach about designing an agent?

Short answer. Bound every loop, keep the model away from control parameters, fetch full content rather than snippets, standardise citations, isolate noisy work in subagents, run code in a sandbox, keep secrets in host-side tools, and tell the agent exact paths.

EXPLANATION

Lesson	How the research agent shows it	How the data agent shows it
Separate orchestration from work	Orchestrator plans and writes; the researcher searches	The agent plans and runs code; a host tool posts to Slack
Bound the loop	2 to 3 searches (5 max), 3 parallel units, 3 rounds	Opt-in todos track a short fixed workflow
Hide control parameters	`InjectedToolArg` on `max_results` and `topic`	The Slack channel is fixed in the tool
Keep secrets out	API keys in host tools	Slack token **never enters the sandbox**
Fail softly	A fetch error returns text, not an exception	The agent recovers from a bad path

STEP BY STEP (THE RESEARCH DESIGN)

1. The orchestrator writes a todo plan and saves the request to `/research_request.md`.
2. It delegates **every** research task through `task()`. It never searches itself.
3. The default is **one** subagent. It parallelises only for explicit comparisons or clearly separate aspects.
4. Each unique URL gets one citation number. It writes `/final_report.md` and checks it against the saved request.

STEP BY STEP (THE DATA DESIGN)

1. Create a sandbox backend (`LangSmithSandbox`, or Daytona, E2B, Modal, Runloop, AgentCore) and upload the CSV.
2. The agent writes and runs a pandas and matplotlib script.
3. `slack_send_message` downloads the artifact from the sandbox and posts it with the Slack SDK.
4. In the run, a relative path failed (`/data/...` instead of `/root/data/...`), and the agent spent several steps finding the file.

ADVANTAGES

- Cost control, clean context, auditable files, and safe execution.

LIMITATIONS

- Prompts for the research workflow are long, and the tutorial's runs still depend on model judgement.

EDGE CASES AND TRAPS

- A blocked page returns an `Error fetching` string that the model might cite. Instruct the researcher to discard it.
- Very long pages can overflow the researcher's context. Truncate in the fetch helper.
- Fetched content can carry prompt injection. Treat researcher output as **data**.
- If `download_files` returns an error entry, the Slack call fails. Check `error` first. Large plots can exceed upload limits.
- `InMemorySaver` loses state on restart. Use a durable checkpointer in production.
- `LocalShellBackend` is unrestricted shell access: development only.

LIKELY FOLLOW-UPS

- *Why use a cheaper model for the researcher?* It does bounded, repetitive work, and its output is summarised before it reaches the orchestrator.

Q119. In the content-builder agent, why are behaviour, memory and subagents kept in files?

Short answer. Because the three have different lifetimes and costs. `AGENTS.md` (memory) carries standing brand voice and is always loaded. `skills/*/SKILL.md` carry workflows that load only when relevant. `subagents.yaml` defines an isolated researcher. Editing Markdown and YAML changes behaviour without touching Python.

EXPLANATION

Layer	File	Loaded	Purpose
Memory	`AGENTS.md`	Every run	Voice, standards, research rules
Skills	`skills/blog-post/SKILL.md`, `skills/social-media/SKILL.md`	On demand, by description	Step-by-step workflows and output contracts
Subagent	`subagents.yaml` (`researcher`)	When delegated	Cheap, isolated research with one `web_search` tool
Tools	Python	Always	`generate_cover`, `generate_social_image` (Gemini images)

STEP BY STEP

1. The user asks for a blog post.
2. The agent matches the `blog-post` skill by description and reads it.
3. The skill requires research via `task` with `subagent_type: "researcher"`, so the agent delegates and receives notes in `research/<slug>.md`.
4. It writes `blogs/<slug>/post.md`, then calls `generate_cover` for `hero.png`.
5. Output contracts such as "a post is incomplete without its image" make completeness checkable.

ADVANTAGES

- Non-engineers can edit behaviour, skills stay cheap until needed, and the researcher keeps raw search out of the writer's context.

LIMITATIONS

- Deep Agents does **not** load subagents from files by default. You write a small `load_subagents` helper that reads the YAML and maps tool names to functions.

EDGE CASES AND TRAPS

- **Name drift.** The tutorial has inconsistencies: the skill mentions `generate_image` though the tools are `generate_cover` and `generate_social_image`; the researcher prompt mentions `write_file` though only `web_search` is listed; one passage says artifacts go to a temp directory while the code writes under `EXAMPLE_DIR`; `SKILLS.md` appears where the file is `SKILL.md`. Align tool names between skills and code, or the agent will call tools that do not exist.
- `FilesystemBackend` can read, write and delete under `root_dir`. Run it in a dedicated directory and review output before publishing.
- Platform limits live in the skills: a LinkedIn post is capped at 1,300 characters and an X post at 280.

LIKELY FOLLOW-UPS

- *Why not put brand voice in a skill?* It must apply to every task, so it belongs in always-loaded memory.

Scenario Questions

Restate, diagnose, fix, then say what could go wrong with the fix.

Q120. Your nightly OpenWiki job writes a blank wiki through the company gateway, and on other nights opens no pull request. Diagnose both.

Scenario. You run `openwiki code --update --print` in CI against an `openai-compatible` gateway. The first run commits empty pages with no error. Weeks later the job is green but no PR has appeared for days.

Short answer. The blank wiki is the streaming mismatch: the gateway returns HTTP 200 with empty content for non-streaming calls. Set `OPENWIKI_OPENAI_COMPATIBLE_STREAMING=true`. The missing PR is expected behaviour when neither the wiki Markdown nor the Claims changed. Confirm with `.last-update.json` and the job log.

STEP BY STEP

1. **Blank wiki.** OpenWiki sends non-streaming requests. Enable streaming support with the environment variable and re-run. Token counts become estimates.
2. **Check the basics.** `OPENWIKI_PROVIDER`, `OPENWIKI_MODEL_ID`, the base URL (and Entra ID settings if used), and a key present even if the gateway ignores it.
3. **Reset.** Because the earlier run wrote empty pages, run `--init` once to regenerate from scratch. Note that `--init` preserves `INSTRUCTIONS.md`.
4. **No PR.** A clean `--update` skips model work, leaves content unchanged and still refreshes `openwiki/.last-update.json`. If neither the wiki nor `.claims/` changed, **no PR is opened**. That is the design.
5. **Verify it is working.** Look at the job log and the timestamp in `.last-update.json`. Introduce a small code change on a test branch and confirm that a PR is created.
6. **Partial failures.** OpenWiki can publish partial progress if some pages finished. An ephemeral runner that failed mid-run starts fresh next time unless you preserve the workspace.

7. **Secrets and telemetry.** Store keys as repository secrets. Set `OPENWIKI_TELEMETRY_DISABLED=1` if policy requires it.

ADVANTAGES

- Silent success on quiet nights keeps noise and cost down.

LIMITATIONS

- The documentation does not give the token cost per refresh, the cron syntax of your CI, or how a wiki PR conflicts with new commits on the default branch. Plan these yourself.

EDGE CASES AND TRAPS

- Bitbucket Pipelines needs you to schedule the custom pipeline yourself.
- A freshness check that only reads `.last-update.json` will say "fresh" on a day nothing changed, which is correct.
- Setting a strict token limit to save money can truncate long pages.

LIKELY FOLLOW-UPS

- *How do you stop a bad wiki merging?* Review the PR like code. The Claims and `verified` fields tell you which statements have evidence.

Q121. The RAG agent answers from memory, and sometimes says it cannot find the saved files. What is wrong?

Scenario. Your documentation agent is meant to answer only from retrieved chunks. Testers see two problems: answers that cite nothing in the docs, and `read_file` errors for paths the search tool just returned.

Short answer. The shared-backend rule is probably broken, so the files exist in a different store than the one `read_file` uses. Separately, tighten the prompt and verification so the agent answers only from retrieved content, and treat retrieved text as data.

STEP BY STEP

1. **Check the backend.** The tool writes with `backend.upload_files(...)`. The same instance must be passed to `create_deep_agent(backend=backend)`. A second instance is a different filesystem.
2. **Check the paths.** The tool returns `/retrieved/<batch>/chunk_n.md`. Confirm the analyst is given exactly that string.
3. **Check the subagent.** `chunk-analyst` has `tools=[]` and relies on the built-in file tools. It inherits the backend, so the fix in step 1 applies to it too.
4. **Answers from memory.** The prompt must say to answer from documentation rather than memory, to cite the source URL from each summary, and to say "not found" when search returns nothing relevant. Add a `RubricMiddleware` criterion such as "every claim cites a source URL from a chunk summary" (requires `deepagents>=0.6.5`, beta).
5. **Planning.** Add `TodoListMiddleware` if the agent skips searching. Planning is opt-in.
6. **Injection.** Retrieved text may contain instructions. Keep the "treat as data" rule, keep the `# Source:` header, and do not allow tools with side effects in this agent.
7. **Retrieval quality.** If the right chunk is never returned, change `k`, chunk size or the embeddings model, not the agent.

ADVANTAGES

- Each failure has a testable cause: wiring, prompt, or retrieval.

LIMITATIONS

- A rubric costs extra model calls and a bad rubric loops until the cap.

EDGE CASES AND TRAPS

- An in-memory store and `StateBackend` are rebuilt or lost on restart, so a deployed agent may find nothing on the first query.
- `k=4` chunks of 1,000 characters may not hold a whole procedure. Summaries then miss steps.
- Do not log retrieved chunks that contain private data.

LIKELY FOLLOW-UPS

- *How do you measure it?* Build a small set of questions with known answers and check both the answer and the cited URL.

Q122. You are taking the data-analysis agent to production. What changes?

Scenario. The tutorial agent works on a laptop with an `InMemorySaver`, a sandbox created by hand, and a Slack token in an environment variable. A customer wants it to analyse uploaded files for their team.

Short answer. Isolate execution per thread, scope uploads and credentials per user, make state durable, bound cost, and handle failures at each boundary: upload, execution, download and delivery.

STEP BY STEP

1. **Execution.** Use a sandbox, not `LocalShellBackend`. Create a sandbox per thread or per customer and delete it when finished, or set an idle TTL.
2. **Secrets.** Keep the Slack token (and any database credentials) in host-side tools. The model sees only the tool.
3. **Paths.** Tell the agent absolute paths in the system prompt (`/root/data/...`) so it does not waste steps.
4. **Input.** Upload files with `upload_files` and validate size and type first. Check each result's `error`.
5. **Output.** Retrieve artifacts with `download_files` and check for error entries before posting. Resize or compress large plots to fit upload limits.
6. **State.** Replace `InMemorySaver` with a durable checkpointer and pass a stable `thread_id`.
7. **Cost and loops.** Add `ModelCallLimitMiddleware` and `ToolCallLimitMiddleware`, and set a sandbox lifetime.
8. **Tenancy.** The Slack channel should come from per-request context, not a constant, and files must not cross customers.
9. **Review.** Add `interrupt_on` for the Slack tool if posting externally is sensitive.
10. **Verify model names.** Tutorial model identifiers may differ from your provider's current list.

ADVANTAGES

- The structure (sandbox plus host tool) already separates untrusted code from credentials.

LIMITATIONS

- Sandboxes cost money while they run and need lifecycle management.

EDGE CASES AND TRAPS

- A leaked sandbox keeps billing. Always clean up in a `finally` block or use TTLs.
- Generated code can read anything in its sandbox, including other users' files if you share one sandbox.

- A script that loads the full dataset may exceed the sandbox's memory. Give guidance on sampling.
- Model-written analysis can be wrong. Tell users that numbers come from code that is in the file list, and keep the script.

LIKELY FOLLOW-UPS

- *Where would a code interpreter fit instead?* For small, fast calculations that need no filesystem or packages, the interpreter is cheaper than a sandbox.

Q123. Capstone review, part 1: memory, permissions and subagents. What is wrong with the design?

Scenario. Three review questions from the capstone. The `StoreBackend` namespace is `lambda rt: (rt.server_info.user.identity, "memories")`, a deny rule protects `/skills/**` and `/memories/policy/**` for writes, and `researcher` and `fact-checker` are subagents.

Short answer. The namespace must include the user, otherwise every user shares one memory. The deny rule covers only built-in file writes, so tools such as `execute`, MCP tools and custom tools can still change behaviour. A subagent that sets `permissions` **replaces** the parent's list, so the deny rule disappears unless you repeat it.

STEP BY STEP

1. **Why the namespace includes the identity.** A namespace is the partition key in the store. Without the user in it, all users read and write the same `/memories/` partition: one person's tone preferences, or something sensitive, appear for another. A factory such as `lambda rt: (rt.server_info.user.identity, "memories")` partitions per user. If the identity is missing (an unauthenticated run), fail closed rather than falling back to a shared namespace.
2. **What still changes behaviour.** The deny rule is evaluated for the built-in filesystem tools (first match wins, no match allows). It does not cover `execute` (a sandbox shell could edit files directly), MCP tools, or your custom tools that write to storage. It also does nothing for `/memories/AGENTS.md`, which is writable and loaded into every prompt: a poisoned PR description could persuade the agent to store an instruction there.
3. **Subagent permissions.** `permissions` set on a subagent **replace** the parent's rules. Define the base rules once and use `BASE_RULES + extra` for each subagent. `tools` replace too, while `interrupt_on` and `model` inherit, so `publish_release_notes` approval still applies to a subagent that has the tool.
4. **Fix list.** Add the user to the namespace and fail closed. Prefer read-only tools for subagents. Keep `execute` out of this agent, or put it behind `interrupt_on`. Make the policy files read-only at the backend or storage level, not just by a rule. Treat memory content as untrusted notes.

ADVANTAGES

- Each fix is small and checkable.

LIMITATIONS

- Permissions are a convenience guard for file tools, not a sandbox.

EDGE CASES AND TRAPS

- Denying `write` does not block `read`. If policy files are sensitive, deny reads too.
- Path rules apply to the path the agent uses. Test with traversal attempts such as `..`.
- In a CompositeBackend, the route prefix decides which store a path reaches. Check which backend handles `/memories/policy/`.

LIKELY FOLLOW-UPS

- *How do you test it?* Run two users in parallel and assert that user B never sees user A's memory. Run an injected prompt and assert that the deny rule holds.

Q124. Capstone review, part 2: reliability order, rubric exhaustion and moving to Managed Deep Agents.

Scenario. The capstone stack is `TodoList`, `Rubric`, `ModelRetry`, `ModelFallback`, `ModelCallLimit`, `ToolCallLimit`, with `interrupt_on` for publishing. Three questions remain: how retry, fallback and the limits interact, what to do when the rubric cap is reached, and what changes in Managed Deep Agents.

Short answer. The documented stack is retry, then fallback, then call limits; when the last model in the chain fails the exception propagates, so verify the nesting with a fake model. After `max_iterations_reached` treat the draft as failed. On MDA the backend, store and checkpointer disappear and memory, skills and instructions become files.

STEP BY STEP

1. **Order.** The documentation recommends the stack retry, fallback, then call limits. Retry handles transient failures of one model. Fallback switches model when the provider is down. The limits are separate guards on the number of loop calls. Because wrap-style hooks nest with the first middleware outermost, the list order decides which layer wraps which, so **test it**: use a fake model that fails N times and count real calls.
2. **When the fallback also fails.** No layer is left to recover, so the exception propagates and the run fails visibly. That is correct: alert on it. `ModelCallLimitMiddleware` and `ToolCallLimitMiddleware` do not turn it into a success.
3. **Limits.** Choose `run_limit` from real runs (here 60 model calls and 200 tool calls), and use `thread_limit` only with a checkpointer.
4. **Rubric exhaustion.** After three iterations the last evaluation still reads `needs_revision`. Check `result.get("_rubric_status")` for `max_iterations_reached`. The application should not publish: show the draft as "needs human edit", log the unmet criteria, or retry with stronger instructions. The interrupt for `publish_release_notes` stays as a final human gate.
5. **Rubric quality.** Keep criteria checkable from the transcript (a version header, a "Breaking changes" section or "None", a link per claim).
6. **Resume.** The publish approval needs a checkpointer and the same `thread_id`. Send one decision per action request in the same order.
7. **Moving to MDA.** `system_prompt` becomes `instructions.md`; `skills` become `skills/<name>/SKILL.md`; memory becomes `memory.py` with `define_memory(...)`; the filesystem and shell become `sandbox/` if needed; the `backend`, `store` and `checkpointer` are supplied by the runtime; the agent gets a static `name`; middleware must be async. Permissions are cleared if you declare a sandbox.

ADVANTAGES

- The same harness runs in both places, so the capstone moves with small changes.

LIMITATIONS

- On MDA you cannot keep a custom store layout (`CompositeBackend` with a per-user `StoreBackend`). Per-user memory becomes the user memory layer, with its access policy.

EDGE CASES AND TRAPS

- A fallback model may call tools differently. Test the whole chain.

- Retrying `publish_release_notes` can publish twice. The tool is gated by `interrupt_on`, and it should also be idempotent.
- `InMemorySaver` and `InMemoryStore` lose everything on restart. Use durable ones in production.

LIKELY FOLLOW-UPS

- *What do you alert on?* Fallback activations, limit hits, `max_iterations_reached` and propagated exceptions.

CHAPTER 75

System-Design Walkthroughs

At a glance: Whiteboard questions test whether you can assemble the pieces into a coherent system and defend the trade-offs. Each walkthrough below follows the same shape: **requirements**, a **design in numbered steps**, **edge cases**, and **what to say about trade-offs**. Practise them aloud with a timer: ten minutes each.

D1. A research assistant that remembers preferences and cites sources

Requirements. Multi-source web research; a report with numbered citations; remembers each user's preferences across conversations; must not send anything externally without approval; must not leak one user's preferences to another.

DESIGN

1. **Coordinator plus research subagent.** The coordinator plans (add `TodoListMiddleware`), saves the request to `/research_request.md`, delegates to a `research-agent` subagent, consolidates citations and writes `/final_report.md`.
2. **Full-page retrieval tool.** The search tool finds URLs, then *fetches each page and converts it to markdown*, so the subagent reads real sources rather than snippets.
3. **Memory.** A `CompositeBackend` with `default=StateBackend()` and `routes={"/memories/": StoreBackend(namespace=lambda rt: (rt.server_info.user.identity,))}`; `memory=["/memories/AGENTS.md"]`; pass `store=`.
4. **Safety.** `interrupt_on={"send_email": True}` with a checkpointer; `PIIMiddleware("email", strategy="redact", apply_to_input=True)`; `ModelCallLimitMiddleware` and `ToolCallLimitMiddleware` to bound cost.
5. **Citation rule.** One citation number per unique URL, assigned by the coordinator after consolidating the subagent's results.
6. **Observability.** LangSmith tracing; filter by `lc_agent_name`.

EDGE CASES

- Bias toward *one* comprehensive research subagent rather than many tiny ones; cap the number of tool calls, rounds and parallel units.
- Pages that block scrapers return error strings that a model may cite as sources. Instruct the researcher to discard them.
- Retrieved web text can contain prompt-injection; treat it as data and never as instructions.
- Preferences written by the agent must not be shared across users; verify with two test users.

Trade-offs to state. More subagents means more parallelism but more cost and more citations to reconcile. Per-user memory is safe but cannot learn across users; assistant-scoped learning needs read-only controls and review.

D2. A data-analysis agent that posts charts to Slack, with no token leakage

Requirements. Analyse uploaded CSVs by running code; post a formatted report and a chart to Slack; the Slack token must never be reachable by the agent.

DESIGN

1. **Sandbox backend** (for example LangSmith sandboxes) so the agent can write and run a pandas/matplotlib script with `execute`.
2. **Seed data on the application plane.** `backend.upload_files([("/root/data/sales_data.csv", csv_bytes)])` before the run.
3. **Slack as a host-side tool.** `slack_send_message(text, file_path=None)` runs on your server, downloads the chart with `backend.download_files([file_path])`, and uploads it to Slack with the bot token. The agent calls the tool by name and never sees the token.
4. **Persistence.** A checkpointer so the thread survives; optional `TodoListMiddleware` to show progress.
5. **Instructions.** Tell the agent to use **absolute paths** so it does not waste turns on `pwd` and `ls`.
6. **Teardown.** `finally: delete_sandbox(...)` and an idle TTL.

EDGE CASES

- If `download_files` returns an error entry instead of content, the Slack call fails; check `error` first.
- Large plots can exceed Slack upload limits.
- `InMemorySaver` loses state on restart; use a durable checkpointer in production.
- A sandbox protects the host but not against prompt injection or network exfiltration, so restrict egress and keep every credential outside it.

Trade-offs to state. Agent-outside-sandbox keeps keys on your server and survives sandbox death, at the cost of per-call latency. Agent-inside-sandbox mirrors local development but puts keys inside.

D3. A coding agent that must never delete without approval

Requirements. The agent edits a repository and runs tests; destructive actions need a human; test failures should trigger self-correction.

DESIGN

1. **Sandbox backend** scoped to the **thread**, cloned from a snapshot that contains the repository (in MDA, via `setup.sh`).
2. **Guard the built-in tools.** Permissions treat `write_file`, `edit_file` and `delete` as one `write` operation, so they cannot single out deletion. Use them for what they do well: deny writes to protected paths such as `/workspace/.env` first, then allow `/workspace/**`, then a catch-all deny. For deletion itself, gate the `delete` tool by name with `interrupt_on={"delete": True}` (human-in-the-loop matches tools by name) and confirm in a test that your version intercepts it. An anchored `mode="interrupt"` rule (for example on a `/secrets/**` prefix) is the right tool for paths that need approval for every write.
3. **Guard what permissions do not cover.** `execute` can run `rm -rf`, so add `interrupt_on` for `execute` with a `when` predicate that matches destructive patterns, or interrupt on every `execute` for the first version.
4. **Checkpointer and stable** `thread_id` so approvals can wait hours.
5. **Quality loop.** `RubricMiddleware` with a grader that has a test-runner tool and criteria such as "all tests pass" and "no files outside `/workspace` changed".
6. **Frontend.** A sandbox IDE: refresh the file tree after `execute`, show diffs, and surface the interrupt as an approval card.

EDGE CASES

- Directory `delete` is all-or-nothing: one denied descendant denies the whole operation.
- If the default backend is a sandbox, permission paths must sit under route prefixes or `NotImplementedError` is raised.

- `respond` is not a denial; use `reject` with a message.
- A dynamic-subagent `eval` bypasses `interrupt_on`; do not enable the interpreter with a broad allowlist here, or gate `eval`.

Trade-offs to state. Approving every `execute` is safe and slow; predicate-based approval is faster and only as good as the predicate. Defence in depth: permissions, approvals, thread-scoped sandbox, network restriction.

D4. Review 200 documents in parallel with high precision

Requirements. Find issues in each document, minimize false positives, produce a single consolidated report.

DESIGN

1. **Dynamic subagents.** `CodeInterpreterMiddleware(ptc=["glob"], subagents=True)`; subagents `reviewer`, `verifier` and `synthesizer` configured on the agent.
2. **Fan out.** `Promise.all(files.map(f => task({description: ..., subagentType: "reviewer", responseSchema})))`, with a typed schema so no JSON parsing is needed.
3. **Adversarial verification.** For each finding, dispatch a separate verifier with a skeptical prompt and a verdict schema. Keep only confirmed findings (a majority of three verifiers is the docs' illustration).
4. **Synthesize.** One final `task` merges confirmed findings.
5. **Control.** Gate the `eval` tool with approval; cap batch size in the code; add call limits.

EDGE CASES

- The docs give **no concurrency caps or quotas**; chunk the file list (say 20 at a time) to avoid rate-limit storms.
- `task()` skips per-call HITL, so approval must happen on `eval`.
- Interpreter limits: 5 seconds per `eval` call, 4,000 characters returned, 256 PTC calls. Long batches must be chunked across several `eval` calls with state kept in interpreter variables (`mode="thread"`).
- Verifiers can be wrong too; spot-check a sample.

Trade-offs to state. Verification roughly doubles or triples cost to cut false positives. A tournament is better for ranking subjective options; fan-out plus verify is better for finding defects.

D5. An hours-long job that the user can steer

Requirements. A deep investigation that runs for hours; the user should be able to check progress, redirect it and cancel it, while the supervisor stays responsive.

DESIGN

1. **Async subagents**, co-deployed (ASGI) in one `langgraph.json`, one per specialist (researcher, coder).
2. **Supervisor prompt**: after `start_async_task`, return control to the user; never poll immediately; always call `check` or `list` before reporting status.
3. **Durable persistence**: Agent Server's checkpointer; each task has its own thread (the task ID).
4. **Capacity**: size workers so every active run has a slot (supervisor plus subagents).
5. **Notifications** (`>=0.7.26`): handle `deepagents_notification` messages, including the `waiting` status.
6. **Steering**: `update_async_task` restarts the subagent on the same thread with full history plus new instructions.

EDGE CASES

- Worker-slot starvation queues launches.
- Models truncate task IDs; stale statuses in chat history mislead; always re-query.
- A waiting task must be answered through the subagent's own thread by the user, not by the supervisor.
- `parent_thread_id` metadata is editable by anyone with thread access.
- Summarization will not forget tasks because `async_tasks` is a separate channel.

Trade-offs to state. ASGI co-deployment is simple and fast; HTTP lets you scale and own subagents independently but adds auth and network concerns.

D6. A multi-tenant SaaS with strict organization isolation

Requirements. Many organizations and users; no data may cross boundaries; the agent acts on users' behalf with their own credentials.

DESIGN

1. **Authenticate** with custom auth; **authorize** with handlers that tag resources with `owner` and filter every query; return 403 on violations.
2. **Namespace everything**: stores keyed by `(org_id, user.identity)` through a factory using `rt.context` and `rt.server_info`.
3. **Sandboxes per thread or per user**; avoid assistant-scoped sandboxes unless nothing sensitive accumulates; set TTLs.
4. **Credentials**: Agent Auth for per-user OAuth, the sandbox auth proxy for outbound API keys, workspace secrets for server keys. No secrets in the sandbox.
5. **Shared policy** read-only, written from application code; `permissions` deny writes to shared memory.
6. **RBAC** separately for your own team.

EDGE CASES

- Namespaces isolate storage, not identity; a spoofable `context.user_id` is not authorization.
- Missing namespace shares data. Test with two users in two orgs.
- Shared writable memory is a prompt-injection channel across tenants.

D7. Surviving a provider outage mid-run

Requirements. The primary model provider degrades during long runs; users should rarely notice; costs must stay bounded.

DESIGN

1. `InMemoryRateLimiter` on the model to stay under provider limits.
2. `ModelRetryMiddleware(max_retries=3, backoff_factor=2.0, initial_delay=1.0)` for transient failures.
3. `ModelFallbackMiddleware("gpt-5.5")` for a full outage (a different provider, ideally).
4. `ModelCallLimitMiddleware(run_limit=...)` and `ToolCallLimitMiddleware(...)` to bound spend.
5. A durable checkpointer, so if a worker dies the run resumes from the latest checkpoint.
6. Alerting on fallback usage (log in a wrap hook).

EDGE CASES

- Retries burn calls on non-transient errors; respect `is_retryable`.
- A fallback model may have different tool-calling behaviour or context size; test the combination, and note that summarization thresholds come from the model profile.
- Non-idempotent tools should not be retried blindly.

D8. A customer-support agent on Managed Deep Agents

Requirements. Answers order questions in Slack; looks up each requester's own account data; remembers team conventions; posts a morning digest; escalates refunds to a human.

DESIGN

1. **Project files.** `agent.py` (`define_deep_agent`), `instructions.md`, `skills/refunds/SKILL.md`, `tools/orders.py`, `channels/slack.py`, `schedules/digest.py`, `memory.py`, `identity.py`.
2. **Slack channel.** `channels.slack(name=..., description=...)`; default triggers (mentions and DMs).
3. **Per-user access.** A user-owned OAuth connection to the order system, resolved with `connections.get(slug, {"type": "user"})`. The first request raises the authorization interrupt; Slack handles it.
4. **Memory.** `define_memory(agent=MemoryLayer(), user=MemoryLayer())`; Slack DMs mount both layers, channels mount only the agent layer. The instructions forbid storing personal data in the agent layer.
5. **Refund approval.** An agent-owned interrupt: the tool posts a Block Kit approve or reject card, calls `interrupt()` with a correlation ID, and guards the posting step against re-execution on resume. (Platform-owned `interrupt_on` in Slack supports approve and reject only.)
6. **Schedule.** `define_schedule(cron="0 8 * * 1-5", timezone=..., prompt=..., deliver_to=<channel id>)`; deploy without `--no-wait`.
7. **Deploy.** `mda dev` for Studio tests, then `mda deploy`; set `--context-strategy` in CI.

EDGE CASES

- On channel runs `context` is `None`; authorize against `runtime.server_info.principal`.
- Agent memory is shared by everyone; nothing personal goes there.
- Hooks must be async.
- A dismissed modal leaves the interrupt parked, so write instructions for the abandoned case.
- Declaring a sandbox clears `permissions`.

D9. Durable repository knowledge for coding agents

Requirements. Several coding agents (Claude Code, Codex, Cursor) work in the same repository; they waste time and tokens rediscovering architecture; docs go stale.

DESIGN

1. Install OpenWiki, run `openwiki --init`, and review the generated pages.
2. Edit `openwiki/INSTRUCTIONS.md` for scope and terminology; add `.openwikiignore` for sensitive or generated paths.
3. Confirm the managed block in `AGENTS.md` (and `CLAUDE.md` if present) so every agent discovers the wiki.
4. Optionally install an integration (`openwiki integrations install claude`) so the host agent can run updates and use `openwiki_search` and `openwiki_read`.

5. Add a scheduled CI job running `openwiki code --update --print`; it opens a PR only when the wiki or Claims changed.
6. Link related repositories into a workspace for cross-repo search.

EDGE CASES

- The CI cadence and token cost are undocumented; measure them and choose a schedule.
- Ignored paths can still be inferred; do not rely on `.openwikiignore` for secrecy.
- Gateways that reject non-streaming requests produce a blank wiki; enable streaming.
- Telemetry is on by default; disable it in CI if policy requires.

D10. A UI that shows parallel subagents, a plan and approvals

Requirements. Users see the plan, watch several specialists work, and approve risky actions.

DESIGN

1. Backend: `TodoListMiddleware`, subagents, `interrupt_on` for risky tools, checkpointer.
2. Frontend: `useStream` with `apiUrl` and `assistantId`; `stream.messages` for the coordinator; `stream.subagents` for cards; `stream.values.todos` for the plan.
3. Link each card to its spawning message by tool-call ID; mount `useMessages` and `useToolCalls` only while a card is open.
4. Render interrupts as approval cards that submit the decisions in order and use the same thread.
5. Configure reconnection, raise `recursionLimit`, enable `streamSubgraphs`.

EDGE CASES

- Handle failure per subagent; reading `output` of a failed subagent can raise.
- Progress counted across all turns misleads; scope per turn.
- Persist the thread ID so a reload resumes.

CHAPTER 76

Spot the Bug

At a glance: Each snippet below contains one real-world mistake. Cover the answer, find the bug, then check. These are the same traps that appear in the earlier chapters, in disguise.

B1. The approval that never pauses

```python
agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
    tools=[send_email],
    interrupt_on={"send_email": True},
)
```

Bug. No `checkpointer`. Interrupts and resume need persistence; without it the pause cannot be saved or resumed.

Fix. Pass `checkpointer=MemorySaver()` in development (a durable one in production) and invoke with a `thread_id`.

B2. The resume that starts over

```python
def resume(decisions):
    config = {"configurable": {"thread_id": str(uuid.uuid4())}}
    return agent.invoke(Command(resume={"decisions": decisions}), config=config)
```

Bug. A new `thread_id` is generated on resume, so there is no checkpoint to resume.

Fix. Reuse the original thread's `thread_id`.

B3. The `.env` that stayed readable

```python
permissions = [
    FilesystemPermission(operations=["read", "write"], paths=["/workspace/**"], mode="allow"),
    FilesystemPermission(operations=["read", "write"], paths=["/workspace/.env"], mode="deny"),
]
```

Bug. The broad allow matches first, so the specific deny is unreachable. There is also no catch-all deny, so paths outside `/workspace` are allowed by default.

Fix. Put the specific deny first, then the allow, then a final `deny` on `/**`.

B4. Everyone shares one memory

```python
backend = CompositeBackend(
    default=StateBackend(),
    routes={"/memories/": StoreBackend()},
)
```

Bug. No namespace factory, so data is not partitioned per user (and from 0.7 an explicit namespace is required).

Fix. `StoreBackend(namespace=lambda rt: (rt.server_info.user.identity,))`, and pass `store=` to `create_deep_agent`.

B5. The skills that never load

```python
agent = create_deep_agent(model=..., skills=["/skills/blog-post/"], backend=backend)
```

Bug. The entry points at a *skill* directory. It must point at the directory that *contains* skill directories.

Fix. `skills=["/skills/"]`. No error is raised when it silently loads nothing.

B6. The subagent that lost its tools

```python
research = {"name": "research-agent", "description": "...", "tools": [fetch_url]}
agent = create_deep_agent(model=..., tools=[search, calculator], subagents=[research])
```

Bug (if the intent was "search plus fetch"). A subagent's `tools` **replaces** the parent's. It now has only `fetch_url`.

Fix. List every tool the subagent should have: `[search, calculator, fetch_url]`. The same replace rule applies to `permissions`.

B7. One decision for two requests

```python
payload = result["__interrupt__"][0].value        # two action_requests
decisions = [{"type": "approve"}]
agent.invoke(Command(resume={"decisions": decisions}), config=config)
```

Bug. The decision count does not match `action_requests`. Decisions must be one per request in the same order.

Fix. Iterate `payload["action_requests"]` and append a decision for each.

B8. "Respond" used as denial

```python
decisions = [{"type": "respond", "message": "No, do not delete that file."}]
```

Bug. `respond` turns the reply into a synthetic *tool result* and nothing executes; the model may read it as a successful delete.

Fix. Use `{"type": "reject", "message": "..."}` for denials. Reserve `respond` for tools where the human is effectively the tool.

B9. The counter that races

```python
class CallCounter(AgentMiddleware):
    def __init__(self):
        self.calls = 0
    def before_model(self, state, runtime):
        self.calls += 1
```

Bug. Mutating `self` is unsafe: subagents, parallel tools and threads share the instance concurrently, and the count also does not survive a checkpoint.

Fix. Keep the counter in graph state with a custom `state_schema` and return `{"model_call_count": state.get("model_call_count", 0) + 1}`.

B10. The early exit that is ignored

```python
@before_model
def stop_if_long(state, runtime):
    if len(state["messages"]) > 50:
        return {"messages": [AIMessage("Conversation limit reached.")], "jump_to": "end"}
```

Bug. The hook returns `jump_to` without declaring `can_jump_to`.

Fix. `@before_model(can_jump_to=["end"])`. On class methods use `@hook_config(can_jump_to=["end"])`.

B11. The skills reset that disabled skills

```python
agent.update_state(config, {"skills_metadata": []})
```

Bug. An empty list means "no skills" for the rest of the thread.

Fix. Reset with `None` (`deepagents>=0.7.16`), and do it from `after_agent` when using a hook.

B12. The approval that tools.* skipped

```python
agent = create_deep_agent(
    model=...,
    tools=[send_email],
    interrupt_on={"send_email": True},
    middleware=[CodeInterpreterMiddleware(ptc=["send_email"])],
    checkpointer=MemorySaver(),
)
```

Bug. Calling `tools.sendEmail(...)` from `eval` bypasses the normal tool path, so `interrupt_on` never fires for it.

Fix. Remove `send_email` from the PTC allowlist, or gate the `eval` tool itself (`interrupt_on={"eval": True}`).

B13. The "local" backend in production

```python
backend = FilesystemBackend(root_dir="/srv/app/data")      # deployed to the cloud
```

Bug. `virtual_mode` defaults to false, so there is no path restriction; the backend also writes internal artifacts (offloads, history) to real disk, and filesystem backends must never be used in deployed agents.

Fix. Use a store-backed `CompositeBackend` for durable data and a sandbox for code execution.

B14. The Slack form that posts twice

```python
@tool
async def ask_refund_approval(order_id: str, runtime):
    await post_block_kit_card(runtime, order_id)           # posts an approve/reject card
    answer = interrupt({"correlation": make_id(), "order": order_id})
    return answer
```

Bug. On resume the tool re-executes from the top, so the card is posted again.

Fix. Guard the posting step: skip it when the incoming channel event is already a `user_prompt_response`, and put the correlation ID in each button's `value`.

CHAPTER 77

Rapid-Fire Numbers and Defaults

> **At a glance:** These are the facts interviewers expect you to recall instantly. Read them as flash cards: cover the right column, say the answer, check.

Defaults and thresholds

Question	Answer
Default backend?	`StateBackend()` (thread-scoped)
General-purpose subagent by default?	On
`write_todos` by default?	Off (opt-in since 0.7)
`execute` by default?	Off (sandbox backends only)
`eval` interpreter by default?	Off (opt-in, beta)
Permission when no rule matches?	Allow
Subagent default mode?	`isolated`
Offload threshold?	Over 20,000 tokens; model sees path plus first 10 lines
Summarization trigger and keep?	85% of `max_input_tokens`; keep 10%
Summarization fallback?	170,000 tokens; keep 6 messages
Rubric `max_iterations`?	3
Interpreter limits?	64 MB heap, 5 s per eval, 4,000 chars, 256 PTC calls
`LocalShellBackend` limits?	120 s timeout, 100,000 output bytes
Tool and model retry defaults?	2 retries, backoff 2.0, initial delay 1 s, max delay 60 s, ±25% jitter
`ContextEditing` defaults?	Trigger 100,000 tokens, keep 3, placeholder `[cleared]`
Prompt-caching TTL?	5 minutes (override `1h`)
Frontend `recursionLimit` in Deep Agents?	10,000
`openai:` model strings use?	The Responses API
Skill `name` and `description` limits?	Name 1 to 64 chars matching the directory; description at most 1,024 chars
Skill body guidance?	About 5,000 tokens / 500 lines; files over 10 MB skipped
`read_file` default page?	2,000 lines
Async subagent default transport?	ASGI when `url` is omitted
Managed sandbox `idle_ttl_seconds` and `default_timeout`?	600 and 600
Max runtime schedules per owner per agent?	20

Question	Answer
MDA build size limit?	200 MB
OpenWiki Node requirement?	22.22.0 or later
OpenWiki page concurrency?	1 to 8, default 1

Version gates

Feature	Needs
Permissions	`deepagents>=0.5.2`
`rt.server_info` in factories	`>=0.5.0`
Subagent `response_format`	`>=0.5.3`
Rubric middleware	`>=0.6.5`
Custom `state_schema`	`>=0.6.6`
`mode="interrupt"` permissions	`>=0.6.8`
Fork subagents	`>=0.7.13`
Skills reset with `None`	`>=0.7.16`
Binary content offloading	`>=0.7.21`
Skill tools	`>=0.7.22`
Pinned skills	`>=0.7.23`
Async-subagent notifications	`>=0.7.26`
`when` predicate	`langchain>=1.3.3`
`ToolErrorMiddleware`	`langchain>=1.3.14`
Per-call HITL	`langchain>=1.4.6`
`langchain.mcp`	`langchain[mcp]>=1.4.0`
MDA memory and sandbox `runtime.backend`	`managed-deepagents>=0.8.0`
MDA runtime schedules	`>=0.9.0`

One-line answers

Question	Answer
What does `create_deep_agent` return?	A compiled LangGraph graph
Four capability categories?	Execution environment, context management, delegation, steering
First thing HITL needs?	A checkpointer, and the same `thread_id` on resume
Four HITL decisions?	approve, edit, reject, respond
Which subagent fields replace the parent's?	`tools` and `permissions`
Which do not inherit at all?	`middleware` and `skills`
Can a fork call `task`?	No

Question	Answer
`CompositeBackend` routing rule?	Longest matching path prefix wins
Where does `store=` go?	`create_deep_agent`, not the composite
What do permissions cover?	Built-in file tools only
Rule evaluation?	In order; first match wins; no match allows
Does a sandbox hold secrets?	Never; use host-side tools or an auth proxy
What does PTC skip?	The normal tool path, including `interrupt_on`
Why is memory after caching?	So memory edits do not invalidate the cached prefix
Summarization versus media?	Summarization drops media; offloading keeps it
`skills_metadata` reset value?	`None`, never `[]`
What replaces a built-in middleware?	A same-`.name` instance (0.7+)
Hook order for `after_*`?	Reverse (last to first)
What is `thread_id` versus `context`?	Conversation key versus per-run data
Subagent card link key?	Snapshot `id` equals the spawning `tool_call_id`
MCP versus ACP versus A2A versus AG-UI?	Tools, editor, other agents, frontend
A2A error for a terminal task ID?	`-32004`
A2A method not implemented?	`-32601`
Where do MDA instructions and skills live?	Context Hub, synced on deploy
What does `define_deep_agent` omit?	`backend`, `store`, `checkpointer`, `state_schema`
MDA hooks must be?	Async
MDA agent memory risk?	Shared by all callers and influenceable by them
What does a `202` from an HTTP channel mean?	Accepted, not completed
What does `--no-wait` skip?	Cron reconciliation (and is incompatible with Slack)
What does OpenWiki's Claim update decide?	Confirm, revise, add or retract

CHAPTER 78

Mock Interview Rounds

At a glance: Use these scripts with a friend, or read the questions aloud and answer without notes. Each round has a time limit and a set of "must-hit" points.

Round 1: 20-minute screen

1. *What is Deep Agents and when would you use it over a plain agent?* (Q1, Q9)
2. *What does* `create_deep_agent` *return, and what does that give you?* (Q2, Q3)
3. *Explain the default backend and how you would add durable per-user memory.* (Q14, Q18, Q19)
4. *How does human-in-the-loop work and what are its requirements?* (Q61, Q62)
5. *Name two security boundaries you would be careful about.* (Permissions do not cover `execute`; secrets never in a sandbox.)

Must-hit points. Harness on LangGraph; `StateBackend` default; composite plus namespace; checkpointer plus `thread_id`; permissions and sandbox limits.

Round 2: 45-minute deep dive

1. *Walk me through one turn of the agent.* (Q6)
2. *State the subagent inheritance rules, then design a research system with subagents.* (Q50, D1)
3. *Summarize how context is kept small.* (Q37, Q43)
4. *Review this permission list and this HITL code for bugs.* (Q15, Q21, B1 to B8)
5. *When would you pick dynamic over async subagents?* (Q55, Q56)
6. *How do retries, fallbacks and limits fit together?* (Q73, Q76, Q77)

Must-hit points. Middleware order; replace-not-merge; offload and summarize numbers; the `eval` bypass; non-idempotent tools.

Round 3: system-design loop (60 minutes)

Choose two of D1 to D10 and defend them. Interviewers look for: requirements clarified first; backend and scoping choices stated with reasons; security considered before they ask; failure modes (crash, outage, injection, double execution) named; and a sensible answer to "what would you cut for a first version?"

Round 4: 30-minute applied round (Managed Deep Agents, OpenWiki and build-alongs)

1. *What is Managed Deep Agents, and what do you give up compared with* `create_deep_agent`*?* (Q101, Q110)

2. *How is user memory decided for a Slack channel versus a DM?* (Q103, Q109)
3. *How do you secure an HTTP channel webhook?* (Q107, Q111)
4. *What are Claims in OpenWiki and why do updates stay cheap?* (Q114)
5. *Explain retrieve, offload, delegate and why the search tool returns paths.* (Q117, Q121)
6. *Walk me through the capstone and what you would harden first.* (Q123, Q124)

Must-hit points. Static `name` and files-to-capabilities map; memory defaults by run source; `verify` as the only barrier; confirm, revise, add or retract; shared backend instance; namespace includes the user; subagent `permissions` replace; rubric `max_iterations_reached` is a failure.

Questions you can ask the interviewer

- Which features are you running in beta, and how do you handle version drift?
- How do you evaluate agent quality (rubrics, Harbor evals, traces)?
- What is your tenancy model: thread, user or assistant scoped?
- Who owns approvals for destructive actions, and what is the latency budget for them?

APPENDICES

Appendices

Reference material to keep within reach: a cheat sheet, the trap list, version notes, a troubleshooting catalogue, a glossary, and the licences and notices that apply to this book.

IN THIS SECTION

A	Cheat Sheet: Code, Defaults and Decisions
B	The Trap List: Thirty Mistakes to Avoid
C	Version Notes and Documentation Differences
D	Troubleshooting and Edge-Case Catalogue
E	Glossary
F	Notices, Licences and Acknowledgements

APPENDIX A

Cheat Sheet: Code, Defaults and Decisions

```python
# ── Create ──────────────────────────────────────────────────────────
from deepagents import create_deep_agent, FilesystemPermission
from deepagents.backends import CompositeBackend, StateBackend, StoreBackend
from langchain.agents.middleware import TodoListMiddleware
from langgraph.checkpoint.memory import MemorySaver
from langgraph.store.memory import InMemoryStore

agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",           # or a model object
    tools=[my_tool],                                # + MCP via MCPAdapter
    system_prompt="...",
    memory=["/memories/AGENTS.md"],                 # always loaded
    skills=["/skills/"],                            # progressive disclosure
    backend=CompositeBackend(default=StateBackend(),
            routes={"/memories/": StoreBackend(namespace=lambda rt: (rt.server_info.user.identity,))}),
    permissions=[FilesystemPermission(operations=["write"], paths=["/skills/**"], mode="deny")],
    subagents=[{"name": "...", "description": "...", "system_prompt": "...", "tools": [...]}],
    middleware=[TodoListMiddleware()],              # opt-in planning
    interrupt_on={"send_email": True},              # HITL
    context_schema=Ctx, state_schema=MyState,
    checkpointer=MemorySaver(), store=InMemoryStore(),
)

# ── Run ─────────────────────────────────────────────────────────────
cfg = {"configurable": {"thread_id": "t1"}}
agent.invoke({"messages": [{"role": "user", "content": "..."}]}, config=cfg, context=Ctx(...))

# ── Resume after HITL ───────────────────────────────────────────────
from langgraph.types import Command
agent.invoke(Command(resume={"decisions": [{"type": "approve"}]}), config=cfg)

# ── Stream ──────────────────────────────────────────────────────────
for c in agent.stream(inp, stream_mode="updates", subgraphs=True, version="v2", config=cfg): ...
stream = agent.stream_events(inp, version="v3")      # → stream.subagents / stream.messages / stream.values
```

Defaults at a glance

Thing	Default
Backend	`StateBackend()` (thread-scoped)
General-purpose subagent	**On** (auto-added)
Task planning (`write_todos`)	**Off** (opt-in, since v0.7)
Code execution (`execute`)	**Off** (sandbox backends only)
JS interpreter (`eval`)	**Off** (opt-in, beta)
Permissions	**Allow** if nothing matches; first match wins
Summarization	Trigger 85% of `max_input_tokens` (fallback 170k tokens / 6 messages); keep 10%
Offload threshold	> 20,000 tokens → file + first 10 lines
Subagent mode	`isolated`
Rubric `max_iterations`	3

Thing	Default
Interpreter limits	64 MB · 5 s/eval · 4,000 chars · 256 PTC calls
Frontend `recursionLimit`	10,000
Prompt caching	On for Anthropic & Bedrock; TTL 5 min (override `1h`)
`openai:` strings	Responses API
Skill name / description	1–64 chars, matches directory / ≤ 1,024 chars
Skill body	≲ 5,000 tokens / 500 lines; files > 10 MB skipped
LocalShellBackend	120 s timeout, 100,000 output bytes
ToolRetry / ModelRetry	2 retries, backoff 2.0, initial delay 1 s, max 60 s, jitter
ContextEditing	trigger 100,000, keep 3, placeholder `[cleared]`

The decision table you'll be asked for

Need	Use
Persist across conversations	`CompositeBackend` + `StoreBackend` on `/memories/`
Per-user isolation	Namespace factory with `rt.server_info.user.identity`
Run code / install packages	Sandbox backend (`execute`)
In-process orchestration of tools/subagents	`CodeInterpreterMiddleware` (PTC, `task()`)
Restrict file access	`permissions=[FilesystemPermission(...)]`
Human approval	`interrupt_on` + checkpointer
Parallel workers	Multiple `task` calls · dynamic subagents · async subagents
Background, steerable work	Async subagent (`start/check/update/cancel`)
Quality gate	`RubricMiddleware` (beta)
Survive provider outage	`ModelFallbackMiddleware`
Survive transient errors	`ModelRetryMiddleware` / `ToolRetryMiddleware`
Stop runaway loops	`ModelCallLimitMiddleware` / `ToolCallLimitMiddleware`
Tool errors the model can fix	`ToolErrorMiddleware`
Mask PII	`PIIMiddleware`
Many tools, few relevant	`LLMToolSelectorMiddleware` / `ProviderToolSearchMiddleware`
Editor integration	ACP (`deepagents-acp`, `AgentServerACP`)
Tool servers	MCP (`langchain.mcp.MCPAdapter`)
Cross-framework agent calls	A2A (`/a2a/{assistant_id}`)
Frontend integration	`useStream` or AG-UI (`@ag-ui/langgraph`)
Hosted production	Managed Deep Agents / LangSmith Deployment
Repo wiki for coding agents	OpenWiki

APPENDIX B

The Trap List: Thirty Mistakes to Avoid

Backends & files

1. `FilesystemBackend` without `virtual_mode=True` → no path restriction; with shell access `virtual_mode` protects nothing.
2. `FilesystemBackend` / `LocalShellBackend` → never in deployed agents.
3. `StoreBackend` without a namespace → all users share data.
4. `StateBackend` writes outside a graph run don't take effect until the graph runs; big files get checkpointed every step.
5. Skill/memory files must be **seeded** (`create_file_data`) for `StateBackend`.
6. Stale `ContextHubBackend` writes fail → re-pull and retry.

Permissions & safety

7. Permissions cover only **built-in file tools**: not `execute`, MCP or custom tools.
8. Rule order matters; no match = **allow**; subagent `permissions` **replace** the parent's.
9. Secrets never go in a sandbox (auth proxy or host-side tools).
10. PTC and `task()` inside `eval` **bypass per-call HITL** → gate the `eval` tool.

Subagents & delegation

11. Subagent `tools` / `permissions` **replace**; `middleware` / `skills` **don't inherit**.
12. Forks can't call `task`; a fork `system_prompt` appends and reduces cache reuse.
13. Async: one worker slot per active run; premature polling; truncated task IDs; stale status.
14. Re-adding a `general-purpose` spec replaces the default.

HITL & steering

15. `interrupt_on`/resume need a **checkpointer** and a **stable** `thread_id`.
16. Decisions must match `action_requests` in count and order. `respond` ≠ deny.
17. Rubric: check `_rubric_status`; `on_evaluation` errors are swallowed; no `rubric` in state = no-op.

Middleware & profiles

18. Override by `.name` **replaces**, doesn't merge; never mutate `self` in hooks; `after_*` run in reverse.
19. `FilesystemMiddleware`, `SubAgentMiddleware`, permission middleware **can't be excluded**.
20. Profiles: register **before** creating agents; no wildcard key; provider profiles don't apply to model objects.

Context & memory

21. Memory is always loaded: keep it small, put procedures in skills.
22. `skills_metadata = []` ≠ `None`.
23. Shared writable memory = prompt-injection risk.
24. Cron interval must match the consolidation lookback; cron is UTC.
25. Summarization strips media from the model's view.

Models & protocols

26. `openai:` strings — Responses API by default.
27. MCP: `langchain[mcp]>=1.4.0`, beta; `MCPConfig` forces one protocol era; cache isolation per user; sampling/roots unsupported.
28. A2A: don't resend a terminal `taskId`; one enum-case family per client; `SubscribeToTask` not implemented.
29. AG-UI: set `emitInterruptOutcome`; adapter is TypeScript.
30. ACP: absolute path in Zed; stdio belongs to the client.

APPENDIX C

Version Notes and Documentation Differences

> **At a glance:** Documentation for a framework that ships often will disagree with itself in places. These are the differences worth knowing about, so that they do not surprise you in a code review or an interview. When two pages disagree, trust the package you have installed, and treat each item here as a prompt to check.

Naming and signatures that differ between pages

Topic	What differs	What to do
Sandbox protocol	The backends page uses `SandboxBackendProtocol`; the sandboxes page also mentions `SandboxBackendProtocolV2`.	Import from your installed package and read its docstring.
`files_update`	Described as deprecated, yet it still appears in the protocol reference.	Avoid it in new code; handle it if you maintain an older backend.
The `task` tool arguments	Some pages show `task(name=..., task=...)`, others show `task({subagent_type, description})`.	Inspect the tool schema your version generates.
Streaming chunk shape	Examples mix dictionary-style chunk access and tuple unpacking.	The shape depends on the `version=` argument you pass; test it.
`ContextHubBackend`	One page constructs it with no argument, another with a name (`ContextHubBackend("my-agent")`).	Check the constructor signature in your version.
Interpreter package version	The dynamic-subagents page asks for `langchain-quickjs>=0.2.0`; the interpreters page asks for `>=0.3.0`.	Use the newer minimum.
Skill file name	One tutorial's prose says `SKILLS.md`; the file is `SKILL.md`.	The file name is `SKILL.md`.
Custom middleware sample	A class example uses `SystemMessage` without importing it.	Import it from the messages module.

Things that drift over time

- **Version pins.** Examples pin versions such as `deepagents>=0.7.x` and `langchain>=1.3.x`. Features arrive in specific releases (this book flags the ones it knows), so confirm the gate before you quote a version.
- **Beta labels.** Managed Deep Agents is in public beta, and the interpreter, event-streaming, profiles, dynamic-subagents, rubric and MCP-adapter features are labelled beta. Beta features can change shape between releases.
- **Model names.** Model identifiers in examples (for instance `anthropic:claude-sonnet-4-6`) are placeholders for whichever model you use.
- **Regional and plan wording.** Pages about Managed Deep Agents word regional availability and plan limits slightly differently. Confirm region, quotas and pricing in the current documentation before you commit.
- **Code samples are condensed.** Where a sample was long, this book shows the essential shape. Treat condensed samples as a guide to structure rather than something to paste.

APPENDIX D

Troubleshooting and Edge-Case Catalogue

At a glance: When something misbehaves, the cause is almost always one of a few repeating patterns: **missing persistence** (no checkpointer, no store route), **wrong scope** (namespace, thread, assistant), **replace-not-merge** (subagent fields, middleware, permissions), **ordering** (permission rules, middleware), or a **boundary the feature does not cross** (permissions versus `execute`, interrupts versus `eval`). Use the tables below as a symptom-first index; each row names the likely cause and the fix.

How to debug a deep agent

1. **Reproduce with a thread.** Use a fixed `thread_id` and a checkpointer so you can inspect state and replay.
2. **Open the trace.** In LangSmith, filter by `lc_agent_name` to separate the coordinator from subagents; memory writes appear as tool calls.
3. **Inspect state.** Look at `files`, `todos`, `skills_metadata` and any custom keys before blaming the model.
4. **Check the layer.** Ask whether the fault is in the runtime (LangGraph), the framework (LangChain middleware) or the harness (Deep Agents defaults).
5. **Check versions.** Many features are beta or version-gated (see the version table in the chapter "Rapid-Fire Numbers and Defaults").

Persistence, interrupts and resume

Symptom	Likely cause	Fix
`interrupt_on` never pauses, or resume does nothing	No checkpointer	Pass `checkpointer=`; durable one in production
Resume starts a fresh conversation	`thread_id` changed between calls	Reuse the original `thread_id`
Resume errors on decision shape	Decision count or order differs from `action_requests`	Build one decision per request, same order
Model proceeds as if a deletion succeeded	`respond` used instead of `reject`	Use `reject` with a specific message
In-tool `interrupt()` resume rejected	Sent `{"decisions": ...}` instead of your payload	In-tool resume values are free-form
Side effect runs twice after resume	The interrupted node re-executes from the top	Make pre-`interrupt` work idempotent or guard it
Approval skipped for tools called from code	`task()` or `tools.*` inside `eval` bypass the normal path	Gate the `eval` tool; narrow the PTC allowlist
Approvals do not work with a persistent shell	`ShellToolMiddleware` sessions do not yet work with HITL interrupts	Use a sandbox `execute` tool with `interrupt_on`
Cannot downgrade after upgrading to 0.6	Delta checkpoints are unreadable by older versions	Plan upgrades as one-way

Files, backends and permissions

Symptom	Likely cause	Fix
Files lost between conversations	Path not routed to a store	`CompositeBackend` route to `StoreBackend`, pass `store=`
One user sees another's memory	Missing or too-coarse `StoreBackend` namespace	Namespace factory on `user.identity`
`/memories/` empty after redeploy	`InMemoryStore` or non-durable path	Use a durable store; seed shared files from application code
Skill or memory file "not found" with `StateBackend`	Files not seeded	Pass `files={path: create_file_data(text)}` at invoke time
A denied file is still readable	Earlier broad allow, or access via `execute`	Put specific denies first; add a catch-all deny; control `execute` separately
`NotImplementedError` from permissions	Sandbox default backend and a rule path outside any route	Keep every permission path under a route prefix
Directory delete refused	One denied descendant denies the whole operation	Adjust rules or delete files individually
Interrupt rule fires on every `ls` and `glob`	Unanchored pattern such as `/**/secrets`	Anchor with a literal prefix: `/secrets/**`
`execute` tool missing	Backend does not implement the sandbox protocol	Use a sandbox backend
Agent can read the whole disk	`FilesystemBackend(virtual_mode=False)`	`virtual_mode=True`, and never deploy filesystem backends
Stale write to a Hub-backed backend fails	Optimistic `parent_commit` conflict	Re-pull and retry
Large files slow every step	`StateBackend` checkpoints files on each step	Offload large artifacts to a store or sandbox
Parsers break after 0.7	`read_file` has no line-number gutter; empty `ls` returns "No files found"	Update the parsers

Skills, memory and context

Symptom	Likely cause	Fix
Skill never used	Wrong path level, `name` not matching directory, or vague description	`skills=["/skills/"]`; fix the name; sharpen the description
Skill edits not visible in an old thread	Skills cached per thread	Reset `skills_metadata` to `None`
All skills vanished after a reset	Reset to `[]` instead of `None`	Use `None`
Skill tools fail mid-conversation	Gated until `SKILL.md` was read; summarization dropped the read	Have the agent re-read the skill, or pin it
Wrong skill chosen	Overlapping descriptions	Make descriptions distinct; reduce the number of similar skills
Custom subagent ignores skills	Skills do not inherit	Give the subagent its own `skills`
Memory grows and every run slows	Always-loaded memory bloat	Keep `AGENTS.md` tiny; move procedures to skills

Symptom	Likely cause	Fix
Agent behaves strangely after reading shared memory	Prompt injection through shared writable memory	Make shared scopes read-only; deny writes; review
Consolidation duplicates or drops memories	Cron interval differs from lookback window; cron is UTC	Match them and convert time zones
Earlier screenshots "forgotten"	Summarization is text-only	Save media to files; delegate image inspection to a subagent
Context overflows despite offloading	Many medium results below 20,000 tokens each	Lower summarization trigger, use context editing, or delegate to subagents
Provider profile has no effect	Applied to a model object, or registered after the agent was built	Use a `provider:model` string and register first
`openai:` model behaves differently	Responses API is the default	Pre-build the model with `use_responses_api=False`

Delegation and steering

Symptom	Likely cause	Fix
Subagent cannot use a tool the parent has	`tools` replaces, not extends	List every tool the subagent needs
Subagent ignores parent's permission rules	Its own `permissions` replaced them	Restate the rules in the subagent
Fork subagent cannot delegate	Forks cannot call `task`	Use an isolated subagent
Compiled subagent fails to exchange results	State has no `messages` key	Add it
No subagent events in the stream	Missing `subgraphs=True` or wrong `version`	Set both, or use event streaming
UI cards missing	Snapshot `id` does not match the spawning `tool_call_id`	Index subagents by tool-call ID
Todos missing in the UI	`TodoListMiddleware` not added (opt-in since 0.7)	Add it
Dynamic fan-out is slow or rate-limited	No built-in concurrency caps	Chunk the batch; add limits
`JSON.parse` fails on a `task()` result	`responseSchema` already returns an object	Use the result directly
Async launches wait in a queue	Not enough worker slots	Provision one slot per active run
Supervisor polls right after launch	Premature polling	Instruct it to return control to the user
`check` or `cancel` says task not found	Truncated task ID	Copy IDs exactly; `list` first
Supervisor reports an old status	Stale chat history	Always `check` or `list`
Waiting async task never answered	Supervisor cannot answer it	The user answers through the subagent's own thread
Rubric loop ends but output still fails	`max_iterations_reached`	Check `_rubric_status` after `invoke`
Rubric never runs	No `rubric` key in state	Provide it in the input

Middleware, reliability and cost

Symptom	Likely cause	Fix
Duplicate-middleware error	Older than 0.7	Upgrade or avoid same-name instances
`ValueError` when excluding middleware	Cannot exclude Filesystem, SubAgent or permission middleware	Restrict via configuration instead
Replaced middleware loses settings	Replacement is not merged	Fully configure the replacement
Hook state corrupted under load	Mutated `self`	Store values in graph state
`jump_to` ignored	`can_jump_to` not declared	Declare it
Costs spike	Retries on non-transient errors, fan-out, forks	Retry only retryable errors; add call limits; review forks
Duplicate emails or charges	Retrying a non-idempotent tool	Idempotency keys; exclude from retry; gate with approval
Tool errors halt the whole run	No `ToolErrorMiddleware`	Convert recoverable errors to messages
Retry never triggers	Error converted to a message before the retry layer	Re-check layering and test it
Run hits a limit unexpectedly	`thread_limit` accumulates across the conversation	Use `run_limit` for per-turn caps
`ToolCallLimitMiddleware` raises `NotImplementedError`	`exit_behavior="end"` on a global limit	Use it only with a single named tool
Provider tool search raises `ValueError`	Unsupported provider	Use Anthropic or OpenAI as documented, or `LLMToolSelectorMiddleware`

Sandboxes and secrets

Symptom	Likely cause	Fix
Surprise cloud bill	Sandbox not torn down	`finally` teardown, idle TTL
Sandbox contents disappeared (MDA)	`idle_ttl_seconds` elapsed; deletion is unrecoverable	Raise the TTL; persist artifacts out of the sandbox
Agent sees API keys	Secrets placed in env vars or files	Host-side tools or the auth proxy
Files leak between users	Assistant-scoped sandbox shared across users	Thread-scoped sandbox; TTL; snapshots
Sandbox cannot see skill scripts	Files not synced into the sandbox	Upload before the run
`download_files` returns no content	Per-path `error` set	Check `error` first
`runtime.backend` is `None` (MDA)	No sandbox declared	Guard every call
`mda dev` works but deployed fails	Local temp-folder fallback masked a provider failure	Test on a development deployment

Protocols and platform

Symptom	Likely cause	Fix
MCP target launches a subprocess	A string naming a local script was treated as a path	Pass only http(s) URLs; `MCPAdapter` rejects other strings
User B sees user A's MCP tools	Cache not isolated by identity	Isolate by verified identity
MCP elicitation repeats work	Tool re-runs from the start on resume	Idempotent pre-request work
`MCPConfig` forced everyone to legacy	One legacy-only server in the fleet	Use `ClientGroup`
MCP transport failure crashes the run	Transport errors raise; only `isError=True` becomes a message	Retry middleware around the tool; handle exceptions
A2A `-32004`	Resent a terminal `taskId`	Send only `contextId` for follow-ups
A2A `-32601`	Unsupported v0.3 name, `SubscribeToTask` or push config	Use implemented v1.0 methods
A2A enum values inconsistent	Mixed v1.0 and v0.3 families	One family per client
AG-UI interrupt arrives in a legacy shape	`emitInterruptOutcome` not set	Set it to `true`
AG-UI graph fails to load	`graphId` mismatch	Match a key in `langgraph.json`
Zed cannot start the ACP agent	Relative path in `command`	Use an absolute path
Frontend run stops early	`recursionLimit` too low	Raise it (Deep Agents defaults to 10,000)

Managed Deep Agents

Symptom	Likely cause	Fix
Search never appears in the trace	Provider tool dictionary does not match the model	Match the dictionary to the provider
Hooks work locally but fail on MDA	Synchronous middleware hooks	Make hooks async
`permissions` seems ignored	Declaring `sandbox/` clears `permissions`	Restrict via the tools you provide
Deploy halts on a Context Hub conflict	Hub edits since the last deploy	Choose `--context-strategy overwrite` or `keep-hub` in non-interactive shells
Hub edit disappears after deploy	Project files are the source of truth	Edit the project files
Schedule not created or removed	`--no-wait` skipped reconciliation	Deploy without `--no-wait`
"schedule is not static"	Computed values in a schedule	Use literals
HTTP 422 on a schedule's thread	Persistent thread ID is not an existing UUID	Create the thread in Studio and copy its UUID
HTTP channel returns 400	`thread_id` is not a UUID	Map external IDs with `uuid.uuid5`
HTTP channel returns 401	`verify` returned false	Fix the signature check over raw bytes
401 on `/mcp`	Missing `x-api-key`	Send the header

Symptom	Likely cause	Fix
403 on `/mcp`	Revoked key, or key from another workspace	Use a valid key from the owning workspace
User memory never mounts	Caller is a service principal, or the run is a channel or API run with default policy	Authenticate a person; supply an `allow` policy that widens within the authenticated caller
User connection fails on a channel	Slack and Studio cannot collect per-caller API keys	Use OAuth grants
Factory behaves differently between runs	Factory has side effects or non-deterministic output	Make it pure; it also runs when the server reads schemas
`context` is `None` in a tool	Channel runs carry no application context	Authorize from `runtime.server_info.principal`
Slack file never arrives	Attachment is never automatic	Tell the agent when to call `attach_file`; sandbox required
Build rejected	Build over 200 MB	Remove generated or large files

OpenWiki

Symptom	Likely cause	Fix
Blank wiki, no error	Gateway rejects non-streaming requests and returns empty content	`OPENWIKI_OPENAI_COMPATIBLE_STREAMING=true`
`--init` removed my edits	`--init` regenerates the wiki and Claims	Keep edits in `INSTRUCTIONS.md`, which is preserved
Ignored directory still described	`.openwikiignore` is a read boundary, not a topic guarantee	Remove sensitive material from the repository
No CI pull request	Neither wiki nor Claims changed	Expected behaviour
Visualizer shows nothing offline	Libraries load from a public CDN	Needs internet access
Personal-mode schedule unavailable	Scheduling is macOS-only	Use another scheduler

APPENDIX E

Glossary

> **At a glance:** Short definitions of the terms used across the book, in alphabetical order. Where a term has a chapter of its own, the entry points you to it by name.

A2A (Agent2Agent). A protocol for agents from different frameworks to exchange messages and tasks over JSON-RPC 2.0. LangSmith Agent Server serves each assistant at `/a2a/{assistant_id}`.

ACP (Agent Client Protocol). A standard for communication between a coding agent and a code editor. Deep Agents provides `AgentServerACP`, run over stdio.

Agent. A language model running in a loop with tools: call the model, run requested tools, append results, repeat until no tool is requested.

Agent Auth. LangSmith's managed OAuth for agents that act on a user's behalf: the first use interrupts with a consent URL, then a token is stored and refreshed.

Agent Card. A discovery document each A2A agent publishes (at `/.well-known/agent-card.json`) describing its capabilities.

Agent harness. An opinionated, pre-assembled agent built on a framework and runtime. Deep Agents is a harness.

Agent Server. LangSmith's hosting runtime for graphs: threads, runs, a store, cron jobs, authentication and streaming APIs.

AG-UI (Agent User Interaction Protocol). An open, event-based protocol connecting agents to user-facing applications.

AGENTS.md. A conventional Markdown file of persistent instructions for agents. Deep Agents loads memory files of this kind at startup; OpenWiki links its wiki through it.

Assistant. A configured instance of an agent. Memory and files can be scoped to one assistant or shared across all.

ASGI transport. The in-process way an async subagent is reached when it is co-deployed in the same `langgraph.json` (no `url`).

Async subagent. A subagent that runs as a background task on an Agent Protocol server. The supervisor receives a task ID and can check, update, cancel and list tasks.

Auth proxy (sandbox). A component outside a sandbox that injects credentials into outbound requests by destination host, so secrets never enter the sandbox.

Backend. The pluggable layer behind the file tools (`ls`, `read_file`, `write_file`, `edit_file`, `glob`, `grep`, `delete`) that decides where the bytes live.

`BackendProtocol`. The interface a custom backend implements. Results carry an `error` field; implementations return errors rather than raising.

Batched interrupt. The default HITL behaviour: all tool calls needing approval in one model turn produce a single pause.

Checkpoint. A saved snapshot of graph state after a step. Checkpoints enable resume, time travel and crash recovery.

Checkpointer. The component that stores checkpoints (for example `MemorySaver` in development or a database-backed saver in production).

Claim (OpenWiki). A material proposition behind a factual wiki page, citing exact repository evidence. Updates confirm, revise, add or retract Claims.

`CodeInterpreterMiddleware`. Middleware (package `langchain-quickjs`) adding an `eval` tool that runs JavaScript in a QuickJS WebAssembly workspace.

`CompiledSubAgent`. A wrapper that turns any compiled LangGraph graph with a `messages` state key into a subagent.

`CompositeBackend`. A backend that routes by path prefix to other backends; the longest matching prefix wins.

Connection (MDA). A workspace-level record that holds credentials for an external service, resolved at run time as agent-owned or user-owned.

Content block. A typed piece of message content (text, image, audio, file) used for multimodality.

Context (runtime). Immutable per-run input (user ID, role, flags) passed with `context=` and read through `runtime.context`. It is not added to the prompt automatically.

Context engineering. Everything that decides what the model sees on each call and how the harness keeps that window small: input shaping, offloading, summarization, isolation and memory.

Context Hub. LangSmith's store for a managed agent's deploy-owned context (instructions, skills, optional memory) and the repository behind `ContextHubBackend`.

Context quarantine. The effect of subagents: intermediate tool calls never enter the parent's window; only one final report returns.

`contextId` / `taskId` (A2A). `contextId` groups messages into one conversation and maps to a LangGraph `thread_id`; `taskId` identifies a single request within it.

Cron. A scheduled trigger using a standard five-field expression; times are UTC unless a time zone is given.

`create_deep_agent`. The function that builds a Deep Agent and returns a compiled LangGraph graph.

`DeepAgentState`. The base state class for custom state schemas; subclassing it keeps the messages reducer.

`define_deep_agent`. The Managed Deep Agents constructor that returns a definition for the platform to compile.

Delta checkpoint. A beta storage mode (since 0.6) that stores per-step deltas to keep checkpoints small; it prevents rolling back to older versions.

Dynamic subagents. Subagents dispatched from interpreter code via `task()`, enabling fan-out, verification, tournaments and loops.

Durable execution. The property that a run can resume from its last checkpoint after a crash, timeout or pause.

Elicitation (MCP). A server's request for input from the user, converted by the adapter into a LangGraph interrupt.

Event streaming. The typed projection API (`stream_events`, `version="v3"`) with a first-class `stream.subagents`.

`eval` tool. The interpreter's tool for running JavaScript. Gating it is how you require approval for programmatic tool calls and dynamic dispatch.

Fork (subagent). A subagent that inherits the parent's conversation and state; it cannot call `task`.

Framework. A library of building blocks for agents. LangChain is the framework layer.

General-purpose subagent. The subagent added by default with filesystem tools and inherited skills.

Graph factory. A function that builds a graph (or definition) per run, used to manage resources such as sandboxes or MCP connections.

Grounded Claims. See *Claim*.

Harbor. The task format and runner used for Managed Deep Agents evals.

Harness profile. Registered defaults that shape the agent for a provider or model: prompt text, tool overrides, excluded tools and middleware, extra middleware and the general-purpose subagent.

HITL (human-in-the-loop). Pausing before specific tool calls so a human can approve, edit, reject or respond.

Hot and cold memory (MDA). `AGENTS.md` at a mount root is hot (loaded every call); other memory files are cold (read on demand).

Interrupt. A pause in a graph created by `interrupt(value)`; the value is returned under `__interrupt__` and `Command(resume=...)` continues the run.

Isolated (subagent mode). The default: a fresh instance that sees only the task description and its own prompt.

LangChain. The framework layer: chat models, messages, tools, `create_agent` and middleware.

LangGraph. The runtime layer: state graphs, persistence, interrupts, streaming and deployment.

LangSmith. The platform for tracing, evaluation, deployment, sandboxes, Context Hub and managed agents.

LLM Gateway. A LangSmith service that routes model calls to providers; model IDs use `langsmith:` with a slash between provider and model.

MCP (Model Context Protocol). An open standard for giving applications' tools and context to language models.

`MCPAdapter`. The `langchain.mcp` class that discovers an MCP server's tools and converts them into LangChain tools.

MDA (Managed Deep Agents). LangSmith's hosted way to run Deep Agents defined as a project of files, deployed with the `mda` CLI.

Memory. Persistent, agent-updatable knowledge loaded at startup (`memory=[...]`) and, when routed to a store, shared across threads.

Middleware. Hooks that wrap the agent loop (before and after the model, around model and tool calls) to add behaviour without changing the loop.

Multimodality. Inputs and outputs beyond text: images, audio, video and files.

Namespace (store). A tuple of strings that partitions store data, produced by a namespace factory.

Node-style hook. A hook that runs at a fixed point in sequence (`before_agent`, `before_model`, `after_model`, `after_agent`).

OKF (Open Knowledge Format). The Markdown-with-front-matter format OpenWiki emits (version 0.2).

Offloading. Moving large tool inputs and results into files, leaving a path and a preview in context.

OpenWiki. An open-source CLI that writes and maintains a Markdown wiki of a repository or personal sources for coding agents.

Permissions. Declarative allow, deny and interrupt rules (`FilesystemPermission`) on the built-in file tools; first match wins and no match allows.

`PatchToolCallsMiddleware`. Middleware that repairs dangling or malformed tool calls so the next model call is valid.

Principal (MDA). The verified caller on `runtime.server_info`: an `id`, a `kind` (person, service or channel) and claims.

Progressive disclosure. Loading skills in stages: metadata at startup, full instructions on use, resources as needed.

Prompt caching. Provider-side caching of the static prompt prefix, automatic for Anthropic and Bedrock models.

Provider profile. Registered defaults for how a model object is constructed (init kwargs).

PTC (programmatic tool calling). Calling allowlisted tools from interpreter code as `tools.camelCaseName(...)`.

QuickJS. The small JavaScript engine, compiled to WebAssembly, behind the interpreter.

RBAC. Role-based access control for your team on a LangSmith workspace (Admin, Editor, Viewer, custom roles).

Reducer. A function that decides how a node's update merges into a state key (for example append for messages).

Responses API. OpenAI's API that `openai:` model strings use by default in Deep Agents.

Rubric. A newline-delimited checklist of verifiable criteria used by `RubricMiddleware` to grade output.

Runtime. The execution layer. In LangGraph it executes graphs; in MDA the `runtime` object also exposes `server_info`, `channel` and `backend`.

Sandbox. An isolated environment with its own filesystem and shell, exposed as a backend that adds the `execute` tool.

Skill. A folder with a `SKILL.md` (frontmatter plus instructions) and optional scripts, references and assets, loaded on demand.

State. The typed, mutable, checkpointed data a graph passes between nodes (`messages`, `files`, `todos`).

Store. A cross-thread key-value space organized by namespaces; the basis of long-term memory.

Streaming namespace. The tuple that identifies which (sub)graph a streamed event came from; a `tools:<id>` segment marks a subagent.

Subagent. A child agent the main agent invokes through `task`, with its own prompt, tools and context.

Summarization. Compacting older history into a text summary when the context nears the model's limit, while saving the full text to the filesystem.

`task` tool. The built-in tool that spawns a subagent and returns its final report.

Thread. One conversation, identified by `thread_id` and persisted by the checkpointer.

`TodoListMiddleware`. Middleware that adds the `write_todos` tool and a `todos` state key (opt-in since 0.7).

Tool. A function the model may call; the schema is inferred from type hints and docstring.

`ToolRuntime`. The runtime object given to a tool, exposing context, state and store.

Wrap-style hook. A hook that runs around a model or tool call and decides whether and how often the call executes.

APPENDIX F

Notices, Licences and Acknowledgements

At a glance: This book describes software owned by others and adapts material from openly licensed documentation. This chapter records the licences, trademarks and acknowledgements that apply, including the notice the MIT License asks for.

Independent publication

This book is an independent, unofficial guide. It is not published, sponsored or endorsed by LangChain, Inc., and it is not part of the official documentation. When this book and the official documentation disagree, the official documentation and the software you have installed are authoritative.

Source material and the MIT License

Parts of this book summarise, adapt and quote from two open-source works: the LangChain documentation (published at docs.langchain.com from the repository `langchain-ai/docs`) and the Deep Agents software repository (`langchain-ai/deepagents`). Both are published under the MIT License. As that licence requires, its notice is reproduced here in full. It applies to the adapted source material, not to this book as a whole.

> MIT License
>
> Copyright (c) 2025 LangChain
>
> Copyright (c) LangChain, Inc.
>
> Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:
>
> The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.
>
> THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Trademarks

LangChain, LangGraph, LangSmith and Deep Agents are product names of LangChain, Inc. All other product, company and service names mentioned in this book, including those of model providers, cloud services, editors and messaging tools, belong to their respective owners. They are used only to identify the software being described, and their use does not imply any affiliation or endorsement.

How this book was made

The text, diagrams and code samples were drafted with the assistance of an AI model (Claude, from Anthropic), working from the public documentation named above and under the direction of the author. The author is responsible for the book's contents. Diagrams were drawn for this book; those labelled *schematic illustration* show the shape of an interface rather than a literal screenshot.

Code, corrections and updates

The code samples are collected in a companion repository, organised by part and chapter and released under the MIT License:

> **github.com/sadiqgpasha89/deep-agents-guide-code**

Report mistakes or request updates by opening an issue in that repository. Versions, beta labels and model names reflect the documentation as it stood in October 2026.

Acknowledgements

Thanks to the LangChain team for publishing documentation that is open, detailed and candid about its own rough edges, to the maintainers of LangChain, LangGraph, LangSmith and Deep Agents, and to everyone who answers questions in the community.